

Scientific Data Analysis and
Performance Optimization in Python

Pythonによる データ解析と高速化

Ichiro KOMAE

2026/08/24

目次

はじめに	1
第1章 このテキストの使い方	3
1.1 対象と到達目標	3
1.2 章のつながり	3
1.3 コード例と実行結果の読み方	4
1.4 章末問題の進め方	4
1.5 検証と記録	5
1.6 参考資料の使い分け	5
1.7 総合課題で作る解析の例	6
第2章 シェルとファイル操作	7
2.1 シェルの基本と環境	7
2.1.1 Windows では WSL を使う	7
2.2 コマンドの基本と使い方	8
2.2.1 コマンド、オプション、引数	8
2.2.2 困ったときは説明を読む	8
2.3 ファイルとディレクトリの操作	9
2.3.1 今どこにいるかを確認する: <code>ls</code>	9
2.3.2 移動する: <code>cd</code>	12
2.3.3 絶対パスと相対パス	13
2.3.4 ディレクトリを作る: <code>mkdir</code>	14
2.4 コピー、移動、削除	15
2.4.1 コピーする: <code>cp</code>	15
2.4.2 移動する、名前を変える: <code>mv</code>	16
2.4.3 削除する: <code>rm</code>	17
2.4.4 ワイルドカード	18

2.5	ファイルの内容確認とデータ抽出	18
2.5.1	cat	19
2.5.2	less	19
2.5.3	head	20
2.5.4	tail	20
2.5.5	wc	20
2.5.6	grep	21
2.5.7	awk	21
2.5.8	パイプ	22
2.6	URL からファイルを取得する	22
2.6.1	curl	23
2.6.2	wget	23
2.7	圧縮ファイルを扱う	24
2.7.1	gzip: 1 つのファイルを圧縮する	24
2.7.2	tar: 複数ファイルやディレクトリを 1 つに束ねる	25
2.7.3	zip: 配布しやすい形式	25
2.8	環境変数と PATH	26
2.8.1	環境変数を使う	26
2.8.2	展開と引用	27
2.8.3	PATH と実行ファイル	28
2.9	リダイレクトとログ保存	28
2.9.1	echo、標準出力、標準エラー	28
2.9.2	> と >>	29
2.9.3	1> と 2>	29
2.9.4	tee で見ながら保存する	30
2.10	権限とスクリプトの実行	30
2.10.1	権限を読む	30
2.10.2	スクリプトを直接実行する	31
2.11	章末問題	32
第 3 章	Python 環境を整える	35
3.1	Python の確認と管理方針	35
3.1.1	何を分けて考えるべきか	35
3.1.2	今の Python を確認する	35
3.2	pyenv による複数バージョンの管理	36
3.2.1	macOS での準備	36
3.2.2	WSL / Ubuntu での準備	36
3.2.3	pyenv を入れてシェルへ組み込む	37

3.2.4	バージョンを入れて切り替える	38
3.3	venv でプロジェクトごとの仮想環境を作る	39
3.3.1	pip と python -m pip	40
3.4	解析用ライブラリと JupyterLab	41
3.4.1	JupyterLab を併用する	41
3.5	プロジェクト管理ツール uv の利用	42
3.5.1	プロジェクトの初期化	42
3.5.2	依存関係の追加と環境の同期	43
3.5.3	コマンドの実行とバージョン固定	43
3.5.4	標準ライブラリと外部ライブラリ	44
3.5.5	解析用のライブラリを入れる	44
3.6	引数・設定ファイルとコードの分離	45
3.6.1	コマンドライン引数と設定ファイルを使い分ける	45
3.6.2	設定ファイルとマジックナンバーの排除	46
3.7	章末問題	47
第 4 章	Python の基本	49
4.1	データ型と基本操作	49
4.1.1	変数と基本型	49
4.1.2	数値と演算子	51
4.1.3	文字列	52
4.2	データ構造	52
4.2.1	リスト、タプル、辞書	52
4.2.2	リストのメソッド	53
4.2.3	インデックスとスライス	54
4.3	制御構文	55
4.3.1	条件分岐	55
4.3.2	反復処理	57
4.4	関数とモジュール	59
4.4.1	関数を分ける理由	59
4.5	保守と実践的な書き方	62
4.5.1	エラーは失敗ではなく情報である	62
4.6	章末問題	65
第 5 章	入出力とファイル	67
5.1	テキストファイルの入出力と探索	67
5.1.1	ファイルを開き、読み書きの基本を押さえる	67
5.2	文字列データの解析と整形	70
5.2.1	CSV を列名で安全に読む	70

5.2.2	文字列を値へ変換し、結果を書き出す	71
5.3	オブジェクトと構造化データの保存	72
5.3.1	JSON と Pickle / PKL	72
5.4	発展的なトピック	74
5.4.1	日付と時刻を数値として扱う	74
5.4.2	保存形式について: ベストプラクティスへのステップ	74
5.5	章末問題	76
第 6 章	NumPy による数値計算	79
6.1	NumPy の考え方	79
6.1.1	配列計算の基本概念	79
6.1.2	ndarray と属性	80
6.1.3	Python, C, NumPy でのデータ型の違い	80
6.1.4	NumPy が速くなりやすい理由	81
6.2	最小限のコード例	82
6.2.1	配列を作る	82
6.2.2	よく使う操作	82
6.2.3	要素ごとの演算とユニバーサル関数	83
6.2.4	集計と axis	84
6.2.5	添字、スライス、条件抽出	85
6.2.6	形を変える	85
6.2.7	配列をつなぐ、分ける	86
6.2.8	形とブロードキャスト	87
6.2.9	統計量とヒストグラム	88
6.3	よくある落とし穴	88
6.3.1	演算子の意味を取り違える	88
6.3.2	axis と条件式	89
6.3.3	dtype と np.empty	89
6.3.4	コピーとビュー	90
6.3.5	浮動小数点数の比較	90
6.4	解析へのつながり	91
6.4.1	解析で NumPy を使うときの視点	91
6.4.2	差分、ソート、ヒストグラム	91
6.4.3	線形代数の入口	91
6.4.4	公式資料の使い方	93
6.5	章末問題	93
第 7 章	pandas による表形式データ解析	95
7.1	pandas の考え方	95

7.1.1	Series と DataFrame	95
7.1.2	最初に見るべき情報	96
7.1.3	index と列名の役割	97
7.1.4	pandas が速くなりやすい理由	97
7.2	最小限のコード例	97
7.2.1	列選択と行抽出	97
7.2.2	列を追加する	98
7.2.3	欠損値と型	99
7.2.4	要約統計と groupby	99
7.2.5	表の形を変える	100
7.2.6	時刻、文字列、カテゴリ	101
7.2.7	入出力	102
7.2.8	大きな CSV を分割して集計する	103
7.2.9	表を LaTeX へ出力する	104
7.2.10	plot() は確認用として使う	105
7.3	よくある落とし穴	105
7.3.1	SettingWithCopy と loc	105
7.3.2	object 型のまま計算しない	105
7.3.3	行ループは最後の手段である	105
7.3.4	欠損値と比較演算を混同しない	107
7.4	解析へのつながり	108
7.5	章末問題	108
第 8 章	Polars による高速な表処理	111
8.1	Polars の考え方	111
8.1.1	index を持たない DataFrame	111
8.1.2	式で列変換を書く	112
8.1.3	eager と lazy	112
8.2	最小限のコード例	113
8.2.1	select() と filter()	113
8.2.2	with_columns() で列を増やす	113
8.2.3	group_by() と集計	114
8.2.4	join と表の変形	115
8.2.5	欠損値、null、NaN	116
8.2.6	文字列と条件分岐	116
8.2.7	lazy API と scan_parquet()	117
8.2.8	CSV と Parquet の入出力	117
8.3	よくある落とし穴	118

8.3.1	pandas の書き方をそのまま持ち込まない	118
8.3.2	map_elements() や Python の lambda を多用しない	118
8.3.3	lazy API では collect() を忘れない	118
8.3.4	欠損値は null と NaN を区別する	118
8.4	解析へのつながり	118
8.5	章末問題	119
第 9 章	Matplotlib による可視化	121
9.1	基本的な作図と可視化	121
9.1.1	最小限のヒストグラム	121
9.1.2	軸ラベル、凡例、目盛り	122
9.1.3	plot 関数へ渡すデータ	123
9.1.4	最初に覚える主要な描画関数	123
9.1.5	ヒートマップとカラーバー	124
9.1.6	全天図と投影: Hammer 図法と Mollweide 図法	125
9.2	実践的な作図 (データに応じた図の選択)	127
9.2.1	相関の可視化: 論理的改善ステップ	127
9.2.2	複数の図を並べる (subplots)	129
9.2.3	単純な横並び以外: subplot_mosaic と GridSpec	130
9.2.4	凡例とレイアウト	131
9.2.5	エラーバーと対数軸 (log-log プロット)	132
9.2.6	結果例の読み方	133
9.2.7	画像フォーマットの選択: ラスタとベクタ	133
9.3	オブジェクト指向 API による複雑な図の管理	134
9.3.1	Figure, Axes, Axis, Artist の役割	134
9.3.2	オブジェクト指向 API で描く	138
9.3.3	補助関数で作図をまとめる	138
9.4	見た目とスタイルの管理	139
9.4.1	マーカー、線種、色の指定	139
9.4.2	rcParams で見た目をそろえる	141
9.4.3	cmap の選び方	141
9.4.4	設定ファイル (.mplstyle) で見た目を共通化する	142
9.4.5	共通スタイルの一部だけを上書きする	143
9.5	比較しやすい図を作る	143
9.5.1	Matplotlib の数式表記を使う	143
9.6	よくある落とし穴	144
9.6.1	plt と ax を混ぜて迷子になる	144
9.6.2	文字列がカテゴリ軸になる	144

9.6.3	<code>savefig</code> と余白調整の順序	144
9.6.4	公式資料から似た図を探す	145
9.7	章末問題	145
第 10 章	SciPy による数値計算とフィット	149
10.1	SciPy の考え方	149
10.1.1	NumPy の上にある専門工具箱	149
10.1.2	関数形を仮定して数値を取り出す	150
10.1.3	この本でよく使う分布	150
10.2	最小限のコード例	151
10.2.1	<code>scipy.stats</code> による要約統計と分布	151
10.2.2	<code>curve_fit</code> によるパラメータ推定	152
10.3	フィットを進める手順	153
10.3.1	初期値、制約、残差	153
10.3.2	<code>sigma</code> を使った重み付きフィット	154
10.3.3	<code>scipy.interpolate</code> による補間	156
10.3.4	<code>scipy.signal</code> による平滑化とピーク検出	156
10.3.5	<code>scipy.integrate</code> と <code>scipy.constants</code>	157
10.4	よくある落とし穴	157
10.4.1	モデル選択の理由がないままフィットしない	157
10.4.2	初期値と制約を無視しない	157
10.4.3	補間と外挿を混同しない	158
10.4.4	信号処理ではサンプリング間隔を意識する	158
10.5	解析へのつながり	158
10.6	章末問題	159
第 11 章	Astropy による時刻・単位・天体データ処理	161
11.1	Astropy の考え方	161
11.1.1	数値だけでなく意味も持たせる	161
11.1.2	時刻系と座標系を自前で持たない	162
11.2	最小限のコード例	162
11.2.1	<code>units</code> と <code>Quantity</code>	162
11.2.2	物理定数	163
11.2.3	<code>Time</code> による時刻表現	163
11.2.4	時刻差と <code>TimeDelta</code>	164
11.2.5	<code>SkyCoord</code> による座標変換	164
11.2.6	<code>Table</code> と <code>QTable</code>	165
11.2.7	Astropy Table と pandas の行き来	165
11.2.8	<code>astropy.io.fits</code> による FITS の読み書き	166

11.2.9	ASCII や ECSV も読める	166
11.3	よくある落とし穴	167
11.3.1	.value を早く使い過ぎない	167
11.3.2	時刻系を省略しない	167
11.3.3	座標変換には観測条件が必要なことがある	167
11.3.4	DataFrame への変換で情報が落ちる	167
11.4	解析へのつながり	167
11.5	章末問題	168
第 12 章	自作モジュールを作って再利用する	169
12.1	なぜ自作モジュールにするのか	169
12.2	uv でパッケージ開発を始める	170
12.2.1	最初の初期化	170
12.2.2	pyproject.toml の役割	171
12.3	具体例: ヒストグラムとフィットをまとめたパッケージ	171
12.3.1	モデル関数を分ける	172
12.3.2	pytest で最小のテストを書く	173
12.3.3	ヒストグラムを作って保存する関数	174
12.3.4	公開インタフェースを __init__.py で整える	177
12.3.5	使い方の例	177
12.4	docstring、型ヒント、README	179
12.4.1	docstring と型ヒント	179
12.4.2	README に書くべきこと	179
12.5	pip install . と pip install -e .	180
12.5.1	通常インストール	180
12.5.2	editable install	180
12.5.3	インストール確認	181
12.6	Git リポジトリへ置いて共有する	181
12.7	依存関係をどう書くか	182
12.8	章末問題	182
第 13 章	Python 解析の高速化	185
13.1	まず測る	185
13.1.1	Jupyter では %timeit も使える	185
13.2	計算量の見積もり	186
13.2.1	オーダーとは何か	186
13.2.2	オーダーが改善する例: 重複判定	188
13.2.3	オーダーは同じで定数倍だけ改善する例	188
13.3	実装を軽くする	189

13.3.1	NumPy と pandas による高速化	189
13.3.2	メモリとコピーの管理	189
13.3.3	分割読込み、並列化、ストリーミング	190
13.4	プロファイリングと低レベル実装	191
13.4.1	プロファイラを使う	191
13.4.2	C や C++ へ書き換えるという選択肢	192
13.4.3	Python から C/C++ を呼び出す	192
13.4.4	外部実装を参考にする時の見方	193
13.5	結果例と報告の仕方	194
13.6	章末問題	195
第 14 章	課題: 同時観測イベントの探索	197
14.1	課題の準備	197
14.1.1	まず配布コードを動かす	197
14.1.2	gzip ファイルの中身を見る	198
14.1.3	小さいデータで確認する	198
14.2	課題で求めること	199
14.2.1	出力としてほしいもの	199
14.2.2	正しさ確認の目安	200
14.3	提出物と発展課題	200
14.3.1	発展課題	201
付録 A	SSH, 鍵認証, ファイル転送	203
A.1	SSH の基本	203
A.1.1	サーバーへ接続する	203
A.1.2	SSH 鍵を作る	203
A.1.3	公開鍵をサーバーへ登録する	204
A.1.4	ssh-agent を使う	204
A.2	接続設定と GitHub 連携	205
A.2.1	~/.ssh/config を書く	205
A.2.2	GitHub に SSH 鍵を登録する	205
A.3	ファイル転送	205
A.3.1	scp の最小例	206
A.3.2	rsync を推奨する理由	206
付録 B	Git と GitHub の基本	209
B.1	Git の基本概念	209
B.1.1	リポジトリと公開範囲	209
B.1.2	Git で何をしているのか	210
B.2	ローカルで履歴を作る	210

B.2.1	最初の設定	210
B.2.2	新しいリポジトリを作る	210
B.2.3	編集、確認、記録の流れ	211
B.2.4	.gitignore を使って除外する	211
B.3	GitHub と接続する	212
B.3.1	既存のリポジトリを持ってくる	212
B.3.2	手元のリポジトリを GitHub へ初めて載せる	213
B.3.3	GitHub と同期する	214
B.3.4	GitHub で使う URL には HTTPS と SSH がある	215
B.4	認証と最小ワークフロー	215
B.4.1	GitHub の認証: SSH 鍵を基本にし、必要なら PAT	215
B.4.2	初心者が押さえるべき最小ワークフロー	216
付録 C	pybind11 による C++ 拡張モジュール	217
C.1	どのようなときに使うか	217
C.2	最小構成	217
C.2.1	pyproject.toml	218
C.2.2	CMakeLists.txt	218
C.2.3	C++ の実装	219
C.2.4	Python 側の入口	220
C.3	ビルドとインストール	220
C.4	Python 側からの使い方	220
C.5	どこまで C++ へ出すべきか	221
付録 D	正規表現の基本	223
D.1	正規表現とは何か	223
D.1.1	文字どおりの一致とメタ文字	223
D.1.2	ワイルドカードとの違い	224
D.2	よく使う部品	224
D.2.1	アンカー (行頭と行末)	224
D.2.2	文字クラス [...] と代表的な省略記法	224
D.2.3	量指定子 *, +, ?	225
D.3	物理・データ解析での具体例	225
D.3.1	ログから必要な行だけを探す	225
D.3.2	検出器 ID やセンサー名を選別する	226
D.3.3	Python で抽出と変換を同時に行う	226
D.4	Python の re と grep/awk の使い分け	226
D.4.1	Python では raw string を使う	226
D.4.2	search, match, fullmatch を使い分ける	227

D.4.3	コマンドラインで十分な場面と、Python に移すべき場面	227
付録 E	シェルスクリプトによる自動化と一括処理	229
E.1	シェルスクリプトの基礎構造	229
E.1.1	シバンと実行権限	229
E.1.2	コマンドライン引数	230
E.2	変数、演算、条件式	230
E.2.1	変数の定義と配列	230
E.2.2	数値計算（算術式展開）	231
E.2.3	条件式（test コマンド）	231
E.3	条件分岐 (if, case)	232
E.3.1	if 文を用いた分岐	232
E.3.2	case 文による分岐	232
E.4	反復処理と一括自動化 (for)	233
E.4.1	配列やリストの要素に対する for ループ	233
E.4.2	実践: 複数ファイルのパス置換とバッチ処理	233
E.5	sed を用いたテンプレート書き換えと動的実行	234
E.5.1	テンプレートファイルの準備	235
E.5.2	sed による置換の実行	235
参考文献		237
索引		241

はじめに

自然科学のデータ解析では、Python の文法を知っているだけでは十分ではない。ファイルと実行環境を確認し、データを適切な型で読み込み、計算結果を検証して図にまとめる必要がある。データ量が増えた場合には、結果を変えずに処理時間とメモリ使用量を改善する。本書は、この一連の作業を自力で組み立てられるようになることを目的とする。

本書の前半では、シェルによるファイル操作、Python の導入と仮想環境、Python の基本構文、データの入出力を扱う。シェルと環境管理は導入時だけの準備ではなく、入力の確認、スクリプトの実行、ログの保存、別の計算機への転送を含む解析基盤である。

中盤では、NumPy、pandas、Polars、Matplotlib、SciPy、Astropy を扱う。NumPy で数値配列を計算し、pandas と Polars で表形式データを整形・集計し、Matplotlib で結果を可視化する。SciPy は統計、フィット、補間、信号処理などの数値計算を担い、Astropy は物理単位、時刻系、天球座標、FITS 形式を扱う。それぞれのライブラリを孤立した道具として覚えるのではなく、同じ解析の中で役割を分担させる方法を説明する。

後半では、繰り返し使う処理を自作モジュールやパッケージとして整理し、テストと依存関係を含めて再利用する方法を学ぶ。その後、計算量、ベクトル化、メモリ使用量、逐次処理、プロファイリングを基に性能を改善する。巻末の総合課題では、これらの内容を用いて、複数の観測装置が 250 ns 以内に記録したイベントを同時観測とみなして探索する。

コードが例外を出さずに終了したことは、解析結果が正しいことを意味しない。本書では、既知の答えを持つ小さな入力を作る、配列の形状や型を表示する、図と残差を調べる、高速化の前後で同じ結果が得られることを確認する、といった検証を重視する。自然科学の解析では、どの値を得ただけでなく、どの条件と手順で得たかを説明できることが重要である。

Python の言語仕様と標準ライブラリには、公式チュートリアル [1] と公式ライブラリリファレンス [2] を用い、各ライブラリの公式資料とともに巻末へまとめた。

シェルの実行例は macOS、Linux、Windows Subsystem for Linux (WSL) で共通する書き方を基本とし、プロンプトを \$ で示す。実装によってオプションが異なる場合は、手元の環境で --help または man を確認する。

第 1 章

このテキストの使い方

1.1 対象と到達目標

本書は、Python を初めて学ぶ読者と、短い解析コードは書けるものの、実行環境の管理、結果の検証、再利用できる構成、高速化に不安がある読者を対象とする。読み終えた後には、次の作業を一つの解析手順として実行し、その選択理由を説明できることを目標とする。

- シェルでファイルとディレクトリを操作し、コマンドの標準出力と標準エラー出力を保存する。
- Python のバージョンと仮想環境を管理し、解析に必要なライブラリとそのバージョンを記録する。
- テキスト、表、バイナリデータを読み込み、型、欠損値、単位、時刻系を確認する。
- 配列や表を整形・集計し、図、統計量、フィット結果を用いてデータの性質を調べる。
- テストと測定に基づいて、解析結果を変えずにコードを整理し、高速化する。

すでに一部の内容を理解している場合は、必要な章から読み始めてもよい。ただし、後半の例では前の章で導入した用語とファイルを使用する。理解が曖昧な用語が現れた場合は、索引または本文中の参照先へ戻って確認する。

1.2 章のつながり

本書は、解析作業の順序に沿って四つの段階に分かれている。

1. シェルの章では、現在位置、パス、ファイル、実行ファイル、環境変数、入出力の流れを学ぶ。Python 環境の章では、その知識を用いて Python のバージョンとプロジェクトごとの環境を管理する。

2. Python の基本構文とデータ入出力の章では、短いスクリプトを読み書きし、ファイルから値を取り出して Python のオブジェクトへ変換する。
3. NumPy から Astropy までの章では、数値配列、表、図、統計処理、物理単位、時刻、天体データを扱う。各ライブラリの役割を比較し、同じデータを次の処理へ受け渡す。
4. 自作モジュールと性能改善の章では、解析処理をテスト可能な部品へ分け、実行時間とメモリ使用量を測定する。最後の総合課題では、ここまでの操作を一つの解析として組み合わせる。

この順序は、特定のライブラリを使うこと自体ではなく、入力を確認してから計算し、結果を検証して保存する流れを身につけるためにある。例えば、pandas で表を読み込めても、ファイルの所在、区切り文字、文字コード、列型を確認できなければ、読み込み結果の妥当性を判断できない。基礎的な操作は、後半の解析を検証するための手段でもある。

1.3 コード例と実行結果の読み方

コード例を読むときは、最初から各行の文法を完全に理解しようとせず、次の順序で処理の全体像を確認する。

1. 入力を確認する。変数、引数、ファイル名、配列の形状、単位が何かを調べる。
2. 処理を確認する。呼び出している関数と、関数へ渡している引数を追う。
3. 出力を確認する。返り値、表示内容、保存先のファイルを特定する。
4. 実行結果を本文の説明と照合する。数値、型、形状、行数、図の軸が想定どおりかを確認する。

例えば、`values = np.loadtxt("data.txt")` という行では、`data.txt` が入力、`np.loadtxt()` が処理、`values` が返り値の保存先である。次に `values.shape` と `values.dtype` を表示すれば、行列の大きさと要素型を確認できる。関数名だけを覚えるのではなく、入力条件と返り値を実行結果に結び付けて読む。

コマンドやコードの実行例には、可能な範囲で代表的な出力を続けて示す。表示されるパス、バージョン番号、処理時間などは環境によって異なるため、文字列をそのまま一致させるのではなく、各行が何を表し、どの部分を確認すべきかを読む。図についても、コードだけでなく保存された出力例を示し、本文では軸、色、凡例、分布、残差のうち何を確認するかを説明する。

1.4 章末問題の進め方

各章の末尾には、その章だけを読み直して取り組める問題を置いている。問題文には、入力の作り方、使用する主な関数、保存するファイル名、確認すべき値をできる限り示した。

最初に小さな検証用データで処理を確認し、その後に配布データまたは自分のデータへ適用する構成を基本とする。

数値の目安は、答えを書き写すためではなく、実装の誤りを検出するために使う。期待値と一致しない場合は、単位、境界条件、並べ替え、欠損値、データ型を順に確認する。図を提出する問題では、画像ファイルが作られたことだけでなく、軸ラベルと単位、凡例、対象範囲、データとモデルの対応を確認する。

第 14 章の総合課題では、複数の章で作った処理を連結する。課題用のスクリプトとデータの入手先は同章に記載している。まず配布コードをそのまま実行して入力と出力を把握し、小さなデータで正しさを確認した後に、大規模データの処理と性能改善へ進む。

1.5 検証と記録

データ解析では、コードが終了したことと、結果が正しいことを区別する。結果を確認するときは、次の観点をを用いる。

- 件数、最小値、最大値、平均値などを、手計算できる小さな入力と比較する。
- 配列の `shape` と `dtype`、表の列名と欠損値数を表示する。
- 境界値を含む入力を作り、`<` と `<=` などの条件を検証する。
- フィット値だけでなく、データ、モデル、残差を同じ図で確認する。
- 高速化の前後で結果が一致することを確認してから、同じ条件で処理時間を測る。

検証に使った入力、コマンド、ライブラリのバージョン、乱数シード、出力ファイルを記録しておけば、後から同じ結果を再現しやすい。解析コードを自作モジュールへ分け、テストを追加する作業も、この記録を実行可能な形にする方法の一つである。

1.6 参考資料の使い分け

本文は単独で読み進められるように構成しているが、コマンドのオプションやライブラリの API は更新される。概念と代表例は本文で確認し、全引数、戻り値、バージョンごとの差異は公式資料で調べる。本文で引用した資料は巻末の参考文献にまとめている。

Python の言語仕様と標準ライブラリには、Python 公式チュートリアル [1] と公式ライブラリリファレンス [2] を用いる。環境構築には JupyterLab、uv、pyenv の公式資料 [3–5]、主要な解析ライブラリには各プロジェクトの公式資料 [7, 9, 16, 22, 27, 35] を参照する。

シェルコマンドは、OS や実装によってオプションが異なることがある。本文の例を実行する前後に、`command --help` または `man command` で手元の仕様を確認する。Git と GitHub の基本操作は付録 B、SSH とファイル転送は付録 A にまとめている。

1.7 総合課題で作る解析の例

図 1.1 は、本書で扱う操作を組み合わせた解析例である。左の地図は、地震カタログから経度、緯度、マグニチュードを読み取り、震源の分布を色付きの点で示している。右上はイベントの発生間隔から求めた経験生存関数と、単一指数モデルおよび二成分指数モデルの比較である。右下はデータとモデルの比を示し、曲線がデータへ合っているだけでなく、系統的なずれが残っていないかを調べる。

この図は、最初から同じ外観を再現するための見本ではない。シェルによるデータ取得、NumPy または表処理ライブラリによる整形、Matplotlib による可視化、SciPy によるモデル推定を順に学んだ後の到達例である。使用範囲、モデル、軸尺度、色の割当てには解析上の判断が含まれるため、図を作るときは各選択の理由も記録する。

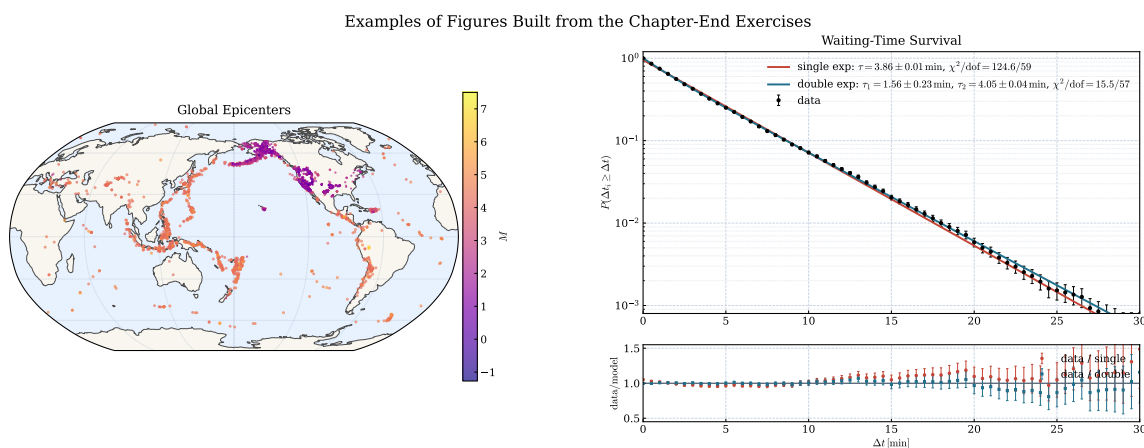


図 1.1 地震カタログを用いた解析例。左は震源の経度と緯度を表し、点の色はマグニチュードに対応する。右上は発生間隔の経験生存関数と二つの指数モデル、右下は各モデルに対するデータの比を示す。

次章では、データとスクリプトを操作するための基盤として、シェルの役割、パス、ファイル操作、入出力、権限を学ぶ。

第 2 章

シェルとファイル操作

2.1 シェルの基本と環境

本書では、ファイル操作やスクリプト実行をターミナルから行う。初めに、ターミナルとシェルの役割を区別する。

- ターミナル: 文字で命令を入力するためのアプリケーション
- シェル: ターミナルの中で動き、入力されたコマンドを解釈して実行するプログラム

macOS では zsh、Linux や WSL では bash を使うことが多い。zsh と bash はどちらもシェルであり、本書の基本コマンドはほぼ同じように使える。自分がどのシェルを使っているかを調べるには、次のように実行する。

```
$ echo $SHELL
```

実行結果の例は次のようになる。

```
/bin/zsh
```

この 1 行は、「いまログイン中のシェルとして zsh が使われている」ことを表す。末尾の zsh や bash が、実際にコマンドを解釈しているプログラム名である。以後、設定ファイルを編集するときは、この結果を見て ~/.zshrc と ~/.bashrc のどちらを触るかを決める。

本書では、シェルで入力するコマンド例の先頭に \$ を付ける。\$ 自体は「ここから先が入力するコマンドである」という印であり、読者が実際に打ち込むのはその後ろの部分である。これに対して、出力結果やファイル内容の例には \$ を付けない。

2.1.1 Windows では WSL を使う

本書のコマンド例は macOS や Linux のシェルを前提にしている。そのため、Windows で読み進める場合は、コマンドプロンプトや PowerShell をそのまま使うより、WSL (Windows

Subsystem for Linux) を入れて Ubuntu などの Linux 環境を使う方が混乱しにくい。導入の際は次のように実行する。

```
$ wsl --install
```

WSL を使う利点は、単にコマンド名がそろふことだけではない。後半で扱う並列処理や周辺ツールの中には、Linux 系環境での利用を前提としているものがある。OS ごとの差でつまずくより、最初から UNIX 系の作法へ寄せた方が、解析そのものに集中しやすい。

2.2 コマンドの基本と使い方

2.2.1 コマンド、オプション、引数

シェルで打つ 1 行は、多くの場合「コマンド本体」「オプション」「引数」から成る。たとえば次のような形である。

```
$ ls -l results
$ cp data/raw.txt backup/raw.txt
```

1 行目では、ls がコマンド本体、-l がオプション、results が引数である。2 行目では、cp がコマンド本体で、data/raw.txt と backup/raw.txt が引数である。一般に、オプションは動作を変え、引数は対象のファイル名やディレクトリ名を与える。

オプションには短い書き方 (- の後に 1 文字) と、長い書き方 (-- の後に英単語) が用意されていることが多い。たとえば、使い方を表示するオプションとしては -h と --help の両方がサポートされていることが多い。

2.2.2 困ったときは説明を読む

コマンドの使い方を丸暗記する必要はない。忘れたときは、その場で説明を読む方が正確である。短い使い方を知りたいときは -h や --help、より詳しい説明を読みたいときは man を使う。実際の調べ方は次のようになる。

```
$ python3 -h
$ python3 --help
$ man rsync
```

python3 -h の出力は長いですが、先頭はたとえば次のようになる。

```
usage: python3 [option] ... [-c cmd | -m mod | file | -] [arg] ...
Options and arguments (and corresponding environment variables):
-b      : issue warnings about str(bytes_instance), str(bytearray_instance)
         and comparing bytes/bytearray with str. (-bb: issue errors)
-B      : don't write .pyc files on import; also PYTHONDONTWRITEBYTECODE=x
-c cmd  : program passed in as string (terminates option list)
```

ここで最初の `usage: ...` は「このコマンドをどういう形で呼び出せるか」を要約した行である。`[-c cmd | -m mod | file | -]` のように角括弧で囲まれた部分は省略可能、`|` で区切られた部分はどれか 1 つを選ぶ、という意味で読む。その次の `Options and arguments ...` 以下では、各オプションの意味が 1 行ずつ並ぶ。`-b` のように短いオプション名、コロンの後ろにその効果、必要なら対応する環境変数が示される。`-c cmd` のように引数を伴うものは、オプションの直後に必要な値の名前も書かれる。

`man rsync` を開くと、画面の先頭付近はたとえば次のようになる。

OPENSRC(1)	General Commands Manual
OPENSRC(1)	
NAME	
opensrc - synchronise local and remote files	
SYNOPSIS	
opensrc [-0468BCDEFHILOPRSTWVabcdefghijklmnopqrstuvwxyz] [-e program] [-f filter] ...	
... source ... directory	

NAME はそのコマンドが何をするものかを 1 行で示す欄である。**SYNOPSIS** は、「このコマンドをどういう形で呼び出せるか」を要約した欄である。ここで角括弧 [...] に入っているものは省略可能であり、すべてを書かなければならないわけではない。この例では、末尾の `source ... directory` を見れば、コピー元を並べて最後にコピー先を書くことが分かる。全オプションを暗記する必要はない。`--exclude` や `-a` のように、必要なものだけを後ろの節や `man` から確認して使えば十分である。

`--help` や `-h` は、そのコマンドが受け付けるオプションや引数の書式を短く表示することが多い。ただし、すべてのコマンドが両方を実装しているわけではない。macOS の `ls` のように `--help` を受け付けないコマンドもあるので、その場合は `man ls`、`man ssh`、`man rsync` のように `man` を引く方が確実である。`man` を開いたあとは、上下キーや `Space` で移動し、`/pattern` で検索できる。`q` を押せば終了する。

検索エンジンや AI を用いてコマンドの使い方を調べることもできるが、同じ名前のコマンドでもバージョンや OS によって使い方が異なることもあるため注意が必要である。検索しても問題が解決しない場合は `--help` や `man` を確認し、手元の環境にあった正しい情報を確認しよう。

2.3 ファイルとディレクトリの操作

2.3.1 今どこにいるかを確認する: `ls`

シェルでは、いま作業しているディレクトリが基準になる。この基準となるディレクトリをカレントディレクトリという。まずは、現在どこで作業しているのか、そしてその場所に

何があるのかを確認できるようになろう。そのためによく使うのが、次の `pwd` と `ls` である。

```
$ pwd
$ ls
$ ls -l
$ ls -lh
$ ls -a
$ ls -lah
```

`pwd` は、現在のカレントディレクトリの絶対パスを表示する。`ls` は、そのディレクトリにあるファイルやディレクトリの一覧を表示する。たとえば `pwd` の結果が次のようであれば、

```
/Users/ikomae/analysis
```

現在の作業場所は `/Users/ikomae/analysis` である。この状態で `ls` を実行すると、その中にある項目が表示される。

```
$ ls
data  scripts
```

この結果から、`/Users/ikomae/analysis` の中には `data` と `scripts` という2つのディレクトリがあることが分かる。

`ls` にはさまざまなオプションがあり、表示内容を切り替えられる。まず、`-l` オプションを付けると、一覧を詳細表示にできる。

```
$ ls -l
total 0
drwxr-xr-x  3 ikomae  staff  96  4月 12 23:19 data
drwxr-xr-x  3 ikomae  staff  96  4月 12 23:19 scripts
```

`ls -l` の最初の行に表示される `total 0` は、この一覧に表示されている項目が使用しているブロック数の合計を表している。`total` の単位は環境によって見え方が分かりにくいので、ここでは「一覧に表示されている項目全体についての使用量をまとめて示したもの」と理解しておけばよい。

`ls -l` の出力では、その下の各行が1つのファイルまたはディレクトリに対応しており、左から順に次の情報が表示される。

- `drwxr-xr-x`：ファイル種別とアクセス権
- `3`：リンク数
- `ikomae`：所有者
- `staff`：グループ
- `96`：サイズ
- `4月 12 23:19`：最終更新日時
- `data`：名前

先頭の1文字は種別を表し、`d` ならディレクトリ、`-` なら通常のファイルである。続く `rwxr-xr-x` はアクセス権を表している。アクセス権の詳しい意味は第2.10.1節で説明する。

リンク数は、ファイルやディレクトリに対して、ファイルシステムの中でいくつ参照があるかを表す値である。通常のファイルでは、ふつうに作成したものであれば多くの場合は 1 であり、入門の段階ではあまり深く気にしなくてよい。ディレクトリでは意味が少し異なる。ディレクトリ構造により値が変化するが、その定義は環境によって異なるため、通常のファイルのように単純には読めない。したがって、リンク数については、通常のファイルでは多くの場合 1 であり、ディレクトリでは構造に関係する値が表示される、と理解しておけば十分である。

`ls -l` で表示されるサイズの欄は、通常はバイト単位で表示される。したがって、ファイルが大きくなると数値も大きくなり、ぱっと見ただけでは大きさをつかみにくいことがある。そこで、サイズを見やすく表示したいときには `-h` オプションを併用する。`h` は `human-readable` の略で、サイズを `K`、`M`、`G` などの読みやすい単位で表示する。

```
$ ls -lh
total 0
drwxr-xr-x 3 ikomae staff 96B 4月 12 23:19 data
drwxr-xr-x 3 ikomae staff 96B 4月 12 23:19 scripts
```

この例ではサイズが `96B` と表示されているが、これはディレクトリ項目そのものの大きさであって、その中に入っているファイル全体の容量を表しているわけではない。ディレクトリのサイズ欄は、通常のファイルのサイズ欄とは同じ感覚では読めない点に注意が必要である。

通常のファイルに対して `ls -lh` を使うと、サイズは `12K` や `3.4M` のように読みやすい単位で表示される。そのため、特にファイルの大きさを確認したいときには `ls -lh` を使うことが多い。

一方、`-a` オプションを付けると、隠しファイルも含めて表示できる。

```
$ ls -a
.      ..      .config data  scripts
```

ファイル名やディレクトリ名が `.`（ドット）で始まるものは隠しファイルとして扱われ、通常の `ls` では表示されない。たとえば `.config`、`.gitignore`、`.bashrc` などがこれにあたる。設定ファイルのように、ふだんは一覧に見えなくてよいものを隠しファイルにしておくことが多い。

上の `ls -a` に現れる `.` と `..` にも意味がある。`.` は現在のディレクトリ自身を表し、`..` は 1 つ上の親ディレクトリを表す。これらは後でディレクトリ移動をするときにも重要になる。

詳細表示、隠しファイル表示、読みやすいサイズ表示をまとめて使うこともできる。たとえば `-lah` を指定すると、詳細情報を隠しファイル込みで、人が読みやすい単位で表示できる。

```
$ ls -lah
total 96
drwxr-xr-x 5 ikomae staff 160B 4月 12 23:20 .
drwxr-xr-x 5 ikomae staff 160B 4月 12 23:18 ..
```

```
-rw-r--r--  1 ikomae  staff   47K  4月 12 23:22 .config
drwxr-xr-x  3 ikomae  staff   96B  4月 12 23:19 data
drwxr-xr-x  3 ikomae  staff   96B  4月 12 23:19 scripts
```

この例では `.config` という通常のファイルも表示されており、`ls -l` のときとは `total` の値が異なっている。`total` は各行の「サイズ」欄を単純に足したものではなく、表示対象が使用しているブロック数に基づいて表示される点に注意したい。

このように、`ls` はオプションを組み合わせて、必要な情報を見やすく確認できる。まずは、現在位置を確認する `pwd`、項目を一覧する `ls`、詳細を見る `ls -l`、隠しファイルも含めて確認する `ls -a`、サイズを読みやすく表示する `ls -lh` や `ls -lah` を使い分けられるようになるとうい。

2.3.2 移動する: `cd`

`cd` はカレントディレクトリを変更するためのコマンドである。`ls` や `cat`、`cp` など多くのコマンドは「いまいる場所」を基準に動くため、`cd` を正しく読めるようになると、その後のファイル操作も理解しやすくなる。次のようなディレクトリ構成を例に考える。

```
analysis/
|-- data/
|   |-- events.txt
|-- scripts/
|   |-- plot.py
```

いま `/Users/ikomae/analysis` にいるとする。まず `pwd` と `ls` を実行すると、現在位置とその場にある項目は次のように分かる。

```
$ pwd
/Users/ikomae/analysis
$ ls
data  scripts
```

この状態で `cd data` を実行すると、`analysis` の下にある `data` ディレクトリへ移動する。

```
$ cd data
$ pwd
/Users/ikomae/analysis/data
$ ls
events.txt
```

`data` は相対パスなので、「いまいる `analysis` から見て `data` に入る」という意味になる。次に `cd ..` を実行すると、1 つ上の親ディレクトリへ戻る。

```
$ cd ..
$ pwd
/Users/ikomae/analysis
```

ここで使っている `..` は、`ls -a` でも見た「親ディレクトリ」を表す特別な名前である。

絶対パスを使って直接移動することもできる。たとえば `~` はホームディレクトリを表すので、次のように書けば現在位置に関係なく `scripts` へ移動できる。

```
$ cd ~/analysis/scripts
$ pwd
/Users/ikomae/analysis/scripts
$ ls
plot.py
```

また、`cd -` は 1 つ前にいた場所へ戻る。たとえば上の状態で実行すると、直前にいた `/Users/ikomae/analysis` へ戻る。

```
$ cd -
/Users/ikomae/analysis
```

さらに、引数なしの `cd` はホームディレクトリへ戻る。深い階層まで移動したあとで基準位置に戻したいときに便利である。

```
$ cd
$ pwd
/Users/ikomae
```

このように、`cd` では「相対パスでたどる」「絶対パスで直接飛ぶ」「`..` や `-` で戻る」という使い分けが重要になる。慣れないうちは、移動した直後に `pwd` や `ls` を実行して、本当に意図した場所へ来ているかを確認めるとよい。

2.3.3 絶対パスと相対パス

ファイルやディレクトリの場所は、パスで表す。パスは階層を `/` でつないだ表記であり、最初の書き方によって意味が変わる。大きく分けると絶対パスと相対パスがある。

表 2.1 よく出てくるパス表記

表記	意味
<code>/abs/data.txt</code>	絶対パス。 <code>/</code> から始まり、どこにいても同じ場所を指す
<code>~/analysis</code>	絶対パス。 <code>~</code> はホームディレクトリ
<code>data.txt</code>	相対パス。今いるディレクトリを基準に解釈される
<code>./x.txt</code>	相対パス。 <code>.</code> は現在のディレクトリ
<code>../x.txt</code>	相対パス。 <code>..</code> は 1 つ上のディレクトリ

表 2.1 は、シェルを読み始めた直後に最低限区別したい表記をまとめたものである。特に重要なのは、絶対パスは `/` や `~` から始まり、相対パスは現在のカレントディレクトリを基準に読む、という違いである。

先ほどと同じく、次のディレクトリ構成を考える。

```
analysis/  
|-- data/  
|   |-- events.txt  
|-- scripts/  
|   |-- plot.py
```

現在位置が `/Users/ikomae/analysis` なら、次の 2 つのコマンドは同じファイルを指している。

```
$ pwd  
/Users/ikomae/analysis  
$ cat data/events.txt  
$ cat /Users/ikomae/analysis/data/events.txt
```

1 つ目の `data/events.txt` は相対パスであり、「いまいる `analysis` の中にある `data` の中にある `events.txt`」という意味である。2 つ目の `/Users/ikomae/analysis/data/events.txt` は絶対パスなので、どこにいても同じ `events.txt` を指す。

一方で、現在位置が `/Users/ikomae/analysis/scripts` に変わると、同じ相対パスでも意味が変わる。

```
$ pwd  
/Users/ikomae/analysis/scripts  
$ cat data/events.txt  
cat: data/events.txt: No such file or directory  
$ cat ../data/events.txt
```

この場所で `data/events.txt` と書くと `scripts/data/events.txt` を探しに行くことになるが、そのようなファイルは存在しないので `data/events.txt: No such file or directory` というエラー文が表示される。相対パスで `events.txt` を指定するには、1 つ上へ戻る `..` を使って `../data/events.txt` と書く必要がある。

同様に、`./plot.py` は「現在のディレクトリにある `plot.py`」、`../data` は「1 つ上にある `data` ディレクトリ」を表す。相対パスは短く書ける反面、現在位置を読み違えると別の場所を指してしまう。したがって、コマンドを読むときは「今どこにいるか」と「そのパスが絶対か相対か」を必ずセットで考える必要がある。

2.3.4 ディレクトリを作る: `mkdir`

`mkdir` は新しいディレクトリを作るコマンドである。解析では、生データ、途中結果、図、ログなどを別々の場所へ整理したいので、作業の最初に出力先のディレクトリを作ることが多い。

たとえば、最初に次のような構成になっているとする。

```
analysis/  
|-- data/  
|   |-- events.txt
```

output というディレクトリを 1 つ作るだけなら、次のように実行すればよい。

```
$ mkdir output
$ ls
data    output
```

1 行目では、現在のカレントディレクトリの直下に output が新しくできる。2 行目の ls で、それが確認できる。

一方、output/2026/04 のように深い階層をまとめて作りたい場面もある。途中の output/2026 がまだ無い場合、単純に mkdir output/2026/04 のように書くとエラーが表示される。-p オプションを付けると、足りない親ディレクトリも含めてまとめて作ることができる。

```
$ mkdir output/2026/04
mkdir: output/2026: No such file or directory
$ mkdir -p output/2026/04
```

実行後は、次のようなディレクトリ構成になる。

```
analysis/
|-- data/
|   |-- events.txt
|-- output/
|   |-- 2026/
|       |-- 04/
```

解析では、観測日ごとの出力先や、figures/summary/ のような途中ディレクトリを一度に作ることが多い。このような場面では mkdir -p を用いるのが便利である。

2.4 コピー、移動、削除

2.4.1 コピーする: cp

cp は元のファイルやディレクトリを残したまま複製を作るコマンドである。最後の引数が既存のディレクトリならその中へ同じ名前でコピーされ、ファイル名ならその名前で複製が作られる。まずは次のようなディレクトリ構造を考える。

```
work/
|-- backup/
|-- raw/
|   |-- events.txt
```

ここで raw/events.txt を backup へコピーするには、たとえば次のように実行できる。

```
$ cp raw/events.txt backup/
$ cp raw/events.txt backup/events_20260412.txt
```

1行目では、`backup/` が既存のディレクトリなので、その中に同じ名前の `events.txt` が作られる。2行目では、コピー先の名前まで明示しているので、同じ内容を `events_20260412.txt` という別名で保存できる。

```
work/
|-- backup/
|   |-- events.txt
|   `-- events_20260412.txt
`-- raw/
    `-- events.txt
```

このように、`cp` は元ファイルを消さない。`raw/events.txt` はそのまま残り、コピー先だけが増える。バックアップを取るときや、元データを残したまま加工用のファイルを作るときによく使う。

ディレクトリごと複製したい場合は `-r` を付ける。

```
$ cp -r raw raw_backup
```

```
work/
|-- backup/
|   |-- events.txt
|   `-- events_20260412.txt
|-- raw/
|   `-- events.txt
`-- raw_backup/
    `-- events.txt
```

`-r` は再帰的にコピーする、という意味である。ディレクトリの中にさらにファイルやサブディレクトリがあっても、その構造ごと複製される。なお、既存ファイルへの上書きが心配なときは `-i` を付けて確認しながらコピーすると安全である。

2.4.2 移動する、名前を変える: `mv`

`mv` は「別の場所へ移す」と「名前を変える」の両方に使う。考え方は `cp` と似ており、最後の引数が既存ディレクトリならその中へ移動し、そうでなければその名前へ変更する。ただし、`cp` と違って元の場所には残らない。

まず、同じディレクトリの中で名前だけを変える例を見る。

```
work/
`-- result.txt
```

```
$ mv result.txt result_old.txt
```

```
work/
`-- result_old.txt
```

この場合は、同じ場所にあるファイルの名前だけが `result_old.txt` へ変わる。

次に、別のディレクトリへ移動しながら名前も変える例を考える。

```
work/
|-- output/
`-- result_old.txt

$ mv result_old.txt output/final.txt

work/
`-- output/
    |-- final.txt
```

`mv result_old.txt output/final.txt` では、`result_old.txt` が `output` の中へ移り、名前も `final.txt` に変わる。もし `mv result_old.txt output/` と書けば、名前はそのまま場所だけが変わる。

このように、`mv` は「移動」と「改名」を1つのコマンドで行えるのが特徴である。整理のためにファイルを別ディレクトリへ移したり、解析段階に応じて `tmp.txt` を `final.txt` に改名したりするときに使う。`mv -i` を使うと、すでにファイルが存在した場合に上書きするかどうかを確認しながら進められる。

2.4.3 削除する: `rm`

`rm` はファイルやディレクトリを削除するコマンドである。重要なのは、通常の `rm` はゴミ箱へ移すのではなく、その場で削除するという点である。Finder での削除と違って、あとから気軽に戻すことはできない。したがって、慣れないうちは削除前に `pwd` や `ls` で場所と対象を確認する方がよい。

たとえば、次のような状態を考える。

```
work/
|-- old_output/
|   |-- plot.png
|-- result_old.txt
```

ここで使う基本形は次の通りである。

```
$ rm result_old.txt
$ rm -i result_old.txt
$ rm -r old_output
$ rm -ri old_output
```

`rm result_old.txt` は通常のファイルを削除する。`rm -i result_old.txt` は削除前に確認を出すので、うっかり消したくないときに有用である。`rm -r old_output` はディレクトリを中身ごと再帰的に削除する。`rm -ri old_output` のように `-r` と `-i` を組み合わせれば、ディレクトリ削除でも確認しながら進められる。

特に `rm -r` は強力である。対象のパスを誤ると大きな被害になるので、削除前に `ls old_output` や `ls result_old.txt` のように対象を一度表示してから実行すると安全である。

2.4.4 ワイルドカード

複数のファイルを一度に指定したいときは、ワイルドカードが役に立つ。よく使うのは `*` であり、これは「0 文字以上の任意の文字列」を表す。重要なのは、`cat` や `rm` が `*` を理解しているのではなく、シェルが先に実際のファイル名へ展開してから各コマンドへ渡しているという点である。

たとえば、次のような構成を考える。

```
data/
|-- run01.csv
|-- run02.csv
|-- run03.txt
|-- test_20260411.csv
|-- test_20260412.csv
|-- test_20260413.csv
`-- subdir/
    |-- run04.csv
```

このとき、次のような書き方ができる。

```
$ ls data/*.csv
$ cp data/*.csv backup/
$ rm test_*.csv
```

`data/*.csv` は、`data` ディレクトリ直下にある `.csv` ファイルすべてを意味する。したがって、この例では `run01.csv` と `run02.csv` は対象になるが、`subdir/run04.csv` は含まれない。`*` は `/` をまたいで階層全体を探すわけではないからである。

`cp data/*.csv backup/` とすると、条件に合う複数の CSV をまとめてコピーできる。`rm test_*.csv` なら、`test_001.csv` や `test_old.csv` のように同じ接頭辞を持つファイルを一括削除できる。

ただし、ワイルドカードは便利な反面、想定より多くのファイルへ一致することもある。特に削除系のコマンドでは、先に `ls test_*.csv` のように実際の対象を表示してから `rm test_*.csv` を実行する方が安全である。また、一致しないときの振る舞いはシェル設定によって異なることがあるので、その意味でもまず `ls` で確認する習慣が有用である。

2.5 ファイルの内容確認とデータ抽出

解析では、データを読む前にまず中身を確認する。そのための基本コマンドと、最低限覚えておきたいオプションを最初に表へまとめる。

表 2.2 は、ファイルの確認と抽出で頻繁に使うコマンドを一覧するためのものである。最初は全てを暗記する必要はないが、「中身を見る」「条件で抜く」「列ごとに整形する」という役割の違いは押さえたい。ここからは、各コマンドの役割と主要なオプションを順に説明

表 2.2 中身確認と抽出でよく使うコマンドと主要オプション

コマンド	主なオプションや書式	主な用途
cat	-n	短いファイルをそのまま読む、複数ファイルを連結する
less	-N, /pattern, q	長いファイルをスクロールしながら読む
head	-n N	先頭の数行だけを見る
tail	-n N, -f	末尾の数行を見る、ログの追記を監視する
wc	-l, -w, -c	行数、単語数、バイト数を数える
grep	-i, -v, -n, -A N, -B N	条件に合う行だけを抽出する
awk	-F, if (...), sum += ...	列単位で取り出す、簡単な整形や集計を行う

する。

2.5.1 cat

cat はファイルの中身をそのまま標準出力へ流す。短い設定ファイルや、数行だけの確認用テキストを一気に見たいときに向く。複数のファイルを並べて渡すと、内容を順に連結して表示できる。

```
$ cat data/events.txt
$ cat header.txt body.txt
$ cat log/*.log
$ cat -n config.txt
```

3 行目のように、前節のワイルドカードを使えば複数のログファイルをまとめて連結して表示できる。ただし、巨大なログや観測データへそのまま使うと画面が一気に流れて読めない。つまり、cat は「短い内容を一気に見る」道具だと考えるとよい。長いファイルは、次に述べる less や head、tail に切り替える。

-n 行番号を付けて表示する。設定ファイルやログの何行目に何があるかを確認したいときに便利である。

2.5.2 less

less は長いファイルをスクロールしながら読むためのコマンドである。大きなデータを「まず中身だけ見たい」とときには、最初に less を使うことが多い。

```
$ less data/events.txt
$ less -N data/events.txt
```

less は単なる表示コマンドではなく、表示中に検索や移動ができる。ログや観測データの形式を確認するときは、次の操作を覚えておくとよい。

-N 行番号を表示する。後で「どの行に何があったか」を参照しやすくなる。

Space 1 画面ぶん先へ進む。

b 1 画面ぶん戻る。

/pattern 指定した文字列を前方検索する。たとえば `/ERROR` と打てば、`ERROR` を含む次の行へ飛べる。

n 直前の検索条件の次の一致へ進む。

q 表示を終了してターミナルへ戻る。

2.5.3 head

`head` はファイルの先頭だけを見るためのコマンドである。どのような形式のデータが入っているか、ヘッダやコメント行はどのようになっているかを確認したいときに向く。

```
$ head data/events.txt
$ head -n 20 data/events.txt
```

-n N 先頭から `N` 行だけ表示する。既定では 10 行なので、少し長めに見たいときに使う。

2.5.4 tail

`tail` はファイルの末尾だけを見るためのコマンドである。ログファイルの最新部分を確認したいときや、計算結果が末尾へ追記される形式のファイルを見るときに使う。

```
$ tail log.txt
$ tail -n 30 log.txt
$ tail -f log.txt
```

-n N 末尾から `N` 行だけ表示する。直近 20 行や 30 行だけ見たいときに使う。

-f ファイルの末尾を監視し、追記された内容を続けて表示する。実行中のジョブのログを見るときに有用である。

2.5.5 wc

`wc` はファイルの行数、単語数、バイト数などを数えるコマンドである。特に `-l` は「何行あるか」を知るためによく使う。データ件数やログの大きさをざっと見積もるときの第一歩になる。

```
$ wc data/events.txt
$ wc -l data/events.txt
$ wc -w memo.txt
$ wc -c output.bin
```

- l 行数を数える。最もよく使う。
- w 単語数を数える。自然言語テキストの確認で使うことが多い。
- c バイト数を数える。ファイルサイズを概算したいときに役立つ。

2.5.6 grep

grep は、条件に合う行だけを抽出するためのコマンドである。ログからエラー行を探す、コメント行を除く、特定の測定条件を含む行だけ取り出す、といった用途で頻繁に使う。

```
$ grep "ERROR" log.txt
$ grep -n "ERROR" log.txt
$ grep -A 2 -B 1 "ERROR" log.txt
$ grep -i "warning" log.txt
$ grep -v "^#" data.txt
$ grep -rn "TODO" src/
```

- i 大文字と小文字を区別せずに検索する。error と ERROR を同じものとして扱いたいときに使う。
- v 一致した行ではなく、一致しなかった行を出す。コメント行やノイズ行を除外したいときに便利である。
- n 一致した行の行番号も表示する。後で元ファイルを見直すときに役立つ。
- A N 一致した行の後ろを N 行ぶん追加で表示する。エラー行の直後にある補足メッセージも一緒に見たいときに便利である。
- B N 一致した行の前を N 行ぶん追加で表示する。エラーの直前に何が起きていたかを見たいときに使う。
- r ディレクトリの中を再帰的に検索する。複数ファイルにまたがって同じ文字列を探したいときに使う。たとえば `grep -rn "TODO" src/` と書けば、src/ 以下のすべてのファイルをたどり、TODO を含む行を行番号付きで表示できる。日常的なログ確認では -A や -B の方を先に使うことが多いが、複数ファイル検索ではこちらも重要である。

grep では正規表現も使える。たとえば ^ は行頭、\$ は行末を表すので、`grep "^#" data.txt` と書けば「# で始まる行」だけを拾える。また、複数のログファイルをまとめて調べたいなら、`grep -r "ERROR" log/` のようにディレクトリごと渡せばよい。ここでは最低限の例だけを示したが、[] や . や * まで含めた正規表現の整理は付録 D にまとめているので、記号の意味が曖昧になったときはそちらを見返すとよい。

2.5.7 awk

awk は、テキストを列データとして扱って抽出や整形を行うコマンドである。空白区切りのログや表形式テキストを扱うときには非常に強力である。

```
$ awk '{print $1}' data.txt
```

```
$ awk '{print $1, $3}' data.txt
$ awk -F, '{print $2}' data.csv
$ awk '{ if ($3 > 10) print $1, $3 }' data.txt
$ awk '{ sum += $3 } END { print sum }' data.txt
```

既定では空白やタブが区切り文字として使われる。そのため、空白区切りの観測データなら \$1 が 1 列目、\$2 が 2 列目という意味になる。条件分岐は if を明示的に書くこともできるので、「3 列目が 10 より大きい行だけ表示する」といった処理の流れが読みやすい。また、各行で値を足し合わせて最後に合計や平均を出す、といった簡単な計算もそのまま書ける。

-F 区切り文字を指定する。CSV のようにカンマ区切りなら -F, のように書く。

{print \$1} 1 列目を表示する最も基本的な形である。**{print \$1, \$3}** のようにすれば複数列を並べて出せる。

{ if (\$3 > 10) print \$1, \$3 } 条件分岐を明示的に書く形である。最初のうちは、このように if を書いた方が処理の意味を追いやすい。

{ sum += \$3 } END { print sum } 各行で 3 列目を足し上げ、最後に合計を表示する。簡単な集計や平均計算の出発点として便利である。

2.5.8 パイプ |

さらに、強力な手段としてパイプ (|) がある。パイプは、あるコマンドの標準出力を次のコマンドの入力へそのまま流し込む機能である。単機能のコマンドを組み合わせたい結果を得るのが UNIX 環境の流儀である。

```
$ cat log.txt | grep "ERROR"
$ grep "ERROR" log.txt | wc -l
$ cat events.txt | grep -v "noise" | awk '{print $2}'
```

1 行目は log.txt の内容を読み出し、その中から ERROR を含む行だけを表示する。2 行目では、抽出結果をさらに wc -l へ渡し、エラー件数を数えている。3 行目では、ノイズ行を除外したうえで 2 列目だけを取り出している。

このように、まず全体を見る、次に条件で絞る、最後に列を抜く、という手順を 1 行の中でつなげられる。

2.6 URL からファイルを取得する

解析では、公開データや設定例、インストーラスクリプトを URL から取得することがある。そのときによく使うのが curl と wget である。大きな違いは、curl は何も付けずに内容を標準出力へ出し、wget は既定でファイルとして保存する点にある。macOS には curl が標準で入っているが、wget は標準では入っていない。必要なら brew install wget などで追加する。

2.6.1 curl

curl は、URL の内容を表示したり、保存したり、別のコマンドへ渡したりするときに使う。まずは次の形を覚えておけばよい。

```
$ curl https://example.com/config.txt | head -n 2
$ curl -O https://example.com/data/run001.csv
$ curl -o events.csv https://example.com/data/run001.csv
$ curl -L -O https://example.com/latest.csv
$ curl -fsSL https://pyenv.run | bash
```

1 行目では、取得した内容をファイルへ保存せず、そのまま画面へ流している。たとえば結果は次のようになる。

```
threshold=5.0
window_ns=250
```

2 行目と 3 行目はファイル保存である。2 行目の `-O` は URL の末尾にあるファイル名をそのまま使う。3 行目の `-o events.csv` は、保存名を自分で決めたいときに使う。4 行目の `-L` はリダイレクト先までたどるオプションであり、短縮 URL や配布用リンクを扱うときによく必要になる。5 行目の `-fsSL` はインストール手順でよく見る組合せで、`-f` は HTTP エラー時に失敗として扱う、`-s` は通常の進捗表示を消す、`-S` は `-s` と併用したときでもエラー文は表示する、`-L` はリダイレクトをたどる、という意味である。

- O** URL の末尾にある名前でも保存する。配布元のファイル名をそのまま使いたいときに向く。
- o** 保存名を自分で指定する。長い URL や分かりにくい名前を短く直したいときに使う。
- L** リダイレクトをたどる。共有 URL や最新版リンクでは付けた方が安全なことが多い。
- f** HTTP エラーを成功扱いにしない。存在しない URL を取ってしまったときの見落としを減らせる。
- ss** 通常表示を減らしつつ、エラーだけは見えるようにする。スクリプトから呼ぶときに便利である。

2.6.2 wget

wget は、URL を指定すると既定でファイルへ保存する。大量のファイルを順に取得したいときや、大きなファイルの取得を再開したいときに使いやすい。

```
$ wget https://example.com/data/run001.csv
$ wget -O events.csv https://example.com/data/run001.csv
$ wget -c https://example.com/archive.tar.gz
```

1 行目では、URL の末尾にある `run001.csv` という名前で保存される。2 行目の `-O` は保存名を自分で決める形であり、`curl -o` と同じ発想である。3 行目の `-c` は途中で止まったダウ

ンロードを再開するときに使う。回線が不安定な環境やサイズの大きい配布物では便利である。

- o 保存名を自分で指定する。既定の名前を使いたくないときに使う。
- c 中断したダウンロードを再開する。大きなファイルで有効である。
- q 表示を減らす。ログを静かにしたいときに使う。
- show-progress 進捗だけは見たいときに使う。静かな実行と相性がよい。

wget が手元にない環境では、curl -O や curl -o で代用できる。また、画面へ流した内容をそのままパイプで次のコマンドへ渡したいときは、wget より curl の方が自然である。

2.7 圧縮ファイルを扱う

2.7.1 gzip: 1 つのファイルを圧縮する

gzip は、基本的には 1 つのファイルを 1 つの圧縮ファイルへ変える道具である。ログ、CSV、設定ファイルのようなテキストを素早く縮めたいときによく使う。

```
$ gzip -k events.csv
$ ls -lh events.csv*
$ gzip -cd events.csv.gz | head -n 2
$ gunzip events.csv.gz
```

1 行目の -k は元ファイルを残す指定である。付けない場合は、通常 events.csv が events.csv.gz に置き換わる。2 行目では圧縮前後のサイズを見比べている。3 行目の -cd は、「展開した内容をファイルへ戻さず標準出力へ流す」という意味であり、head や grep と組み合わせやすい。4 行目の gunzip は展開用コマンドであり、gzip -d と同じ目的で使える。

- k 元ファイルを残す。配布物を壊したくないときに安全である。
- d 展開する。圧縮を戻すときの基本である。
- c 結果を画面へ流す。パイプと相性がよい。
- 1 から -9 圧縮率と速度のバランスを指定する。数字が大きいほど一般に圧縮率は上がるが時間がかかる。-9 は、少しでも小さくしたい配布用ファイルでは意味があるが、日常的なログ圧縮では既定値のままで十分なことが多い。

gzip 自体は複数ファイルを 1 つに束ねる道具ではない。ディレクトリごとまとめたいときは、次の tar と組み合わせることが多い。

2.7.2 tar: 複数ファイルやディレクトリを 1 つに束ねる

tar は、本来は複数ファイルやディレクトリを 1 本のアーカイブへ束ねる道具である。gzip と組み合わせて .tar.gz を作る形が非常によく使われる。

```
$ tar -czf archive.tar.gz raw/ notes.txt
$ tar -tzf archive.tar.gz
$ tar -xzf archive.tar.gz
```

1 行目は、raw/ と notes.txt をまとめて archive.tar.gz にしている。2 行目の -t は中身一覧の表示であり、展開せずに内容だけ確認したいときに便利である。3 行目の -x は展開である。つまり、tar では「作る」「一覧を見る」「展開する」が頻出操作になる。tar では、tar czf archive.tar.gz ... のように先頭の - を省いた書き方も広く使われる。ただし、本書では他のコマンドとの対応を分かりやすくするため、tar -czf のように - を付けた形で統一する。

- c 新しくアーカイブを作る。
- t 中身一覧だけを見る。
- x 展開する。
- z gzip 圧縮を併用する。
- f 直後の文字列をアーカイブ名として扱う。

tar -czf のようにオプションが連なって見えにくいときは、c=create、t=list、x=extract、z=gzip、f=file と分解して読むと混乱しにくい。

2.7.3 zip: 配布しやすい形式

zip も複数ファイルやディレクトリを 1 つにまとめられる。Windows を含む環境で配布しやすいため、共同研究や授業資料では .zip が使われることも多い。

```
$ zip -r archive.zip raw/ notes.txt
$ unzip -l archive.zip
$ unzip archive.zip
```

1 行目の -r はディレクトリを再帰的にたどってまとめる指定である。2 行目の unzip -l は展開せずに中身一覧を表示する。3 行目は実際に展開している。したがって、受け取った圧縮ファイルが .zip なら unzip、.tar.gz なら tar -xzf、.gz なら gunzip や gzip -d を使う、と対応付けて覚えるとよい。

2.8 環境変数と PATH

2.8.1 環境変数を使う

シェルやプログラムには、「名前」と「値」の組として渡される設定がある。これが環境変数である。環境変数は、シェルの中だけで完結するものではなく、そこから起動した Python や各種コマンドにも引き継がれる。現在設定されているよく使う環境変数の値を見るには、次のように打つ。

```
$ echo $HOME
$ echo $SHELL
$ echo $PATH
```

実行結果は、たとえば次のようになる。

```
/Users/ikomae
/bin/zsh
/opt/homebrew/bin:/usr/local/bin:/usr/bin:/bin:/usr/sbin:/sbin
```

1 行目はホームディレクトリ、2 行目は現在のシェル、3 行目は実行ファイルを探すディレクトリの一覧である。PATH はコロン : 区切りで並んでおり、シェルは左から順に探して最初に見つかった実行ファイルを使う。後で出てくる PYENV_ROOT や DATA_ROOT も同じ仲間であり、「プログラムへ外から値を渡す仕組み」と考えると分かりやすい。新たに環境変数を設定するには次のようにする。

```
$ export DATA_ROOT="$HOME/data/events"
$ echo $DATA_ROOT
```

結果の例は次の通りである。

```
/Users/ikomae/data/events
```

export 自体は成功しても通常は何も表示しない。続けて echo \$DATA_ROOT を実行し、意図した値が入ったことを確認する。ここでは \$HOME が先に展開され、その結果として絶対パスが保存されている。

ただし、この設定は通常そのターミナルを閉じると消える。毎回使う値ならシェルの設定ファイルに export ... を書いておく。どのファイルを編集するかは今使っているシェルで決まる。第 2.1 章で述べたように、echo \$SHELL を実行することで使用しているシェルの種類を確認することができる。/bin/zsh と出た場合は ~/.zshrc、/bin/bash と出た場合は ~/.bashrc に export ... を追記すればよい。

2.8.2 展開と引用

シェルでは、コマンドを実行する前に `$HOME` のような変数展開や、`$(pwd)` のようなコマンド置換が行われる。ここで重要なのは、引用符の種類によって展開の有無が変わることである。

```
$ echo $HOME
$ echo "$HOME"
$ echo '$HOME'
$ name="analysis result"
$ echo $name
$ echo "$name"
$ echo "$(pwd)"
$ echo "`pwd`"
```

1 行目と 2 行目は、どちらも `$HOME` を展開する。多くの場合、`HOME` には空白が入らないので、見た目の結果は同じになる。しかし、クォートなしの `$HOME` は展開後に空白で分割されたり、ワイルドカードとして再解釈されたりする可能性がある。これに対して `"$HOME"` は 1 つの引数として扱われるので、安全である。したがって、「変数は基本的にダブルクォートで囲む」と覚えた方がよい。

3 行目のシングルクォート `'...'` は、中の文字列をほぼそのまま扱う。そのため `$HOME` は展開されず、文字どおり `$HOME` と表示される。4 行目と 5 行目の違いは、空白を含む値で見ると分かりやすい。 `name="analysis result"` のあとで `echo $name` と書くと、シェルは `analysis` と `result` を別々の単語として扱う。これに対して `echo "$name"` なら、空白を含んだまま 1 つの値として渡る。6 行目の `$(pwd)` は `pwd` の出力で置き換える書き方であり、7 行目のバッククォート記法も同じ目的で使われる。ただし、新しく書くなら `$(...)` の方が読みやすく、入れ子にもできるので自然である。

この違いは、設定ファイルへ 1 行追記するときに特に重要である。たとえば `echo 'export PATH="$HOME/bin:$PATH"' >> ~/.zshrc` のようにシングルクォートで囲むと、`$HOME` や `$PATH` を展開せず、その文字列をそのまま `~/.zshrc` へ追記できる。もしダブルクォートで囲むと、その場で現在の値へ展開されて書き込まれるので、意図とずれることがある。

また、空白と改行の扱いにも注意が必要である。クォートの外では、空白やタブや改行が単語の区切りになる。そのため、空白を含むファイル名はクォートしないと壊れる。

```
$ mkdir "analysis result"
$ cp "analysis result/data.txt" backup/
```

1 つの長いコマンドを複数行へ分けたいときは、行末に `\` を置いて改行を継続する。

```
$ python analyze.py \
--input data/events.txt \
--output result.txt
```

改行は通常その場でコマンドの終わりになるので、\ を付けずに改行すると別のコマンドとして解釈される。コマンド例を写すときは、空白、クォート、改行のどれが意味を持っているかを意識した方が安全である。

2.8.3 PATH と実行ファイル

シェルが `python3` や `ls` を実行できるのは、それらが実行ファイルとして保存されており、しかもシェルがその場所を知っているからである。その探索に使う環境変数が `$PATH` である。実際に実行ファイルがどこから使われているかを探すには、次のコマンドを用いる。

```
$ which python3
$ which ls
$ echo $PATH
```

出力の例は次のようになる。

```
/opt/homebrew/bin/python3
/bin/ls
/opt/homebrew/bin:/usr/local/bin:/usr/bin:/bin:/usr/sbin:/sbin
```

`which python3` は「いま `python3` という名前で実際に呼ばれる実行ファイルの場所」を返す。ここでは Homebrew で入れた Python が使われている。`which ls` が `/bin/ls` なら、`ls` は OS 標準のコマンドを使っているという意味である。3 行目の `PATH` と見比べると、`/opt/homebrew/bin` が `/usr/bin` より左にあるため、`python3` ではそちらが優先されていることが分かる。

`bin` という名前は、実行ファイルを置くディレクトリによく使われる。後で出てくる `.venv/bin` の `bin` も、この慣習に従っている。

自分が作ったシェルスクリプトや実行ファイルを別のディレクトリ（例えば `~/my_scripts`）に置き、どこからでもコマンド名だけで呼び出したい場合は、次のように現在の `$PATH` の先頭に自分のディレクトリを追加して上書きすればよい。これもシェル起動時に毎回行う必要があるなら、`~/.zshrc` などに記載しておく。

```
$ export PATH="$HOME/my_scripts:$PATH"
```

2.9 リダイレクトとログ保存

2.9.1 echo、標準出力、標準エラー

シェルのコマンドは、単に画面へ文字を出すだけではなく、出力先を切り替えられる。ここで区別したいのは、通常の結果を流す標準出力（standard output, `stdout`）と、エラーメッセージを流す標準エラー（standard error, `stderr`）である。

`echo` は標準出力へ文字列を出す最も基本的なコマンドであり、動作確認や設定ファイルへの追記でよく使う。

```
$ echo "hello"
$ echo "$HOME"
```

1 行目は単に `hello` を出力する。2 行目は前節で述べた変数展開により、`$HOME` の値を出力する。ここで重要なのは、`echo` 自体は画面へ出しているだけであり、その出力先は次の節のリダイレクトで変えられるという点である。

2.9.2 > と >>

`>` は標準出力をファイルへ書き出す。既存のファイルがあれば中身を上書きする。

```
$ echo "alpha" > notes.txt
$ cat notes.txt
alpha
```

この例では、`echo` の結果が画面ではなく `notes.txt` に保存される。すでに `notes.txt` が存在していた場合も、その内容は新しい内容で置き換わる。したがって、`>` は便利だが、うっかり使うと既存ファイルを消してしまう。

`>>` は追記であり、既存の内容を残したまま末尾に足す。

```
$ echo "alpha" > notes.txt
$ echo "beta" >> notes.txt
$ cat notes.txt
alpha
beta
```

設定ファイルの末尾へ 1 行追加したいときは、この `>>` を使う。たとえば `~/.zshrc` に環境変数の設定を足すときに自然である。逆に、`>` を使うと既存の設定を消してしまうので注意が必要である。

2.9.3 1> と 2>

`>` は実際には `1>` と同じ意味であり、標準出力だけをファイルへ送る。標準エラーを別ファイルへ分けたいときは `2>` を使う。

```
$ python analyze.py > output.log
$ python analyze.py 1> output.log 2> error.log
```

1 行目では通常の実出力だけが `output.log` に入り、エラーメッセージは画面に残る。2 行目では、通常の実出力を `output.log` に、エラーメッセージを `error.log` に分けて保存している。長い処理を回すときは、「結果は保存したいが、失敗したときの原因も別に残したい」ということが多いので、この書き方は有用である。

標準出力と標準エラーを同じファイルへまとめたいときは、`2>&1` を使う。

```
$ python analyze.py > run.log 2>&1
```

これは、標準エラーの行き先を標準出力と同じ先へ合わせる、という意味である。ログを1本にまとめたいときに使う。

2.9.4 tee で見ながら保存する

> や >> を使うと出力はファイルへ送られ、画面には出なくなる。実行中の様子も見たいが、同時に保存もしたいときは tee が便利である。tee は標準入力を受け取り、その内容を画面へ出しつつファイルへも書く。

```
$ python analyze.py | tee run.log
$ python analyze.py 2>&1 | tee run.log
```

1 行目は標準出力だけを画面と run.log の両方へ流す。2 行目は標準エラーもまとめてから tee へ渡しているので、実質的に「全部を見ながら全部を保存する」形になる。処理が数分から数十分かかる場合、進捗を画面で見つつ、あとで見返すためのログも残せるので実用的である。

tee に -a を付けると追記になる。

```
$ python analyze.py 2>&1 | tee -a run.log
```

この場合は、既存の run.log を消さずに末尾へ足していく。毎日の実行結果を1つのログへ蓄積したい場合に向く。

2.10 権限とスクリプトの実行

2.10.1 権限を読む

ファイルにはアクセス権がある。読み取れるか、書き込めるか、実行できるかを rwx で表す。ls -l を使うと確認しやすい。

```
-rwxr-xr-x 1 user staff 1234 Apr 8 10:00 run_analysis.sh
-rw-r--r-- 1 user staff 411 Apr 8 10:05 notes.txt
-rw----- 1 user staff 411 Apr 8 10:05 id_ed25519
```

最初の 10 文字のうち、先頭 1 文字はファイル種別であり、残り 9 文字が権限である。3 文字ずつに区切って、所有者、グループ、その他の利用者に対する権限を表す。文字と権限の対応は表 2.3 のとおりである。

権限を変更するには chmod コマンドを用いる。所有者 (user) に実行権限 (x) を付与したければ u+x、その他の利用者 (other) の読み込み権限 (r) と書き込み権限 (w) を剥奪したければ o-rw、グループ (group) とその他 (other) の権限を読み取り (r) のみにしたければ

go=r のように指定すればよい。a+x のようにして、すべてのユーザー (all) の権限を変更することもできる。

```
$ chmod u+x my_script.sh
$ chmod o-rw notes.txt
$ chmod go=r read_only.txt
$ chmod a+x run_analysis.sh
```

表 2.3 権限の記号と数字

記号	数字	意味
r	4	読み取り
w	2	書き込み
x	1	実行

表 2.3 に示すように、権限は数字で指定することもできる。数字指定では権限を足し算で表す。たとえば 7 は rwx、6 は rw-、5 は r-x である。したがって 755 は所有者が rwx、それ以外が r-x を意味し、644 は所有者が rw-、それ以外が r-- を意味する。秘密鍵のように本人だけが読めればよいものには 600 がよく使われる。設定を変更する例を下に示す。

```
$ chmod 755 run_analysis.sh
$ chmod 644 notes.txt
$ chmod 600 ~/.ssh/id_ed25519
```

2.10.2 スクリプトを直接実行する

Python スクリプトは `python3 hello.py` の形で実行するなら、実行権限がなくてもよい。これに対して `./hello.py` のように直接実行する場合には、どのプログラムで解釈するかをファイル先頭を書いておく必要がある。その先頭行の `#!` で始まる記法を、シバン (shebang) という。

```
1 #!/usr/bin/env python3
2
3 print("hello")
```

このように記述した上で、ファイルそのものに対して直接実行できるようにするため、実行権限を付与する。所有者に対して権限を付与する場合は、付与の対象 (u) を省略することができる。

```
$ chmod +x hello.py
$ ./hello.py
```

`#!/usr/bin/env python3` は、「python3 を見つけてこのファイルを実行せよ」という意味である。いきなり言葉だけで覚える必要はないが、「直接実行にはシバンと実行権限の両方が必要だ」という構造だけは理解しておいた方がよい。この考え方をシェルスクリプト全体へ

広げると、自動化や一括処理を書けるようになる。付録 E では、for ループや条件分岐を用いた実践的なバッチ処理の書き方をまとめている。

2.11 章末問題

各課題では、コマンドを実行した事実だけでなく、入力、主要なオプション、出力の読み方を記録する。ネットワーク上のデータを取得できない場合でも、1 問目と、手元の任意の CSV を使った 3 問目は実行できる。

1. 章末問題用の作業ディレクトリを作る。次のコマンドを実行し、どのようなディレクトリが作成されるか確認せよ。

```
$ mkdir -p earthquake_practice/{raw,compressed,scripts,figures,results}
$ cd earthquake_practice
$ pwd
$ ls
```

シェルは `earthquake_practice/{raw,compressed,scripts,figures,results}` を、コンマで区切られた 5 通りのパスへ展開してから `mkdir` へ渡す。この機能をブレース展開という。`-p` は親ディレクトリも作り、対象が既存でもエラーにしない。`ls` の結果に 5 個の名前が現れることを確認し、`pwd` の出力がどのディレクトリを示すかも説明せよ。

2. USGS の地震カタログを取得し、シェルで中身を調べる。本書の章末課題では、アメリカ地質調査所（United States Geological Survey、USGS）が公開している地震カタログを使用する。以下のコマンドを実行し、2024 年のカタログを `raw/` へ保存し、行数、先頭数行、末尾数行、特定の地名を含む行を調べよ。

```
$ curl -L "https://earthquake.usgs.gov/fdsnws/event/1/query?format=csv&starttime=2024-01-01&endtime=2024-01-31&limit=20000&orderby=time-asc" \
-o raw/usgs_global_2024_01.csv
$ ls -lh raw/usgs_global_2024_01.csv
$ wc -l raw/usgs_global_2024_01.csv
$ head -n 3 raw/usgs_global_2024_01.csv
$ tail -n 3 raw/usgs_global_2024_01.csv
$ grep -i "Japan" raw/usgs_global_2024_01.csv | head
```

どのコマンドが「件数確認」「冒頭確認」「末尾確認」「文字列検索」に対応しているかを整理してまとめよ。`curl` の `-L` は転送先が変更された場合に追従し、`-o` は保存先を指定する。`wc -l` の値にはヘッダ行も含まれるので、データ件数との違いを説明せよ。`grep -i` の `-i` と、後段の `head` が検索結果を何件に制限するかも記録すること。

3. 同じ CSV を 3 種類の圧縮形式で扱う。`gzip`、`tar.gz`、`zip` をそれぞれ 1 つずつ作り、圧縮前後の容量を確認せよ。

```
$ gzip -k raw/usgs_global_2024_01.csv
$ tar -czf compressed/usgs_global_2024_01.tar.gz raw/usgs_global_2024_01.csv
```

```
$ zip -r compressed/usgs_global_2024_01.zip raw/usgs_global_2024_01.csv
```

gzip -cd、tar -tzf、unzip -l を使って中身を確認し、それぞれのオプションの意味についてまとめよ。gzip -k は元の CSV を残し、tar -czf は複数ファイルをまとめた書庫を gzip 圧縮し、zip -r は指定ディレクトリ以下を再帰的に格納する。ls -lh で各ファイルの容量を表示し、圧縮率を $\text{圧縮後の容量} / \text{圧縮前の容量}$ として比較せよ。表示単位が丸められるため、厳密な比には ls -l または stat のバイト数を使うこと。

4. **発展課題。** URL の構造について調べよ。カタログの URL (<https://earthquake.usgs.gov/fdsnws/event/1/query?format=csv&starttime=2024-01-01&endtime=2024-01-31&limit=20000&orderby=time-asc>) を確認し、https、earthquake.usgs.gov、usgs.gov、.gov、/fdsnws/event/1/query、?format=csv&starttime=2024-01-01&endtime=2024-01-31&limit=20000&orderby=time-asc のそれぞれについて、名称や役割をまとめよ。ここでは順に、スキーム、ホスト名、登録ドメイン、トップレベルドメイン、パス、クエリ文字列を表す。クエリ文字列は & で複数の 名前=値 を区切る。format、starttime、endtime、limit、orderby が取得結果をどう変えるかを、2 問目の出力と対応させて説明せよ。

シェルでファイルの所在、内容、権限、入出力を確認できれば、Python の導入後も実行されるプログラムと保存先を追跡できる。次章では、PATH と実行ファイルの知識を用いて、Python のバージョン、仮想環境、プロジェクトの依存関係を管理する。

第 3 章

Python 環境を整える

3.1 Python の確認と管理方針

3.1.1 何を分けて考えるべきか

Python を使い始めるときは、次の 3 つを分けて考える必要がある。

1. Python 本体を入れること
2. 複数バージョンの Python を切り替えること
3. プロジェクトごとの仮想環境を作ること

まず Python 本体が必要であり、そのうえで Python 3.11 と 3.12 を切り替えたいならバージョン管理ツールが必要になる。さらに、同じ Python 3.12 でもプロジェクト A と B で依存ライブラリが違えば、仮想環境で分ける必要がある。この 3 段階を分けて理解した方が、あとで混乱しにくい。

3.1.2 今の Python を確認する

macOS でも WSL でも、最初にいま見えている Python を調べておく方がよい。現在利用可能な Python を確認するには、次のように実行する。

```
$ python3 --version
$ which python3
$ python3 -c "import sys; print(sys.executable)"
```

結果の例は次のようになる。

```
Python 3.12.10
/opt/homebrew/bin/python3
/opt/homebrew/Cellar/python@3.12/3.12.10/bin/python3.12
```

1 行目は、いま python3 で呼ばれるバージョンを表す。2 行目は、シェルが実際に見つけた python3 という名前の実行ファイルである。3 行目は、その Python の内部から見た実行ファイルの実体であり、シンボリックリンクの先まで含めて表示されることがある。2 行目と 3 行目が完全に同じ文字列でなくても問題はないが、「どちらも同じ系列の Python を指しているか」は確認した方がよい。複数バージョンを切り替えるようになると、この確認は非常に重要になる。

3.2 pyenv による複数バージョンの管理

Python 本体の導入と切り替えには pyenv が分かりやすい。pyenv は、シェルから見える python をディレクトリ単位で切り替えるための道具であり、複数バージョンの Python を使い分けるときの基準として扱いやすい。

3.2.1 macOS での準備

pyenv は Python 本体をビルドして入れるので、macOS では Xcode Command Line Tools が必要になる。すでに Homebrew が正常に動いているなら、Command Line Tools は入っていることが多い。すでにインストールされているかを確認するには次のように実行する。

```
$ xcode-select -p
```

すでにインストールされていれば、結果の例は次のようになる。

```
/Library/Developer/CommandLineTools
```

このようにパスが表示されれば、Command Line Tools はインストール済みである。逆に、エラー文が出て何も表示されなければ未導入の可能性が高い。未導入の場合は、次のコマンドを実行してインストールする。

```
$ xcode-select --install
```

3.2.2 WSL / Ubuntu での準備

WSL の Ubuntu では、ビルドに必要なライブラリを先に入れておく。一括で導入するには次のコマンドを実行する。

```
$ sudo apt update
$ sudo apt install -y build-essential curl git libssl-dev zlib1g-dev \
  libbz2-dev libreadline-dev libsqlite3-dev libffi-dev liblzma-dev tk-dev
```

apt update の出力は長いが、末尾でたとえば次のように Done が見えれば、パッケージ一覧の更新は完了している。

```
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
```

`apt install` でも多くの行が流れる。今回の例では `-y` を付けているので、通常は確認なしで続行する。`-y` を外して実行した場合は、途中で次のような確認が出ることがある。

```
Need to get 48.2 MB of archives.
After this operation, 201 MB of additional disk space will be used.
Do you want to continue? [Y/n]
```

この表示が出たら、示されているパッケージや使用量に問題がないことを確認し、続けるなら `y` を入力して `Enter` を押す。`n` を入力すると処理は中止され、インストールは行われない。

インストールの終盤では、次のように `Setting up ...` が並ぶことがある。

```
Setting up libffi-dev:amd64 (3.4.4-1) ...
Setting up libsqlite3-dev:amd64 (3.40.1-2ubuntu0.2) ...
Setting up build-essential (12.9ubuntu3) ...
```

これは各パッケージの設定を進めている段階であり、この表示自体は正常である。最後まで流れたあとでエラーが出ずにシェルのプロンプトへ戻れば、インストールは完了している。

3.2.3 pyenv を入れてシェルへ組み込む

macOS では、まず次のように実行する。

```
$ brew install pyenv
```

結果の末尾では、たとえば次のような行が見える。

```
==> Pouring pyenv--2.6.2.arm64_sequoia.bottle.tar.gz
Installed to /opt/homebrew/Cellar/pyenv/2.6.2
```

最後の行は、Homebrew が `pyenv` をどこへ入れたかを示している。エラーなくこの行まで進めば、インストール自体は完了している。

一方、WSL / Ubuntu では通常 Homebrew を前提にしないので、前節の依存パッケージを入れたあとにインストーラスクリプトを使う。

```
$ curl -fsSL https://pyenv.run | bash
```

`curl ... | bash` の出力は長くなりやすいが、エラーなくシェルへ戻ればインストール自体は完了している。

どちらの方法でも、インストール直後はまだ `pyenv` を通常のコマンドとして使える状態にはなっていない。次に、自分のシェルに対応する設定ファイルへ初期化コードを書き込む。設定先は、`/bin/zsh` なら `~/.zshrc`、`/bin/bash` なら `~/.bashrc` である。以下では `>>` を使って

設定ファイルの末尾へ1行ずつ追記している。>ではなく>>を使うのは、既存の設定を消さないためである。

```
$ echo 'export PYENV_ROOT="$HOME/.pyenv"' >> ~/.zshrc
$ echo '[[ -d $PYENV_ROOT/bin ]] && export PATH="$PYENV_ROOT/bin:$PATH"' >> ~/.zshrc
$ echo 'eval "$(pyenv init - zsh)"' >> ~/.zshrc
$ exec "$SHELL"
```

exec "\$SHELL" は、いま動いているシェルを \$SHELL で示されるシェル本体で置き換えて、設定をその場で読み直すためのコマンドである。SHELL は現在使っているシェルのパスを入れた環境変数であり、zsh なら通常 /bin/zsh、bash なら通常 /bin/bash が入っている。ここでは ~/.zshrc に追記した内容をすぐ反映させるために実行している。ターミナルをいったん閉じて開き直しても同じ目的は果たせる。

echo ... >> ~/.zshrc 自体は成功しても何も表示しない。確認は、シェルを読み直した後に pyenv が見えるかで行う。たとえば次のように打つ。

```
$ pyenv --version
pyenv 2.6.2
$ which pyenv
/Users/ikomae/.pyenv/bin/pyenv
```

- 1 行目でバージョン番号が出れば、シェルから pyenv を実行できる状態になっている。
- 2 行目では、pyenv が ~/.pyenv/bin から見つかったことが分かる。ここで command not found になる場合は、設定ファイルへの追記先を間違えたか、exec "\$SHELL" で読み直せていない可能性が高い。

使っているシェルが bash の場合は、次のように ~/.bashrc に対して設定を追記する。

```
$ echo 'export PYENV_ROOT="$HOME/.pyenv"' >> ~/.bashrc
$ echo '[[ -d $PYENV_ROOT/bin ]] && export PATH="$PYENV_ROOT/bin:$PATH"' >> ~/.bashrc
$ echo 'eval "$(pyenv init - bash)"' >> ~/.bashrc
$ exec "$SHELL"
```

3.2.4 バージョンを入れて切り替える

pyenv を使う場合は、pyenv install でバージョンを追加し、pyenv global、pyenv local、pyenv shell で切り替える。たとえば新しいバージョンを追加して確認するには次のように行う。

```
$ pyenv install 3.11.11
$ pyenv install 3.12.10
$ pyenv versions
```

pyenv install は数分かかることがあり、途中でビルドログが多数流れる。完了後に pyenv versions を実行すると、たとえば次のように表示される。

```
* system (set by /Users/ikomae/.pyenv/version)
 3.11.11
```

3.12.10

system は、pyenv で切り替えない場合に OS 側が標準で見せる Python である。3.11.11 と 3.12.10 が一覧に出ていれば、そのバージョンはインストール済みである。先頭の * は、いま有効になっている系統を示す印であり、この段階ではまだ system のままでも問題はない。

その後、システム全体で普段使う標準のバージョンを決めるなら、次のように pyenv global を実行する。

```
$ pyenv global 3.12.10
$ python --version
$ which python
```

結果の例は次のようになる。

```
Python 3.12.10
/Users/ikomae/.pyenv/shims/python
```

pyenv global は普段使うバージョンを決める。ここで python の実体が ~/.pyenv/shims/python になっている点が重要である。pyenv はこの shim を経由して、裏でどのバージョンを起動するかを切り替えている。これに対して pyenv local は、そのディレクトリだけ別のバージョンを使いたいときに使う。特定のプロジェクト用ディレクトリ内でバージョンを固定するには、次のような手順を踏む。

```
$ mkdir analysis311
$ cd analysis311
$ pyenv local 3.11.11
$ python --version
$ cat .python-version
```

結果の例は次のようになる。

```
Python 3.11.11
3.11.11
```

pyenv local を実行すると .python-version が作られ、そのファイルの中身として 3.11.11 が書き込まれる。そのディレクトリでは指定したバージョンが優先される。つまり、analysis311 に入っている間だけ python の結果が 3.11 へ切り替わる。これによって、同じマシンの中でプロジェクト A は 3.11、プロジェクト B は 3.12 という使い分けができる。

pyenv shell 3.11.11 は、そのターミナルを閉じるまでの一時的な切り替えである。検証用に 1 回だけ古いバージョンで動かしたいときに便利である。

3.3 venv でプロジェクトごとの仮想環境を作る

Python 本体のバージョンを決めたら、そのバージョンの上にプロジェクト専用の仮想環境を作る。依存ライブラリの衝突を避けるためである。仮想環境を作り、そこに入るための基本コマンドは次の通りである。

```
$ python -m venv .venv
$ source .venv/bin/activate
$ which python
$ which pip
$ python --version
$ pip --version
```

仮想環境へ入ると、プロンプトの先頭に (.venv) が付くことが多い。ターミナルの表示は次のように変わる。

```
(.venv) ikomae@macbook analysis $
```

which python 以降の結果は、たとえば次のようになる。

```
/Users/ikomae/analysis/.venv/bin/python
/Users/ikomae/analysis/.venv/bin/pip
Python 3.11.11
pip 25.0 from /Users/ikomae/analysis/.venv/lib/python3.11/site-packages/pip
```

which python と which pip を見ると、.venv/bin/python と .venv/bin/pip が使われていることが分かる。これは、source .venv/bin/activate が \$PATH の先頭に .venv/bin を追加しているからである。最後の pip ... from .../.venv/... という表示も、「この仮想環境の中の pip を使っている」ことの確認になる。

3.3.1 pip と python -m pip

仮想環境へ入っているなら、通常は pip と python -m pip は同じ環境の pip を指す。したがって、作業中は pip install numpy と書いてよい。

一方で、いまどの Python が有効か不安なときや、スクリプトや文書の中で「この Python に対応する pip を使う」と明示したいときは、python -m pipの方が誤解が少ない。実務では両方見るが、意味の違いを分かって使うことが重要である。実際にライブラリを入れる例を挙げる。

```
$ pip install --upgrade pip
$ pip install numpy pandas polars matplotlib scipy astropy tqdm
```

pip install の出力は長いが、末尾ではたとえば次のような行が見える。

```
Successfully installed numpy-2.3.0 pandas-2.3.0 scipy-1.15.0
```

ここで Successfully installed ... が出れば、少なくともインストール処理自体は成功している。直後に python -c "import numpy, pandas" のように試して import エラーが出ないことまで確認すると確実である。

作業を終えたら deactivate で仮想環境から抜けられる。

3.4 解析用ライブラリと JupyterLab

3.4.1 JupyterLab を併用する

対話的に試しながら解析したいときは、JupyterLab も便利である。小さなコード片をその場で動かし、数値や図をすぐ確認できるため、探索的な作業では特に役立つ。これを導入・起動するには次のように実行する。

```
$ pip install jupyterlab
$ jupyter lab
```

jupyter lab を実行すると、ターミナルにはたとえば次のような行が出る。

```
[I] Jupyter Server 2.15.0 is running at:
[I] http://localhost:8888/lab?token=abc123...
```

この URL は、ブラウザから接続するためのアドレスである。localhost:8888 は「いま使っているマシン上で待ち受けている」ことを意味し、token=... は最初の接続確認に使う一時的な鍵である。ブラウザが自動で開かなかった場合も、この URL を開けば JupyterLab へ入れる。

JupyterLab の利点は、コードを実行できるだけでなく、その前後に説明文、数式、図の読み方を書き添えられる点にある。解析の途中経過を他人へ見せるときも、自分で後から読み返すときも、コードと説明が同じノートブックに並んでいる方が理解しやすい。たとえば次のような書式で Markdown セルを書くことができる。

```
# 解析メモ

- 入力ファイル: `events_2024_01_01.txt.gz`
- 閾値: `signal > 5.0`

 $\Delta t$  の分布を確認する
```

ただし、JupyterLab は最終成果物の置き場所というより、試行錯誤の場として使う方がよい。解析手順を再現したい段階では、ノートブック中で固めた処理を Python スクリプトへ戻し、入力や出力を明示した方が後で困りにくい。ここまでの「試しながら考える」ための環境であり、次に扱う uv は、その試行錯誤をプロジェクトとして固定するための道具である。

3.5 プロジェクト管理ツール uv の利用

uv は Python 本体の導入もできるが、本書ではそれよりも「プロジェクト単位の管理」に使う道具として位置付ける方が自然である。特に、依存関係、ロックファイル、`.python-version`、Git 管理、パッケージ化、ワークスペース管理といった、プロジェクト全体の整理で力を発揮する。

3.5.1 プロジェクトの初期化

多くの人は、まずシンプルに `uv init` を使えばよい。オプションは後から必要になったときに足せばよい。新しいプロジェクトを始めるには、次のようにディレクトリを作ってから初期化する。

```
$ mkdir analysis_uv
$ cd analysis_uv
$ uv init
```

`uv init` 自体は短い完了メッセージだけで終わることもあるので、結果はファイル構成で確認の方が確実である。たとえば `ls -a` を実行すると、既定では少なくとも次のようなファイルやディレクトリが見える。(tree で確認してもよい。)

analysis_uv/ の初期構成

```
.git/
.gitignore
.python-version
README.md
main.py
pyproject.toml
```

つまり、`uv init` は単に Python ファイルを 1 つ置くだけではなく、Git 管理の土台までまとめて用意する。この点が `venv` 単体との大きな違いである。まずはこの既定構成を理解すれば十分であり、以下のオプションは「最初から構成を変えたい事情がある場合」にだけ見ればよい。

- `uv init --bare`: `pyproject.toml` だけを作る
- `uv init --vcs none`: Git 初期化をしない
- `uv init --no-pin-python`: `.python-version` を作らない
- `uv init --package` や `uv init --lib`: スクリプトではなくパッケージやライブラリとして始める

特に、コードを再利用可能なモジュールやライブラリとして育てたいなら、`--package` や `--lib` を意識した方がよい。単発スクリプトで終わるのか、将来 `import` される部品になるの

かで、適切な初期構成は変わる。

3.5.2 依存関係の追加と環境の同期

`uv add` は依存ライブラリを追加し、`pyproject.toml` とロック情報を更新する。複数同時に指定する場合は次のようにする。

```
$ uv add numpy pandas polars matplotlib scipy astropy tqdm
```

`uv add` を実行すると、依存関係の解決結果が要約表示される。たとえば次のようになる。

```
Resolved 18 packages in 0.42s
Installed 7 packages in 0.31s
+ numpy==2.3.0
+ pandas==2.3.0
+ scipy==1.15.0
```

`Resolved ... packages` は、直接指定したライブラリだけでなく、その依存先も含めて必要な集合を決めたことを表す。`Installed ... packages` は実際に環境へ追加した数であり、その下の `+` 付き行は今回入った主なパッケージ名とバージョンである。

`uv sync` は、`pyproject.toml` とロックファイルに基づいて環境をそろえる。ここで初めて `.venv` が作られることもある。実行コマンドは次の通りである。

```
$ uv sync
```

`uv sync` の後は、たとえば次のような行が見える。

```
Creating virtual environment at: .venv
Installed 18 packages in 0.54s
```

ここで `Creating virtual environment at: .venv` と出れば、そのプロジェクト専用の仮想環境が作られている。過去の自分の解析や、他人から受け取ったプロジェクトを再現するときは、この `uv sync` が重要になる。リポジトリを取得して `uv sync` すれば、必要な依存関係をまとめてそろえやすい。ここで `Git` の基本が必要になるので、付録 B もあわせて読むとよい。

3.5.3 コマンドの実行とバージョン固定

`uv run` は、そのプロジェクトの環境でコマンドを実行する。スクリプトを直接走らせるときと、`Python` 本体を起動したいときとで書き分ける。いくつか例を挙げる。

```
$ uv run main.py
$ uv run python --version
$ uv run python -m pytest
```

たとえば `uv run python --version` の結果は次のようになる。

```
Python 3.12.10
```

この1行は、「いまのプロジェクト設定に従って起動された Python が 3.12.10 である」ことを示している。実行対象が Python スクリプトそのものであるなら、`uv run main.py` と書けばよい。わざわざ `python` を挟む必要はない。一方で、`-m` を使いたいときや、Python 本体へオプションを渡したいときは `uv run python ...` と書く。

ここまでは「その環境でどう実行するか」を見た。次は「そのプロジェクトがどの Python を使うか」を固定する。

`uv` もプロジェクト単位で Python バージョンを固定できる。`pyenv` がシェル全体の切り替えを担当し、`uv` がプロジェクト側の固定を担当する、と分けて考えると分かりやすい。固定や一発実行のコマンド例は次の通りである。

```
$ uv python pin 3.12
$ uv run python --version
$ uv run --python 3.11 python --version
```

結果の例は次の通りである。

```
Python 3.12.10
Python 3.11.11
```

1 行目は、`uv python pin 3.12` によってプロジェクト既定が 3.12 系になっていることを示す。2 行目は、`--python 3.11` を付けたその1回だけ 3.11 系で起動した結果である。`pin` は既定値の設定、`--python` は一時的な上書きとして読むとよい。

3.5.4 標準ライブラリと外部ライブラリ

Python には、最初から使える標準ライブラリと、自分で追加する外部ライブラリがある。

- 標準ライブラリの例: `math`, `pathlib`, `glob`, `argparse`
- 外部ライブラリの例: `numpy`, `pandas`, `polars`, `matplotlib`, `scipy`, `astropy`

ここまでで Python 本体、仮想環境、プロジェクト管理の役割を見たので、次はその環境へ何を入れるのかを整理する。標準ライブラリは Python 本体に含まれている。外部ライブラリは、目的に応じて自分で追加する。解析では外部ライブラリが非常に重要だが、まずは「何が標準で、何が追加なのか」を区別しておくことが大切である。

3.5.5 解析用のライブラリを入れる

本書の後半で使う主なライブラリを表 3.1 に示す。

`venv` を使うなら `pip install`、`uv` を使うなら `uv add` と書けばよい。まず `venv` を使う場合は、次のように書く。

```
$ pip install numpy pandas polars matplotlib scipy astropy tqdm
```

インストールが成功すると、末尾ではたとえば次のように表示される。

```
Successfully installed astropy-7.0.0 matplotlib-3.10.0 numpy-2.3.0 pandas-2.3.0
```

表 3.1 本書で使う主なライブラリと役割

ライブラリ	主な役割
NumPy	数値配列の高速処理
pandas	表形式データの読み込みと加工
Matplotlib	図の作成
SciPy	フィットと数値計算
Astropy	時刻・単位・座標の扱い
Polars	大規模な表データの高速処理
tqdm	進捗表示

ここでは、指定した主要ライブラリがその仮想環境へ入ったことが分かる。バージョン番号まで表示されるので、レポートや再現用メモではこの情報も残しておくに役立つ。

同じライブラリを uv 管理のプロジェクトへ追加するなら、次のように書く。

```
$ uv add numpy pandas polars matplotlib scipy astropy tqdm
```

uv 管理のプロジェクトでは、たとえば次のような要約が表示される。

```
Resolved 18 packages in 0.42s
Installed 7 packages in 0.31s
```

pip install と違って、uv add は環境へ入れるだけでなく、pyproject.toml に依存関係を記録する点が重要である。つまり、「いまだけ入った」のではなく、「このプロジェクトはこれらのライブラリに依存している」と宣言したことになる。

3.6 引数・設定ファイルとコードの分離

3.6.1 コマンドライン引数と設定ファイルを使い分ける

同じ解析スクリプトを何度も使うなら、入力ディレクトリや出力先をコードに埋め込むより、コマンドライン引数で渡す方がよい。Python では argparse が標準で使える。もっともシンプルな構成例を下に示す。

```
1 import argparse
2
3 parser = argparse.ArgumentParser()
4 parser.add_argument("input_dir")
5 parser.add_argument("--outdir", default="output")
6 args = parser.parse_args()
7
8 print(args.input_dir, args.outdir)
```

このスクリプトを analyze.py として保存し、次のように実行するとする。

```
$ python analyze.py data/run001
data/run001 output
$ python analyze.py data/run001 --outdir results/figures
data/run001 results/figures
```

1 行目では、位置引数 `input_dir` に `data/run001` が入り、`--outdir` は省略したので既定値 `output` が使われている。2 行目では、`--outdir` を明示したため、既定値が `results/figures` に上書きされている。引数の説明では、「どの値が必須で、どの値が省略時の既定値を持つか」を結果と対応させて読むことが重要である。

ただし、何でも引数にすればよいわけではない。毎回ほぼ同じ値まで毎回書かせると、使いにくいスクリプトになる。

値の置き場所は、表で機械的に覚えるより、用途ごとに言葉で整理した方が分かりやすい。

コマンドライン引数 実行ごとに変わる値を置く。入力ディレクトリ、出力先、日付範囲、並列数などが典型である。

設定ファイル プロジェクト単位では固定だが、将来変更し得る値を置く。較正ファイルのパス、検出器 ID、閾値、図の保存形式などが当てはまる。

環境変数 マシン依存の値や秘密情報を置く。`DATA_ROOT`、API トークン、認証情報などが典型である。

コード内の定数 定義として固定したい値や単位変換を書く。`NS_PER_SECOND` や物理定数のように、意味が仕様として決まっているものを置く。

入力ファイルのパス、出力先、しきい値、作業ディレクトリのように「実行条件」であるものは、原則としてハードコーディングしない方がよい。コードを書き換えずに切り替えられるようにしておくと、再利用と再現が楽になる。

3.6.2 設定ファイルとマジックナンバーの排除

めったに変わらない値は、設定ファイルへまとめる方が使いやすい。形式は JSON でも TOML でもよい。ここでは標準ライブラリだけで扱える JSON の例 (`config.json`) を示す。

`config.json`

```
{
  "calibration_path": "calib/detector_a.csv",
  "coincidence_window_ns": 250,
  "figure_format": "pdf"
}
```

この JSON ファイルを Python のスクリプトから読み込む場合のコード例は次のようになる。

```
1 import json
2 from pathlib import Path
3
```

```
4 config = json.loads(Path("config.json").read_text())
5 window_ns = config["coincidence_window_ns"]
6 print(window_ns)
```

実行結果は次のようになる。

```
250
```

この 250 は、JSON の中の `coincidence_window_ns` を読み取った結果である。ここで重要なのは、250 という値がコードへ直接書かれていなくても、設定ファイルから読み込んで使える点である。このようにしておけば、同じコードを保ったまま設定だけを差し替えられる。Python 3.11 以降なら `tomllib` を使って TOML を読むこともできる。どの形式を選ぶにしても、「たまに変わるがコード本体ではない値」を分離することが大切である。

250 や 3600 のような数値が説明なしにコードへ裸で出てくると、後から読んだ人は意味を判断しにくい。このような数値はマジックナンバーと呼ばれる。詳しくは第 4.5.1.3 節で述べるが、意味のある数値には名前を付け、実行条件なら引数や設定ファイルへ出す、という整理を早い段階で身に付けた方がよい。

3.7 章末問題

この章の課題では、コマンドだけでなく、作成されたファイルと実際に選ばれた Python を確認する。バージョン番号は環境によって異なるため、特定の番号との一致ではなく、各コマンドの出力が互いに矛盾していないかを検証する。

1. **uv を使って地震データ解析用の新しいプロジェクトを作る。** `earthquake_practice` の下に `usgs_project` を作り、`uv init` を実行せよ。その後で、手元で使える Python を 1 つ選んで `uv python pin ...` で固定し、`uv add numpy matplotlib scipy` を実行せよ。その後で `uv run python --version` と `uv run python -c "import numpy, matplotlib, scipy"` を実行し、どの Python とどの仮想環境が使われているかを確認せよ。`uv init` 後には少なくとも `pyproject.toml` が作られ、Python を固定すると `.python-version` が作られる。`uv add` 後には依存関係が `pyproject.toml` へ記録され、通常は `uv.lock` と `.venv/` も作られる。`uv run python -c "import sys; print(sys.executable)"` を追加で実行し、表示されたパスがプロジェクトの `.venv` 以下にあることを確認せよ。`-c` が後続文字列を Python コードとして実行するオプションであることも説明すること。
2. **pyenv と venv で別の作り方も試す。** `earthquake_practice` の下に `usgs_venv` を作り、必要なら `pyenv local ...` で Python を切り替えてから `python -m venv .venv` を実行せよ。活性化の前後で `which python`、`python --version`、`pip --version` がどう変わるかを記録せよ。活性化後のプロンプト先頭に通常は `(.venv)` が現れ、`which python` は `usgs_venv/.venv/bin/python` を指す。`pip --version` の出力にも同じ `.venv` 以下のパスが含まれることを確認せよ。最後に `deactivate` を実行し、`which python` が活性化前の場所

へ戻ることも記録する。`python -m venv .venv` の `-m` と最後の `.venv` がそれぞれ何を指定するかを説明せよ。

3. **発展課題。** この課題では、同じコマンド名 `python` でもディレクトリごとに別の実体を指せることを確かめる。手元で 2 系統以上の Python が使えるなら、別ディレクトリを 2 つ作り、それぞれで `pyenv local` を設定して `python --version`、`which python`、`cat .python-version` を比較せよ。さらに片方のディレクトリで `uv run python --version` も実行し、`pyenv` による切り替えと `uv` による固定がどう重なるかを整理せよ。比較結果には、シェルが最初に見つけた `python`、`pyenv local` が選ぶバージョン、`uv` がプロジェクト仮想環境で使うバージョンを区別して記述する。`which python` だけで実体を断定せず、`python -c "import sys; print(sys.executable); print(sys.version)"` でも確認せよ。

ここまでで、シェルから呼び出される Python の実体と、プロジェクトごとのライブラリ環境を区別できるようになった。次章では、この環境でインタラクティブモードとスクリプトを実行し、値、条件分岐、反復、関数を用いて処理を組み立てる。

第 4 章

Python の基本

4.1 データ型と基本操作

ターミナルで `python3` と入力すると、Python をインタラクティブモードで起動できる。インタラクティブモードに入ると、ターミナルに以下のような文字列が表示される。

```
$ python3
Python 3.13.5 (main, Jun 11 2025, 15:36:57) [Clang 17.0.0 (clang-1700.0.13.3)] on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

この章の前半では、インタラクティブモードで各例を実行し、入力した式と表示された値を対応させながら読み進める。

4.1.1 変数と基本型

Python では、値に名前を付けて使う。たとえばデータを表すいくつかの変数を定義すると次のようになる。

```
>>> n_rows = 12
>>> threshold = 3600.0
>>> label = "A"
>>> use_log = True
```

`n_rows` は整数、`threshold` は浮動小数点数、`label` は文字列、`use_log` は真偽値である。実際に `type()` で確かめると次のようになる。解析では、整数と浮動小数点数を区別して扱う感覚が重要になる。たとえば、件数は整数だが、フィットで得られるパラメータは浮動小数点数であることが多い。

```
>>> type(n_rows)
<class 'int'>
```

```
>>> type(threshold)
<class 'float'>
>>> type(label)
<class 'str'>
>>> type(use_log)
<class 'bool'>
```

Python では変数に対して明示的な型の宣言（「これは整数です」という指定）を記述しなくても自動的に推論されるため、型を意識せずにコードを書くことができる。これは非常に便利である反面、背後では「どの型のデータとして扱うべきか」を都度 Python 側が判定・管理・包含しているため、計算処理が重くなる原因にもなる。

後述する NumPy などを用いて大量のデータを高速に処理する際には、C 言語と同様に、メモリ上の表現まで区別した型を指定する場面がある。Python の型と NumPy の型の違いは第 6.1.3 節で説明する。基本構文の段階でも、1 は整数、1.0 は浮動小数点数であり、“12” は数値ではなく文字列であることを区別する。特に文字列の “12” と整数の 12 の混同は、ファイルの読み込みや数値計算でエラーの原因になる。

たとえば、見た目が似ていても次の 3 つは別物である。

```
>>> count = 12
>>> threshold = 12.0
>>> text = "12"
>>>
>>> type(count), count
(<class 'int'>, 12)
>>> type(threshold), threshold
(<class 'float'>, 12.0)
>>> type(text), text
(<class 'str'>, '12')
```

12 は整数なので件数や添字に向き、12.0 は浮動小数点数なので平均値やフィット係数のような連続量に向く。一方、“12” は文字列であり、そのままでは数値計算に使えない。ファイルから読んだ値は最初は文字列になっていることが多いので、「見た目が数字でも、まだ文字列かもしれない」と考える癖を付けておくといよい。

また、= は「左辺と右辺が等しい」という意味ではなく、「右辺の値を左辺の変数名へ結び付ける」という代入の記号である。条件の比較で使う == とは意味が異なるので、ここで区別に慣れておきたい。真偽値は条件分岐と組み合わせて使うことが多く、たとえば次のように読める。

```
>>> use_log = True
>>> if use_log:
...     print("log scale")
...
log scale
```

4.1.2 数値と演算子

Python は、対話的な計算機として使い始めるのが理解しやすい。ターミナルで `python3` と打つと、Python を対話モードで使用できる。四則演算、べき乗、剰余、整数除算など、基本的な計算の書き方は次の通りである。

```
$ python3
>>> 2 + 3
5
>>> 2 * 3
6
>>> 7 / 3
2.3333333333333335
>>> 7 // 3
2
>>> 7 % 3
1
>>> 2 ** 10
1024
```

`/` は浮動小数点数として割り算を行い、`//` は整数部分を返す。観測時間を秒へ直す、ビン番号を求める、インデックスを分ける、といった処理では、この違いが重要になる。特に `//` と `%` は、周期的な区切りやグループ分けでよく現れる。演算の優先順位は一般的な数式とほぼ同じであり、掛け算や割り算が足し算や引き算より先に評価される。読み手に迷いを残したくないときは、括弧を使って意図を明示した方が安全である。

```
>>> 2 + 3 * 4
14
>>> (2 + 3) * 4
20
>>> 7 / 3
2.3333333333333335
>>> 7 // 3
2
>>> int(7 / 3)
2
```

この例では、`7 // 3` と `int(7 / 3)` はどちらも 2 になるが、考え方は同じではない。`//` は整数除算そのものを表し、`int(7 / 3)` はまず浮動小数点数の割り算を行った後で整数へ変換している。途中で何が起きているかを区別して読めるようになると、後で NumPy や pandas の演算を読むときにも混乱しにくい。

浮動小数点数の計算では、表示された値が見た目通りに内部で表現されているとは限らない。これは Python 特有の問題ではなく、一般的な計算機の表現上の性質である。後半でフィットや統計量を扱うときにも、この前提を意識しておきたい。

4.1.3 文字列

ファイル名、日付時刻、ヘッダ、ラベルなど、解析コードでは文字列を扱う場面が多い。Python では、`"` や `'` で挟まれたものを文字列として認識する。文字列は単なる表示用のデータではなく、分割、検索、整形の対象でもある。短い操作は、対話モードで 1 つずつ確かめると理解しやすい。

```
>>> name = "detector_A"
>>> date = "2026-04-04"
>>> raw_line = "  detector_A,42,3.5  \n"
>>> name.upper()
'DETECTOR_A'
>>> date.replace("-", "/")
'2026/04/04'
>>> name.split("_")
['detector', 'A']
>>> raw_line.strip()
'detector_A,42,3.5'
>>> f"{name}_{date}.txt"
'detector_A_2026-04-04.txt'
```

文字列では、`split()`、`replace()`、`strip()` のような操作が、ファイル読み込みの前処理で頻繁に現れる。特に `split()` は、空白区切りやカンマ区切りの 1 行を各列へ分ける最初の道具である。`strip()` は行末の改行や前後の空白を落とすときに便利で、テキストファイルを読むとすぐに必要になる。

また、解析コードではファイル名やラベルを組み立てることが多いので、`f` 文字列も早い段階で使えるようにしておくといよい。上の例のように書くと、変数の値を埋め込んだ文字列を簡潔に作れる。ログ出力、図の保存名、結果ファイル名などで頻繁に使うので、単なる飾りではなく実務的な道具として覚えておきたい。

文字列は単なる表示用のデータではなく、検索や分割や整形の対象である。NumPy や pandas を使う前に、この感覚を持っておくとデータ形式の理解が速い。

4.2 データ構造

4.2.1 リスト、タプル、辞書

複数の値をまとめたいときには、いくつかの基本的な入れ物がある。よく使う 3 種類の定義例を下に示す。

```
>>> labels = ["A", "B", "C", "D"]
>>> point = (12, 18)
>>> row = {"id": 42, "value": 3.5, "flag": True}
```

リストは順序付きで変更可能、タプルは順序付きで変更不可、辞書は名前付きの値を持つ。実際に要素を取り出すときは、リストやタプルでは位置を、辞書ではキーを使う。対話モードで試すと次のようになる。

```
>>> labels[0]
'A'
>>> point[1]
18
>>> row["id"]
42
>>> row["value"]
3.5
```

この例では、`labels[0]` は最初のラベル、`point[1]` は 2 番目の座標成分、`row["id"]` は `"id"` というキーに対応する値を表している。最初のうちは辞書が読みやすいが、大きなデータを大量に処理するときはオーバーヘッドが大きくなることもある。この違いは後の高速化章で再び出てくる。

どの入れ物を選ぶかは、見た目の好みではなく、何をしたいかで決まる。位置でアクセスするならリストやタプル、名前でアクセスするなら辞書が自然である。小さなコードでは差が見えにくいのが、処理量が増えると選択の重みが増してくる。

変更できるかどうかの違いも、短い例で見ておくと分かりやすい。リストはあとから要素を書き換えられる。

```
>>> labels[1] = "B_detector"
>>> labels
['A', 'B_detector', 'C', 'D']
```

一方、タプルは作った後で要素を書き換えられない。たとえば次のようにするとエラーになる。

```
>>> point[0] = 15
TypeError: 'tuple' object does not support item assignment
```

座標のように「ひとかたまりの値として持ち、途中で書き換えない」ものにはタプルが向いている。逆に、ラベル一覧や処理対象のファイル一覧のように、あとから追加・削除・更新したいものにはリストが自然である。辞書は、列名や設定名のように「名前で値を引きたい」場面で特に読みやすい。

4.2.2 リストのメソッド

リストにはいくつかの基本メソッドがある。これらを知っていると、簡単な前処理や集約を自然に書ける。主な操作は次のようになる。

```
>>> values = [3, 1, 4]
>>> values
[3, 1, 4]
```

```
>>> values.append(1)
>>> values
[3, 1, 4, 1]
>>> values.sort()
>>> values
[1, 1, 3, 4]
>>> last = values.pop()
>>> last
4
>>> values
[1, 1, 3]
```

`append()` は末尾へ追加し、`sort()` はその場で並べ替え、`pop()` は末尾要素を取り出して削除する。

リストメソッドの多くは「その場で変更する」点に注意が必要である。新しいリストを返すのではなく、元のリスト自体を書き換える操作が多いので、どこで状態が変わるかを意識して読む必要がある。

特に `sort()` は、並べ替えたリストを返すのではなく、元のリストそのものを変更する。そのため、次のように書くと意図とずれることがある。

```
>>> values = [3, 1, 4]
>>> result = values.sort()
>>> result
None
>>> values
[1, 3, 4]
```

新しい並べ替え済みリストがほしいときは、`sorted()` を使う方が分かりやすい。

```
>>> values = [3, 1, 4]
>>> new_values = sorted(values)
>>> values
[3, 1, 4]
>>> new_values
[1, 3, 4]
```

4.2.3 インデックスとスライス

解析では、配列や文字列の一部だけを取り出したい場面が非常に多い。Python では、`[]` の中に位置を書くことで要素を取り出すことができる。このとき、`[]` の中に書く数字のことをインデックスとよぶ。また、`:` を使うことで範囲を切り出すこともできる。このような切り出し方はスライスとよばれる。

```
>>> values = [10, 20, 30, 40, 50]
>>>
>>> values[0]
10
```

```
>>> values[-1]
50
>>> values[1:4]
[20, 30, 40]
>>> values[:3]
[10, 20, 30]
>>> values[2:]
[30, 40, 50]
>>> values[:-1]
[10, 20, 30, 40]
```

リストのインデックスは 0 から始まる。インデックスは負の値を与えることも可能であり、-1 は一番最後の要素を表す。この例から、`values[1:4]` は 1 番目から 3 番目までを取り出し、`values[:3]` は `values[3]` よりも手前を、`values[2:]` は `values[2]` 以降を取り出すと読める。リストのスライスでは、`:` の左側のインデックスに対応する要素は含まれるが、右側のインデックスに対応する要素は含まれないということが重要である。

文字列に対しても同じ考え方が使える。例えば次のようになる。

```
>>> hhmmss = "123045"
>>> hhmmss[0:2]
'12'
>>> hhmmss[2:4]
'30'
>>> hhmmss[4:6]
'45'
```

このように、`hhmmss[0:2]` で時、`hhmmss[2:4]` で分、`hhmmss[4:6]` で秒を取り出せる。この一貫性が Python の扱いやすさの一つである。

4.3 制御構文

4.3.1 条件分岐

条件によって処理を変えるには `if` を使う。最も基本的な形は次の通りである。

```
>>> value = 2.5
>>> threshold = 3.0
>>> if value <= threshold:
...     print("small")
... else:
...     print("large")
...
small
```

比較に使う主な演算子には、`==`、`!=`、`<`、`<=`、`>`、`>=` がある。たとえば `value <= threshold`

は「value が threshold 以下なら真」を意味する。条件式そのものの値は真偽値であり、True か False のどちらかになる。

```
>>> value = 5
>>> threshold = 3
>>> value > threshold
True
>>> value == threshold
False
```

条件が3通り以上に分かれるときは、elif を使うと読みやすい。たとえば値の大小を3段階に分けるなら次のように書ける。

```
>>> value = 3.0
>>> threshold = 3.0
>>> if value < threshold:
...     print("small")
... elif value == threshold:
...     print("equal")
... else:
...     print("large")
...
equal
```

解析コードでは、閾値判定、ファイル名の選別、欠損値の除外など、条件分岐が頻繁に現れる。重要なのは、「なぜその条件で分けているのか」を自分で説明できることだ。

条件分岐が増えて読みにくくなったときは、条件式そのものを変数へ切り出すと分かりやすくなる。たとえば `is_valid_row = value <= threshold` のように一度名前を付けるだけでも、後で読むときの負担がかなり減る。

条件は左から右へ読んで意味が分かる形に書く方がよい。複数の条件を同時に満たすかどうかを調べたいときは `and`、どちらか一方を満たせばよいときは `or`、真偽を反転したいときは `not` を使う。

```
>>> value = 5
>>> lower = 3
>>> upper = 10
>>> use_log = False
>>> lower <= value and value <= upper
True
>>> value < lower or value > upper
False
>>> not use_log
True
```

否定が重なる条件や、複数の `and` と `or` が混ざる条件は、括弧や一時変数で整理した方が安全である。解析コードでは、条件の誤読がそのままバグになる。

4.3.2 反復処理

複数の値を順に調べるには for を使い、次のように取り出す。

```
>>> for label in labels:
...     print(label)
...
A
B_detector
C
D
```

要素の番号（添字）も必要なら enumerate を組み合わせると便利である。

```
>>> for i, value in enumerate(labels):
...     print(i, value)
...
0 A
1 B_detector
2 C
3 D
```

一方で、データ点が非常に多いとき、Python レベルの for ループが速度の瓶頸になることがある。本書では後半で、その回避方法を詳しく扱う。

初学者の段階では、まず正しく読める for 文を書くことが先である。速く書くことを意識するのは、その後でよい。読みやすい基本形を知っているからこそ、後で NumPy や pandas へ置き換えたときに、何を省略しているかが分かる。

4.3.2.1 range と while

for 文と並んで、反復処理でよく使うのが range と while である。range は連続した整数を順に生成し、while は条件が成り立つ間繰り返す。次のように記述する。

```
>>> for i in range(5):
...     print(i)
...
0
1
2
3
4
>>> count = 0
>>> while count < 3:
...     print(count)
...     count += 1
...
0
1
```

2

`range(5)` は 0 から 4 までを返す。ファイル番号やビン番号のような添字処理では便利である。一方、`while` は終了条件を自分で管理する必要があるので、無限ループにしないよう注意が必要である。時刻順に読み進めて、条件を満たす間だけ比較するような処理では `while` が自然な場合もある。

4.3.2.2 break, continue, else

反復処理では、途中で打ち切ったり、ある条件だけ飛ばしたりしたくなる。Python では `break` と `continue` がそのために使える。

```
>>> values = [-1, 1, 3, 5]
>>> threshold = 4
>>> for value in values:
...     if value < 0:
...         continue
...     if value > threshold:
...         break
...     print(value)
...
1
3
```

`continue` はその回だけを飛ばし、`break` はループ全体を終了する。探索範囲を打ち切る処理では特に重要である。さらに Python には `for ... else` という構文もあり、`break` せずに最後まで回り切ったときだけ `else` 節が実行される。頻出ではないが、見かけても驚かないようにしておくといよい。

4.3.2.3 内包表記

短い変換や簡単なフィルタであれば、内包表記を使うとすっきりと構築できる。

```
>>> values = [1, 2, 3, 4, 5]
>>> squares = [x * x for x in values]
>>> large_values = [x for x in values if x >= 3]
>>> squares
[1, 4, 9, 16, 25]
>>> large_values
[3, 4, 5]
```

内包表記は便利だが、何段階もの条件や複雑な変換を 1 行に押し込むと読みにくくなる。初学者のうちは、読みやすさを失わない範囲で使う方がよい。長くなりそうなら、普通の `for` 文や補助関数へ戻した方が親切である。

4.4 関数とモジュール

4.4.1 関数を分ける理由

同じ処理を何度も書かないためだけでなく、処理の意味を分けるためにも関数は重要である。一例として、時刻を表す文字列から秒数を計算する関数を考える。

```
1 def hhmss_to_seconds(hhmss):
2     hour = int(hhmss[0:2])
3     minute = int(hhmss[2:4])
4     second = int(hhmss[4:6])
5     return hour * 3600 + minute * 60 + second
```

この関数は短い、「文字列で与えられた時刻を秒へ変換する」という役割が明確である。ここから先は、複数行の処理をまとまりとして読むため、短いスクリプトをファイルへ保存して実行する形も使う。たとえば次の内容を `time_conversion.py` という名前で保存すれば、ターミナルで `python3 time_conversion.py` と実行できる。

time_conversion.py

```
1 def hhmss_to_seconds(hhmss):
2     hour = int(hhmss[0:2])
3     minute = int(hhmss[2:4])
4     second = int(hhmss[4:6])
5     return hour * 3600 + minute * 60 + second
6
7 print(hhmss_to_seconds("013015"))
```

```
$ python3 time_conversion.py
4515
```

このように、関数は入力を受け取り、決まった規則で出力を返す小さな部品として読むと分かりやすい。解析コードでは、このような小さな関数が積み重なって全体の読みやすさを作る。

関数を切り出す利点は再利用だけではない。入力が何で、出力が何かをはっきりさせることで、テストしやすくなる。短い補助関数ほど後回しにされがちだが、解析の正しさはこうした小さな部品の積み重ねで決まることが多い。

4.4.1.1 関数の引数

関数を書くときは、引数の意味をはっきりさせることが重要である。Python では位置引数だけでなく、デフォルト値付き引数やキーワード引数も使える。

```
1 def histogram_path(name, suffix=".pdf"):
```

```

2     return f"{name}{suffix}"
3
4 print(histogram_path("value_hist"))
5 print(histogram_path("value_hist", suffix=".png"))

```

```

value_hist.pdf
value_hist.png

```

デフォルト引数を使うと、よく使う設定を省略できる一方で、必要なら上書きできる。コマンドライン引数と同じで、解析コードの柔軟性は「固定値をどこに置くか」で決まることが多い。関数でも同じ発想が使える。

ただし、可変オブジェクトをデフォルト引数に置くと予想外の共有が起きる。この罟は Python 公式チュートリアル [1] でも強調されている典型例である。初学者の段階では、リストや辞書をデフォルトにしたくなったら一度立ち止まった方がよい。以下の書き方は避けるべきである。

```

1 def bad_append(value, buffer=[]):
2     buffer.append(value)
3     return buffer

```

この例では、`buffer` が呼び出しのたびに新しく作られるわけではない。そのため、前回の状態が次回にも残る。実際に呼ぶと次のようになる。

```

1 print(bad_append(1))
2 print(bad_append(2))

```

```

[1]
[1, 2]

```

安全に書くなら `None` を使って関数内部で初期化の方がよい。

```

1 def good_append(value, buffer=None):
2     if buffer is None:
3         buffer = []
4     buffer.append(value)
5     return buffer

```

```

1 print(good_append(1))
2 print(good_append(2))

```

```

[1]
[2]

```

初学者がつまづきやすいので、早めに知っておく価値が高い。

4.4.1.2 モジュールと `import`

別ファイルに書かれた関数を使うには次のように `import` する。たとえば標準ライブラリの `math` を使うと、円周率や平方根を次のように扱える。

```

1 import math
2 import glob
3 import argparse
4
5 print(math.pi)
6 print(math.sqrt(2.0))

```

```

3.141592653589793
1.4142135623730951

```

Python の解析では、標準ライブラリと外部ライブラリを組み合わせる。どの名前がどこから来たものかを意識して読む習慣を付けておくと、大きなコードも追いやすくなる。

特に、同じ名前に見えても標準ライブラリと外部ライブラリで役割が大きく異なることがある。インポート文は単なるおまじないではなく、そのファイルが何に依存しているかを示す地図でもある。ここまでは既存のライブラリを読み込む話だったが、同じ仕組みで自分が書いたファイルも読み込める。

4.4.1.3 自作モジュールへ分ける

解析が少し大きくなると、1 つのファイルへ全部を書くのはすぐに苦しくなる。よく使う関数や定数は別ファイルへ分け、モジュールとして読み込む方が見通しが良い。たとえば次のようなファイル構造で分けるとする。

```

analysis_project/
|-- constants.py
|-- physics.py
`-- main.py

```

この構成にすると、physics.py に書いた関数を main.py から読み込める。次の例は、その最小形である。

```

1 # physics.py
2 import numpy as np
3
4 def deg_to_rad(degrees):
5     return degrees * np.pi / 180.0
6
7
8 # main.py
9 import physics
10
11 theta = physics.deg_to_rad(45.0)

```

この形にしておくと、座標変換、時刻変換、定数定義のような再利用しやすい処理を 1 か所へまとめられる。たとえば main.py 側で print(theta) と書けば、結果は次のようになる。

```

0.7853981633974483

```

ファイルを分けると、「他のファイルから読み込んで使う」場面が自然に増える。逆に、そのファイル自身を直接実行して動作確認したいこともある。次に示す `if __name__ == "__main__":` は、その2つを切り分けるための書き方である。

4.4.1.4 スクリプトとして実行する

Python ファイルは、他のファイルから読み込まれることもあれば、直接実行されることもある。その違いを区別する基本形が次である。

```
1 def main():
2     print("run")
3
4 if __name__ == "__main__":
5     main()
```

この形にしておくと、モジュールとして読み込んだときには関数だけを使い、直接実行したときだけ本体が動く。後半の解析スクリプトでも、この形を基本として採用すると整理しやすい。

4.5 保守と実践的な書き方

4.5.1 エラーは失敗ではなく情報である

初心者のうちは、エラーメッセージが出るとそこで止まってしまいがちである。しかし、トレースバックは「どこで何が起きたか」を教えてくれる重要な情報である。

たとえば `FileNotFoundError` ならファイルパスが違うかもしれないし、`ValueError` なら想定した形式でデータを読めていないかもしれない。実際のエラー表示は次のようになる。

```
1 with open("missing.txt") as f:
2     text = f.read()
```

```
Traceback (most recent call last):
  File "read_missing_file.py", line 1, in <module>
    with open("missing.txt") as f:
FileNotFoundError: [Errno 2] No such file or directory: 'missing.txt'
```

大切なのは、英語の文章全体を恐れるのではなく、例外名と最後の数行を見ることである。解析では、最初から一度もエラーを出さずに進む方が珍しい。問題はエラーが出るのではなく、エラーを無視したり、意味を確かめずに場当たりの直したりすることである。例外名、発生行、直前に読んでいたデータの形を確認する習慣を付けておくと、修正の質が安定する。

4.5.1.1 docstring とヘルプ

関数やモジュールの役割を後から思い出せるようにするには、短い説明を書いておくとうい。Python では次のように、関数定義の直後に文字列を書くと docstring（ドキュメント文字列）になる。

```
1 def read_table(path):
2     """空白区切りの表を読み、各行を辞書として返す。"""
3     ...
```

`help(read_table)` を実行すると、この説明を後から確認できる。たとえば要約だけの docstring なら、次のように表示される。

```
1 help(read_table)
```

```
Help on function read_table:
```

```
read_table(path)
    空白区切りの表を読み、各行を辞書として返す。
```

コメントと同じように見えるが、実行時にも参照できる点が異なる。解析コードが増えると、どの関数は何を返すのかを書いておくことの価値が大きくなる。

短い関数なら 1 行の docstring で十分だが、少し長い処理では複数行にして、少なくとも「何をするか」「主要な引数」「返り値」「起こり得る例外」を書いておくとうみやすい。たとえば次のような形である。

```
1 def read_table(path):
2     """空白区切りの表を読み込む。
3
4     Args:
5         path: 入力ファイルへのパス。
6
7     Returns:
8         各行を辞書として格納したリスト。
9
10    Raises:
11        FileNotFoundError: ファイルが存在しない場合。
12    """
13    ...
```

流儀はいくつかあるが、最初の 1 行を「要約」、その後ろを詳細と考えると整理しやすい。特に、他人が `help()` やエディタ上の補完で関数を読むときは、最初の 1 行がそのまま関数の説明になることが多い。

4.5.1.2 名前付けと PEP 8

Python には PEP 8 という代表的なスタイルガイドがある。すべてを最初から覚える必要はないが、少なくとも変数名、関数名、空白の入れ方を次のようにそろえるだけでコードは

かなり読みやすくなる。

```
1 MAX_EVENTS = 1000
2
3 def read_event_table(path):
4     total_count = 0
5     ...
```

一般には、変数名や関数名は `snake_case`、定数は大文字の `UPPER_CASE` にすることが多い。さらに、演算子の前後に空白を入れる、1 行を詰め込みすぎない、といった基本だけでも効果は大きい。スタイルの統一は飾りではなく、バグを見つけやすくするための道具だと考える方が実践的である。

PEP 8 を最初から全部暗記する必要はないが、少なくとも次の規則は守る価値が高い。

- インデントはタブではなく 4 スペースにする
- 関数名や変数名は `snake_case`、クラス名は `CamelCase`、定数は `UPPER_CASE` にする
- 演算子 (=, +, - など) やカンマの後ろには空白を入れる
- トップレベルの関数定義やクラス定義の間には空行を入れて、かたまりを分ける
- 1 行を長くしすぎない。PEP 8 では 79 文字が目安だが、少なくとも横に流れすぎて読みにくい行は分割する
- `import` は標準ライブラリ、外部ライブラリ、自作モジュールの順にまとめる

この程度でも、見た目のばらつきはかなり減る。重要なのは「絶対に厳密であること」より、「同じプロジェクトの中で揃っていること」である。

4.5.1.3 マジックナンバーを避ける

コード中に 250 や 3600 のような数値が突然現れると、その値が何を表すのかを読む側が推測しなければならない。このような、説明なしに裸で現れる数値をマジックナンバーと呼ぶ。これを避けるには、次のように意味のある名前を持たせる。

```
1 NS_PER_SECOND = 1_000_000_000
2 COINCIDENCE_WINDOW_NS = 250
3
4 dt_ns = second_diff * NS_PER_SECOND + ns_diff
5 is_coincident = abs(dt_ns) <= COINCIDENCE_WINDOW_NS
```

250 をそのまま書くより、`COINCIDENCE_WINDOW_NS` と書いた方が意味が明確である。3600 なら `SECONDS_PER_HOUR`、86400 なら `SECONDS_PER_DAY` といった具合に、意味を名前へ出した方が読みやすい。

ただし、どんな値でもコード内定数にすればよいわけではない。物理定数や単位換算のように定義として固定したい値はコード内の定数に向いている。一方で、入力パス、出力先、閾値、利用する検出器番号のように実行条件として変わり得る値は、コマンドライン引数や設定ファイルへ出した方がよい。値の置き場所そのものが設計である、という意識を持つこ

とが重要である。

4.5.1.4 例外処理

エラーは読むべき情報だが、予想される失敗を自分で受け止めたい場面もある。例えば、1行だけ形式が壊れているデータを見つけて記録し、解析全体は続けたいことがある。そのときに使うのが次のような `try` と `except` の組み合わせである。

```
1 try:
2     value = float(text)
3 except ValueError:
4     print("数値へ変換できませんでした")
```

たとえば `text = "abc"` のような文字列に対して `float(text)` を呼ぶと、`ValueError` が起きる。したがって、何でも `except:` で受けてしまうと、本来止まるべきバグまで隠してしまう。どの例外を受けるのかを明示し、何を記録して、何を諦めて、何を継続するのかを自分で決めることが重要である。

4.6 章末問題

この章の課題は、次の小さなイベント列だけで実行できる。前章までのファイルは必要ない。

```
1 events = [
2     {"detector": "A", "time_ns": 1000, "signal": 12.4},
3     {"detector": "B", "time_ns": 1180, "signal": 9.8},
4     {"detector": "A", "time_ns": 1300, "signal": 15.1},
5     {"detector": "B", "time_ns": 1510, "signal": None},
6 ]
```

1. **入力を確認して条件に合う行を表示する。** `for` 文で各辞書を取り出し、`signal` が `None` でない行だけを `A 1000 12.4` の形式で表示せよ。`is not None` を使う理由と、辞書の角括弧に書いたキーの役割を説明せよ。表示される行は3行である。
2. **関数へ分離して集計する。** イベント列と検出器名を受け取り、該当する検出器の有効な `signal` の平均を返す `mean_signal(events, detector)` を作れ。A では 13.75、B では 9.8 が返ることを `print()` で確認せよ。該当する有効値がない場合は `ValueError` を送出し、docstring に引数、返り値、例外条件を書くこと。
3. **時刻差を調べる。** `time_ns` だけをリスト内包表記で取り出し、昇順に並べた後、隣り合う要素の差を求めよ。結果は `[180, 120, 210]` になる。`range(len(times) - 1)` の開始値、終了値、最後の添字がどう対応するかを説明せよ。
4. **コマンドラインから条件を受け取る。** 検出器名を位置引数、信号の下限を `--min-signal` で受け取るスクリプトを `argparse` で作れ。既定値は `0.0` とする。`python`

`filter_events.py A --min-signal 13` を実行したとき、時刻 1300 の 1 件だけが表示されることを確認し、位置引数とオプション引数の違いを説明せよ。

この章で扱った値、コンテナ、条件分岐、反復、関数は、ファイルから読み取ったデータを処理するための部品になる。次章では、これらの部品を入出力と結び付け、テキスト、CSV、JSON、バイナリ形式を Python のオブジェクトへ変換する。

第 5 章

入出力とファイル

この章では、Python でファイルを開き、複数のファイルを集め、1 行ずつ読みながら値へ変換する基本を扱う。ここで重要なのは、特定のデータ形式に慣れることではなく、入力を見て必要な情報を取り出す流れを理解することである。入出力の設計は、解析そのものの再現性にも直結する。

5.1 テキストファイルの入出力と探索

5.1.1 ファイルを開き、読み書きの基本を押さえる

Python でテキストファイルを読むときの基本は `open` である。さらに `with` を組み合わせると、処理が正常に終わった場合だけでなく、途中で例外が発生した場合にもファイルが閉じられる。例として、`sample.txt` に次の 3 行が入っているとする。

sample.txt

```
run 1: 12.4
run 2: 10.1
run 3: 14.8
```

このファイルを 1 行ずつ表示するコードは次のようになる。

```
1 path = "sample.txt"
2
3 with open(path, "r", encoding="utf-8") as f:
4     for line in f:
5         print(line.rstrip())
```

実行結果は次の通りである。

```
run 1: 12.4
run 2: 10.1
```

```
run 3: 14.8
```

第1引数の `path` は開くファイルの場所、`"r"` は読み込みモード、`encoding="utf-8"` は文字を UTF-8 として解釈する指定である。`as f` によって、開いたファイルを表すオブジェクトを変数 `f` で参照する。`for line in f` は全行を一度にメモリへ置かず、1行ずつ取り出す。

`read` でまとめて読む方法もあるが、大きなファイルでは1行ずつ読む方が安全である。まずはこの形に慣れておくとよい。

このとき、`line` には改行文字が含まれている点にも注意したい。表示だけなら `rstrip` で末尾を落とせば十分だが、区切り文字を扱うときには改行の有無が結果に影響することがある。テキスト処理では、見えていない文字を意識することが重要である。

5.1.1.1 pathlib でパスを扱う

ファイルパスを単なる文字列として扱うこともできるが、Python では次のように `pathlib.Path` を使うと見通しが良くなる。

```
1 from pathlib import Path
2
3 root = Path(".")
4 path = root / "sample.txt"
5
6 print(path.exists())
7 print(path.name)
```

この例でファイルが存在する場合、出力は次のようになる。

```
True
sample.txt
```

`exists()` はパスが存在するかを真偽値で返し、`name` は末尾のファイル名だけを返す。`root / "sample.txt"` の `/` は除算ではなく、`Path` オブジェクトに対してパス要素を結合する演算子である。例えば `Path("data") / "2026" / "04" / "sample.txt"` とすれば、複数階層のパスも同じ方法で組み立てられる。

`/` 演算子でパスをつなげられるため、文字列連結より読みやすい。ファイル名だけ取りたい、親ディレクトリへ戻りたい、拡張子を変えたい、といった操作も自然に書ける。解析コードでは、パス操作を文字列で済ませるより安全なことが多い。`pathlib` を用いて新しく出力先を作る処理は次のようになる。

```
1 outdir = Path("output")
2 outdir.mkdir(parents=True, exist_ok=True)
3 outfile = outdir / "summary.txt"
```

`parents=True` は `output` より上位の未作成ディレクトリも必要に応じて作る指定であり、`exist_ok=True` は既存のディレクトリをエラーにしない指定である。ただし、同名の通常ファイルが存在する場合はエラーになる。

ディレクトリがなければ作る、すでにあってもエラーにしない、という処理は解析コードで非常によく使う。こうした定型操作を安全に書ける点でも `pathlib` は便利である。

5.1.1.2 ファイルオブジェクトのメソッド

ファイルを開いた後には、`read`、`readline`、`write` などのメソッドが使える。Python 公式チュートリアル [1] でも強調されているように、何をどの粒度で読むかでコードの性格が変わる。たとえば次のように記述する。

```
1 with open(path, "r", encoding="utf-8") as f:
2     first_line = f.readline()
3
4 with open("out.txt", "w", encoding="utf-8") as f:
5     f.write("hello\n")
```

`readline()` の戻り値には末尾の改行が含まれる。`write()` は書き込んだ文字数を返すが、改行を自動では補わないため、行末が必要なら `"\n"` を明示する。この例を実行すると、`out.txt` の内容は `hello` と改行になる。

`readline` は 1 行だけ確認したいとき、`write` は整形済み文字列をそのまま出したいときに便利である。大きな入力全体を一気に読む前に、最初の数行だけ確認するという使い方もよく行う。

5.1.1.3 複数のファイルを集める

ログや測定結果が複数ファイルに分かれている場合は、まず対象ファイルの一覧を作る必要がある。Python では `glob.glob` がよく使われる。基本的な探索方法は次の通りである。

```
1 import glob
2
3 paths = sorted(glob.glob("data/**/*.txt", recursive=True))
4 print(f"count = {len(paths)}")
5 for path in paths[:3]:
6     print(path)
```

例えば対象が 3 ファイルなら、出力は次のようになる。

```
count = 3
data/2025/run01.txt
data/2026/run01.txt
data/2026/run02.txt
```

パターン中の `*` は同じ階層の任意の文字列、`**` は `recursive=True` と組み合わせたときに複数階層へ一致する。`paths[:3]` は確認のために先頭 3 件だけを表示するスライスであり、処理対象を 3 件へ制限しているわけではない。

`glob.glob` はシェルのワイルドカードに近い感覚で使える。複数ファイルを扱う処理では、最初に見つかった件数や先頭数件のパスを表示して確認する習慣を付けておくとよい。

また、順序が重要な処理では `sorted` を付ける意味も大きい。ファイル探索の結果が環境依存の順序で返ると、処理結果やログの見え方が毎回変わり、デバッグしにくくなる。時系列や連番を扱う場面では、最初に順序を固定する方が安全である。

5.2 文字列データの解析と整形

5.2.1 CSV を列名で安全に読む

CSV は一見単純に見えるが、引用符やカンマを含む列があると `line.split(",")` では壊れやすい。列名付きの表を扱うなら、標準ライブラリの `csv` を使う方が安全である。

```
1 import csv
2 from pathlib import Path
3
4 path = Path("events.csv")
5
6 with path.open("r", encoding="utf-8", newline="") as f:
7     reader = csv.DictReader(f)
8     first_row = next(reader)
9
10 print(first_row["time"])
11 print(first_row["mag"])
```

ここで `events.csv` の先頭が次の内容だとする。

events.csv		
time,mag,place		
2026-04-04T12:00:00Z	3.2	"Offshore, Japan"
2026-04-04T12:05:00Z	2.8	Tokyo

先頭データ行の `time` と `mag` が文字列として読み込まれるため、実行結果は次の通りである。

2026-04-04T12:00:00Z
3.2

`DictReader` は 1 行目を列名として読み、その後の各行を辞書として返す。したがって `row["mag"]` のように列名で値を取り出せる。`newline=""` を付けるのは、CSV モジュールが改行を正しく扱いやすくするためである。

列の中にカンマを含む場合でも、CSV の規則に従って引用されていれば `csv` モジュール側で処理してくれる。つまり、自由記述の文字列や名前の列を含む CSV では、`split(",")` よりこの方法の方が安全である。

5.2.2 文字列を値へ変換し、結果を書き出す

多くの表形式データは、1 行の中に複数の列が並ぶ形で保存されている。テキストとして読んだだけでは、まだ単なる文字列である。そこから列に分け、整数や浮動小数点数へ変換して初めて解析に使える。たとえば空白区切りの 1 行を分解する例は次のようになる。

```
1 line = "17 2026-04-04 12:34:56 15.2 OK"
2 columns = line.split()
3
4 record_id = int(columns[0])
5 date_text = columns[1]
6 time_text = columns[2]
7 value = float(columns[3])
8 status = columns[4]
9
10 print(record_id, date_text, time_text, value, status)
```

`split()` は引数を省略すると連続する空白を一つの区切りとして扱う。`int()` と `float()` が文字列を数値へ変換するため、出力は次のようになる。

```
17 2026-04-04 12:34:56 15.2 OK
```

ここで重要なのは、どの列をどの型で持つかを自分で決めることである。後で `pandas` を使う場合でも、この感覚を持っていると列の意味を見失いにくい。

文字列をどの段階で数値へ変換するかも設計の一部である。すぐ数値化してよい列もあれば、元の文字列表現を残しておいた方が確認しやすい列もある。読みやすい解析コードでは、どこでどの変換を行ったかが追えるようになっている。

5.2.2.1 ファイルへ書き出す

解析では、読むだけでなく結果を書き出す場面も多い。最も単純には、テキストとして 1 行ずつ書いていけばよい。

```
1 lines = ["run_id count mean", "1 120 3.42", "2 118 3.37"]
2
3 with open("summary.txt", "w", encoding="utf-8") as f:
4     for line in lines:
5         f.write(line + "\n")
```

このコードの実行後、`summary.txt` には `lines` の 3 要素が 1 行ずつ保存される。`"w"` は、ファイルが存在すれば内容を空にしてから書き、存在しなければ新しく作るモードである。これに対して `"a"` は既存内容の末尾へ追記する。既存ファイルを上書きしてよいのか、別名で保存すべきかは、実験ノートやレポートの再現性にも関わる。出力ファイル名の設計も解析手順に含まれる。

5.2.2.2 文字列整形

結果を画面やファイルへ出すときには、数値を読みやすく整えることも大切である。Python では、以下のような f-string（フォーマット済み文字列リテラル）が最もよく使われる。

```
1 mean = 3.14159
2 sigma = 0.27182
3
4 text = f"mean = {mean:.3f}, sigma = {sigma:.3f}"
5 print(text)
```

実行結果は `mean = 3.142, sigma = 0.272` となる。f は固定小数点形式、.3 は小数点以下の桁数を指定する。これは表示上の丸めであり、変数 `mean` と `sigma` に保存された値そのものは変化しない。

`:.3f` は小数点以下 3 桁で表示するという意味である。解析結果をレポートへ貼る前に、このような整形で見やすくしておくと、図や本文との対応が取りやすい。

Python 公式チュートリアル [1] では、f-string のほかに `str.format` や手動整形も紹介されている。現在の実務では f-string が最も使いやすいが、古いコードを読むと別の書き方が出てくることもある。複数の流儀があることは知っておきたい。

5.3 オブジェクトと構造化データの保存

5.3.1 JSON と Pickle / PKL

結果や設定を構造化して保存したいときは、JSON が便利である。テキストとして読めて、Python では標準ライブラリ `json` ですぐ扱える。基本的な保存と読み込みの記述は次のように行う。

```
1 import json
2
3 config = {"threshold": 3.5, "bins": 40}
4
5 with open("config.json", "w", encoding="utf-8") as f:
6     json.dump(config, f, indent=2)
7
8 with open("config.json", "r", encoding="utf-8") as f:
9     loaded_config = json.load(f)
10
11 print(loaded_config)
```

`indent=2` は入れ子を 2 空白で字下げし、人間が読みやすい JSON を保存する指定である。読み戻した結果は次のように表示される。

```
{'threshold': 3.5, 'bins': 40}
```

JSON のオブジェクトは Python では辞書として復元される。

大きな表形式データの本体を JSON で持つのは効率が悪いことも多いが、設定や要約結果を保存するには便利である。人間が読める形式と、Python が簡単に読める形式を両立しやすい。

5.3.1.1 Pickle / PKL

Python オブジェクトをそのまま保存したいときは、`pickle` もよく使われる。拡張子は慣習的に `.pkl` を使うことが多い。辞書、リスト、NumPy 配列、学習済みモデル、中間集計結果などを素早く退避するには便利である。処理の記述例を下に示す。

```
1 import pickle
2
3 result = {"mean": 3.14, "sigma": 0.27, "values": [1.0, 2.0, 3.0]}
4
5 with open("result.pkl", "wb") as f:
6     pickle.dump(result, f)
7
8 with open("result.pkl", "rb") as f:
9     loaded_result = pickle.load(f)
10
11 print(loaded_result["mean"])
12 print(loaded_result["values"])
```

復元後の出力は次の通りである。

```
3.14
[1.0, 2.0, 3.0]
```

"wb" と "rb" の b はバイナリモードを意味する。JSON と違って人間には読めないが、Python では構造を保ったまま簡単に保存できる。ただし、`pickle` は Python 専用であり、他言語との受け渡しには向かない。また、信頼できない `.pkl` ファイルを `load` してはならない。共有用の正式な保存形式というより、自分の解析途中結果を手早く再利用するためのローカル形式と考える方が安全である。

ここでいう「危険」とは、単に壊れたファイルを読んでエラーになるという意味ではない。`pickle.load()` は、ファイルの内容に従って Python オブジェクトを復元する過程で任意のコード実行につながる可能性がある。したがって、メール添付や外部配布物として受け取った不明な `.pkl` を気軽に開いてはならない。自分で保存したもの、あるいは十分に信頼できる相手が生成したものだけを読む、という運用が必要である。

5.4 発展的なトピック

5.4.1 日付と時刻を数値として扱う

日付や時刻を含むデータでは、文字列のまま扱うより、比較や計算がしやすい形へ変換した方がよい。標準ライブラリの `datetime` は、そのための基本的な道具となる。実際のコードは次のようになる。

```
1 from datetime import datetime
2
3 text = "2026-04-04 12:34:56.123456"
4 dt = datetime.strptime(text, "%Y-%m-%d %H:%M:%S.%f")
5 unix_time = dt.timestamp()
6 print(dt)
7 print(unix_time)
```

`strptime()` の第 1 引数は変換する文字列、第 2 引数は文字列の形式である。`%Y`、`%m`、`%d` は年、月、日を表し、`%f` はマイクロ秒を表す。`timestamp()` は Unix epoch からの秒数を返すが、タイムゾーン情報を持たない `datetime` の解釈は実行環境のローカルタイムゾーンに依存する。UTC のログでは、タイムゾーンを明示するか、後述する `Astropy` の `Time` を使う必要がある。

秒単位の差を見たいのか、より細かい時間差を見たいのかによって、どの単位で値を持つかは変わる。ただし、どの表現を選んでも、一貫して使うことが大切である。

`datetime` は便利だが、何でも `datetime` オブジェクトのまま持てばよいわけではない。差分計算を大量に繰り返すなら、途中で秒やナノ秒の数値へ変換した方が扱いやすいことも多い。人間に読みやすい表現と計算しやすい表現を分けて考えることが重要である。

さらに、タイムゾーンが関わるデータでは、見かけの時刻だけを比較すると誤解が生じる。研究用のログや観測データでは UTC を使うことが多いが、どの基準時刻を採用しているかを最初に確認する習慣を付けたい。

5.4.2 保存形式について: ベストプラクティスへのステップ

データをファイルに保存・読み出しする際、多くの初学者は何も考えずに `.csv` (CSV 形式) を使い続ける。しかし、データが数百万行に達すると、処理時間の大半が「ファイルの準備」に消えるようになる。

5.4.2.1 Step 1: テキスト形式 (CSV, JSON) の限界

テキスト形式は文字コード（例：UTF-8）を使って人間がそのまま読める形で保存する。CSV/TSV は非常に汎用性が高く、C++ や R、Excel など、どの言語でも読めるのが強みである。

問題点: コンピュータはディスク上の文字列（例：「1.2345」という 6 文字の羅列）を、計算に使える実数値にいちいち変換（パース）しなければならない。データ量が数 GB に及ぶと、この「文字列から実数への変換処理」だけで何十分も待たされることになる。

5.4.2.2 Step 2: バイナリ形式への移行と、どれを選ぶかの判断基準

バイナリ形式は、数値を文字列から変換する処理を省ける場合があり、CSV より短時間で読み書きできることが多い。ただし、実際の時間はファイルサイズ、圧縮方式、記憶装置、列数、読み込む範囲、ライブラリの実装に左右される。バイナリ形式に変えただけで、常に記憶装置の転送速度だけが上限になるわけではない。

しかし、バイナリ形式にはいくつかデファクトスタンダードがあり、目的に合わせて選ばなければならない。

- **NPY / NPZ (NumPy 標準):** NumPy だけで完結する小さな解析なら最も手軽。ただし、Python・NumPy 以外（C++や R など）からは読みづらく使い捨てになりがちである。
- **Pickle / PKL:** Python オブジェクトをそのまま保存できるため、中間結果や学習済みモデルの一時保存に便利である。ただし、Python 以外ではほぼ使えず、信頼できないファイルを読み込んで서는ならない。共同研究の交換形式というより、ローカルな作業用キャッシュとして使うべき形式である。
- **HDF5:** 大規模な数値データを階層構造で管理できる形式である。多次元配列を部分的に読み書きし、データとメタデータを一つのファイルへまとめた場合に向く。C/C++、Fortran、R などからも利用できるが、受渡し相手が使うライブラリとデータ構造を事前に確認する必要がある。
- **Parquet:** 列指向のバイナリ形式である。pandas や Polars から表を保存し、必要な列だけを読む用途に向く。行単位の頻繁な更新より、列の選択、集計、複数ファイルの一括処理と相性がよい。

解析の初期段階では、中身を目視でき、他のソフトウェアでも読みやすい CSV が役立つ。大量のイベントデータを繰り返し読むことが実測上のボトルネックになった場合は、変換手順と列型を記録した上で Parquet や HDF5 を中間形式として導入するとよい。小規模なデータや一度しか読まないデータでは、変換時間と形式管理の手間が利点を上回ることもある。`.pkl` はその補助として、信頼できる環境内で途中結果や作業用キャッシュを保存する用途に向いている。

Parquet は「保存形式」の名前であり、pyarrow はその形式を pandas から読み書きする代表的な実装の一つである。したがって、「Parquet を使う」と「pyarrow を入れる」は同じ話ではない。多くの環境では read_parquet() や to_parquet() の背後で pyarrow が使われるため、これが入っていないと Parquet を扱えないことがある。

5.5 章末問題

第2章で USGS の CSV を取得していない場合は、次の内容を raw/usgs_global_2024_01.csv として保存すれば、同じ課題を小規模な入力で実行できる。

raw/usgs_global_2024_01.csv

```
time,latitude,longitude,mag,place
2024-01-01T00:00:00.000Z,35.0,140.0,3.2,"Offshore, Japan"
2024-01-01T00:05:00.000Z,36.0,141.0,,Japan
2024-01-01T00:12:30.000Z,34.5,139.5,2.8,Tokyo
```

この小規模入力では、ヘッダを除くデータ行数は 3、最初と最後の時刻はそれぞれ 2024-01-01T00:00:00.000Z と 2024-01-01T00:12:30.000Z、空欄を除くマグニチュードの最小値と最大値は 2.8 と 3.2 になる。自分の出力がこれらと一致することを確認してから、実データへ進むこと。

1. USGS の CSV を読み、要約を JSON へ保存する。第2章で取得した raw/usgs_global_2024_01.csv を csv.DictReader で読み、少なくとも次の情報を results/usgs_summary.json へ保存せよ。

- 全行数
- 最初のイベント時刻
- 最後のイベント時刻
- マグニチュードの最小値と最大値

出力には pathlib と json.dump を用いること。Path.mkdir(parents=True, exist_ok=True) で出力ディレクトリを作り、json.dump(..., indent=2) の各引数が何を指定するかも説明せよ。保存後に json.load() で読み戻し、辞書の内容を表示して検証すること。

2. 後で NumPy や Matplotlib へ渡しやすい中間データを Pickle で保存する。CSV から time、latitude、longitude、mag だけを抜き出し、辞書のリストとして results/usgs_preview.pkl へ保存せよ。まずは先頭 200 行だけでよい。読み直して要素数と先頭 1 件を表示し、保存と復元ができていることを確かめよ。小規模入力を使った場合、要素数は 3 であり、先頭辞書の time は 2024-01-01T00:00:00.000Z になる。"wb" と "rb" の b の意味を説明し、この PKL を信頼できない相手から受け取った場合に読み込んではならない理由も述べよ。
3. 発展課題。CSV の中には空欄や想定外の値が入り得る。空欄の mag をスキップする処理を自分で足し、何件スキップしたかを results/skipped_rows.txt へ書き出せ。

`DictReader` では空欄セルは空文字列として読まれるので、たとえば `if row["mag"] == ""` のような判定を入れればよい。小規模入力では 1 件をスキップする。`float(row["mag"])` が失敗する場合も `ValueError` として数えられるようにし、空欄と不正な文字列を区別して記録せよ。

ファイルの構造を確認し、必要な値を Python のオブジェクトへ変換できれば、次は多数の数値を同じ型の配列として計算できる。次章では、ここで読み込んだ時刻や信号値を NumPy 配列へ移し、差分、条件抽出、集計、ヒストグラムを扱う。

第 6 章

NumPy による数値計算

NumPy は、Python で数値配列を扱うときの基本となるライブラリである。本章では、まず配列計算の考え方と ndarray の基本を確認し、その後で配列生成、演算、形の変更、集計、ブロードキャストを順に扱う。後半では、初学者がつまづきやすい点を整理し、最後に解析でどのように使うかを具体例で見る。

6.1 NumPy の考え方

6.1.1 配列計算の基本概念

Python のリストは柔軟だが、数値配列として大量の値を処理するには向いていないことがある。NumPy は、同じ型の数値を多次元配列 ndarray として持ち、配列全体へまとめて演算をかけるためのライブラリである。最も基本的な NumPy 配列の定義と差分計算の例を示す。

```
1 import numpy as np
2
3 times = np.array([10, 15, 23, 41], dtype=np.int64)
4 deltas = times[1:] - times[:-1]
```

このような差分計算は、Python の for ループでも書けるが、NumPy では配列全体に対してまとめて書ける。後の図 13.1 に示すように、この差は速度にも大きく効く。

数式で書けば、これは隣り合う時刻の差

$$\Delta t_i = t_{i+1} - t_i$$

をまとめて計算していることに対応する。NumPy を使うと、このような「添字付きの式」を配列全体に対してそのまま書きやすい。

ここで重要なのは、NumPy が単に「速いリスト」ではないという点である。NumPy 配列は、同じ型の数値をまとめて持ち、一つの式で大量の要素を処理するための道具である。し

たがって、設計の発想も「1 要素ずつ何をするか」から「配列全体にどの変換をかけるか」へ変わる。

6.1.2 ndarray と属性

NumPy の中心は ndarray である。配列を作ったら、まず shape、ndim、dtype を確認する癖を付けると混乱しにくい。

```
1 import numpy as np
2
3 a = np.array([[1.0, 2.0, 3.0],
4               [4.0, 5.0, 6.0]])
5
6 print(a.shape)      # (2, 3)
7 print(a.ndim)       # 2
8 print(a.dtype)      # float64
9 print(a.size)       # 6
10 print(a.itemsize)  # 8
```

shape は各軸の長さ、ndim は次元数、dtype は要素型、size は総要素数、itemsize は 1 要素あたりのバイト数である。NumPy の初学者は「配列の中身」ばかり見がちだが、実際には shape と dtype を見れば、多くの不具合の原因を早い段階で絞り込める。

```
(2, 3)
2
float64
6
8
```

出力を見れば、この配列が「2 行 3 列の float64 配列」であることがすぐ分かる。NumPy のトラブルの多くは、この 5 行のどれかを見れば原因に近づける。

6.1.3 Python, C, NumPy でのデータ型の違い

Python 標準の変数は型を自動で管理して非常に自由度が高いが、NumPy を用いた数値計算では逆に「データサイズを固定した C 言語レベルの厳密な型」を意識することが重要になる。代表的な型の対応を表 6.1 に示す。

C 言語では、扱う変数が「メモリ空間を何ビットの箱として占有するか」を明示的に宣言しなければならない（例えば単精度の float と倍精度の double）。対して Python の標準型である int は桁数に実質的な上限がなく、float は背後で C 言語の倍精度（double 相当の 64 ビット）として確保される。しかし、Python の標準変数は単なるメモリ上の数値だけでなく、「この変数は現在のどこから参照されているか」「これはいま何の型のオブジェクトか」といった管理情報を余分にひとまとめにして抱え込んでいるため、単純な数値演算の際には非常にオーバーヘッドが大きくなる。

これに対して NumPy では、内部構造として C 言語の「純粋な数値の固定長な配列（ポイ

表 6.1 Python, NumPy, C 言語での代表的な数値型の対応

分類	C 言語の型	Python の標準型	NumPy の型 (dtype)
整数型	int, long	int (桁数上限なし)	int32, int64
単精度浮動 小数	float (32 ビット)	(標準では存在せず)	float32
倍精度浮動 小数	double (64 ビット)	float	float64
論理型	bool	bool	bool_

ンタからの連続オフセット)」と同じ状態をメモリ上に展開する。これにより、Python の無駄な管理部分を一切通さず、CPU や C 言語レベルの演算回路にデータを配列ごとそのまま流し込んで一気に処理（ベクタライズ計算）させることが可能になっている。

浮動小数点型は、必要な精度と値域、配列の大きさ、入出力元の型、CPU や GPU の特性を基準に選ぶ。NumPy の多くの配列生成関数や Python の float から作る配列では float64 が既定になり、反復計算や差の小さい値を扱う計算では有力な選択肢になる。一方、float32 は 1 要素が 4 バイトであり、8 バイトの float64 の半分で済むため、大きな画像、センサデータ、GPU 計算、メモリ転送が支配的な処理では有効である。元データが float32 なら、必要性を検討せずに倍精度へ変換するとメモリだけが増える場合もある。

型を変えるときは、速度だけでなく結果が必要な精度を満たすかを確認する。例えば `np.finfo(np.float32).eps` と `np.finfo(np.float64).eps` を表示すると、1 付近で区別できる相対的な刻み幅を比較できる。どちらの型でも、桁落ちや丸め誤差そのものがなくなるわけではない。基準値が決まっていない段階では float64 から始め、メモリや転送が問題になった時点で float32 でも解析結果が変わらないかを検証する方法が堅実である。

6.1.4 NumPy が速くなりやすい理由

NumPy が速くなりやすい理由は、単に「ライブラリだから」ではない。主な理由は三つある。

1. 数値を連続したメモリにまとめて持てること
2. Python レベルのループを減らせること
3. 内部で低レベルに最適化された処理を使えること

特に重要なのは、Python で 1 要素ずつ処理する回数が減ることである。大きなデータでは、この差がそのまま処理時間の差になる。

もう一つの利点は、式がそのまま数学的な記述に近づくことである。たとえば差分、閾値判定、正規化、平均、分散といった操作は、配列演算で書いた方が意図が明確になることが多い。読みやすさと速度が同時に改善する場面は少なくない。

6.2 最小限のコード例

6.2.1 配列を作る

NumPy では、最初に「どの形の配列を、どの型で作るのか」を決める。np.array 以外にも、一定値で埋める、等間隔の値を作る、といった入口が多い。

```
1 a = np.array([1, 2, 3], dtype=np.float64)
2 zeros = np.zeros((2, 3))
3 ones = np.ones((2, 3))
4 tmp = np.empty((2, 3))
5 grid = np.arange(0, 10, 2)
6 x = np.linspace(0.0, 1.0, 5)
7
8 print(a)
9 print(zeros.shape)
10 print(grid)
11 print(x)
```

```
[1.  2.  3.]
(2, 3)
[0  2  4  6  8]
[0.   0.25 0.5  0.75 1.  ]
```

np.array は既存の Python リストなどから配列を作る。np.zeros と np.ones は初期値が明確なので安全である。np.arange は整数刻みの列、np.linspace は区間を等分した実数列を作る。解析では、ビン中心、時間軸、周波数軸を作る場面で linspace をよく使う。

zeros.shape が (2, 3) なので、zeros と ones は 2 行 3 列の配列である。grid は 0 から 8 まで 2 刻み、x は 0 から 1 を 5 点で等分した座標列である。np.empty はメモリだけ確保して値を初期化しないため高速だが、初期値は未定である。後で必ず全要素を上書きする場合にだけ使う方が安全である。

6.2.2 よく使う操作

解析でよく使う NumPy の操作として、条件に合う要素だけを取り出すフィルタ、順序をそろえるソート、隣り合う要素の差を取る差分、分布を数えるヒストグラムがある。これらは別々の操作に見えるが、解析ではまとめて使うことが多い。

```
1 times = np.array([41, 10, 23, 15, 42])
2
3 selected = times[times > 20]
4 sorted_times = np.sort(times)
5 deltas = np.diff(sorted_times)
```

```

6 hist, edges = np.histogram(times, bins=[0, 20, 40, 60])
7
8 print(selected)
9 print(sorted_times)
10 print(deltas)
11 print(hist)
12 print(edges)

```

```

[41 23 42]
[10 15 23 41 42]
[ 5  8 18  1]
[2 1 2]
[ 0 20 40 60]

```

この例で行っていることは次のとおりである。

- フィルタ: `times > 20` は `[True, False, True, False, True]` に相当する条件配列を作り、その位置にある要素だけを `selected` として取り出す。
- ソート: `np.sort(times)` は値を昇順に並べる。観測時刻やエネルギーのように順序が意味を持つ量では、後続の処理の前に並べ替えることが多い。
- 差分: `np.diff(sorted_times)` は隣り合う値の差を返す。返る配列の長さは元の配列より1つ短い。イベント時刻が昇順に並んでいれば、これは到着間隔として読める。
- ヒストグラム: `np.histogram(times, bins=[0, 20, 40, 60])` は、区間ごとの個数を `hist`、区間境界を `edges` として返す。この例では `[0, 20)` に2個、`[20, 40)` に1個、`[40, 60]` に2個入っている。

ブール配列によるフィルタは NumPy の基本である。条件式の結果をそのまま添字にできるので、`(times > 20) & (times < 40)` のような複数条件も簡潔に書ける。一方で、差分は順序に依存するので、時刻列のようなデータでは `np.diff(times)` をいきなり使わず、まずソートが必要かどうかを確認した方が安全である。

ヒストグラムは、ビン境界 $b_0, b_1, \dots, b_n$ に対して

$$h_k = \#\{t_i \mid b_k \leq t_i < b_{k+1}\}$$

を数える操作と見なせる。ただし、NumPy では最後のビンだけ右端も含むので、最終区間は $[b_{n-1}, b_n]$ と読めばよい。

6.2.3 要素ごとの演算とユニバーサル関数

NumPy の演算子や多くの関数は、原則として要素ごとに働く。NumPy の公式ドキュメント [6] では、`exp`、`sqrt`、`sin` のような要素ごと関数をユニバーサル関数 (ufunc) と呼んでいる。

```

1 a = np.array([1.0, 2.0, 3.0])
2 b = np.array([10.0, 20.0, 30.0])

```

```

3
4 print(a + b)
5 print(a * 2)
6 print(a ** 2)
7 print(np.exp(a))
8 print(np.sqrt(a))

```

```

[11. 22. 33.]
[2. 4. 6.]
[1. 4. 9.]
[ 2.71828183  7.3890561 20.08553692]
[1.          1.41421356 1.73205081]

```

配列どうしの四則演算は、形がそろっていれば要素ごとに実行される。`np.exp(a)` や `np.sqrt(a)` も同様である。関数が 1 要素に対して何をするかを知っていれば、その関数が配列全体へどう作用するかも追やすい。

たとえば、配列 $a = (a_i)$ 、 $b = (b_i)$ に対しては

$$(a + b)_i = a_i + b_i, \quad (a^2)_i = a_i^2, \quad (\exp(a))_i = e^{a_i}$$

のように、各要素へ同じ規則が適用されると考えればよい。

6.2.4 集計と axis

NumPy の集計関数は、配列全体に対しても、特定の軸ごとにも使える。`axis` の意味は初学者がつまづきやすいので、コードで確認するのがよい。

```

1 matrix = np.array([[1.0, 2.0, 3.0],
2                     [4.0, 5.0, 6.0]])
3
4 total = matrix.sum()
5 col_sum = matrix.sum(axis=0)
6 row_sum = matrix.sum(axis=1)
7 col_max = matrix.max(axis=0)
8
9 print(total)
10 print(col_sum)
11 print(row_sum)
12 print(col_max)

```

```

21.0
[5. 7. 9.]
[ 6. 15.]
[4. 5. 6.]

```

`total` は全要素の和である。`axis=0` は「0 軸をつぶす」、すなわち各列ごとの集計になり、`axis=1` は各行ごとの集計になる。言葉だけで覚えるより、「どの軸が消えて、何が残るか」を `shape` で確認する方が理解しやすい。

行列 $A = (a_{ij})$ に対しては、

$$(\text{col_sum})_j = \sum_i a_{ij}, \quad (\text{row_sum})_i = \sum_j a_{ij}$$

に対応する。つまり、axis=0 は列方向の和、axis=1 は行方向の和を返している。

6.2.5 添字、スライス、条件抽出

NumPy の添字は Python リストと似ているが、多次元配列とブール配列が自然に使える点が重要である。

```
1 times = np.array([10, 15, 23, 41, 42])
2 print(times[0])
3 print(times[-1])
4 print(times[1:4])
5 print(times[times > 20])
6
7 grid = np.arange(12).reshape(3, 4)
8 print(grid[1, 2])
9 print(grid[:, 1])
10 print(grid[1, :])
```

```
10
42
[15 23 41]
[23 41 42]
6
[1 5 9]
[4 5 6 7]
```

times[1:4] は 1 番目から 3 番目までを取り出す。grid[:, 1] は全行の 2 列目、grid[1, :] は 2 行目全体である。多次元配列に対して for row in grid: と書くと、最初の軸に沿って 1 行ずつ取り出される。反復自体はできるが、NumPy を使うなら、まず配列演算で書けないかを先に考える方が自然である。

6.2.6 形を変える

NumPy を使い始めたら、配列の shape を意識することが重要である。1 次元配列なのか、行列なのか、列ベクトルとして扱いたいのかで、同じ演算でも結果が変わる。

```
1 a = np.arange(12)
2 matrix = a.reshape(3, 4)
3 flat = matrix.ravel()
4 transposed = matrix.T
5 column = np.array([1.0, 2.0, 3.0])[:, None]
6
7 print(matrix.shape)
```

```

8 print(matrix)
9 print(flat)
10 print(transposed)
11 print(column.shape)
12 print(column)

```

```

(3, 4)
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
[ 0  1  2  3  4  5  6  7  8  9 10 11]
[[ 0  4  8]
 [ 1  5  9]
 [ 2  6 10]
 [ 3  7 11]]
(3, 1)
[[1.]
 [2.]
 [3.]]

```

`reshape` は形を変える。`ravel` は可能ならビューとして 1 次元へ潰す。`T` は転置である。`[:, None]` は新しい軸を 1 本追加し、1 次元配列を列ベクトルのように扱う書き方である。後で述べるブロードキャストでは、この 1 本の軸追加が非常に重要になる。

6.2.7 配列をつなぐ、分ける

複数の配列をつないだり、1 つの配列を分割したりする操作もよく使う。特に、前処理済みデータを縦に積む、特徴量を列方向に並べる、といった場面が必要になる。

```

1 x = np.array([1, 2, 3])
2 y = np.array([4, 5, 6])
3
4 joined = np.concatenate([x, y])
5 stacked = np.stack([x, y], axis=0)
6
7 matrix = np.arange(12).reshape(3, 4)
8 left, right = np.split(matrix, 2, axis=1)
9
10 print(joined)
11 print(stacked.shape)
12 print(stacked)
13 print(left)
14 print(right)

```

```

[1 2 3 4 5 6]
(2, 3)
[[1 2 3]
 [4 5 6]]

```

```
[[0 1]
 [4 5]
 [8 9]]
[[ 2  3]
 [ 6  7]
 [10 11]]
```

`concatenate` は既存の軸に沿ってつなぐ。`stack` は新しい軸を追加して積む。`split` は指定した軸に沿って分割する。見た目が似ていても結果の `shape` は異なるので、実行後に必ず `shape` を見る方がよい。

6.2.8 形とブロードキャスト

NumPy では、配列の形状をそろえなくても、自動拡張の規則に従って演算できる場合がある。この規則をブロードキャストという。

```
1 x = np.array([1.0, 2.0, 3.0])
2 y = np.array([10.0, 20.0, 30.0])
3
4 z = x + y
5 matrix = x[:, None] + y[None, :]
```

後半の `matrix` では、1 次元配列を軸付きで広げることで、3 行 3 列の和の表を作っている。こうした自動拡張の規則をブロードキャストという。最初は少し難しいが、二つの配列の全組合せを調べるような場面で非常に強力である。上記の操作によるブロードキャストの結果は次のようになる。

```
z =
[11. 22. 33.]

matrix =
[[11. 21. 31.]
 [12. 22. 32.]
 [13. 23. 33.]]
```

この結果から、`z` は要素ごとの和、`matrix` は全組合せの表であることが分かる。便利ではあるが、配列サイズが大きいと一気にメモリを消費するので、実データでは常に形と大きさを見積もる必要がある。

数式で書けば、

$$z_i = x_i + y_i, \quad M_{ij} = x_i + y_j$$

である。後者の M_{ij} が、ブロードキャストによって作られる「全組合せの表」に対応している。

6.2.9 統計量とヒストグラム

NumPy には、平均、分散、最小値、最大値、ヒストグラムなど、解析の基本となる集計関数がそろっている。代表的な統計量とヒストグラムを計算する例を下に示す。

```
1 values = np.array([1.0, 2.0, 2.0, 3.0, 4.0])
2 mean = values.mean()
3 std = values.std(ddof=0)
4 hist, edges = np.histogram(values, bins=[0.0, 2.0, 4.0, 6.0])
5
6 print(f"mean = {mean:.2f}")
7 print(f"std = {std:.2f}")
8 print(hist)
9 print(edges)
```

`np.histogram` は図を描かずに分布だけを数えたいときに便利である。ビン境界 `edges` も同時に返るため、後で自分で曲線を重ねたり、複数データを比較したりしやすい。可視化と数え上げを分けて考えると、処理の見通しが良くなる。実際にこれらの統計量を出力した結果は次のようになる。

```
mean = 2.40
std = 1.02
[1 3 1]
[0. 2. 4. 6.]
```

この例では、`[0, 2)` に 1 個、`[2, 4)` に 3 個、`[4, 6]` に 1 個入っている。平均や標準偏差だけでは分布の形は分からず、ヒストグラムだけでは条件間比較がしにくい。数値要約と分布表示を組み合わせる読むことが、データ解析では基本になる。

`values.mean()` と `values.std(ddof=0)` は、それぞれ

$$\bar{x} = \frac{1}{N} \sum_{i=1}^N x_i, \quad \sigma = \sqrt{\frac{1}{N} \sum_{i=1}^N (x_i - \bar{x})^2}$$

を計算している。ここで `ddof=0` は、標本標準偏差ではなく母標準偏差の定義を使うという意味である。

6.3 よくある落とし穴

6.3.1 演算子の意味を取り違える

* は要素ごとの積であり、行列積ではない。行列積が欲しいときは `@` か `np.dot` を使う。

```
1 A = np.array([[1.0, 2.0],
2               [3.0, 4.0]])
```

```

3 B = np.array([[5.0, 6.0],
4               [7.0, 8.0]])
5
6 print(A * B)
7 print(A @ B)

```

この違いを理解せずには書くと、コードは動いているのに意味だけが間違っている、という厄介な状態になる。特に線形代数や座標変換では注意が必要である。

6.3.2 axis と条件式

axis の意味は、初学者が最初に混乱しやすい箇所である。axis=0 は列ごとの集計、axis=1 は行ごとの集計になる。また、配列どうしの条件結合では and や or ではなく、& や | を使う。

```

1 matrix = np.array([[1, 2, 3],
2                    [4, 5, 6]])
3 print(matrix.sum(axis=0))
4 print(matrix.sum(axis=1))
5
6 times = np.array([10, 15, 23, 41, 42])
7 mask = (times > 10) & (times < 40)
8 print(mask)
9 print(times[mask])

```

```

[5 7 9]
[ 6 15]
[False  True  True False False]
[15 23]

```

times > 10 & times < 40 のように括弧を省略すると、優先順位の違いで意図しない式になる。NumPy の条件式では「各比較を括弧で囲む」を基本にした方が安全である。

6.3.3 dtype と np.empty

NumPy では、混在した型を入れると配列全体の dtype が自動的に決まる。この挙動を意識していないと、数値計算のつもりが文字列配列になっていた、ということが起こる。

```

1 a = np.array([1, 2.5])
2 b = np.array([1, "2"])
3 tmp = np.empty(5)
4
5 print(a.dtype)
6 print(b.dtype.kind)
7 print(np.issubdtype(a.dtype, np.number))
8 print(np.issubdtype(b.dtype, np.number))
9 print(tmp.shape)

```

```
float64
U
True
False
(5,)
```

a は浮動小数点へそろえられる。これに対して b は `dtype.kind == "U"`、つまり Unicode 文字列の配列になっており、もはや数値計算用ではない。tmp は 0 で初期化されないで、値を見る前に必ず全要素を書き込む必要がある。配列生成直後に `dtype` と初期値の有無を確認することが重要である。

6.3.4 コピーとビュー

NumPy は便利だが、配列をむやみにコピーすると逆に重くなる。一方で、意図せずビューを通じて元配列を書き換えてしまうこともある。この違いを理解せずに値を書き換えると、意図しない副作用や余分なメモリ使用を招く。

```
1 a = np.arange(6)
2
3 view = a[1:4]
4 view[:] = -1
5
6 copy = a[[0, 2, 4]]
7 copy[:] = 99
8
9 safe = a.copy()
```

この例では、スライスで作った `view` は元の `a` とデータを共有するので、`view[:] = -1` とすると `a` も変わる。これに対して、整数配列やブール配列による取り出しは新しい配列を返すので、`copy[:] = 99` としても元の `a` は変わらない。`reshape` や `ravel` も可能ならビューを返すため、「独立した配列が欲しい」ときは `copy()` を明示した方が安全である。

```
a after view = [ 0 -1 -1 -1  4  5]
copy          = [99 99 99]
safe          = [ 0 -1 -1 -1  4  5]
```

この出力では、`view` を書き換えた時点で元配列 `a` が変化しているのに対し、`copy` は独立していることが見て取れる。

6.3.5 浮動小数点数の比較

浮動小数点数の丸め誤差にも注意が必要である。理論上は等しいはずの値でも、計算手順によって最後の数桁がずれることがある。そのため、`a == b` のような直接比較より、許容誤差を設けた比較を使う方が安全なことが多い。

```
1 a = 0.1 + 0.2
```

```
2 print(a == 0.3)
3 print(np.isclose(a, 0.3))
```

```
False
True
```

6.4 解析へのつながり

6.4.1 解析で NumPy を使うときの視点

大規模データでは、型を適切に選ばないとメモリを無駄に使う。たとえば、識別子や時刻差なら `int64` が自然だが、カテゴリ番号が小さい範囲なら `int16` や `int32` でも十分なことがある。大規模データでは、この積み重ねが効いてくる。

また、平均や分散だけでなく、差分、閾値判定、並べ替え、ヒストグラムまで NumPy の中で一貫して処理できると、コードの往復が減り、解析の流れを追いやすくなる。

6.4.2 差分、ソート、ヒストグラム

解析では、まず時刻順に並べ、次に差分を取り、その後で条件を掛けて分布を見る、という流れがよく現れる。NumPy ではこの流れをほぼそのまま式として書ける。

```
1 order = np.argsort(times)
2 sorted_times = times[order]
3 deltas = np.diff(sorted_times)
4 mask = (deltas >= 0) & (deltas <= 250)
5 hist, edges = np.histogram(deltas[mask], bins=50)
6 peak_bin = hist.argmax()
```

`np.argsort` は並べ替え順の添字を返す。`np.diff` は隣接差分である。`hist.argmax()` は最も高いビンの位置を返す。イベント時刻、スペクトル、波形特徴量のいずれでも、この種の処理は頻出である。

```
sorted_times = [10 15 23 41 42]
deltas       = [ 5  8 18  1]
peak_bin     = 0
```

6.4.3 線形代数の入口

NumPy は単なる配列操作だけでなく、線形代数の入口としても重要である。小規模な連立方程式や座標変換なら、まず NumPy の `linalg` を使うだけで十分なことが多い。

```
1 A = np.array([[1.0, 1.0],
2               [1.0, -1.0]])
```

```
3 b = np.array([3.0, 1.0])
4 x = np.linalg.solve(A, b)
```

校正係数の決定、座標変換、最小二乗法の前処理などでは、こうした線形代数の操作が自然に現れる。大規模で高度な最適化は後の SciPy 章で扱うが、入口は NumPy で十分に理解できる。

6.4.3.1 Python リストを用いた数値計算の限界

Python の標準リストは非常に柔軟であり、どんな型のものでも放り込める。しかしいざ大量の数値を計算しようとする、この「なんでも入る」という性質が最大のネックとなる。避けるべきリストを用いた数値計算の例を示す。

```
1 # 100万個のリストを作る
2 py_list = list(range(1000000))
3
4 # すべての要素を2倍にする。forループと内包表記が必要
5 doubled_list = [x * 2 for x in py_list]
```

一見普通に動くが、Python のリストは「あちこちに散らばっている多様なサイズの箱へのポインタ（地図）」の集まりである。CPU は一つの値を計算するたびに「この値は何型か」「どこに置かれているか」を確認しなければならない、絶望的に遅い。

6.4.3.2 NumPy のベクタライズ化

これに対し NumPy デイメンショナル配列 (ndarray) は、同じ型の数値 (int64 や float64 など) をメモリ上に「すき間なく一列に並べた団地」として構成される。型確認が不要で、C 言語レベルの一括演算回路が直結する。NumPy 配列へ変換してから一括計算するコードは次の通りである。

```
1 import numpy as np
2
3 # 1. リストから NumPy アレイへの変換 (メモリ上にきれいに並べ直す)
4 np_array = np.array(py_list)
5
6 # 2. アレイはベクトル演算が可能 (for文を書かなくても全要素を一気に計算できる)
7 doubled_array = np_array * 2
8
9 # 3. アレイからリストへ戻すことも可能だが、重いので通常は最後だけにする
10 back_to_list = doubled_array.tolist()
```

6.4.3.3 pandas DataFrame との変換

後述する pandas DataFrame は、列名と行ラベルを持つ表形式のデータ構造であり、列の内部では NumPy 配列や拡張配列が使われる。解析の途中で NumPy の関数を使いたい

場合は、DataFrame や Series を NumPy 配列へ変換できる。pandas の DataFrame・Series と NumPy 配列を相互変換する基本形は次の通りである。

```
1 import pandas as pd
2
3 # NumPy 配列から pandas の DataFrame (二次元表) に変換
4 df = pd.DataFrame(np_array, columns=["Value"])
5
6 # pandas の列から NumPy 配列を得る
7 values_array = df["Value"].to_numpy()
```

to_numpy() がコピーを作るかどうかは、列の型、列数、必要な型変換に依存する。単一の数値列では元データとメモリを共有する場合もあるが、常に共有されるとは限らない。コピーの有無が性能や書換えに影響する場面では、np.shares_memory() で確認し、必要なら to_numpy(copy=True) で独立した配列を要求する。

6.4.4 公式資料の使い方

NumPy の基礎を復習したいときは、NumPy quickstart [7] を併読すると整理しやすい。特に、配列生成、添字とスライス、shape の変更、ブロードキャスト、コピーとビューは、解析コードを書くたびに戻って確認したくなる項目である。本章では解析で困りやすい点を補って説明しているので、まず本章で全体像をつかみ、その後で API 名や細かな仕様を quickstart で確認する読み方がよい。

6.5 章末問題

USGS の CSV をまだ用意していない場合は、次の配列を入力として 1 問目と 2 問目を実行する。この配列は秒単位の発生時刻であり、意図的に未整列にしてある。

```
1 times = np.array([720.0, 0.0, 60.0, 4200.0, 180.0, 780.0])
```

1. 地震の発生時刻から発生間隔配列を作る。第 5 章で扱った CSV から time 列を読み、秒単位の数値配列へ変換せよ。その後で NumPy 配列へまとめ、np.sort と np.diff を使って隣り合う地震の発生間隔 Δt を求めよ。小規模入力では、整列後の時刻が [0., 60., 180., 720., 780., 4200.], Δt が [60., 120., 540., 60., 3420.] になる。np.sort() が返す配列と np.diff() が返す配列について、shape、dtype、要素数が 1 減る理由を説明せよ。
2. 発生間隔の基本統計とヒストグラムを NumPy だけで調べる。 Δt 配列に対して、平均値、0 秒超だけに絞った要素数、[0, 60) s、[60, 600) s、[600, 3600) s の 3 区間の個数を求めよ。さらに np.histogram を用いて 0 秒から 3600 秒までを等間隔ビンに分け、最も多いビンの範囲を報告せよ。小規模入力の平均値は 840 秒であり、0 秒超の要素数は 5 である。3 区間の個数は順に 0、4、1 になる。区間の右端を含めるかどうか条件式と一致

していることを確認し、`np.histogram()` が返す `counts` と `bin_edges` の長さの関係も説明せよ。

3. **発展課題。** 発生間隔は短時間側に密集し、長時間側にはまばらに点が残ることが多い。そのため、等間隔ビンだけでは短時間側の構造が見えにくいことがある。たとえば `np.logspace(np.log10(1), np.log10(3600), 30)` のようにビン境界を作り、 Δt のヒストグラムを対数ビンで作り直して、固定幅ビンの場合と見え方を比較せよ。余裕があれば、日本近海データを別に取得して同じ処理を行い、全球データとの違いも比較せよ。

NumPy 配列は同じ型の数値を計算する場合に適している。一方、観測時刻、検出器名、信号値のように、意味と型が異なる列をまとめて扱うには表形式の構造が適している。次章では、NumPy の列演算を土台として、pandas で列名、欠損値、グループ集計、結合を扱う。

第 7 章

pandas による表形式データ解析

pandas は、行と列からなる表形式データを扱うための代表的なライブラリである。観測ログ、イベント一覧、校正表、集計結果のように「1 行が 1 件の記録」であるデータは、まず pandas の DataFrame として読むのが自然である。本章では、DataFrame の意味、最小限の操作、落とし穴、解析へのつながりを順に整理する。

7.1 pandas の考え方

7.1.1 Series と DataFrame

pandas の基本構造は Series と DataFrame である。Series はラベル付き 1 次元配列、DataFrame は列名付き 2 次元表である。NumPy の配列が「同じ型の値を高速に並べる道具」だとすれば、pandas は「列ごとに意味を持つデータをまとめて扱う道具」である。

```
1 import pandas as pd
2
3 events = pd.DataFrame(
4     {
5         "detector": ["A", "A", "B", "B"],
6         "time_ns": [1032, 1188, 1055, 1244],
7         "signal": [12.4, 10.1, 14.8, 9.7],
8         "valid": [True, True, False, True],
9     }
10 )
11
12 print(type(events).__name__)
13 print(events)
```

このコードを実行すると、次のような DataFrame が表示される。

```
DataFrame
```

	detector	time_ns	signal	valid
0	A	1032	12.4	True
1	A	1188	10.1	True
2	B	1055	14.8	False
3	B	1244	9.7	True

この表では、行番号 0, 1, 2, 3 は index、"detector" や "signal" は列名である。index は「行を識別する補助ラベル」であり、列名は「何の量なのか」を示す。初学者は中身の数値ばかり見がちだが、実際には列名と型を確認した方が理解は速い。

DataFrame の利点は、列ごとに意味を持たせながら、まとめて処理を書けることにある。「何列目か」を覚える必要が減り、列名を通して処理の意味が見えやすくなる。特に、ファイル読み込み、列変換、フィルタ、集計、書き出しまでを一つの流れで扱える点は大きい。解析の途中で確認用の表を少し表示したいときにも便利である。

DataFrame の基本操作は、Python 公式チュートリアル「データ構造」章に相当する考え方を、表形式へ拡張したものだと考えると理解しやすい。リストや辞書を知っていると、列名によるアクセスや行集合に対するフィルタの意味が見えやすい。

7.1.2 最初に見るべき情報

DataFrame を作ったり読んだりした直後は、まず `head()`、`shape`、`dtypes`、`info()` を見る癖を付けるとよい。

```
1 print(events.head())
2 print(events.shape)
3 print(events.dtypes)
```

この確認では、先頭数行、形、各列の型がまとめて分かる。出力は次の通りである。

```
detector  time_ns  signal  valid
0         A    1032    12.4   True
1         A    1188    10.1   True
2         B    1055    14.8  False
3         B    1244     9.7   True
(4, 4)
detector    object
time_ns      int64
signal      float64
valid        bool
dtype: object
```

`shape` は行数と列数、`dtypes` は各列の型である。ここで `signal` が `float64` なのか、`detector` が文字列列として読まれているのかを確かめるだけで、多くの不具合を早い段階で防げる。特に `object` は「文字列かもしれないし、型が混ざっているかもしれない」という合図なので、数値に見える列が `object` になっていたら注意が必要である。

7.1.3 index と列名の役割

pandas の操作は、整数位置だけでなくラベルでも書ける。これが NumPy との大きな違いである。

```
1 print(events["signal"])
2 print(events.loc[1:2, ["detector", "signal"]])
3 print(events.iloc[1:3, 0:2])
```

events["signal"] は列名による選択、loc はラベルによる選択、iloc は整数位置による選択である。pandas の indexing ガイド [11] でも、対話的な試行錯誤では [] が便利だが、整理されたコードでは loc、iloc、at、iat を意識することが勧められている。解析コードでは「列名で取りたいのか」「何行目を取りたいのか」を曖昧にしない方が安全である。

7.1.4 pandas が速くなりやすい理由

pandas が速くなりやすいのは、列単位でデータを処理し、内部でベクトル化された操作を使えるからである。Python の辞書リストを 1 行ずつ回して書くより、列としてまとめて扱う方が速く、コードも短くなりやすい。

ただし、すべての場面で pandas が最適とは限らない。行ごとに複雑な条件分岐を大量に行う場合や、データ構造自体を細かく制御したい場合には、NumPy や自作構造の方が向くこともある。したがって、「とりあえず pandas を使う」よりも、「列として扱える問題かどうか」を考える方が重要である。表として整理できるなら pandas は強力だが、近傍探索や状態遷移のように行どうしの関係が複雑になると、別の設計の方が自然なこともある。

7.2 最小限のコード例

7.2.1 列選択と行抽出

最初に身に付けたいのは、必要な列だけを残し、条件に合う行だけを取り出す操作である。

```
1 selected = events[["detector", "signal"]]
2 valid_events = events.loc[events["valid"], :]
3 strong = events.loc[events["signal"] > 11.0, ["detector", "signal"]]
4
5 print(selected)
6 print(valid_events)
7 print(strong)
```

それぞれの変数にどの行と列が残ったかは、実際の出力を見ると追やすい。

```
detector  signal
0         A    12.4
```

```

1      A    10.1
2      B    14.8
3      B     9.7
   detector  time_ns  signal  valid
0      A    1032    12.4   True
1      A    1188    10.1   True
3      B    1244     9.7   True
   detector  signal
0      A    12.4
2      B    14.8

```

ブール配列で行を絞る操作は NumPy と似ているが、pandas では列名を介して意味を保ったまま書ける。events["signal"] > 11.0 の結果は行ごとの真偽値であり、それを loc に渡すと条件に合う行だけが残る。

7.2.2 列を追加する

解析では、元の列から比や差、フラグ列を作ることが多い。pandas では列全体に対する式として書くのが自然である。

```

1 events["signal_ratio"] = events["signal"] / events["signal"].mean()
2 events["is_late"] = events["time_ns"] > 1100
3
4 print(events)

```

列を追加すると、元の表へ新しい列が右側へ増える。出力は次のようになる。

```

   detector  time_ns  signal  valid  signal_ratio  is_late
0      A    1032    12.4   True      1.058874   False
1      A    1188    10.1   True      0.862202    True
2      B    1055    14.8  False      1.263652   False
3      B    1244     9.7   True      0.827111    True

```

assign() を使うと、途中の DataFrame を明示しながら複数列をまとめて追加できる。

```

1 derived = (
2     events
3     .assign(
4         dt_ns=lambda df: df["time_ns"] - df["time_ns"].min(),
5         quality=lambda df: pd.Series(["high", "mid", "low", "mid"])
6     )
7 )

```

assign() は処理をパイプラインの形で並べたいときに読みやすい。一方で、単に 1 列だけ追加するなら events["new_col"] = ... の方が直感的である。

7.2.3 欠損値と型

現実のデータでは、空欄や壊れた値が入り込む。pandas ではこれを NaN や NA として扱うことが多い。

```
1 df = pd.DataFrame(
2     {
3         "id": [1, 2, 3, 4],
4         "signal": [12.4, None, 14.8, 9.7],
5         "comment": ["ok", "bad", None, "ok"],
6     }
7 )
8
9 print(df.isna())
10 print(df.dropna(subset=["signal"]))
11 print(df["signal"].fillna(df["signal"].mean()))
```

isna() の真偽表、dropna() の結果、fillna() 後の列を順に表示すると、欠損処理の違いが見えやすい。

```
   id  signal  comment
0  False  False   False
1  False   True   False
2  False  False    True
3  False  False   False
   id  signal  comment
0    1   12.4      ok
1    3   14.8    None
2    4    9.7      ok
0   12.400000
1   12.300000
2   14.800000
3    9.700000
Name: signal, dtype: float64
```

欠損値は == で判定しない。isna()、notna()、dropna()、fillna() を使う。また、整数列に欠損が入ると float64 に変わることがあるので、必要なら pandas の nullable integer 型である Int64 を使う。

```
1 df["id"] = df["id"].astype("Int64")
```

7.2.4 要約統計と groupby

describe() は列の全体像を手早く見るのに便利であり、groupby() はカテゴリごとの集計で中心になる。

```
1 print(events.describe())
```

```

2
3 summary = (
4     events
5     .groupby("detector")
6     .agg(
7         count=("signal", "size"),
8         mean_signal=("signal", "mean"),
9         std_signal=("signal", "std"),
10        late_fraction=("is_late", "mean"),
11    )
12 )
13
14 print(summary)

```

`describe()` は全体の要約、`groupby()` の結果は検出器ごとの集計である。出力は次の通りである。

	time_ns	signal	signal_ratio
count	4.000000	4.000000	4.000000
mean	1129.750000	11.750000	1.002960
std	99.494975	2.306513	0.196299
min	1032.000000	9.700000	0.827111
25%	1049.250000	10.000000	0.853429
50%	1121.500000	11.250000	0.960538
75%	1202.000000	13.000000	1.110069
max	1244.000000	14.800000	1.263652

	count	mean_signal	std_signal	late_fraction
detector				
A	2	11.25	1.626346	0.5
B	2	12.25	3.606245	0.5

`groupby()` は「検出器ごと」「日ごと」「run ごと」に同じ集計を繰り返したいときの基本である。`agg()` に名前付き集計を書くと、列名が分かりやすくなる。

7.2.5 表の形を変える

Getting started tutorials [9] でも、表を縦長から横長へ変える操作、複数表を結合する操作が重視されている。解析でも、観測値と較正值を結び付けたり、日別表をまとめたりする場面で必ず使う。

```

1 gain = pd.DataFrame(
2     {
3         "detector": ["A", "B"],
4         "gain": [1.02, 0.97],
5     }
6 )
7
8 merged = events.merge(gain, on="detector", how="left")
9 stacked = pd.concat([events.iloc[:2], events.iloc[2:]], ignore_index=True)

```

```

10
11 long_df = pd.DataFrame(
12     {
13         "detector": ["A", "A", "B", "B"],
14         "kind": ["raw", "cal", "raw", "cal"],
15         "value": [12.4, 12.7, 14.8, 14.4],
16     }
17 )
18 wide = long_df.pivot(index="detector", columns="kind", values="value")
19
20 print(merged[["detector", "signal", "gain"]])
21 print(wide)

```

merge() で較正係数が付き、pivot() で縦長の表が横長に変わる様子は、出力を並べると分かりやすい。

	detector	signal	gain
0	A	12.4	1.02
1	A	10.1	1.02
2	B	14.8	0.97
3	B	9.7	0.97

	kind	cal	raw
detector			
A		12.7	12.4
B		14.4	14.8

merge() は SQL の join に近い。concat() は同じ形の表を縦や横につなぐ。pivot() や pivot_table() は、縦長データを比較しやすい横長形式へ変えるときに使う。逆に横長から縦長へ戻すなら melt() が便利である。

7.2.6 時刻、文字列、カテゴリ

時刻や文字列は、そのままでは解析しにくい。pandas には時刻用の dt アクセサ、文字列用の str アクセサ、カテゴリ型がある。

```

1 log = pd.DataFrame(
2     {
3         "timestamp": ["2026-04-01 12:00:00", "2026-04-01 12:00:05", "2026-04-02 09:30:00"],
4         "run_name": ["run_A_001", "run_A_002", "test_B_001"],
5         "detector": ["A", "A", "B"],
6     }
7 )
8
9 log["timestamp"] = pd.to_datetime(log["timestamp"], utc=True)
10 log["date"] = log["timestamp"].dt.date
11 log["prefix"] = log["run_name"].str.extract(r"^(run_[A-Z])")
12 log["detector"] = log["detector"].astype("category")
13

```

```
14 print(log[["timestamp", "date", "prefix", "detector"]])
```

時刻列と文字列列を分解すると、日付や接頭辞を独立した列として扱える。出力は次の通りである。

	timestamp	date	prefix	detector
0	2026-04-01 12:00:00+00:00	2026-04-01	run_A	A
1	2026-04-01 12:00:05+00:00	2026-04-01	run_A	A
2	2026-04-02 09:30:00+00:00	2026-04-02	None	B

`to_datetime()` を早めに使うと、差分計算、日付抽出、時刻順ソートが自然に書ける。文字列列には `str.contains()`、`str.extract()`、`str.strip()` などのベクトル化メソッドがある。カテゴリ型は、取り得る値が少ない列のメモリ削減や並び順の明示に役立つ。

7.2.7 入出力

Getting started tutorials [9] でも、入出力は最初期の重要項目である。本書では特に CSV、Parquet、圧縮テキストを意識する。

```
1 df = pd.read_csv(
2     "events.csv.gz",
3     compression="gzip",
4     usecols=["detector", "time_ns", "signal"],
5     dtype={"detector": "category", "time_ns": "int64", "signal": "float64"},
6 )
7
8 df.to_parquet("events.parquet")
9 small = pd.read_parquet("events.parquet", columns=["detector", "signal"])
```

この例は、ヘッダ付きのカンマ区切り CSV を gzip 圧縮した `events.csv.gz` を想定している。第 1 引数は入力パス、`compression="gzip"` は圧縮方式、`usecols` は読み込む列、`dtype` は列ごとの型である。`to_parquet()` は表全体を Parquet へ保存し、`read_parquet(..., columns=...)` は指定した 2 列だけを読み込む。

`read_csv()` を使うときは、必要な列だけ読む、型を指定する、コメント行を飛ばす、といった工夫が重要である。たとえば次のように書くと、圧縮付きの空白区切りファイルから必要列だけを型付きで読める。

```
1 df = pd.read_csv(
2     path,
3     compression="gzip",
4     sep=r"\s+",
5     comment="#",
6     header=None,
7     usecols=[0, 1, 2, 3],
8     names=["id", "date", "time", "value"],
9     dtype={"id": "int64", "value": "float64"},
10 )
```

このように列を絞るだけでも、読み込み時間とメモリ消費はかなり変わる。さらに `dtype=` を指定しておく、読み込み後の型変換を減らせる。列数や行数が大きいデータでは、どの列が本当に必要かを考えるだけで、後続の処理全体が軽くなる。読み込みは解析の最初の一步だが、同時に最適化の出発点でもある。

2 目の例は、ヘッダのない空白区切りファイルを想定している。`sep=r"\s+"` は 1 個以上の空白を区切りとし、`comment="#"` は # から始まる行の残りをコメントとして扱う。`header=None` は列名行がないことを示し、`names` が読み込んだ 4 列へ名前を付ける。`usecols=[0, 1, 2, 3]` の番号は入力ファイル中の列位置であり、0 から数える。

巨大なテキストファイルを何度も読むより、一度整形したら Parquet にして再利用の方がよい。特に `read_parquet()` は列選択と相性が良く、反復的な解析で待ち時間を減らせる。日々の解析では、この「最初に 1 回だけテキストを整理して、以後は Parquet を読む」という流れを作るだけで、待ち時間が大きく減る。

ここで区別したいのは、Parquet がファイル形式の名前であり、`pyarrow` はその読み書きを実装するライブラリだという点である。したがって、`to_parquet()` や `read_parquet()` が失敗したときは、「Parquet が壊れている」のではなく、「必要な実装がまだ入っていない」ことがある。

列同士の演算をそのまま式として書ける点も `pandas` の強みである。

```
1 df["ratio"] = df["value"] / df["value"].mean()
2 selected = df[df["ratio"] > 1.5]
```

行ごとの `for` ループを書く前に、列単位で表現できないかを考える習慣を持つとよい。

7.2.8 大きな CSV を分割して集計する

`pandas` の `DataFrame` は主としてメモリ上で処理されるが、`read_csv()` の `chunksize` を使えば、CSV を一定行数ずつ読みながら集計できる。例として、次の `events.csv` を考える。

events.csv

```
detector,signal
A,12.4
B,14.8
A,10.1
B,9.7
A,13.0
```

検出器ごとの件数と信号値の合計は、次のように 2 行ずつ読んでも計算できる。

```
1 counts = pd.Series(dtype="int64")
2 sums = pd.Series(dtype="float64")
3
4 for chunk in pd.read_csv(
5     "events.csv",
6     usecols=["detector", "signal"],
7     dtype={"detector": "string", "signal": "float64"},
```

```

8     chunksize=2,
9 ):
10     grouped = chunk.groupby("detector")["signal"]
11     counts = counts.add(grouped.count(), fill_value=0)
12     sums = sums.add(grouped.sum(), fill_value=0)
13
14 summary = pd.DataFrame({"count": counts, "mean": sums / counts})
15 print(summary)

```

実行結果は次の通りである。

	count	mean
detector		
A	3.0	11.833333
B	2.0	12.250000

`chunksize=2` は、反復ごとに最大 2 行の `DataFrame` を返す指定である。実データでは 10 万行など、利用可能なメモリと処理時間を見ながら値を決める。`Series.add(..., fill_value=0)` は、あるチャンクに片方の検出器しか含まれない場合にも、存在しない集計値を 0 として足す。平均値をチャンク平均の単純平均として求めると、チャンクごとの件数が違う場合に誤るため、全体の合計を全体の件数で割っている。

分割処理が適するのは、総和、件数、最小値、最大値のようにチャンクごとの結果を統合できる計算である。全行を同時に並べ替える処理、全行どうしを結合する処理、全データに依存する順位計算では、単純な分割だけでは同じ結果にならない。その場合は、必要列を減らす、Parquet を使う、Polars の lazy API やデータベースを使うといった別の方法も検討する。

7.2.9 表を LaTeX へ出力する

集計結果をレポートへ貼るときは、手で打ち直さずに `to_latex()` を使う方が再現性が高い。

```

1 tex_table = summary.reset_index().to_latex(
2     index=False,
3     caption="検出器ごとの平均信号",
4     label="tab:detector-summary",
5     float_format="%.3f",
6     escape=False,
7 )
8
9 with open("tables/detector_summary.tex", "w", encoding="utf-8") as f:
10     f.write(tex_table)

```

ここで `index=False` は左端の連番インデックスを出力しないための指定であり、`reset_index()` は `groupby` 結果の添字を普通の列へ戻すための操作である。さらに `escape=False` は、セル中にすでに TeX 記法が入っている場合にそのまま通したいときだけ使う。通常の文字列だけなら既定値のままエスケープさせた方が安全である。

`groupby()`、`agg()`、`to_latex()` をつなぎ、最後にファイルへ保存して本ドキュメント側で `\input` すれば、解析スクリプトからレポート用表までを自動化できる。表を手で打ち直さないことは、再現性の面でも大きい。

7.2.10 `plot()` は確認用として使う

pandas にも `plot()` があるが、これは Matplotlib の薄いラップである。

```
1 summary.plot(kind="bar", y="mean_signal")
```

手早い確認には便利だが、最終的な提出図や論文用図では、Matplotlib の Figure と Axes を明示して描く方が制御しやすい。本書でも最終的な作図は第 9.3 節以降の方法を基準とする。

7.3 よくある落とし穴

7.3.1 `SettingWithCopy` と `loc`

pandas 初学者がよく踏むのが、切り出した表へ代入したつもりで元の DataFrame を安全に更新できていない問題である。

```
1 subset = events[events["signal"] > 11.0]
2 subset["flag"] = True
```

この書き方では `SettingWithCopyWarning` が出ることがある。意図が「元の表の条件に合う行へ代入する」であるなら、次のように `loc` で書く方が明確である。

```
1 events.loc[events["signal"] > 11.0, "flag"] = True
```

7.3.2 `object` 型のまま計算しない

見た目が数値でも、空白や記号が混ざると `object` 型で読まれることがある。そのまま足し算や比較をすると、文字列連結や例外が起きる。大きなファイルでは `read_csv()` の `dtype=` を使い、読み込み直後に `dtypes` を確かめる方が安全である。

7.3.3 行ループは最後の手段である

pandas の DataFrame では、同じ処理でも列演算と行単位の Python 関数で実行時間が大きく異なることがある。特に、`iterrows()`、`apply(axis=1)`、並列化の使い分けは、処理内容と測定結果に基づいて判断する必要がある。

7.3.3.1 Step 1: iterrows() の限界

Python 初学者が最も陥りやすい罠が、「DataFrame の行を 1 つずつ取り出して処理する」という直感的な書き方である。

```
1 def heavy_computation(val):
2     return val ** 2
3
4 results = []
5 for index, row in df.iterrows():
6     results.append(heavy_computation(row["value"]))
7 df["value_sq"] = results
```

この書き方は、pandas が DataFrame の列をまとめて処理できる利点を活かしにくい。さらに iterrows() は各行を毎回 Series として作り直すため、列ごとの型情報が崩れやすく、オブジェクト生成コストも大きい。行ループが避けられない場合は、itertuples() の方が軽いことが多い。それでも、まず列演算や既存のベクトル化メソッドで表現できないかを確認する。

7.3.3.2 Step 2: apply(axis=1) と列演算

行ごとに処理を適用したい場合、pandas が用意している apply() メソッドを使うと、コードを DataFrame の処理として記述できる。axis=1 は「各行へ関数を適用する」という意味であり、axis=0 なら各列に適用する。

```
1 df["value_sq"] = df.apply(lambda row: heavy_computation(row["value"]), axis=1)
```

ただし、axis=1 を付けた apply() も各行に対して Python 関数を呼ぶため、列演算ほど速いわけではない。つまり、apply() は「for 文を隠した便利関数」であって、万能の高速化ではない。単に二乗したいだけなら、次のように列演算で書く方が自然で速い。

```
1 df["value_sq"] = df["value"] ** 2
```

apply() は「列演算だけでは書きにくい」が、まだ pandas の文脈の中で処理したい」場合の中間手段として理解の方が正確である。まず列演算、次に apply()、それでも重ければ並列化、という順で判断すると迷いにくい。

7.3.3.3 Step 3: 並列化と pandarallel

通常の apply() は、Python 関数を行ごとに順番に呼び出す。本当に計算が重く、列演算や既存の NumPy 関数へ置き換えられない場合に、マルチプロセスによる並列化が選択肢に入る。

```
1 from pandarallel import pandarallel
2
3 pandarallel.initialize(progress_bar=True)
```

```

4 df["value_sq"] = df.parallel_apply(
5     lambda row: heavy_computation(row["value"]),
6     axis=1,
7 )

```

図 7.1 は、同じ種類の処理でも `iterrows()`、`apply()`、`pandarallel` で待ち時間がどう変わるかを模式的に示している。特に、「まず列演算を疑い、その後で `apply()` や並列化を考える」という順序の重要性を読み取ってほしい。

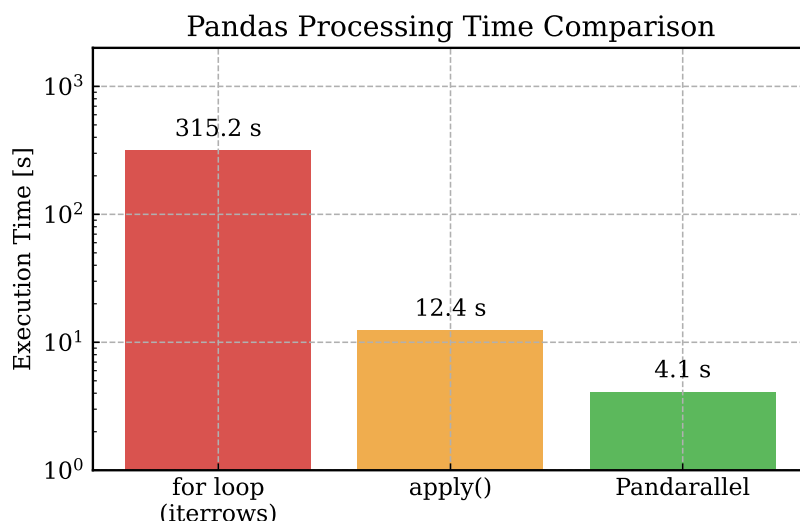


図 7.1 同じ重い計算処理を、Python のループ、`apply()`、`pandarallel` で行ったときの模式的な比較。手段の選び方で処理時間が大きく変わる。

`pandarallel` は、`DataFrame` を複数プロセスへ分割して送り、`apply()` に近い書き方のまま並列化するためのライブラリである。つまり、「既存の `pandas` コードを大きく壊さずに、`parallel_apply()` へ置き換えて試せる」ことが利点である。一方で、背後ではプロセス生成やデータコピーが起きるため、軽い処理ではほとんど得をしない。

`pandarallel` は Linux 系環境での利用が比較的安定している一方で、ネイティブの Windows 環境では設定や挙動で悩みやすいことがある。Windows で本書の流れを再現するなら、前章で触れた WSL を使う方が混乱しにくい。

並列化は魔法の杖ではない。`pandarallel` は背後で「データを複数のプロセスにコピーして分配する」ため、通信オーバーヘッドがかかる。「単純な足し算」などのすぐに終わる軽い処理を `parallel_apply()` にかけても、計算時間よりもコピー時間の方が長くなり、かえって遅くなる。CPU 1 コアでは時間がかかりすぎる重い処理でのみ並列化を採用するという見極めが必要である。

7.3.4 欠損値と比較演算を混同しない

`NaN == NaN` は `False` である。欠損判定に `==` を使わず、`isna()` を使う。また、`None`、`NaN`、`NA` の違いは列型によって見え方が変わるので、「欠損をどう表現しているのか」を確認した

方がよい。

7.4 解析へのつながり

観測データの解析では、pandas はしばしば次の流れの中心になる。

1. テキストや CSV を `read_csv()` で読む
2. `head()` と `dtypes` で構造を確認する
3. 必要な列だけを残し、欠損や異常値を整理する
4. `groupby()` や `merge()` で集計・結合する
5. Matplotlib へ渡して図を描く
6. 集計表を `to_latex()` で書き出す

この流れが成立するのは、pandas が「列名と型を保ったまま処理を書ける」からである。NumPy だけで済む問題もあるが、観測条件、検出器 ID、run 名、欠損フラグのような異種の列が混ざると、DataFrame の方が自然である。

一方で、数千万行規模になり、CSV 読み込みや集計が待ち時間の支配項になってきたら、次章で扱う Polars を検討する価値がある。pandas は基準になる道具であり、その上で「どこから別の道具へ切り替えるべきか」を判断できるようになることが重要である。

また、pandas の多くの操作は、DataFrame が 1 台の計算機のメモリに収まることを前提とする。ただし、`read_csv(chunksize=...)` を使えば、総和や件数のように部分結果を統合できる処理は全件を同時に保持せずに実行できる。必要な表がメモリに収まらない場合は、列と行を早い段階で絞る、分割集計を使う、Parquet に変換する、次章の Polars やデータベースを使う、という順に処理の性質に合う方法を選ぶ。

ここまでの内容を自分で確かめ直したくなったら、公式資料へ戻るとよい。入口としては Getting started tutorials [9] と 10 minutes to pandas [10] が分かりやすい。本章はそれらを解析向けの順序へ並べ替えて説明している。列選択や代入で迷ったときは Indexing [11]、欠損値処理は Missing data [12]、表の結合は Merge, join, concatenate and compare [13]、時刻列は Time series / date functionality [14] を見ると対応する話題に戻りやすい。

7.5 章末問題

次の CSV を `events.csv` として保存し、すべての課題で共通して使う。

events.csv

```
detector,time_ns,signal,valid
A,1000,12.4,true
B,1180,9.8,true
A,1300,15.1,true
```

```
B,1510,,false
A,1760,11.0,true
B,2050,13.2,true
```

1. **読み込み直後の表を検査する。** `pd.read_csv()` で読み、`shape`、`columns`、`dtypes`、`head(3)`、`isna().sum()` を表示せよ。行数は 6、列数は 4、`signal` の欠損数は 1 になる。`head(3)` の 3 と、`isna().sum()` の `sum()` がどの方向に集計しているかを説明せよ。
2. **条件抽出と列追加を行う。** `valid` が真で `signal >= 12.0` の行を `loc` で抽出し、`signal / signal.mean()` を `signal_ratio` 列として追加せよ。抽出行は 3 件になる。複数条件を括弧で囲み、`and` ではなく `&` を使う理由を説明せよ。
3. **検出器ごとに集計して別表と結合する。** `groupby("detector")` と `agg()` を使い、有効な信号の件数、平均、標準偏差を求めよ。平均は A が約 12.8333、B が 11.5 になる。さらに、A と B の校正係数を持つ `DataFrame` を作り、`merge(..., on="detector", how="left")` で結合せよ。`on` と `how` の役割を説明すること。
4. **一括読み込みと分割読み込みを比較する。** 本文の `chunksize=2` の方法で件数と平均を求め、一括読み込みから得た結果と `pd.testing.assert_frame_equal()` または `np.allclose()` で一致を確認せよ。チャンク平均の単純平均が一般には誤る理由を、件数が異なる 2 チャンクの例を使って説明せよ。
5. **結果を再利用できる形式で保存する。** 集計表を CSV と Parquet の両方へ保存し、読み戻して列名と値を確認せよ。`index=False` を付けた場合に何が保存されないかを説明し、Parquet の読み込みに追加ライブラリが必要な環境では、表示された例外名も記録せよ。

`pandas` で表処理の基本と結果の検証方法を確認した後は、同じ処理を別の `DataFrame` 実装で書くと、設計上の違いが明確になる。次章では `Polars` を用い、`index` に依存しない表、列式、遅延実行による処理を扱う。

第 8 章

Polars による高速な表処理

Polars は、列指向の DataFrame 処理を高速に行うためのライブラリである。pandas と同じく表形式データを扱うが、index を持たず、式を組み合わせて列変換を書く点が異なる。さらに、LazyFrame では、実行前にクエリ全体を最適化できる。本章では、Polars の考え方、最小限の書き方、落とし穴、解析で使う場面を整理する。

8.1 Polars の考え方

8.1.1 index を持たない DataFrame

Polars の DataFrame は、常に「行番号は整数位置でだけ参照する 2 次元表」である。pandas のような index に意味を持たせないため、reset_index() や loc を前提にした設計にはならない。

```
1 import polars as pl
2
3 df = pl.DataFrame(
4     {
5         "detector": ["A", "A", "B", "B"],
6         "time_ns": [1032, 1188, 1055, 1244],
7         "signal": [12.4, 10.1, 14.8, 9.7],
8     }
9 )
10
11 print(df)
```

このコードを実行すると、Polars の表は次のように表示される。

```
shape: (4, 3)
+-----+-----+-----+
| detector | time_ns | signal |
```

---	---	---
str	i64	f64
A	1032	12.4
A	1188	10.1
B	1055	14.8
B	1244	9.7

この出力でも、重要なのは `shape: (4, 3)` と列名である。Polars は「何行目を label で指すか」よりも、「どの列にどの式を適用するか」を中心に据えている。

8.1.2 式で列変換を書く

Polars の中心概念は `expression` である。`pl.col("signal")` は「`signal` 列を参照する式」であり、これを四則演算や比較で組み合わせる。

```
1 expr = pl.col("signal") / pl.col("signal").mean()
2 print(expr)
```

表示されるのは、概ね `[(col("signal")) / (col("signal").mean())]` のような式であり、4 行分の数値ではない。この段階では、まだ計算結果ではなく「どう計算するか」を表している。実際の値を得るには、`df.select(expr.alias("signal_ratio"))` のように `DataFrame` の文脈へ式を渡す。Polars の公式ドキュメント [17] でも、まず `expression` を理解することが最重要とされている。`select()`、`with_columns()`、`filter()`、`group_by()` の多くは、式を受け取る API で統一されている。

8.1.3 eager と lazy

Polars には、すぐ実行する `eager API` と、最後まで式をためて最適化してから実行する `lazy API` がある。

```
1 eager_df = pl.read_csv("events.csv")
2 lazy_df = pl.scan_csv("events.csv")
```

`read_csv()` はすぐに `DataFrame` を作るが、`scan_csv()` は `LazyFrame` を返す。`lazy API` を使うと、必要な列だけ読む、不要な行を早い段階で落とす、といった最適化が入りやすい。巨大なログを扱うときは、この違いがそのまま待ち時間の差になる。

8.2 最小限のコード例

8.2.1 `select()` と `filter()`

Polars では、列選択は `select()`、行抽出は `filter()` が基本である。

```
1 selected = df.select("detector", "signal")
2 strong = df.filter(pl.col("signal") > 11.0)
3
4 print(selected)
5 print(strong)
```

`select()` で列が減り、`filter()` で条件に合う行だけが残る。出力は次の通りである。

```
shape: (4, 2)
+-----+-----+
| detector | signal |
+=====+=====+
| A        | 12.4   |
| A        | 10.1   |
| B        | 14.8   |
| B        | 9.7    |
+-----+-----+
shape: (2, 3)
+-----+-----+-----+
| detector | time_ns | signal |
+=====+=====+=====+
| A        | 1032    | 12.4   |
| B        | 1055    | 14.8   |
+-----+-----+-----+
```

`df["signal"]` のような書き方もできるが、Polars では「表に対する操作」を `select()` や `filter()` の文脈で書く方が自然である。特に複数列をまとめて変換したいときは、この違いが大きく効く。

8.2.2 `with_columns()` で列を増やす

新しい列は `with_columns()` で追加する。複数列を同時に書けるので、Polars の式ベース設計と相性が良い。

```
1 derived = df.with_columns(
2     (pl.col("signal") / pl.col("signal").mean()).alias("signal_ratio"),
3     (pl.col("time_ns") > 1100).alias("is_late"),
4 )
5
6 print(derived)
```

列追加後の表を見ると、式で作った列がどの位置へ入るかを確認できる。

```
shape: (4, 5)
+-----+-----+-----+-----+-----+
| detector | time_ns | signal | signal_ratio | is_late |
+=====+=====+=====+=====+=====+
| A        | 1032    | 12.4   | 1.055319     | false   |
| A        | 1188    | 10.1   | 0.859574     | true    |
| B        | 1055    | 14.8   | 1.259574     | false   |
| B        | 1244    | 9.7    | 0.825532     | true    |
+-----+-----+-----+-----+-----+
```

`alias()` は列名を付ける操作である。Polars では式の結果へ名前を付けるという発想が頻繁に現れるので、早めに慣れておく方がよい。

8.2.3 `group_by()` と集計

集計も式として書く。`group_by()` と `agg()` は、pandas の `groupby()` に対応する中心的な道具である。

```
1 summary = (
2     derived
3     .group_by("detector")
4     .agg(
5         pl.len().alias("count"),
6         pl.col("signal").mean().alias("mean_signal"),
7         pl.col("signal").std().alias("std_signal"),
8         pl.col("is_late").mean().alias("late_fraction"),
9     )
10    .sort("detector")
11 )
12
13 print(summary)
```

集計後は、検出器ごとに 1 行ずつの表へまとまる。出力は次の通りである。

```
shape: (2, 5)
+-----+-----+-----+-----+-----+
| detector | count | mean_signal | std_signal | late_fraction |
+=====+=====+=====+=====+=====+
| A        | 2     | 11.25       | 1.626346   | 0.5           |
| B        | 2     | 12.25       | 3.606245   | 0.5           |
+-----+-----+-----+-----+-----+
```

Polars の `group_by()` は、集計対象が `expression` なので、同じ書き方のまま統計量を増やしやすい。`count`、`mean`、`std` のような基本操作は、式の名前を見れば意味を追いやすい。

8.2.4 join と表の変形

別表の校正係数や run 情報を結び付けるなら `join()` を使う。縦長と横長の変換も行える。

```

1 gain = pl.DataFrame(
2     {
3         "detector": ["A", "B"],
4         "gain": [1.02, 0.97],
5     }
6 )
7
8 merged = derived.join(gain, on="detector", how="left")
9 wide = (
10     pl.DataFrame(
11         {
12             "detector": ["A", "A", "B", "B"],
13             "kind": ["raw", "cal", "raw", "cal"],
14             "value": [12.4, 12.7, 14.8, 14.4],
15         }
16     )
17     .pivot(index="detector", on="kind", values="value")
18     .sort("detector")
19 )
20
21 print(merged.select("detector", "signal", "gain"))
22 print(wide)

```

2 つの出力は次のようになる。

```

shape: (4, 3)
+-----+-----+-----+
| detector | signal | gain |
+=====+=====+=====+
| A        | 12.4   | 1.02 |
| A        | 10.1   | 1.02 |
| B        | 14.8   | 0.97 |
| B        | 9.7    | 0.97 |
+-----+-----+-----+

shape: (2, 3)
+-----+-----+-----+
| detector | raw  | cal  |
+=====+=====+=====+
| A        | 12.4 | 12.7 |
| B        | 14.8 | 14.4 |
+-----+-----+-----+

```

`join()` の `on="detector"` は両方の表で照合するキー列を指定し、`how="left"` は左側の `derived` の全行を残す結合方法を指定する。`pivot()` では、`index="detector"` が出力表で行を識別する列、`on="kind"` が新しい列名になる値、`values="value"` がセルへ入る値を表す。

Polars には pandas のような index がないため、結合や変形に使う列を引数で明示する。

8.2.5 欠損値、null、NaN

Polars は欠損値の扱いが比較的厳密である。null と NaN を区別して考える。

```

1 missing = pl.DataFrame(
2     {
3         "signal": [12.4, None, 14.8, float("nan")],
4     }
5 )
6
7 print(missing.with_columns(
8     pl.col("signal").is_null().alias("is_null"),
9     pl.col("signal").is_nan().alias("is_nan"),
10 ))

```

判定結果のうち、主要部分は次のようになる。

signal	is_null	is_nan
12.4	false	false
null	true	null
14.8	false	false
NaN	false	true

null は値が存在しないこと、NaN は浮動小数点の特殊値である。is_nan() へ null を渡した結果も null になる点に注意する。両方を除くなら、例えば pl.col("signal").is_not_null() & pl.col("signal").is_finite() をフィルタ条件にする。どちらを落とすかによって使う式が変わる。

8.2.6 文字列と条件分岐

式ベースの API では、文字列処理や条件分岐も列全体に対する式として書く。

```

1 names = pl.DataFrame(
2     {
3         "run_name": ["run_A_001", "run_A_002", "test_B_001"],
4         "signal": [12.4, 10.1, 14.8],
5     }
6 )
7
8 classified = names.with_columns(
9     pl.col("run_name").str.extract(r"^(run_[A-Z])", 1).alias("prefix"),
10    pl.when(pl.col("signal") > 12.0)
11        .then(pl.lit("high"))
12        .otherwise(pl.lit("normal"))
13        .alias("quality"),
14 )
15

```

```
16 print(classified)
```

追加した 2 列を含む結果は次の通りである。

run_name	signal	prefix	quality
run_A_001	12.4	run_A	high
run_A_002	10.1	run_A	normal
test_B_001	14.8	null	high

`str.extract()` の第 1 引数は正規表現、第 2 引数の 1 は取り出すキャプチャグループの番号である。一致しない `test_B_001` の `prefix` は `null` になる。`when`、`then`、`otherwise` は、条件、真の場合の値、偽の場合の値を順に組み立てる。

8.2.7 lazy API と `scan_parquet()`

大きなデータでは、eager API より lazy API の方が自然である。

```
1 lf = (
2     pl.scan_parquet("events/*.parquet")
3     .filter(pl.col("signal") > 0.0)
4     .select("detector", "signal", "time_ns")
5     .with_columns((pl.col("time_ns") - pl.col("time_ns").min()).alias("dt_ns"))
6     .group_by("detector")
7     .agg(pl.col("signal").mean().alias("mean_signal"))
8 )
9
10 result = lf.collect()
11 print(result)
```

`scan_parquet()` の引数 `"events/*.parquet"` は、一致する複数の Parquet ファイルを入力にする。戻り値は `LazyFrame` であり、この時点ではデータを実体化しない。`collect()` を呼ぶとクエリが実行され、戻り値として `DataFrame` が得られる。実行時には、`select()` で使う列だけを読む列射影や、`filter()` を早い段階へ移す最適化が適用されることがある。最適化内容を確認したい場合は、実行前に `print(lf.explain())` として計画を表示できる。

8.2.8 CSV と Parquet の入出力

Polars でも CSV と Parquet は基本である。

```
1 df = pl.read_csv("events.csv")
2 df.write_parquet("events.parquet")
3
4 lf = pl.scan_csv("events.csv")
5 result = lf.filter(pl.col("signal") > 0).collect()
6
7 print(type(df).__name__)
8 print(type(lf).__name__)
9 print(type(result).__name__)
```

表示される型名は順に `DataFrame`、`LazyFrame`、`DataFrame` である。`read_csv()` はその場で読み込み、`scan_csv()` は読み込み計画を作り、`collect()` が計画を実行する。`write_parquet()` の第 1 引数は保存先であり、同名のファイルがある場合は上書きされるため、保存先を確認してから実行する。

`read_csv()` と `write_parquet()` は eager API、`scan_csv()` は lazy API の入口である。`pandas` と同じ名前関数もあるが、遅延実行では処理を式として蓄積してから `collect()` で評価する点異なる。

8.3 よくある落とし穴

8.3.1 `pandas` の書き方をそのまま持ち込まない

Polars 公式の migration guide [20] でも強調されているが、Polars は `pandas` ではない。`pandas` に似た書き方が一部動いても、それが最も自然な書き方とは限らない。特に、`index` 前提の考え方、`loc`、`apply(axis=1)` へ寄った設計は Polars では合わない。

8.3.2 `map_elements()` や Python の `lambda` を多用しない

Polars の利点は、式が query optimizer に見えることにある。Python の `lambda` や `map_elements()` へ逃げると、その最適化が効きにくくなる。まずは `pl.col()`、`when`、`str`、`dt`、`over` などの組み込み式で書けないかを考える方がよい。

8.3.3 lazy API では `collect()` を忘れない

`scan_csv()` や `scan_parquet()` は `LazyFrame` を返す。ここで `print(lf)` としても、実データではなく計画しか見えない。結果が欲しいなら `collect()` が必要である。

8.3.4 欠損値は `null` と `NaN` を区別する

Polars は型に厳密である。欠損をまとめて扱っているつもりでも、実際には `null` と `NaN` が混ざっていることがある。フィルタや集計の前に、どちらが入っているかを確認した方が安全である。

8.4 解析へのつながり

Polars が特に有効なのは、次のような場面である。

- 数千万行の CSV や Parquet を条件付きで読みたいとき

- 同じ表に対する列変換と集計を一つの query としてまとめたいとき
- pandas では待ち時間が長く、入出力と集計が支配的になっているとき
- 複数ファイルをまとめて scan し、必要な列だけを残したいとき

一方で、小規模データで index を活かした対話的操作や、既存の pandas ベース資産をそのまま流用したいなら、pandas の方が自然なことも多い。大切なのは「pandas を捨てる」ことではなく、「どの規模、どの処理で Polars へ切り替えるべきか」を判断できるようになることである。

Polars の公式資料では、Concepts [16]、Expressions and contexts [17]、Lazy API [18]、IO [19]、Coming from Pandas [20] が土台になる。本章もその流れを踏まえているが、解析で必要な判断が先に見えるように並べ替えてある。式の書き方で迷ったときは expression、遅い処理をまとめて最適化したいときは lazy API、pandas との違いを整理したいときは migration guide を見返すとよい。

8.5 章末問題

前章の events.csv を使う。ファイルがない場合は、第 7 章の章末問題に示した 6 行の CSV を作成する。

1. **式を組み立てて列と行を選ぶ。** `pl.read_csv()` で表を読み、`select()` で `detector` と `signal` だけを選べ。次に、`signal` が `null` でなく 12.0 以上である行を `filter()` せよ。結果は A が 2 行、B が 1 行の計 3 行になる。`pl.col()` が値ではなく式を返すことと、`is_not_null()` を条件に明示する理由を説明せよ。
2. **列追加と集計を一つのクエリにする。** `with_columns()` で各検出器の平均からの差 `signal_centered` を作る。この計算には `pl.col("signal").mean().over("detector")` を使う。その後、`group_by()` と `agg()` で件数、平均、最大値を求め、検出器名で並べ替えよ。`over("detector")` と `group_by("detector")` の違いを、出力行数に着目して説明せよ。
3. **lazy API の計画と結果を分けて確認する。** `pl.scan_csv("events.csv")` から始め、必要な 2 列の選択、欠損除去、検出器別平均を記述せよ。`collect()` の前に `explain()` を表示し、その後に `collect()` した `DataFrame` を表示する。`LazyFrame` と `DataFrame` のどちらが実データを保持するかを説明せよ。
4. **結合と縦横変換を確認する。** 検出器と校正係数を持つ表を作り、`join(..., on="detector", how="left")` で結合して校正後信号を計算せよ。さらに、検出器を行、`valid` を列、件数を値とする表を `group_by()` と `pivot()` で作れ。結合キーが重複している場合に出力行数が増える理由も説明すること。
5. **pandas と結果を照合する。** 前章の検出器別集計と同じ統計量を Polars で求め、CSV に保存して pandas で読み戻せ。平均値が一致することを許容誤差付きで確認し、処理速度だけでなく、欠損値、列型、行順序も比較項目になる理由を述べよ。

pandas と Polars で整形・集計した表は、数値を並べただけでは分布、外れ値、変数間の関係を把握しにくい。次章では、それらの列を Matplotlib へ渡し、解析途中の検証図と最終的な説明図を作成する。

第 9 章

Matplotlib による可視化

Matplotlib は、Python で図を描くための代表的なライブラリである。本章では、最小限の作図コードを確認した後、データの性質に応じた図の選び方を扱う。後半では、複数のパネルや構成要素を制御するために、オブジェクト指向 API、Figure、Axes、Axis、Artist の関係とスタイルの管理方法を説明する。

図はレポートの飾りではない。ヒストグラムや散布図は、計算結果の妥当性を自分で確かめるための道具でもある。数値だけを並べるより、まず図にした方が異常に気が付きやすいことは多い。

例えば、平均値や標準偏差が妥当に見えても、分布の裾が長い、二峰性がある、外れ値が混ざっている、といった特徴は図にしないと分かりにくい。可視化は最後の仕上げではなく、途中の確認作業としても重要である。

9.1 基本的な作図と可視化

9.1.1 最小限のヒストグラム

最初に、正規乱数を 2500 個生成し、40 本のビンへ数え上げて PDF に保存する。入力値は `values`、作図処理は `plt.hist()`、出力先は `histogram.pdf` である。

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 values = np.random.normal(loc=0.0, scale=1.2, size=2500)
5
6 plt.hist(values, bins=40)
7 plt.xlabel("value")
8 plt.ylabel("count")
9 plt.savefig("histogram.pdf")
```

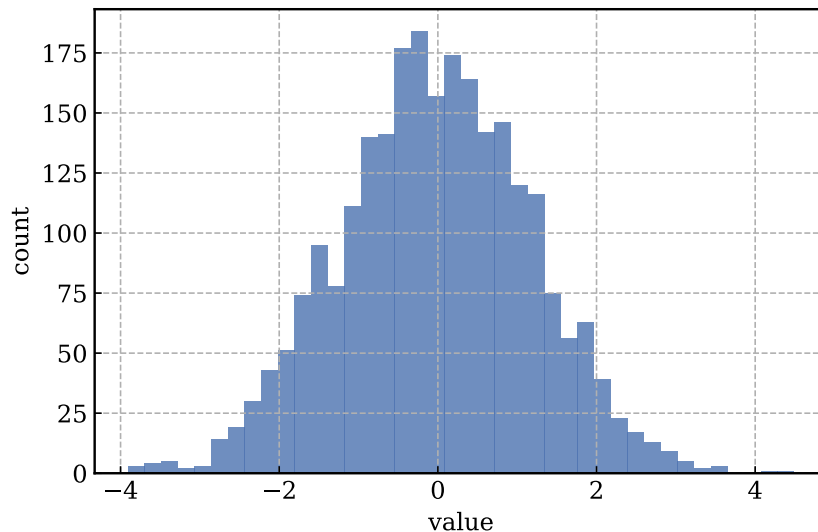


図 9.1 最小限のヒストグラム出力例。最初は、入力配列 `values` が何本の棒へ数え上げられているかだけを追えばよい。

解析では PDF 出力が便利である。拡大しても崩れにくく、LaTeX レポートへそのまま入れやすいからである。図 9.1 は、この最小例をそのまま実行したときの典型的な出力である。

`plt.show` による対話的な確認は便利だが、再現可能な解析では `savefig` でファイルへ残す方が重要になる。図を保存するたびに同じファイル名へ上書きするのか、条件ごとに別名で保存するのかも、早い段階で決めておくと混乱しにくい。

9.1.2 軸ラベル、凡例、目盛り

図として最低限必要なのは、軸ラベル、必要なら単位、系列名、そして比較に必要な目盛りである。ここを省くと、図が描けても読めない。

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 x = np.linspace(0.0, 10.0, 60)
5 y = np.exp(-x / 4.0) * (1.0 + 0.08 * np.sin(2.5 * x))
6
7 fig, ax = plt.subplots(figsize=(6, 4))
8 ax.plot(x, y, marker="o", label="sample A")
9 ax.set_xlabel(r"$t$, [\mathrm{s}]$")
10 ax.set_ylabel("signal")
11 ax.set_title("Labeled line example")
12 ax.set_xticks([0, 2, 4, 6, 8, 10])
13 ax.legend(loc="upper right")
14 fig.savefig("labeled_plot_example.pdf")

```

図 9.2 のように、ラベルと凡例と目盛りがそろっただけで、何を見せたい図なのかが一気に分かりやすくなる。最初は配色に凝るより、まずラベルと凡例を外さない方が重要である。

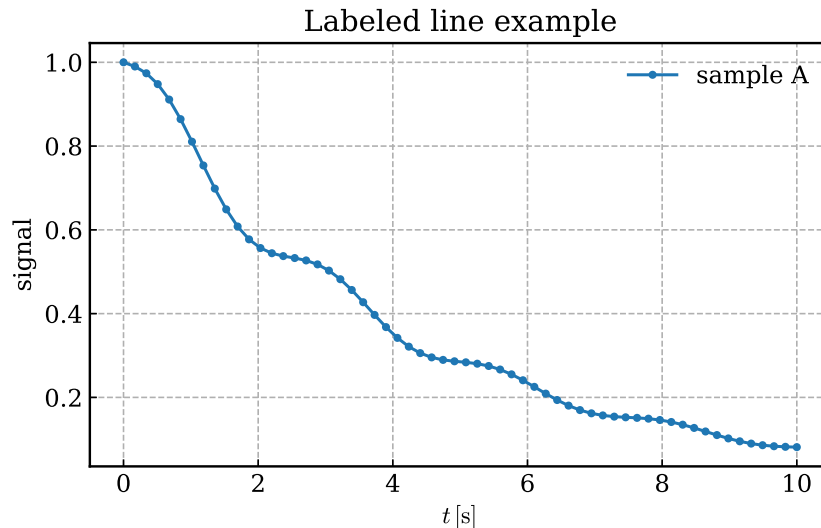


図 9.2 ラベル、凡例、目盛りを付けた最小限の線図。横軸の単位と系列名があるだけで読みやすさが大きく変わる。

9.1.3 plot 関数へ渡すデータ

Matplotlib の多くの作図関数は、NumPy 配列や Python リストのような「配列に変換できるもの」を受け取る。加えて、辞書や DataFrame のように列名で参照できるオブジェクトなら、`data=` を使って列名指定で描くこともできる。

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 times = [0, 1, 2, 3]
5 values = [1.0, 2.0, 1.5, 3.0]
6
7 fig, ax = plt.subplots(figsize=(6, 4))
8 x = np.asarray(times)
9 y = np.asarray(values)
10 ax.plot(x, y)
11
12 table = {"time": x, "value": y}
13 ax.plot("time", "value", data=table)
```

`np.asarray` で NumPy 配列へそろえておくと、型のぶれを減らせる。`data=` は短いコードを書くのに便利だが、列名の打ち間違いに気がつきにくいこともあるので、最初は `x` と `y` を明示して渡す方が安全である。

9.1.4 最初に覚える主要な描画関数

Matplotlib を使い始めた段階で、まず頻繁に使うのは、線図を描く `plot()`、散布図を描く `scatter()`、参照線を引く `axvline()` と `axhline()`、誤差付きの点を描く `errorbar()` である。

これらをまとめて見ておくと、「何をしたいときにどの関数を選ぶか」が整理しやすい。

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 rng = np.random.default_rng(1234)
5 t = np.linspace(0.0, 10.0, 80)
6 y = np.sin(t) * np.exp(-t / 12.0)
7 y_noise = y + 0.12 * rng.normal(size=t.size)
8 y_err = 0.10 + 0.03 * np.cos(t / 2.0) ** 2
9
10 fig, axes = plt.subplots(2, 2, figsize=(10, 7))
11
12 axes[0, 0].plot(t, y, marker="o")
13 axes[0, 1].scatter(t, y_noise, c=t, cmap="viridis")
14 axes[1, 0].plot(t, y_noise)
15 axes[1, 0].axvline(3.0, linestyle="--")
16 axes[1, 0].axhline(0.0, linestyle=":")
17 axes[1, 1].errorbar(t[:4], y_noise[:4], yerr=y_err[:4], fmt="o")
18
19 fig.savefig("basic_plot_types_example.pdf")

```

図 9.3 は、この最小例で使った主要な描画関数を並べたものである。線図は時系列や関数の形を見るときに、散布図は 2 変量の相関やばらつきを見るときに、参照線はしきい値や基準値を示すときに、エラーバーは測定誤差を明示したいときに使う。

`plot()` は既定で点を線で結ぶため、測定点そのものを見せたいのか、連続関数の形を見せたいのかを意識して使う必要がある。対して `scatter()` は点同士を結ばないので、イベント列やサンプル点をそのまま表示したい場合に向く。縦線や横線は、図に補助的な意味を足すためのものであり、基準値を視覚的に共有する手段と考えるとよい。

9.1.5 ヒートマップとカラーバー

2 次元配列や格子状のデータを表示するには、ヒートマップが自然である。Matplotlib では、配列をそのまま色で表示する `imshow()`、散布点の数密度を色へ変換する `hist2d()` がよく使われる。色の意味を図へ明示するには、カラーバーも同時に付ける。

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from matplotlib.colors import LogNorm
4
5 x = np.linspace(-3.0, 3.0, 160)
6 y = np.linspace(-2.5, 2.5, 140)
7 xx, yy = np.meshgrid(x, y)
8 image = np.exp(-(xx**2 + 1.6 * yy**2)) + 0.02
9 rng = np.random.default_rng(1234)
10 sample_x = rng.normal(loc=0.0, scale=1.0, size=80000)
11 sample_y = 0.7 * sample_x + rng.normal(loc=0.0, scale=0.8, size=80000)

```

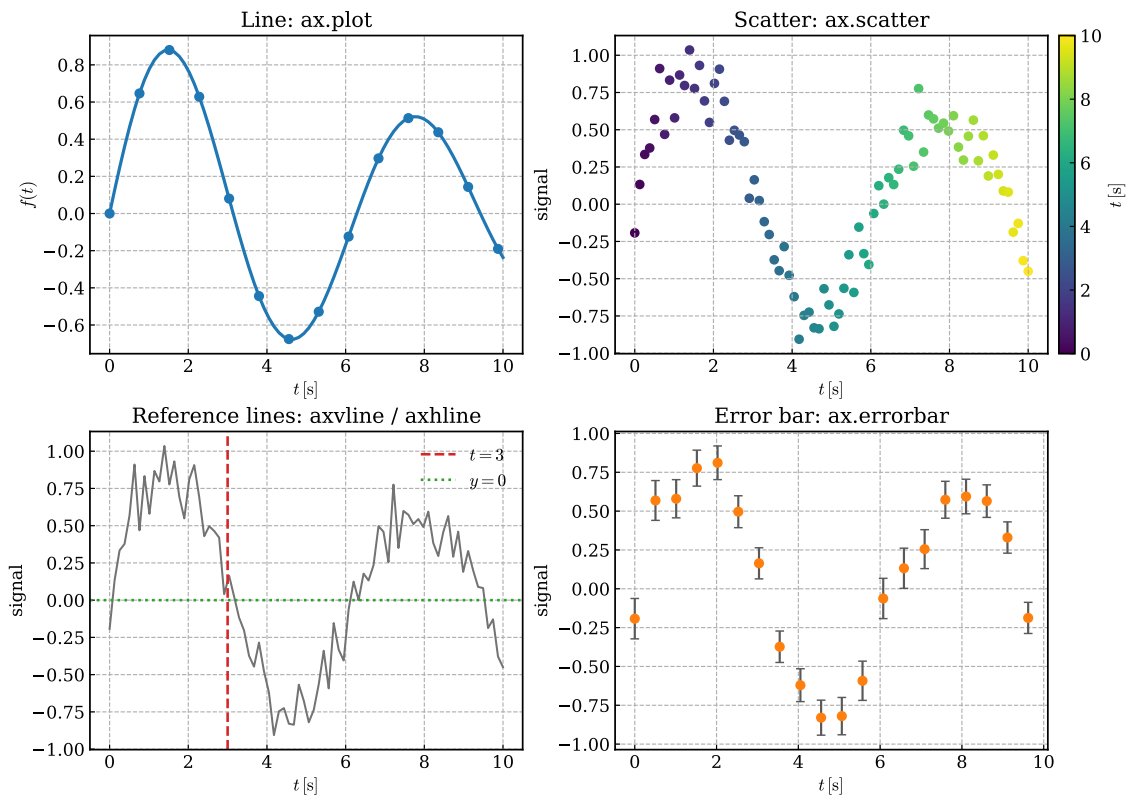


図 9.3 Matplotlib で最初に覚えないといけない主要な描画関数。左上が `plot()`、右上が `scatter()`、左下が `axvline()` と `axhline()`、右下が `errorbar()` の例である。

```

12
13 fig, axes = plt.subplots(1, 2, figsize=(10, 4))
14 im = axes[0].imshow(image, origin="lower", cmap="viridis", aspect="auto")
15 fig.colorbar(im, ax=axes[0], label="intensity")
16
17 h = axes[1].hist2d(sample_x, sample_y, bins=90, norm=LogNorm(), cmap="magma")
18 fig.colorbar(h[3], ax=axes[1], label="counts")

```

図 9.4 の左は `imshow()` によるヒートマップ、右は `hist2d()` と `LogNorm` を使った 2 次元ヒストグラムである。ここで重要なのは、`cmap=` で色の割り当てを決め、`fig.colorbar(...)` でその色が何を意味するかを図の中へ戻している点である。

`imshow()` は「すでに格子上へ並んだ値」を描く関数であり、シミュレーション結果や画像データと相性がよい。一方、`hist2d()` は「点群から格子へ数え上げた結果」を描く関数である。点の密度が数桁にまたがるなら、右図のように `LogNorm()` を使ってカラーバーを対数にする方が内部構造を見失いにくい。

9.1.6 全天図と投影: Hammer 図法と Mollweide 図法

経度と緯度のような球面上のデータを平面へ描くには、投影法が必要になる。Matplotlib には、地図ライブラリなしでも使える全天図向けの投影として、Hammer 図法や Mollweide 図法が用意されている。

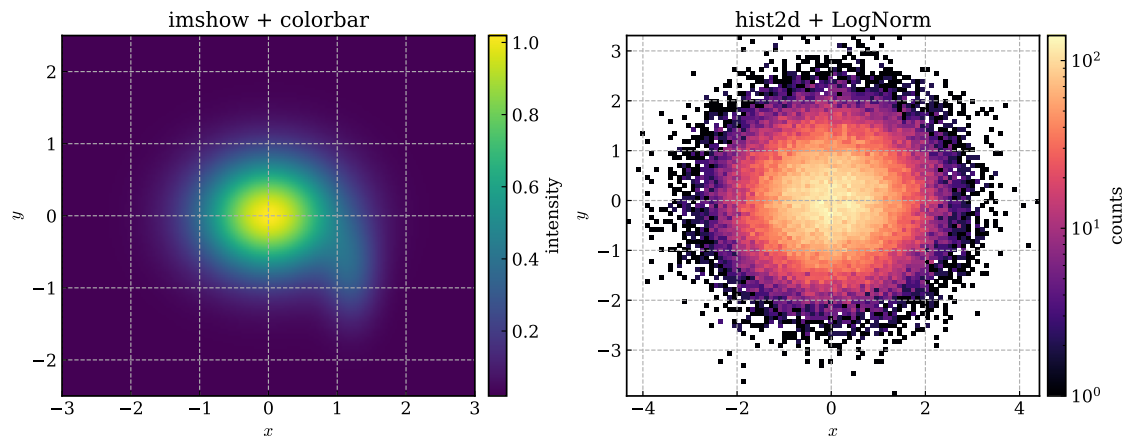


図 9.4 ヒートマップとカラーバーの例。左は `imshow()`、右は `hist2d()` と `LogNorm` を使った例であり、どちらもカラーバーによって色の意味を示している。

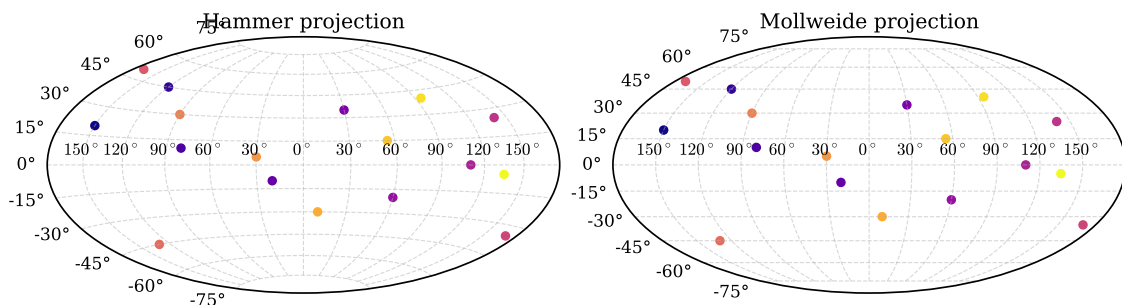


図 9.5 Hammer 図法と Mollweide 図法の例。どちらも球面全体を 1 枚の図へ収めるための投影であり、全天分布を俯瞰したいときに使いやすい。

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 lon_deg = np.linspace(-180.0, 180.0, 241)
5 lat_deg = 25.0 * np.sin(np.deg2rad(2.0 * lon_deg))
6 lon = np.deg2rad(lon_deg)
7 lat = np.deg2rad(lat_deg)
8
9 fig = plt.figure(figsize=(10, 4))
10 ax1 = fig.add_subplot(1, 2, 1, projection="hammer")
11 ax2 = fig.add_subplot(1, 2, 2, projection="mollweide")
12
13 ax1.scatter(lon, lat)
14 ax2.scatter(lon, lat)

```

図 9.5 は、同じ点群を Hammer 図法と Mollweide 図法へ載せた例である。どちらも全周を 1 枚へ収めやすく、全天イベント分布や銀河座標図のような用途で使われる。注意すべき点は、`projection="hammer"` や `projection="mollweide"` を使う場合、角度は通常ラジアンで渡す必要があることである。

地図上の国境や海岸線を描く用途なら `Cartopy` の方が自然だが、球面上の分布そのものを俯瞰したいだけなら、`Matplotlib` の標準投影で十分なことも多い。座標系が度なのかラジアン

ンなのか、周期境界をまたぐかどうかを最初に確認する方が、投影名を覚えるより重要である。

9.2 実践的な作図（データに応じた図の選択）

2 変数の関係を調べるときに、データ量を確認せず散布図を描くだけでは、点の重なりによって密度構造を見落とす場合がある。ここでは、データ点数と確認したい性質に応じて、散布図、透過表示、2次元ヒストグラムを選ぶ手順を説明する。

9.2.1 相関の可視化：論理的改善ステップ

9.2.1.1 Step 1: 直感的な散布図 (Scatter) の限界

2 変数の相関を見るのに最も基本となるのが散布図 (scatter) である。しかし、データ点が数十万規模になると、これはすぐに「役に立たない図」と化す。

```
1 # s=10 で大量の点（約60万点）をそのまま描画すると...
2 fig, ax = plt.subplots(figsize=(6, 5))
3 ax.scatter(x, y, s=10, c='black')
4 fig.savefig("scatter_bad.png", dpi=150)
```

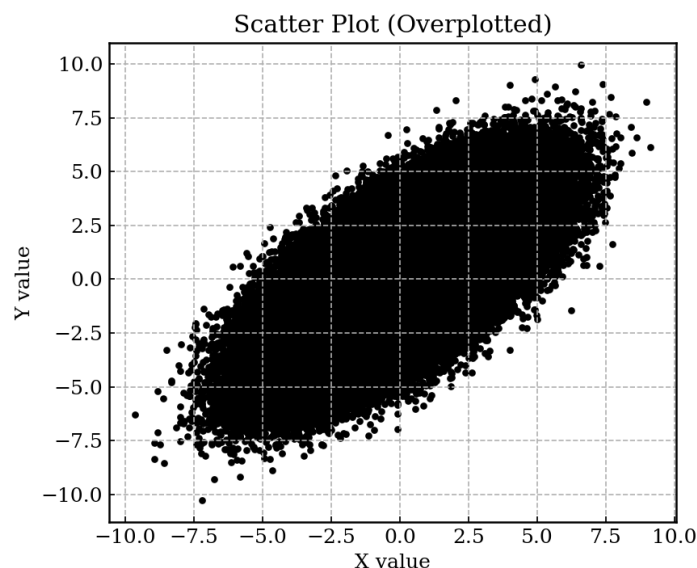


図 9.6 大量データに対する散布図。全てが黒く沈んでおり内部構造が一切見えない。

図 9.6 のような図を作ってしまったてはならない。データがどこにどれだけ集中しているのか（密度）が全く分からないからである。

9.2.1.2 Step 2: 2次元ヒストグラムの導入と、線形スケールの限界

点が潰れるなら、「その領域にいくつ点があるか」を色で表現する 2 次元ヒストグラム (hist2d) が有効である。

```
1 fig, ax = plt.subplots(figsize=(6, 5))
2 h_lin = ax.hist2d(x, y, bins=50, cmap='viridis')
3 cbar = fig.colorbar(h_lin[3], ax=ax, label="Counts (Linear)")
4 fig.savefig("hist2d_linear_bad.pdf")
```

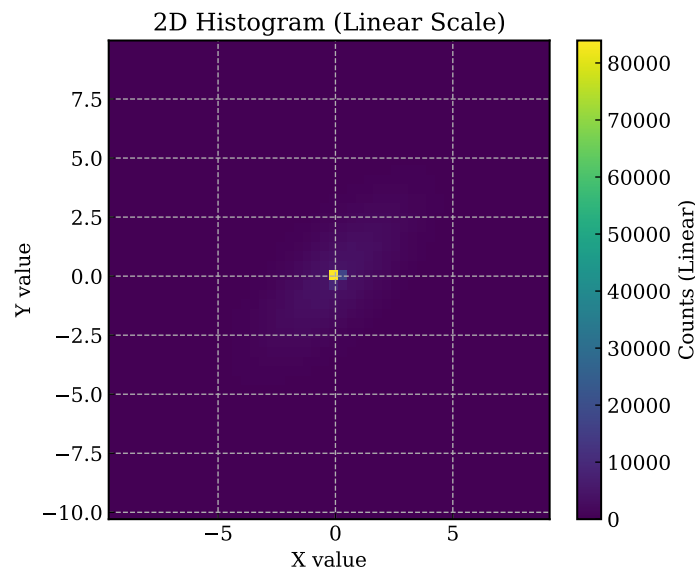


図 9.7 線形スケールを用いた 2D ヒストグラム。最も集中した中心部だけが目立ち、周辺が潰れる。

図 9.7 は中心部が明るくなっているため、散布図よりはマシに見える。しかし、極端に集中している一部（例: 2 万カウント）の色に引っ張られ、10 カウント、100 カウントといった周囲のノイズや裾野の広がりやすべて濃紺に塗りつぶされてしまっている。

9.2.1.3 Step 3: 大規模データに必須となる LogNorm (対数カラースケール)

現実の物理イベントは、頻度が「1, 10, 100, 1000」のように桁単位で変化することが多い。この何桁にもまたがる広がりをもつデータを一つの図に収めるための判断基準が「カウントのカラーバーを対数 (log) にするべきか否か」である。

```
1 import matplotlib.colors as mcolors
2
3 fig, ax = plt.subplots(figsize=(6, 5))
4 # norm=mcolors.LogNorm() により色の割当を対数スケールにする
5 h_log = ax.hist2d(x, y, bins=50, norm=mcolors.LogNorm(), cmap='viridis')
6 cbar = fig.colorbar(h_log[3], ax=ax, label="Counts (Log Scale)")
7 fig.savefig("hist2d_lognorm_good.pdf")
```

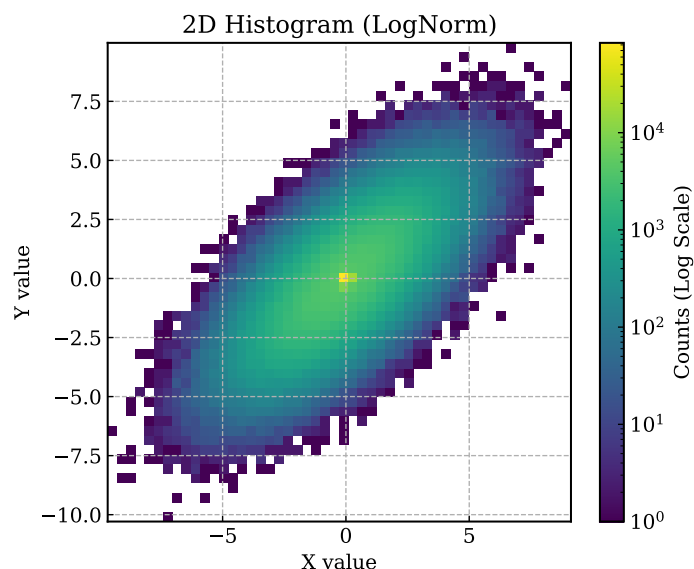


図 9.8 LogNorm を用いた 2D ヒストグラム。中心の密集部分と、周囲の広がりが同時に視認できる。

図 9.8 を見れば、単一のピークの周りに広く緩やかなイベントの相関層が広がっていることが一目で分かる。これが「なぜ LogNorm を知っていなければならないか」の強力な理由である。

9.2.2 複数の図を並べる (subplots)

レポートや論文では、関連する分布を左右や上下に並べて比較することが多い。この場合、1 枚のキャンバス (Figure) の中に複数のパネル (Axes) を配置する機能を利用する。

```

1 # 1行2列（左右に2つ）のパネルを作成する。
2 # fig はキャンバス全体、axes は2つのパネル（配列）を表す。
3 fig, axes = plt.subplots(1, 2, figsize=(10, 4))
4
5 # 左のパネルには axes[0] を使って描画する
6 axes[0].hist(data_a, bins=30)
7 axes[0].set_title("Data A")
8
9 # 右のパネルには axes[1] を使って描画する
10 axes[1].hist(data_b, bins=30)
11 axes[1].set_title("Data B")
12
13 # 複数のパネルを描くと軸の数字やラベルが隣の図と重なりやすいため、
14 # 最後に必ず tight_layout() を呼んで余白を自動調整する。
15 fig.tight_layout()
16 fig.savefig("subplots_example.pdf")

```

単純な `plt.plot()` ばかり使っていると、この「特定のパネル (`axes[0]`) に対して描画を命令する」という書き方に慣れるのに少し時間がかかるが、複雑な比較図を作る上では避けて通れない技法である。Figure や Axes についての詳細は、第 9.3 節以降で述べる。

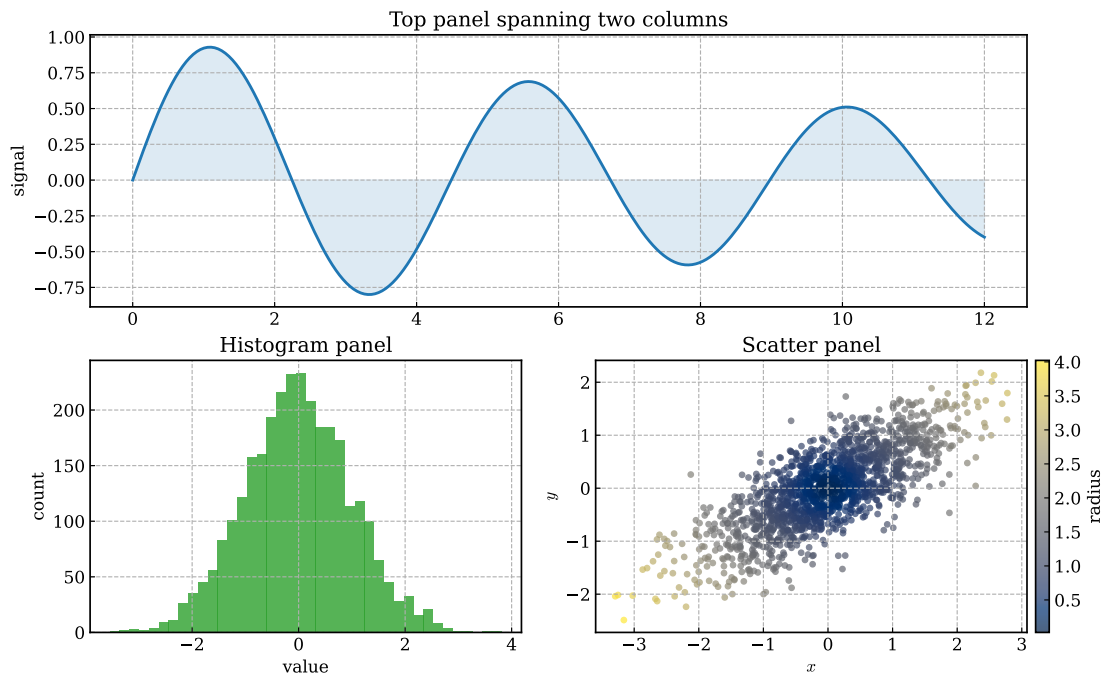


図 9.9 `subplot_mosaic()` を使った複合レイアウトの例。上段は 2 列をまたぐ主図、下段は補助図 2 枚であり、単純な `plt.subplots(1, 2)` では作れない配置になっている。

9.2.3 単純な横並び以外: `subplot_mosaic` と `GridSpec`

実際の解析では、すべてのパネルを同じ大きさで並べたいとは限らない。例えば、上段に主結果を大きく 1 枚置き、下段に補助図を 2 枚並べたい場合がある。このようなレイアウトでは、`subplot_mosaic()` や `GridSpec` が役立つ。

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 values = np.random.default_rng(42).normal(size=2000)
5 x = np.linspace(0.0, 10.0, 150)
6 y_top = np.exp(-x / 5.0) * np.cos(2.0 * x)
7 x_sc = np.linspace(0.0, 1.0, 25)
8 y_sc = x_sc**2 + 0.05 * np.sin(20.0 * x_sc)
9
10 layout = [["A", "A"], ["B", "C"]]
11 fig, axes = plt.subplot_mosaic(layout, figsize=(10, 6))
12
13 axes["A"].plot(x, y_top)
14 axes["B"].hist(values, bins=40)
15 axes["C"].scatter(x_sc, y_sc)

```

図 9.9 のように、"A" を 2 回書くと上段全体を 1 枚の大きな Axes として使える。これは、主図と補助図の役割が異なるときに便利である。左右に同じ大きさで並べるだけが subplot ではない、という点を早めに押さえておくと、比較図の自由度が上がる。

`subplot_mosaic()` は名前付きで Axes を取り出せる点が読みやすく、比較的小規模な複雑

レイアウトに向く。一方、列幅や行高を数値で細かく制御したい場合は `GridSpec` の方が柔軟である。つまり、「名前で配置を考えたいか」「格子の比率を直接いじりたいか」で使い分けるとよい。

9.2.4 凡例とレイアウト

複数の系列やフィット曲線を同じ図へ重ねるなら、凡例は不可欠である。凡例をどこへ置
くか、枠を付けるか、何をどの順に書くかで、読みやすさは大きく変わる。

ここでいう `values1` と `values2` は、同じ物理量を別条件で測った 1 次元配列だと考えれば
よい。例えば、温度条件 A と B で得た信号振幅を左右のパネルへ並べ、同じビン数でヒス
トグラムを描いて分布形状を比較する場面である。単にコードの書式を見るのではなく、同
じ量を別条件で見比べるための雛形だと捉えると、このリストの意図が分かりやすい。

```
1 fig, axes = plt.subplots(1, 2, figsize=(10, 4), sharey=True)
2 axes[0].hist(values1, bins=40, alpha=0.7, label="sample A")
3 axes[0].set_title("sample A")
4 axes[0].set_xlabel("value")
5 axes[0].set_ylabel("count")
6
7 axes[1].hist(values2, bins=40, alpha=0.7, label="sample B")
8 axes[1].set_title("sample B")
9 axes[1].set_xlabel("value")
10
11 for ax in axes:
12     ax.legend(loc="upper right")
13     ax.set_axisbelow(True)
14
15 fig.tight_layout()
16 fig.savefig("subplots_example.pdf")
```

この例では、`axes` は二つの `Axes` を並べた配列であり、`for ax in axes:` の部分で「凡例位
置をそろえる」という共通処理を両パネルへまとめて適用している。`sharey=True` を付け
ると図 9.10 のように縦軸の尺度がそろうため、左右の高さをそのまま比較しやすい。ここでは
各パネルに系列が一つしかないため、凡例はやや大げさに見えるかもしれないが、後から
フィット曲線や参照線を重ねるとそのまま拡張できる書き方である。

`tight_layout` は余白調整の基本であるが、すべてを自動で完璧に直してくれるわけではな
い。タイトルや長いラベルが多い場合は、図のサイズや余白を自分で調整した方がよい。複
数パネルの図では、どのパネルを比較させたいのかを意識して配置することが大切である。

凡例がデータを隠すなら、`loc="best"` で自動配置に任せるか、`bbox_to_anchor` を使って枠
の外へ逃がす。レイアウト調整は「自動で整うかどうか」ではなく、「比較したい情報が隠れ
ていないかどうか」で判断する方が実践的である。

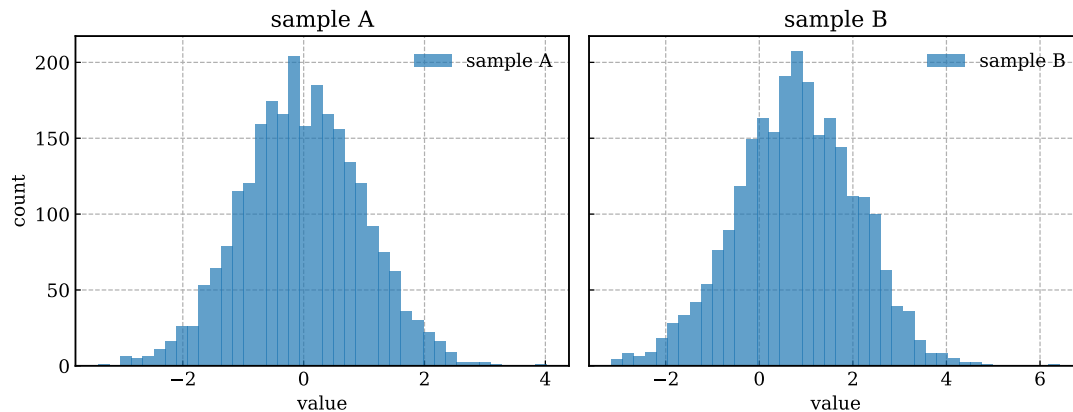


図 9.10 左右に並べたヒストグラムの出力例。縦軸を共有すると、高さの差をそのまま比較しやすい。

9.2.5 エラーバーと対数軸 (log-log プロット)

実際の物理データの多くは測定誤差を伴うため、単なる散布図ではなくエラーバー付きのプロットが必要になる。また、横軸や縦軸のスケールが指数関数的に落ち幅を変える場合、対数 (log) のグラフでプロットしなければ分布の様子が潰れてしまうことが多い。

図 9.11 にエラーバーと対数プロットの例を示す。

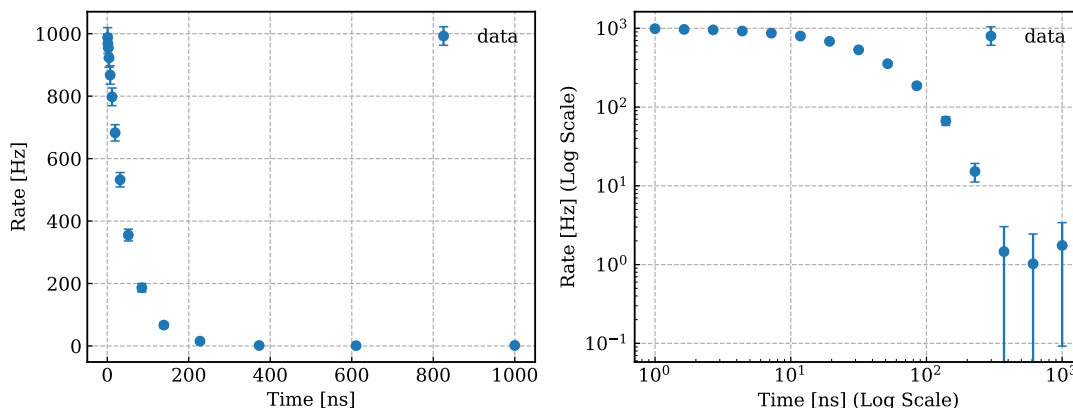


図 9.11 エラーバーと対数軸の設定例。左は通常の線形軸、右は X、Y 両方を対数にした図。

```

1 fig, axes = plt.subplots(1, 2, figsize=(10, 4)) # 横10インチ、縦4インチで 1行2列のパネルを
   作成
2
3 # 左のパネル (axes[0]) へのプロット
4 # x_val, y_val を点ごとにエラー y_err でプロット。
5 # fmt='o' は丸い点を意味し、capsize=3 はエラーバーの終端の横棒の長さ。
6 axes[0].errorbar(x_val, y_val, yerr=y_err, fmt='o', capsize=3, label="data")
7 axes[0].set_xlabel("Time [ns]")
8
9 # 右のパネル (axes[1]) へのプロット
10 axes[1].errorbar(x_val, y_val, yerr=y_err, fmt='o', capsize=3, label="data")
11 # 対数スケール化

```

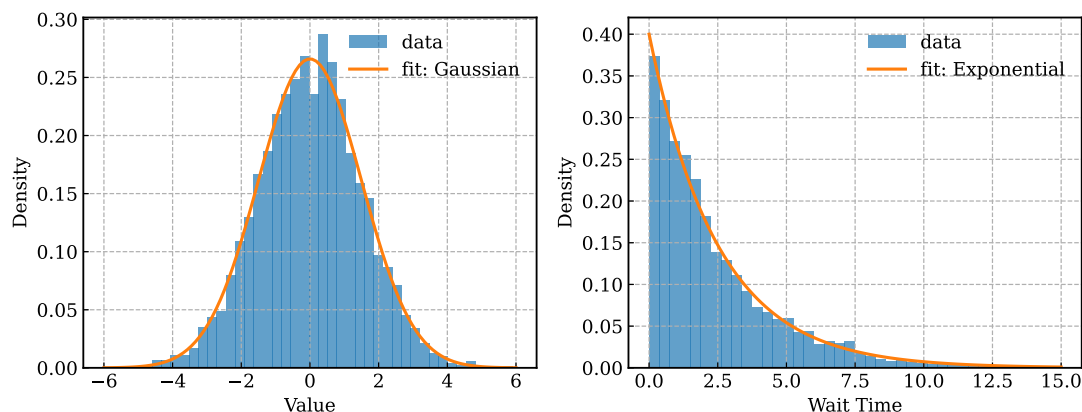


図 9.12 Matplotlib による分布図とフィットの例。左はガウシアン型、右は指数分布であり、どちらもヒストグラムの上にフィット曲線を重ね、凡例に関数形とパラメータを書いている。

```

12 axes[1].set_yscale('log')
13 axes[1].set_xscale('log')
14
15 fig.tight_layout() # ラベルや目盛りが重ならないように余白を自動調整
16 fig.savefig("errorbar_example.pdf")

```

9.2.6 結果例の読み方

図を読めるようになるには、単に描けるだけでは足りない。まず軸と単位を確認し、次に分布の中心と幅、最後に裾や外れ値を見る、という順で眺めると整理しやすい。

例えば、図 9.12 の左パネルでは、ヒストグラムの山がフィット曲線とよく重なっているか、左右対称性がどの程度あるかを見る。右パネルでは、短時間側の立ち上がりと長時間側の減衰が同じ定数で説明できるかを見る。こうした読み方を本文中で言葉にしておくと、図は単なる飾りではなく考察の根拠になる。

9.2.7 画像フォーマットの選択：ラスタとベクタ

解析図を保存する際、適切な画像フォーマットを選ぶことは極めて重要である。

- **ベクタ形式 (.pdf, .svg, .eps)**：どれだけ拡大しても線や文字がぼやけない。論文やレポートで図を用いる場合、文字やプロット点を高品質なまま印刷できるためこれが基本となる。関数 `savefig("plot.pdf")` で保存できる。
- **ラスタ形式 (.png, .jpg)**：ピクセル（点）の集まりで画像を描画する。拡大するとジャギ（ピクセルのギザギザ）が見える。ただし、例えば数十万点に及ぶ密集した散布図や、ヒートマップのような巨大な 2 次元データをベクタ形式で保存すると、点の一つ一つがファイルに記録されるためファイルサイズがギガバイトを超え、PDF ビューアがフリーズする原因になる。そのような複雑な図の場合は、`savefig("plot.png", dpi=150)` のように解像度（dpi）を指定して PNG で出力する方が適切である。

```

1 fig, ax = plt.subplots(figsize=(6, 4))
2 ax.plot(x, y)
3 ax.set_xlabel("time [ns]")
4 ax.set_ylabel("count")
5
6 fig.savefig("spectrum.pdf")
7 fig.savefig("spectrum.png", dpi=150)

```

同じ図でも、spectrum.pdf は拡大しても線と文字が崩れにくく、spectrum.png はスライド貼り付けや Web 掲載で扱いやすい。どちらが良いかは「図の複雑さ」と「最終的な使い道」で決まる。線図や通常のヒストグラムなら PDF を第一候補にし、巨大散布図やヒートマップでは PNG を検討する方が実務的である。

9.3 オブジェクト指向 API による複雑な図の管理

9.3.1 Figure, Axes, Axis, Artist の役割

Matplotlib の初学者が最も混乱しやすいのは、Figure と Axes と Axis の区別である。特にややこしいのは、Axes という名前が複数形に見えるにもかかわらず、Matplotlib では「1 枚の作図パネル」を表すクラス名として使われている点である。まずは次の対応で覚えるとよい。

Figure 図全体である。1 つの PDF や PNG、1 つの表示ウィンドウに相当する。savefig、suptitle、図全体の凡例や colorbar などは主に Figure 側の仕事である。

Axes データを実際に描く 1 つのパネルである。plot、hist、scatter、set_xlabel、legend など、日常的な作図命令の多くは Axes のメソッドとして呼ぶ。

Axis (x Axis, y Axis) 目盛り、目盛りラベル、スケールを管理する部品である。通常は ax.set_xscale("log") や ax.set_xlim(...) のように Axes 経由で触るが、内部には ax.xaxis と ax.yaxis がある。

Artist 図の中に描かれるすべての要素の総称である。線は Line2D、文字は Text、凡例は Legend、ヒストグラムの棒は Rectangle といった Artist として管理される。以下なども Artist に含まれる。

- Title, Legend: 図のタイトル、および各系列を示す凡例。
- Grid, Spine: グリッド線と、座標系を囲む枠線。
- Line, Markers: 折れ線や散布図の点。
- xlabel, ylabel: x 軸・y 軸のラベル。

Figure、Axes、Axis、Artist の階層関係だけを先に図解の構図に合わせて木構造で書くと、次のようになる。

```
Figure
```

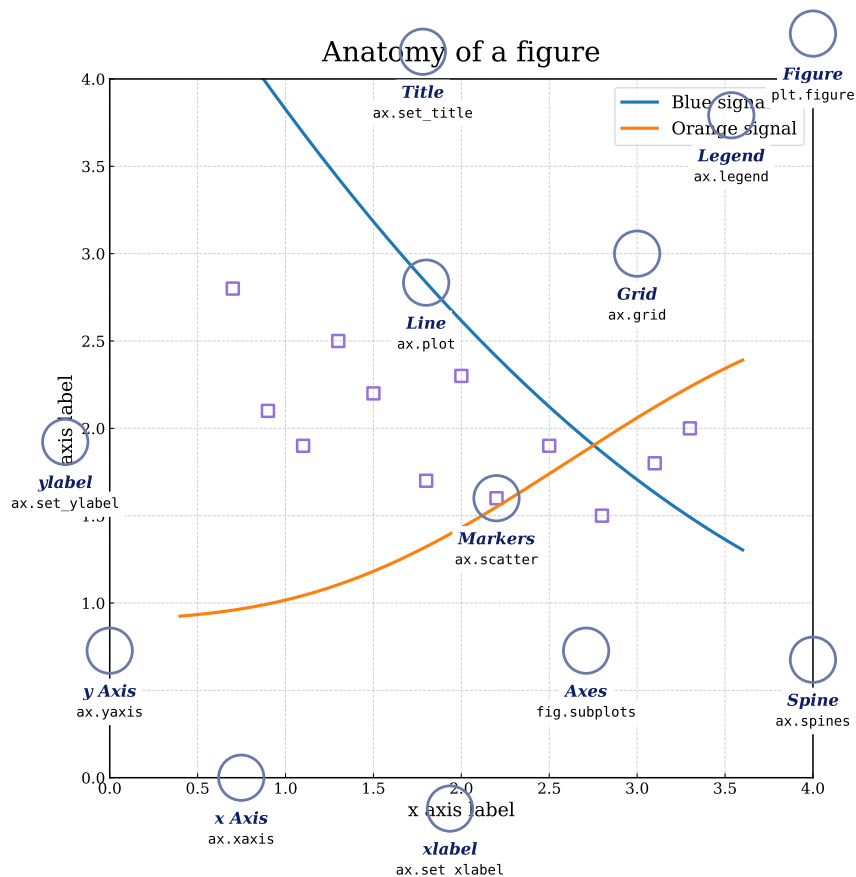


図 9.13 Figure、Axes、Axis、Artist の関係を示した例。1 つの Figure の中に Axes があり、その Axes の中で線、文字、凡例などの Artist が描かれている。

```

|-- Axes
|   |-- Title
|   |-- Spine, Grid
|   |-- xaxis, yaxis (x Axis, y Axis / xlabel, ylabel)
|   |-- Line2D, Markers
|   |-- Legend
|-- Figure-level Text / Colorbar / ...

```

図 9.13 に、Figure、Axes、Axis、Artist の位置関係を示す。まずは「Figure の中に Axes があり、その中に Axis と各種 Artist がある」という階層だけ押さえれば十分である。この対応を、実際の Python オブジェクトの型で確かめる短いコードを次に示す。

```

1 import matplotlib.pyplot as plt
2
3 x = [0, 1, 2, 3]
4 y = [1.0, 2.0, 1.5, 3.0]
5
6 fig, ax = plt.subplots(figsize=(6, 4))
7 line, = ax.plot(x, y, label="signal")
8 ax.set_title("Axes title")
9 ax.set_xlabel("time")
10 ax.set_ylabel("signal")

```

```

11 legend = ax.legend()
12
13 print(type(fig).__name__)
14 print(type(ax).__name__)
15 print(type(ax.xaxis).__name__)
16 print(type(line).__name__)
17 print(type(legend).__name__)
18 print(len(fig.axes))
19 fig.savefig("line.pdf")

```

このコードを実行すると、次の出力が得られる。

```

Figure
Axes
XAxis
Line2D
Legend
1

```

最後の 1 は、この Figure の中に Axes が 1 枚だけ入っていることを表す。もし `plt.subplots(2, 2)` と書けば、Figure は 1 つのままで、Axes が 4 枚入る。

Artist という言葉も、線だけを指すわけではない。タイトルや軸ラベルは Text、ヒストグラムの各棒は Rectangle として管理される。次のコードで、どの型が返るかを確かめる。

```

1 import matplotlib.pyplot as plt
2
3 fig, ax = plt.subplots(figsize=(6, 4))
4 title = ax.set_title("Histogram")
5 xlabel = ax.set_xlabel("value")
6 bars = ax.hist([1, 1, 2, 3], bins=[0.5, 1.5, 2.5, 3.5])[2]
7
8 print(type(title).__name__)
9 print(type(xlabel).__name__)
10 print(type(bars).__name__)
11 print(type(bars[0]).__name__)

```

このコードの出力は次のとおりである。

```

Text
Text
BarContainer
Rectangle

```

ここで bars 全体は BarContainer であり、その中の 1 本 1 本は Rectangle である。したがって、「ヒストグラムの棒だけ色を変える」といった細かい調整は、最終的には Artist を触る操作だと理解できる。

Figure 側の要素も、コードで見ると役割が分かりやすい。図全体タイトル `suptitle` や `colorbar` は、特定の Axes だけでなく Figure 全体の見た目に関わることが多い。次のコードでは、その型を確認する。

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 fig, ax = plt.subplots(figsize=(5, 4))
5 img = ax.imshow(np.arange(4).reshape(2, 2))
6 suptitle = fig.suptitle("Figure title")
7 cbar = fig.colorbar(img, ax=ax)
8
9 print(type(suptitle).__name__)
10 print(type(cbar).__name__)

```

このコードでは、次の出力が得られる。

```

Text
Colorbar

```

`suptitle` は Figure 全体に付く Text であり、`colorbar` は Figure に追加される補助要素である。複数パネルの図で共通タイトルや共通 `colorbar` を付けたいときは、Axes よりも Figure 側の仕事だと考えると整理しやすい。

Axis は少し見えにくい部品だが、実際には目盛りと表示範囲を管理している。たとえば次のような操作は Axis に関わる設定である。

```

1 fig, ax = plt.subplots(figsize=(6, 4))
2 ax.plot([1, 10, 100], [2, 3, 5], marker="o")
3 ax.set_xlim(1, 100)
4 ax.set_xscale("log")

```

`set_xlim` は表示範囲を、`set_xscale("log")` は目盛りの刻み方を変える。ふだんは `ax.xaxis` を直接操作しなくても、Axes メソッド経由で Axis の設定を変えていると理解すれば十分である。

どのオブジェクトを操作すべきかは、何を変えたいかで決まる。次の対応で考えると迷いにくい。

- 図全体を保存する、図全体タイトルを付ける: Figure
- 線やヒストグラムを描く、軸ラベルや凡例を付ける: Axes
- 対数軸、目盛り位置、目盛りラベルを変える: Axis 関連。ただし実際の操作は Axes メソッド経由が多い
- 特定の線だけ色や太さを変える、凡例や文字を細かく調整する: Artist

この区別が付くと、「保存はなぜ `fig.savefig(...)` なのか」「凡例はなぜ `ax.legend()` なのか」「対数軸はなぜ `ax.set_xscale("log")` で変えるのか」が自然につながる。Matplotlib を使い始めたら、まず `fig` と `ax` のどちらへ命令しているのかを追うだけでも、コードの見通しはかなり良くなる。

9.3.2 オブジェクト指向 API で描く

Matplotlib には `plt.hist(...)` のような簡単な書き方と、Figure と Axes を明示的に扱う書き方がある。図が複雑になるほど、後者の方が整理しやすい。

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 values = np.random.normal(loc=0.0, scale=1.2, size=2500)
5
6 fig, ax = plt.subplots(figsize=(6, 4))
7 ax.hist(values, bins=40)
8 ax.set_xlabel("value")
9 ax.set_ylabel("count")
10 fig.savefig("histogram.pdf")
```

`ax` に対して操作を書く形にしておくと、複数図や複数パネルへ発展しやすい。解析スクリプトを後で拡張したり、提出用の図を安定して作り直したりする場面では、この書き方に慣れておく価値が高い。

また、Figure と Axes を区別して考えると、図全体に効く設定と各パネルにだけ効く設定を整理しやすい。タイトルや余白は図全体、軸ラベルや凡例は各パネル、というように役割を分けて考えると見通しが良い。

9.3.3 補助関数で作図をまとめる

同じ体裁の図を何枚も作るなら、Axes を引数にする補助関数へまとめると管理しやすい。

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def plot_hist(ax, values, label):
5     ax.hist(values, bins=40, alpha=0.7, label=label)
6     ax.set_xlabel("value")
7     ax.set_ylabel("count")
8     ax.legend(loc="upper right")
9
10 rng = np.random.default_rng(0)
11 values1 = rng.normal(loc=0.0, scale=1.0, size=2500)
12 values2 = rng.normal(loc=0.8, scale=1.3, size=2500)
13
14 fig, axes = plt.subplots(1, 2, figsize=(10, 4), sharey=True)
15 plot_hist(axes[0], values1, "sample A")
16 plot_hist(axes[1], values2, "sample B")
17 fig.savefig("subplots_example.pdf")
```

重要なのは、`plt.hist(...)` のようなグローバルな状態へ書くのではなく、「どの Axes に描くか」を引数で受け取る点である。比較図、条件ごとの反復描画、関数の再利用がしやす

くなる。

9.4 見た目とスタイルの管理

図を読みやすくするには、少なくとも次の点を意識する。

- 軸ラベルに量と単位を書く
- タイトルを簡潔に付ける
- 余白を調整する
- 色と線幅を統一する
- レポート全体で表記を揃える

色やフォント、線幅をばらばらにすると、個々の図は読めてもレポート全体として落ち着かない。逆に、軸ラベルの位置、単位の書き方、凡例のスタイルをそろえるだけで、見た目の質は大きく上がる。可視化では、派手さより統一感の方が効くことが多い。

9.4.1 マーカー、線種、色の指定

Matplotlib では、点の形、線種、色をそれぞれ独立に指定できる。まずよく使う引数は次の 4 つである。

```
1 ax.plot(  
2     x,  
3     y,  
4     marker="o",  
5     markersize=5,  
6     linestyle="--",  
7     linewidth=2.0,  
8     color="C3",  
9 )
```

- marker: 点の形を決める。
- markersize: 点の大きさを決める。
- linestyle: 実線、破線、点線などを決める。
- color: 線や点の色を決める。

図 9.14 は、よく使う marker、line style、既定の color cycle を並べたものである。見た目の違いを先に知っておくと、コード中の短い記号だけを見ても何が起きるかを想像しやすい。

図 9.14 では、マーカー、線種、色を組み合わせたときの見え方を比較できる。個々の指定と用途は、マーカーを表 9.1、線種を表 9.2、色を表 9.3 にまとめた。系列を区別するときは色だけに依存せず、マーカーや線種も併用すると、白黒印刷や色覚特性が異なる読者にも情報が伝わりやすい。

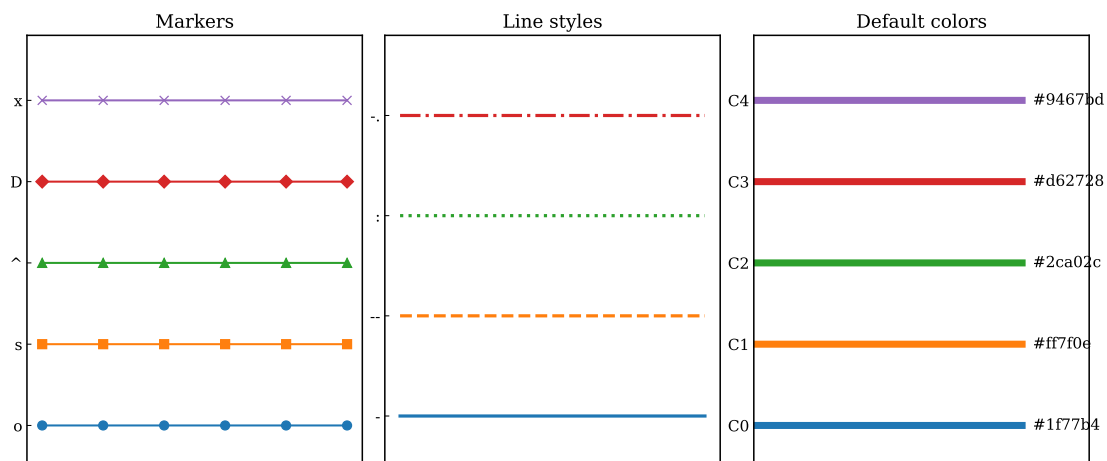


図 9.14 marker、line style、既定色の例。短い指定でも見た目はかなり変わるので、まず代表的なものだけ覚えて使い分けるとよい。

表 9.1 よく使う marker の指定

指定	見た目	用途の例
"o"	丸	測定点、既定の散布図
"s"	四角	条件の違いを強調したい点
"^"	上向き三角	系列ごとの差別化
"D"	ひし形	参照データや補助系列
"x"	バツ	強調した外れ値や選択点

表 9.2 よく使う line style の指定

指定	見た目	用途の例
"-"	実線	主結果、連続関数
"--"	破線	フィット曲線、比較対象
":"	点線	補助線、閾値
"-."	一点鎖線	参照モデル、別条件

表 9.3 色指定の基本

指定	例	備考
"C0" など	"C3"	既定の color cycle を順に使う
名前付き色	"black", "tab:blue"	手早く書ける
16 進数	"#d62728"	図全体の配色を固定したいときに便利

色だけで系列を区別しようとする、印刷や色覚特性によって読みにくくなることがある。系列が2つ以上ある場合は、色に加えて marker や line style も変える方が安全である。

9.4.2 rcParams で見た目をそろえる

Matplotlib では、図のフォントや線幅などの既定値の多くを rcParams という辞書オブジェクトで一元管理している。図を描くたびに同じ設定を繰り返すより、作図スクリプトの冒頭で一度だけこの辞書を更新するのが基本である。

```
1 import matplotlib as mpl
2
3 mpl.rcParams.update(
4     {
5         "font.family": "serif",
6         "mathtext.fontset": "cm",
7         "font.size": 12,
8         "axes.grid": True,
9         "grid.linestyle": "--",
10        "xtick.direction": "in",
11        "ytick.direction": "in",
12        "axes.linewidth": 1.2,
13        "legend.frameon": False,
14    }
15 )
```

- font.family: 既定フォント族を決める。本文と図の雰囲気をもそろえたいときの入口になる。
- font.size: 文字の基本サイズである。発表スライド用と論文用で最初に変えたい項目である。
- axes.grid, grid.linestyle: グリッドを既定で表示するか、その線種をどうするかを決める。
- xtick.direction, ytick.direction: 目盛り線を内向きにするか外向きにするかを決める。
- axes.linewidth: 枠線の太さである。細すぎると印刷時に弱く見える。
- legend.frameon: 凡例の枠を出すかどうかを決める。

rcParams で設定した値は、そのスクリプトで後から作る図へまとめて効く。したがって、「毎回同じ見た目にしたい項目」をここへ集め、「この図だけ変えたい項目」は後で `ax.set_*` などで上書きする、と分けると整理しやすい。

9.4.3 cmap の選び方

ヒートマップや散布図で色を値へ対応付けるときは、`cmap=` でカラーマップを選ぶ。どの色を選んでもよいわけではなく、データの意味に応じて使い分ける必要がある。代表例を

表 9.4 よく使うカラーマップ

種類	例	向いている用途
Sequential	viridis, plasma, cividis	0 から大へ単調に増える量
Diverging	coolwarm, RdBu_r	0 や基準値をはさんで符号が変わる量
Grayscale	Greys	印刷や下地としての濃淡表示
Categorical	tab10	離散的な系列の識別

表 9.4 に示す。

例えば、カウント数や強度のように「小さいから大きいへ一方向に変化する量」には viridis のような sequential map が自然である。正負のずれや差分のように中心値が重要な量には diverging map が向く。系列を分類したいだけなのに sequential map を使うと、値の大小に意味があるように見えてしまうので注意が必要である。

9.4.4 設定ファイル (.mplstyle) で見た目を共通化する

しかし、解析テーマごとに作図スクリプトが増えていくと、すべての Python ファイルの先頭に上記の長い辞書をコピーするのは冗長である。Matplotlib では、これらの設定をプレーンテキストに切り出し、まとめて読み込める。

作業ディレクトリに my_style.mplstyle という名前で、次のような設定ファイルを作成する。

```
font.family: serif
mathtext.fontset: cm
font.size: 12
axes.grid: True
grid.linestyle: --
xtick.direction: in
ytick.direction: in
axes.linewidth: 1.2
legend.frameon: False
```

そして、各解析スクリプトの先頭でこれを一度だけ呼び出せばよい。本テキストに含まれる図も、すべてこの設定ファイル（スタイルシート）を読み込んで生成されている。

```
1 import matplotlib.pyplot as plt
2
3 # スクリプトの冒頭で自分のスタイルシートを読み込む
4 plt.style.use("./my_style.mplstyle")
5
6 # 以降の描画はすべて統一されたスタイル (Serif フォント等) が反映される
7 fig, ax = plt.subplots(figsize=(6, 4))
```

このように設定をファイルに分離しておくことで、新しい解析スクリプトを書くたびにフォントやグリッドの有無がバラバラになる事態を防ぐことができる。また、学会発表用の

大きなフォント設定といった別のファイルを容易に切り替えることも可能になる。

9.4.5 共通スタイルの一部だけを上書きする

共通スタイルを決めておくと便利だが、発表スライド用に文字だけ大きくしたい、下書きではグリッドだけ消したい、といった局所的な変更もよく起こる。そのたびに元の `my_style.mplstyle` を書き換えると、別の図まで見た目が変わってしまう。こういう場合は、基準となるスタイルを読み込んだ後に、必要な項目だけを重ねる方が管理しやすい。

```
# presentation_style.mplstyle
font.size: 16
axes.grid: False
legend.fontsize: 11

1 import matplotlib as mpl
2 import matplotlib.pyplot as plt
3
4 plt.style.use(["./my_style.mplstyle", "./presentation_style.mplstyle"])
5
6 # スタイルファイルを増やしたくないなら、その場で個別に上書きしてもよい
7 mpl.rcParams["axes.titlesize"] = 14
8
9 fig, ax = plt.subplots(figsize=(6, 4))
10 ax.grid(False) # この Axes だけグリッドを消す
```

二つの `.mplstyle` を配列で渡すと、後ろに書いたファイルの設定が優先される。したがって、基準となる `my_style.mplstyle` を保ったまま、文字サイズやグリッドの有無だけを差し替えられる。さらに、`mpl.rcParams` を直接書き換えれば、そのスクリプト以降の図だけを変更でき、`ax.grid(False)` のような Axes メソッドを使えば、単一のパネルにだけ適用できる。設定の効く範囲が、スタイルファイル、スクリプト全体、個別の Axes の三段階に分かれると理解しておくで混乱しにくい。

9.5 比較しやすい図を作る

9.5.1 Matplotlib の数式表記を使う

Matplotlib では、軸ラベルや凡例に数式風の表記を書ける。既定では LaTeX そのものではなく、TeX に似た `MathText` という仕組みが使われる。

```
1 plt.xlabel(r"$\Delta t$, [\mathrm{ns}]$")
2 plt.ylabel(r"$N$")
```

単位まで数式の中へ入れるなら、上の例のように `mathrm` を使って立体で書く方が自然である。レポートを LaTeX で書くなら、図中の記号も

本文と整合するようにそろえると見通しがよい。

`MathText` は便利だが、`LaTeX` そのものではない。よく使う分数、上付き・下付き、ギリシャ文字、

`mathrm` などは扱える一方で、複雑なマクロや一部のパッケージ依存の記法は使えない。通常は追加の `LaTeX` 環境なしで動くのが利点であり、本文と完全に同じ組版結果まで求めるときだけ `text.usetex=True` のような設定を検討する。

また、図中で使う記号は本文での定義と一致させるべきである。本文で τ を時定数として定義したなら、図の凡例やキャプションでも同じ記号を使う方が親切である。解析内容と図の表現が食い違っていると、読者は不要なところで迷う。

9.6 よくある落とし穴

9.6.1 `plt` と `ax` を混ぜて迷子になる

Matplotlib では `plt.xlabel(...)` のような状態依存の書き方もできるが、複数パネルや複数 Figure を扱い始めると、「いまどの Axes が現在の対象なのか」が分かりにくくなる。複雑な図では `fig, ax = plt.subplots(...)` を起点にして、以後は `ax.set_xlabel(...)` のように書く方が安全である。

9.6.2 文字列がカテゴリ軸になる

数字に見える列でも、実際には文字列として読まれていると、Matplotlib は数値軸ではなくカテゴリ軸として解釈する。すると、1, 2, 10, 20 が数値間隔ではなく、単なる 4 つのラベルとして等間隔に並ぶ。

```
1 import matplotlib.pyplot as plt
2
3 x_str = ["1", "2", "10", "20"]
4 y = [1.0, 4.0, 2.0, 5.0]
5 fig, ax = plt.subplots(figsize=(6, 4))
6 ax.plot(x_str, y, marker="o")
```

図 9.15 の左では文字列がカテゴリ軸として扱われ、右では数値へ変換したので数値軸になっている。CSV を読んだ直後に軸の間隔が変だと感じたら、まず `dtype` を疑う方がよい。

9.6.3 `savefig` と余白調整の順序

`savefig` の直前に `tight_layout()` や `constrained_layout` を使わないと、長いラベルや凡例が切れることがある。また、対話環境では `show()` の後に状態が変わることもあるため、再現可能なスクリプトでは「描く、余白を整える、保存する」の順を守る方がよい。対数軸を使

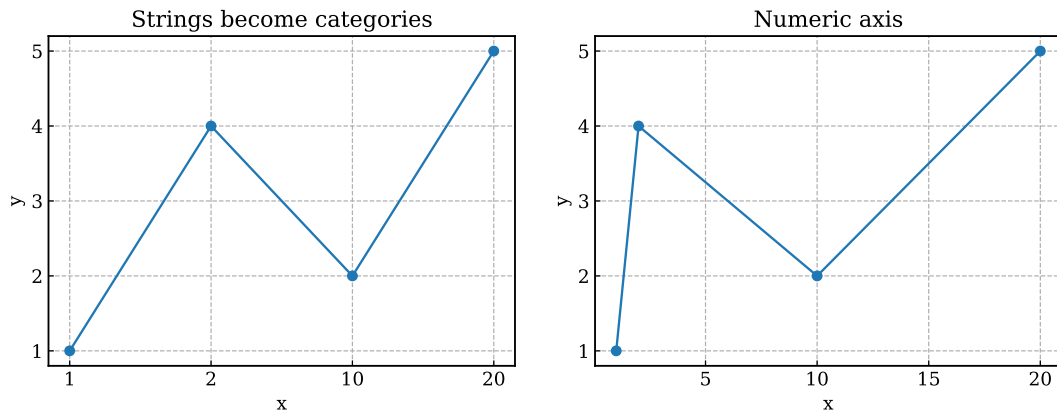


図 9.15 文字列軸と数値軸の違い。左は "1", "2", "10", "20" をカテゴリとして並べており、右は数値として扱っている。

う場合は、0 以下の値をそのまま載せられない点にも注意が必要である。

9.6.4 公式資料から似た図を探す

ここまでで、基本的な描画、レイアウト、オブジェクトの役割、保存まで一通り見た。実際の作図では、その先で「似た例をどこから探すか」が次の問題になる。Matplotlib は機能が広いため、作りたい図の名前が分からなくても、公式資料から近いものを探す方が早いことが多い。特に次の 4 つは参照頻度が高い。

- Quick start [22]
- Plot types [23]
- Gallery [24]
- Cheatsheets [25]

作りたいグラフのイメージが先にあるなら、まず Plot types で図の種類を当て、次に Gallery で近い完成例を探し、最後に Cheatsheets で引数名や書式を確認すると迷いにくい。Quick start は、'Figure' と 'Axes' の関係を短時間で思い出したいときに向いている。初学者が最初から完全なコードを自力で組み立てるより、近い例を見つけて必要な部分だけ差し替える方が現実的である。

9.7 章末問題

USGS データと前章の Δt 配列がない場合は、次の小規模データを使う。課題では画面表示だけで終わらせず、指定したファイルを保存し、保存後に `Path(...).exists()` で存在を確認する。

```
1 import numpy as np
2
3 longitude = np.array([140.0, 141.2, 139.5, 142.0, 138.7])
```

```

4 latitude = np.array([35.0, 36.1, 34.8, 37.0, 33.9])
5 magnitude = np.array([3.2, 4.1, 2.8, 4.8, 3.7])
6 dt = np.array([1.0, 4.0, 12.0, 60.0, 180.0, 900.0, 3600.0])

```

1. **全球地震分布の散布図を作る。** USGS の CSV から longitude、latitude、mag を読み、横軸を経度、縦軸を緯度とする散布図を作れ。点の色でマグニチュードを表し、軸ラベル、カラーバー、タイトルを付けたうえで figures/usgs_global_scatter.pdf へ保存せよ。fig, ax = plt.subplots() の返り値、scatter = ax.scatter(...) の返り値、fig.colorbar(scatter, ax=ax) の第 1 引数と ax 引数の役割を説明せよ。scatter.get_array() がマグニチュード配列と一致し、len(fig.axes) がカラーバーを含めて 2 になることを確認すること。
2. **発生間隔ヒストグラムを対数軸で描く。** 第 6 章で作った Δt 配列を用い、発生間隔ヒストグラムを描け。少なくとも縦軸は対数表示にし、必要なら横軸も対数にして、どの表示が見やすいかを比較せよ。図中にはビンの意味が分かる軸ラベルと単位を書くこと。線形ビンの図と np.logspace() で作った対数ビンの図を 2 枚の Axes に並べ、figures/interval_comparison.pdf へ保存せよ。ax.set_yscale("log") と ax.set_xscale("log") がデータ値そのものを変換するのではなく、Axis の表示尺度を変えることを説明する。0 以下の値を対数軸へ置けない理由も述べよ。
3. **発展課題。** 通常の散布図では、点がどの海溝や列島に沿って並んでいるかは読み取りにくい。海岸線付きの地図へ重ねたい場合は、Matplotlib だけではなく地図投影を扱う cartopy が必要になる。cartopy を導入し、同じ地震データを海岸線付きの地図へ重ねよ。必要な出発点は、
 - projection=ccrs.PlateCarree() で地図 Axes を作る
 - transform=ccrs.PlateCarree() を scatter 側に付ける
 - ax.coastlines() で海岸線を描く
 - 全球図では ax.set_global() を使う
 - 日本近海図では ax.set_extent([122, 154, 20, 50], crs=ccrs.PlateCarree()) を使う

である。これらの使い方を検索し、地図付きの散布図を作成せよ。最小コードは、`import cartopy.crs as ccrs, fig, ax = plt.subplots(subplot_kw={"projection": ccrs.PlateCarree()})`、`ax.set_global()`、`ax.coastlines()`、`ax.scatter(longitude, latitude, c=magnitude, transform=ccrs.PlateCarree())` の順に組み立てられる。projection は地図を描く Axes の座標系、transform は入力データの座標系を指定する。figures/usgs_map.pdf を保存し、両者を同じ値にしても役割が異なることを説明せよ。Cartopy が未導入なら、実行した導入コマンドと、導入前に出た例外名を記録すること。

可視化によって分布の形、外れ値、残差の傾向を確認できると、次に検討すべき数値計算を選びやすくなる。次章では、図から得た仮説を SciPy の統計分布、フィット、補間、信号

処理によって定量化する。

第 10 章

SciPy による数値計算とフィット

SciPy は、NumPy を土台にして数値計算の道具をまとめたライブラリ群である。pandas が表、Matplotlib が図を担当するのに対して、SciPy は統計、補間、信号処理、最適化、積分といった「計算そのもの」を担当することが多い。本章では、SciPy の考え方、最小限のコード例、落とし穴、解析へのつながりを順に見る。

10.1 SciPy の考え方

10.1.1 NumPy の上にある専門道具箱

SciPy の多くの関数は、NumPy 配列を入力に取り、NumPy 配列やスカラーを返す。したがって、SciPy を理解するには NumPy の ndarray と配列演算に慣れていることが前提になる。その上で、「平均や標準偏差を出したいのか」「曲線を当てたいのか」「ピークを探したいのか」に応じて、適切な subpackage を選ぶ。

- `scipy.stats`: 分布、検定、要約統計
- `scipy.optimize`: 最適化、非線形最小二乗、`curve_fit`
- `scipy.interpolate`: 補間
- `scipy.signal`: フィルタ、ピーク検出、相関、スペクトル処理
- `scipy.integrate`: 数値積分
- `scipy.constants`: 物理定数

何でも SciPy で解くのではなく、「NumPy だけで足りるか」「SciPy のどの subpackage が自然か」を切り分けることが重要である。

10.1.2 関数形を仮定して数値を取り出す

SciPy が特に役立つのは、図や配列を「見た目」で終わらせず、数値へ落とし込む場面である。ヒストグラムから幅を取りたい、減衰曲線から時定数を取りたい、ノイズの多い波形からピーク時刻を取りたい、といった場面では、NumPy の基本操作だけでは足りないことが多い。

ヒストグラムを描いただけでは、分布の特徴を定量化できないことがある。そこで、ある関数形を仮定してパラメータを求める。これがフィットである。フィットは万能ではない。まず、なぜその関数形を選ぶのかという物理的、統計的な理由が必要である。さらに、ヒストグラムの範囲、ビン幅、外れ値処理によって見え方が変わるため、「フィット結果が出た」とことと「解釈してよい」ことは別である。

フィットの目的は、曲線をきれいに重ねることではない。分布の中心、幅、時定数などを、比較可能な量として取り出すことが目的である。したがって、フィット前に図を見ておくこと、フィット後に残差やずれを確認することが重要になる。

10.1.3 この本でよく使う分布

データ解析では、ガウシアンと指数分布がよく現れる。

- ガウシアン: 時刻差のばらつき
- 指数分布: 待ち時間や発生間隔

もちろん、実際のデータがいつもきれいな理想分布になるとは限らない。重要なのは「なぜその関数を当てようとしたのか」を説明することである。分布の名前を覚えるだけでなく、どの量に対して自然に現れるかを知ることが重要である。ガウシアンは独立なばらつきの和として、指数分布はランダムな待ち時間として現れやすい。こうした背景を知っていると、単なる形合わせで終わらない。

たとえば、平均 μ 、標準偏差 σ のガウシアンは

$$p(x) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left[-\frac{(x-\mu)^2}{2\sigma^2}\right]$$

で書ける。また、時定数 τ の指数分布は

$$p(t) = \frac{1}{\tau} \exp\left(-\frac{t}{\tau}\right) \quad (t \geq 0)$$

である。SciPy の分布オブジェクトは、このような確率密度やその積分を関数として評価する道具だと捉えると分かりやすい。

10.2 最小限のコード例

10.2.1 scipy.stats による要約統計と分布

まずは、配列の全体像を数値で確認する方法から始める。

```
1 import numpy as np
2 from scipy import stats
3
4 values = np.array([1.2, 0.9, 1.5, 1.1, 0.8])
5
6 summary = stats.describe(values)
7 print(summary.nobs)
8 print(summary.mean)
9 print(summary.variance)
```

この例では、要素数、平均、分散が順に表示される。

```
5
1.1
0.075
```

`stats.describe()` は、要素数、平均、分散、最小値、最大値、歪度、尖度をまとめて返す。最初の確認として便利だが、返り値は名前付きタプルなので、何が入っているかを読んで使う必要がある。

SciPy の分布オブジェクトは、確率密度関数や累積分布関数を同じ API で扱える。

```
1 x = np.linspace(-3.0, 3.0, 7)
2 pdf = stats.norm.pdf(x, loc=0.0, scale=1.0)
3 cdf = stats.norm.cdf(x, loc=0.0, scale=1.0)
4
5 print(pdf)
6 print(cdf)
```

pdf と cdf を同じ点列で評価すると、次のような値が得られる。

```
[0.00443185 0.05399097 0.24197072 0.39894228 0.24197072 0.05399097
 0.00443185]
[0.0013499 0.02275013 0.15865525 0.5          0.84134475 0.97724987
 0.9986501 ]
```

pdf、cdf、rvs、ppf がそろっているので、理論分布との比較、乱数生成、しきい値計算を統一的に書ける。

ここで pdf は確率密度 $p(x)$ 、cdf はその積分

$$F(x) = \int_{-\infty}^x p(t) dt$$

に対応する。したがって、`stats.norm.cdf(x)` は「 x 以下に入る確率」を返している。

10.2.2 `curve_fit` によるパラメータ推定

SciPy で最もよく使われる関数の一つが `scipy.optimize.curve_fit` である。モデル関数を自分で定義し、データへ当てはめる。

SciPy には、分布やフィットのための道具が複数ある。代表的なのは `scipy.stats` と `scipy.optimize.curve_fit` である。図 9.12 のような曲線を重ねると、単に形を見るだけでなく、平均や幅、時定数を数値として比較しやすくなる。単純な場合は、自分で平均や分散を計算するだけでも十分なことがある。一方で、より一般的なモデルや誤差評価をしたいなら SciPy が役立つ。

```

1 from scipy.optimize import curve_fit
2
3 def linear_model(x, a, b):
4     return a * x + b
5
6 xdata = np.array([0.0, 1.0, 2.0, 3.0, 4.0])
7 ydata = np.array([1.1, 2.0, 3.2, 4.1, 5.1])
8
9 params, cov = curve_fit(linear_model, xdata, ydata)
10 errors = np.sqrt(np.diag(cov))
11
12 print(params)
13 print(errors)

```

この線形フィットでは、傾きと切片、およびその標準誤差の目安が次のように出る。

```

[1.02 1.04]
[0.04472136 0.10954451]

```

実際の解析では、ガウシアンや指数分布の平均や幅、時定数を次のような形で整理して比較することが多い。

```

mean = 3.02
sigma = 0.79
tau = 2.47

```

`params` は推定パラメータ、`cov` は共分散行列である。対角成分の平方根が標準誤差の目安になる。上の例なら `params[0]` が傾き、`params[1]` が切片に対応する。一方、非対角成分はパラメータ間の相関を表し、値が大きいと「片方を少し動かすと、もう片方で補えてしまう」ような不安定さがあることを示す。重要なのは、`curve_fit()` が単に曲線をきれいに重ねる道具ではなく、「仮定したモデルのパラメータを数値として取り出す道具」だという点である。

上の直線モデル $f(x; a, b) = ax + b$ なら、`curve_fit()` は概ね

$$S(a, b) = \sum_i [y_i - (ax_i + b)]^2$$

を小さくする a と b を探していると考えればよい。より一般には、モデル $f(x; \mathbf{p})$ に対して

$$S(\mathbf{p}) = \sum_i [y_i - f(x_i; \mathbf{p})]^2$$

という残差二乗和を最小化している。

こうした数値が得られたら、次に考えるべきは「図の見た目と整合するか」「別条件のデータと比較して意味があるか」「単位は妥当か」である。数値が出たこと自体より、どう読むかの方が重要である。SciPy を使うときも、ライブラリ名だけを覚えるのではなく、「どの関数が何を入力として何を返すか」を確認する読み方が重要である。

10.3 フィットを進める手順

実際の解析では、次の順に進めると混乱しにくい。

1. まず図を描き、どの関数形が候補かを考える
2. フィット範囲を決める
3. 初期値が必要なら妥当な値を与える
4. フィット曲線を図へ重ねる
5. 推定値と残差を確認する

この順序を踏まずにいきなり `curve_fit()` を呼ぶと、得られた数値が妥当なのか判断しにくい。フィットは関数呼び出しではなく、仮説と検証の手続きとして理解した方がよい。

10.3.1 初期値、制約、残差

モデルによっては、初期値や制約を与えないと不安定になる。

```

1 def exp_model(x, a, tau, c):
2     return a * np.exp(-x / tau) + c
3
4 params, cov = curve_fit(
5     exp_model,
6     xdata,
7     ydata,
8     p0=[5.0, 2.0, 0.0],
9     bounds=([0.0, 0.0, -np.inf], [np.inf, np.inf, np.inf]),
10 )

```

`p0` は初期値、`bounds` はパラメータ範囲である。どちらも万能ではないが、物理的にあり得ない範囲を避けるときには有効である。モデルによっては、初期値の与え方で収束先が変わることがある。また、明らかに負になってほしくないパラメータなどには制約を入れた方が自然なこともある。

フィット結果を図へ重ねるときは、関数形と推定パラメータを凡例に書く方が見やすいこ

とが多い。図の上に直接テキストを置くと、データや曲線と重なって読みにくくなりやすい。特に、比較する図が複数ある場合は、凡例の書式をそろえると見通しが良くなる。

```

1 fig, ax = plt.subplots()
2 ax.hist(values, bins=40, density=True, alpha=0.7, label="sample")
3 ax.plot(
4     x,
5     gaussian_pdf(x, mean, sigma),
6     linewidth=2.0,
7     label=(
8         "fit: "
9         r"$f(x)=\frac{1}{\sqrt{2\pi}}\sigma$"
10        r"\exp\{-\frac{(x-\mu)^2}{2\sigma^2}\right}$"
11        "\n"
12        + fr"$\mu={mean:.2f}, \sigma={sigma:.2f}$"
13    ),
14 )
15 ax.legend(loc="upper right")

```

図 9.12 でも、フィット曲線の式と推定パラメータは凡例へ入れている。本文、キャプション、凡例の役割を分けると、図全体が読みやすくなる。

```

1 model_y = linear_model(xdata, *params)
2 residual = ydata - model_y
3 print(residual)

```

数値が出たことと、解釈してよいことは別である。残差に系統的な構造が残っていないかを見ないまま結論へ進むべきではない。フィットが見た目に良さそうでも、特定の範囲だけ大きくずれていることがあるため、可能なら残差も確認したい。

また、外れ値や非物理的な領域を含めると、パラメータが不安定になることがある。どの範囲でフィットしたかを書き残すことは、再現性のためにも解釈のためにも重要である。

```

1 mask = (xdata >= 0.0) & (xdata <= 10.0)
2 params, cov = curve_fit(model, xdata[mask], ydata[mask])

```

フィット範囲を切るという行為は、都合の悪い部分を隠すことではなく、どの領域でそのモデルが有効だと思えるかを明示することである。したがって、結果を報告するときは範囲も本文に書くべきである。

10.3.2 σ を使った重み付きフィット

物理の実データでは、各データポイントが常に同じ信頼度を持つとは限らない。カウント数が少ない点は誤差が大きく、カウント数が多い点は誤差が小さい。この情報を無視してフィットを行うと、信頼性の低い点に引っ張られて誤った結果を生む。各点の信頼度が違うなら、 σ を使って重み付けする。

```

1 y_err = np.array([0.2, 0.2, 1.5, 0.2, 0.2])
2

```

```

3 params, cov = curve_fit(
4     linear_model,
5     xdata,
6     ydata,
7     sigma=y_err,
8     absolute_sigma=True,
9 )

```

sigma が大きい点は「ずれても仕方がない点」として弱く効く。図 10.1 は、その差を模式的に示している。

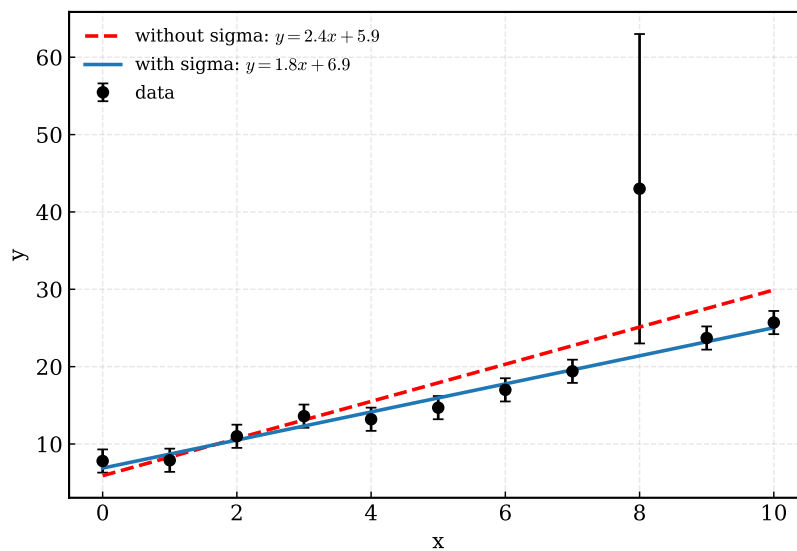


図 10.1 sigma の有無によるフィット結果の違い。誤差の大きい点を適切に弱く扱わないと、結果全体が不安定になる。

sigma には、通常 ydata の各点に対応する 1 次元配列を渡す。このとき最小化される量は

$$\chi^2(\mathbf{p}) = \sum_i \left(\frac{y_i - f(x_i; \mathbf{p})}{\sigma_i} \right)^2$$

と書けるので、 σ_i が大きい点ほど「ずれても仕方ない点」として弱く効き、 σ_i が小さい点ほど強く効く。カウント数 N が十分大きいポワソン統計なら、典型的には $\sqrt{N}$ を誤差の近似として使う。

absolute_sigma=True は、「今渡した sigma は本当に物理的な標準偏差である」と解釈させる指定である。これを False のままにすると、sigma は相対的な重みとしては使われるが、共分散行列の大きさは残差のばらつきに合わせてあとから再スケーリングされる。そのため、推定値の重み付けだけでなく誤差評価まで行いたいなら、sigma の意味を明確にした上で absolute_sigma の設定も意識する必要がある。

図 10.1 から明らかなように、sigma の指定を怠ると結果全体が台無しになる。

10.3.3 scipy.interpolate による補間

測定点の間を滑らかにつなぎたいときは補間を使う。SciPy には複数の補間法があるが、まずは 1 次元補間の基本を押さえるとよい。

```
1 from scipy.interpolate import interp1d
2
3 x = np.array([0.0, 1.0, 2.0, 4.0])
4 y = np.array([0.0, 2.0, 1.0, 3.0])
5
6 linear_interp = interp1d(x, y, kind="linear")
7 x_new = np.array([0.5, 1.5, 3.0])
8 y_new = linear_interp(x_new)
9
10 print(y_new)
```

補間点 x_{new} に対しては、次の値が返る。

```
[1.  1.5 2. ]
```

補間は「元データの間をつなぐ」操作であり、「外側まで正しく予測する」操作ではない。外挿まで含めて解釈すると危険である。

線形補間では、各区間 $x_i \leq x \leq x_{i+1}$ で

$$y(x) = y_i + \frac{y_{i+1} - y_i}{x_{i+1} - x_i}(x - x_i)$$

という一次式を使っている。したがって、測定点の間では滑らかにつながるが、測定範囲の外ではこの式をそのまま信じる根拠はない。

10.3.4 scipy.signal による平滑化とピーク検出

波形や時系列では、ピークの検出や軽い平滑化がよく必要になる。

```
1 from scipy.signal import savgol_filter, find_peaks
2
3 x = np.linspace(0.0, 20.0, 200)
4 signal = np.sin(x) + 0.15 * np.random.default_rng(7).normal(size=x.size)
5
6 smooth = savgol_filter(signal, window_length=11, polyorder=3)
7 peaks, props = find_peaks(smooth, height=0.7, distance=15)
8
9 print(peaks[:5])
10 print(props["peak_heights"][:5])
```

この例では、検出されたピーク位置とその高さが次のように表示される。

```
[ 15  77 140]
[0.94402164 1.14918478 1.00061725]
```

`savgol_filter()` は局所多項式で平滑化する。`find_peaks()` は高さ、距離、prominence などの条件でピーク候補を探す。ピーク数を自分で数える前に、こうした既製の道具があることを知っておくとよい。

10.3.5 `scipy.integrate` と `scipy.constants`

積分や定数も SciPy でまとめて扱える。

```
1 from scipy import constants
2 from scipy.integrate import simpson
3
4 x = np.linspace(0.0, np.pi, 101)
5 y = np.sin(x)
6
7 area = simpson(y, x=x)
8 print(area)
9 print(constants.c)
```

この積分値と光速定数は、次のように出力される。

```
2.0000000108245044
299792458.0
```

`simpson()` は、ここでは

$$\int_0^{\pi} \sin x \, dx = 2$$

という積分を、離散点 (x_i, y_i) から数値的に評価している。したがって、`simpson(y, x=x)` は「 $y(x)$ をサンプル点から積分する関数」と読めばよい。また、`constants.c` は

$$c = 299792458 \text{ m s}^{-1}$$

を表す定数である。ただし、単位付き計算や時刻系まで含めて厳密に扱いたいなら、次章の Astropy の方が自然な場面も多い。

10.4 よくある落とし穴

10.4.1 モデル選択の理由がないままフィットしない

ガウシアン、指数、直線のどれを使うかは、物理的・統計的な理由が必要である。見た目がそれらしく重なったという理由だけでモデルを採用すると、パラメータの意味も崩れる。

10.4.2 初期値と制約を無視しない

非線形フィットは、初期値次第で別の極小へ落ちたり、収束しなかったりする。`curve_fit()` が値を返したからといって、それが唯一の解だとは限らない。複数の初期値で

試す、図へ重ねて見る、残差を見る、といった確認が必要である。

10.4.3 補間と外挿を混同しない

`interp1d()` や `spline` は、観測点の間を埋めるための道具である。測定範囲の外で値を予測すると、もっともらしい数字が出ては根拠は薄い。外側までモデル化したいなら、補間ではなく関数フィットとして考えた方がよい。

10.4.4 信号処理ではサンプリング間隔を意識する

`find_peaks()` の `distance` やフィルタ長は、サンプル数で指定することが多い。したがって、時間軸が 1 サンプル 1 ns なのか 10 ns なのかで、同じ数値でも意味が変わる。コード中の数字をそのまま流用せず、軸の単位へ戻して考える必要がある。

10.5 解析へのつながり

SciPy は、解析の中で次のような役割を持つ。

1. `stats` で分布の形や要約統計を確認する
2. `signal` でピーク候補や平滑化を行う
3. `interpolate` で校正曲線や補助曲線を作る
4. `optimize` でモデルパラメータを推定する
5. `integrate` や `constants` で計算を閉じる

ここで重要なのは、SciPy が「魔法の黒箱」ではないという点である。入力配列がどういう意味を持ち、返ってきた値をどう解釈するかは自分で決めなければならない。特にフィットでは、図を見る、範囲を切る、重み付けを考える、残差を見る、という手順を省略すべきではない。

ここまで見てきた SciPy は、主に「数値をどう計算するか」を受け持つ道具である。時刻系、単位、座標系、FITS のように「数値そのものの意味付け」まで含めて扱いたくなったら、次章の Astropy が必要になる。SciPy は計算アルゴリズム、Astropy は物理量や観測量の表現を担当すると考えると、章のつながりも見えやすい。

本文を読み終えた後で関数名や引数を確認したいときは、SciPy User Guide [27] を起点にすると探しやすい。分布や検定は Statistics [28]、フィットや最適化は Optimization [29]、補間は Interpolation [30]、波形処理は Signal processing [31]、数値積分は Integration [32] が対応している。本章は、それらを解析で使う順に並べ替えて説明した。

10.6 章末問題

この章の課題では、まず既知の分布から作った小規模な検証用データで API と結果の読み方を確認し、その後に実データのモデルを検討する。検証用データは次のコードで作る。乱数シードを固定しているため、同じライブラリ環境では同じ入力を再現できる。

```
1 import numpy as np
2
3 rng = np.random.default_rng(20260412)
4 synthetic_dt = rng.exponential(scale=120.0, size=5000)
5 x = np.linspace(0.0, 10.0, 80)
6 y_true = 2.5 * x + 1.2
7 sigma = np.full_like(x, 0.5)
8 y_obs = y_true + rng.normal(scale=sigma)
```

実データについては、どの分布にどの関数を当ててるかを最初から決め打ちしない。まず図を描いて特徴を観察し、どの量を比べるべきか、どの範囲で近似を試すべきかを判断する。ヒストグラムにするか累積分布にするか、短時間側を含めるか、対数軸で見るかも、選択した理由とともに記録する。

1. **既知の指数分布で推定方法を検証する。** `scipy.stats.expon.fit(synthetic_dt, floc=0)` で指数分布のパラメータを推定せよ。`floc=0` は位置母数を 0 に固定する指定であり、戻り値は `loc` と `scale` の順である。推定された `scale` が生成時の 120 秒に近いことを確認し、標本平均とも比較せよ。許容範囲を 115 秒から 125 秒として自動確認する `assert` も書くこと。
2. **重み付き直線フィットと残差を確認する。** `curve_fit()` を使い、`y_obs` を $y = ax + b$ へフィットせよ。`sigma=sigma`、`absolute_sigma=True` を指定し、推定値 `popt` と共分散行列 `pcov` を受け取る。傾きが 2.3 から 2.7、切片が 0.7 から 1.7 の範囲に入ることを確認せよ。`np.sqrt(np.diag(pcov))` が各パラメータの標準不確かさを与えること、`y_obs - model(x, *popt)` が残差になることを説明し、データ、フィット線、残差を 2 段の図へ保存せよ。
3. **実データの発生間隔分布を観察し、fit の方針を決める。** 第 6 章で作った Δt 配列がある場合は、等間隔ビンのヒストグラム、対数ビンのヒストグラム、経験累積分布のうち少なくとも 2 通りを作成せよ。その上で、指数分布を含む候補を比較し、採用するモデルとフィット範囲を決める。1 問目の検証用データと異なり、実データが単一の指数分布に従う保証はない。モデルを採用した根拠、外した範囲、残差または適合度を記録すること。
4. **解析上の選択に対する感度を調べる。** 3 問目と少なくとも 1 条件を変えて推定を繰り返せ。ビン幅、フィット範囲、短時間側の扱い、重み、関数形などから選び、推定値と不確かさがどの程度変化したかを表にまとめる。結果が変わったという事実だけでなく、

どのデータ点の寄与が変化したためかを図と残差から説明せよ。

5. **発展課題。** マグニチュード分布について、度数分布と累積分布を作り、解析可能な範囲を確認せよ。分布の下端では観測の完全性が損なわれる可能性があるため、全範囲を無条件に一つの関数へ当てはめない。日本近海データと全球データのどちらかで始め、使用範囲、モデル、推定方法、残差を明示すること。

SciPy は数値配列に対する計算を担うが、物理単位、時刻系、天球座標の意味までは自動的に補わない。次章では Astropy を用い、数値に単位や座標系を保持したまま SciPy や表処理ライブラリへ受け渡す方法を扱う。

第 11 章

Astropy による時刻・単位・天体データ処理

Astropy は、天文学や宇宙物理の解析で頻出する「単位付き物理量」「時刻系」「座標系」「FITS」「天文向け表」を安全に扱うためのライブラリである。NumPy や SciPy だけでも数値計算はできるが、単位や時刻系を自前で文字列管理すると、解釈違いや換算ミスが入りやすい。本章では、Astropy の考え方、最小限のコード例、落とし穴、解析へのつながりを整理する。

11.1 Astropy の考え方

11.1.1 数値だけでなく意味も持たせる

Astropy の中心的な発想は、「数値そのもの」ではなく「何の単位で、どの時刻系で、どの座標系なのか」までオブジェクトへ持たせることである。例えば 100 という値だけでは、100 ns なのか 100 ms なのか分からない。Astropy の Quantity を使えば、この曖昧さを減らせる。

```
1 import astropy.units as u
2
3 time_window = 250 * u.ns
4 speed = 299792458 * u.m / u.s
5 distance = speed * time_window
6
7 print(distance)
8 print(distance.to(u.m))
```

この計算では、単位付きのまま距離が評価され、次のように表示される。

```
74.9481145 m
74.9481145 m
```

この例では、`250 * u.ns` という時点で「250 ns」という意味が値に埋め込まれている。式の途中で単位を落とさない限り、単位換算はライブラリが引き受ける。

数式で書けば、ここで行っているのは

$$d = v\Delta t$$

である。特に $v = c$ とみなすなら、250 ns の時間窓に対応する距離 $d = c\Delta t$ をそのまま安全に計算できる。

11.1.2 時刻系と座標系を自前で持たない

観測データでは、UTC、TAI、TT、MJD、JD、Unix time が混在することがある。これを自前の文字列処理でつなぐと、秒境界やうるう秒で破綻しやすい。Astropy の Time と SkyCoord は、こうした表現を専用オブジェクトとして扱う。

11.2 最小限のコード例

11.2.1 units と Quantity

単位付き計算の基本は Quantity である。

```
1 import astropy.units as u
2
3 length = 3.2 * u.m
4 time = 12.5 * u.ns
5 velocity = length / time
6
7 print(velocity)
8 print(velocity.to(u.m / u.s))
```

単位変換の結果は次の通りである。

```
0.256 m / ns
256000000.0 m / s
```

to() を使えば単位変換を明示できる。解析メモの段階では、いきなり .value で生の数値へ落とさず、できるだけ単位付きのまま式を組んだ方が安全である。

この計算は

$$v = \frac{L}{t}$$

に対応しており、具体的には

$$v = \frac{3.2 \text{ m}}{12.5 \text{ ns}} = 0.256 \text{ m/ns} = 2.56 \times 10^8 \text{ m/s}$$

である。コードと単位換算を一緒に見ておくと、桁や接頭辞の取り違えに気付きやすい。

11.2.2 物理定数

Astropy には物理定数も入っている。SciPy の `constants` と似ているが、こちらは単位付きで返る点が多い。

```
1 from astropy.constants import c, k_B
2
3 print(c)
4 print(k_B)
```

Astropy の定数オブジェクトを表示すると、値だけでなく単位と参照元も確認できる。

```
Name   = Speed of light in vacuum
Value   = 299792458.0
Uncertainty = 0.0
Unit    = m / s
Reference = CODATA 2018
Name    = Boltzmann constant
Value   = 1.380649e-23
Uncertainty = 0.0
Unit    = J / K
Reference = CODATA 2018
```

数値と単位が一緒に出るので、式の中で何を使っているかを見失いにくい。

11.2.3 Time による時刻表現

Astropy の `Time` は、複数のフォーマットと複数の時刻系をまたいで扱える。

```
1 from astropy.time import Time
2
3 t = Time("2026-04-12 12:34:56.123456789", format="iso", scale="utc")
4
5 print(t.mjd)
6 print(t.unix)
7 print(t.tai.iso)
```

同じ時刻を複数の表現へ変換すると、次の値が得られる。

```
61142.52426068816
1775997296.1234567
2026-04-12 12:35:33.123
```

`format="iso"` は入力形式、`scale="utc"` は時刻系を表す。mjd、unix、tai.iso のように別形式・別時刻系へ変換できる。重要なのは、`datetime` の代替品というより、「観測時刻を正しい時刻系のまま持つ道具」として捉えることである。

たとえば、修正ユリウス日 MJD は

$$\text{MJD} = \text{JD} - 2400000.5$$

で定義される。Astropy は、このような時刻表現どうしの対応を自前で管理せずに済むようにしてくれる。

11.2.4 時刻差と TimeDelta

差分計算も Time のまま行える。TimeDelta によって、差分の単位変換も一貫して扱える。

```
1 from astropy.time import Time
2
3 t1 = Time("2026-04-12T12:00:00.000000100", format="isot", scale="utc")
4 t2 = Time("2026-04-12T12:00:00.000000320", format="isot", scale="utc")
5 dt = t2 - t1
6
7 print(dt)
8 print(dt.to_value("ns"))
```

時刻差をそのまま表示した結果と、ns へ変換した結果は次の通りである。

```
2.2e-07
220.0
```

このように書くと、秒境界や日付境界をまたいでも同じ考え方で差分が取れる。課題のように 250 ns の窓を扱うときも、自前で桁を切り貼りするより安全である。

ここで計算しているのは単純に

$$\Delta t = t_2 - t_1 = 220 \text{ ns}$$

である。重要なのは、コード上では単に引き算しているだけでも、背後で時刻系と単位が保たれている点である。

11.2.5 SkyCoord による座標変換

Astropy の `coordinates` は、天球上の座標系を変換する。SkyCoord はその中心となるクラスである。

```
1 from astropy.coordinates import SkyCoord
2 import astropy.units as u
3
4 coord = SkyCoord(ra=150.0 * u.deg, dec=20.0 * u.deg, frame="icrs")
5 gal = coord.galactic
6
7 print(gal.l.deg)
8 print(gal.b.deg)
```

座標変換後の銀河経度と銀河緯度は次の通りである。

```
213.80097741291977
50.26374216488792
```

重要なのは、`ra` と `dec` に単位を付けている点である。度なのかラジアンなのかを曖昧にしない方が安全である。

この操作は、赤道座標 (α, δ) から銀河座標 (l, b) への変換に対応する。天球上の回転そのものは自分で式を書かなくても、入力座標系と出力座標系を明示すれば `Astropy` が一貫して扱える。

11.2.6 Table と QTable

`Astropy` には表処理のために `Table` と `QTable` がある。`QTable` は列へ単位を保持できる。

```
1 from astropy.table import QTable
2 import astropy.units as u
3
4 table = QTable()
5 table["time"] = [0.0, 1.0, 2.0] * u.s
6 table["signal"] = [12.4, 10.1, 14.8] * u.mV
7 table["detector"] = ["A", "A", "B"]
8
9 print(table)
```

`QTable` を表示すると、列名だけでなく単位も一緒に確認できる。

time	signal	detector
s	mV	
0.0	12.4	A
1.0	10.1	A
2.0	14.8	B

`QTable` は `pandas` の `DataFrame` より単位との相性がよい。一方で、複雑な `groupby` や `join`、列演算の豊富さでは `pandas` や `Polars` の方が便利なのも多い。どちらを中心にするかは、何を守りたいかで決まる。

11.2.7 Astropy Table と pandas の行き来

`Astropy` の表と `pandas` の `DataFrame` は相互変換できるが、単位や時刻の情報が完全には残らないことがある。

```
1 df = table.to_pandas()
2 print(df)
```

`to_pandas()` は便利だが、どの情報が失われるかを理解してから使う方がよい。特に単位やマスク情報を持つ列は、変換後の解釈を確認した方が安全である。

11.2.8 astropy.io.fits による FITS の読み書き

天文データでは FITS が標準的である。Astropy なら NumPy 配列やヘッダをそのまま扱える。

```
1 import numpy as np
2 from astropy.io import fits
3
4 image = np.arange(9).reshape(3, 3)
5 hdu = fits.PrimaryHDU(image)
6 hdu.header["OBJECT"] = "test"
7 hdu.writeto("image.fits", overwrite=True)
8
9 with fits.open("image.fits") as hdu1:
10     data = hdu1[0].data
11     header = hdu1[0].header
12
13 print(data.shape)
14 print(header["OBJECT"])
```

FITS を書いてから読み直すと、配列形状とヘッダ値は次のように確認できる。

```
(3, 3)
test
```

`fits.open()` は HDU のリストを返す。画像なら `PrimaryHDU`、表なら `BinTableHDU` である。最初は「どの HDU に何が入っているか」を確認し、その後で必要なデータだけ読む方が混乱しにくい。

11.2.9 ASCII や ECSV も読める

Astropy の表入出力は FITS だけではない。`Table.read()` は ASCII、CSV、ECSV などにも対応している。

```
1 from astropy.table import Table
2
3 tbl = Table.read("events.ecsv", format="ascii.ecsv")
4 print(tbl.colnames)
5 print(len(tbl))
```

`Table.read()` の第 1 引数は入力パス、`format="ascii.ecsv"` は ECSV として解釈する指定である。例えば 3 行で `time` と `signal` の 2 列を持つ表なら、`['time', 'signal']` と 3 が表示される。ECSV は単位やメタデータを持たせやすいので、テキスト形式のまま情報を落としにくい。共同研究や中間生成物の受け渡しでは便利である。保存するときは `table.write("events.ecsv", format="ascii.ecsv", overwrite=True)` と書ける。`overwrite=True` は同名ファイルの置換を許可する指定である。

11.3 よくある落とし穴

11.3.1 `.value` を早く使い過ぎない

`Quantity.value` を使うと、生の NumPy 配列やスカラーへ戻る。これは便利だが、その瞬間に単位情報は消える。式の早い段階で `.value` を多用すると、「何 ns のつもりだったか」「何 mV のつもりだったか」が見えなくなる。

11.3.2 時刻系を省略しない

`Time("2026-04-12 12:00:00")` とだけ書くと、どの format と scale を想定しているのかが曖昧になる。特に UTC、TAI、TT をまたぐときは、`format=` と `scale=` を明示した方が安全である。

11.3.3 座標変換には観測条件が必要なことがある

赤道座標から銀河座標への変換だけなら比較的単純だが、地平座標へ変換するには観測地点や観測時刻が必要になる。座標系だけを変えているつもりでも、背後では時刻と場所が意味を持つことがある。

11.3.4 DataFrame への変換で情報が落ちる

Astropy Table を pandas や Polars へ変換すると、単位、マスク、時刻系、メタデータが薄まることもある。逆に pandas の強い `groupby` や `join` を使いたいからといって、何でも最初から DataFrame にするべきではない。まず何を保存したいのかを決める方がよい。

11.4 解析へのつながり

Astropy を使うべき場面は、数値とともに単位、時刻系、座標系、天文形式の情報を保持する必要がある場合である。代表的な用途を次に示す。

- 観測時刻を UTC、MJD、JD、Unix time の間で変換したいとき
- ns、 μ s、mV、MeV などの単位を混ぜて計算したいとき
- 天球座標を別の座標系へ変換したいとき
- FITS 画像や FITS テーブルを直接読み書きしたいとき
- 表データに単位やメタデータを持たせたまま運びたいとき

一方で、巨大な表の集計や複雑な join を高速に回すなら、pandas や Polars の方が便利である。実務では、まず Astropy で単位と時刻を正しく解釈し、その後に必要な部分だけを pandas や Polars へ渡して集計し、最後に Matplotlib や SciPy へつなぐことが多い。

ここまでの内容を公式資料で引き直すなら、まず Getting Started [34] と User Guide [35] を入口にするとよい。その上で、単位は Units and Quantities [36]、時刻は Time and Dates [37]、座標は Coordinates [38]、表は Tables [39]、FITS は FITS File Handling [40] が対応している。本章では、それらの中でも観測データ解析で頻繁に使う順に絞って説明した。

11.5 章末問題

1. **単位を保ったまま時間窓を比較する。** `window = 250 * u.ns` とし、`dt = np.array([120, 220, 250, 251, 800]) * u.ns` の各要素が時間窓以内かを判定せよ。真になる要素は 120、220、250 ns の 3 個である。境界の 250 ns を含めるために `<` ではなく `<=` を使う理由と、`u.ns` を付けずに比較した場合の問題を説明せよ。
2. **秒境界をまたぐ時刻差を求める。** `2026-04-12T11:59:59.999999900` と `2026-04-12T12:00:00.000000120` を UTC の Time として作り、差を ns で表示せよ。結果は約 220 ns になり、250 ns 以内である。`format="isot"`、`scale="utc"`、`to_value("ns")` の役割を説明すること。
3. **UTC と TAI を区別して同一時刻を表す。** 同じ Time オブジェクトについて `utc.iso` と `tai.iso` を表示し、文字列表現が異なっても同じ瞬間を表していることを確認せよ。文字列を直接引き算する方法が不適切である理由を、時刻系とうるう秒の観点から説明せよ。
4. **単位付き表を ECSV と FITS へ保存する。** `time`、`signal`、`detector` を持つ QTable を作り、`time` に ns、`signal` に mV を付ける。ECSV へ保存して読み戻し、各列の単位と行数を表示せよ。さらに同じ信号値を FITS 画像として保存し、`fits.open()` で形状とヘッダを確認せよ。`overwrite=True` を付ける場合の挙動も説明すること。
5. **座標系を明示して変換する。** 赤経 150 度、赤緯 20 度を ICRS の SkyCoord として作り、銀河座標へ変換せよ。銀河経度は約 213.801 度、銀河緯度は約 50.264 度になる。`ra`、`dec`、`frame` の意味を述べ、地平座標への変換では観測時刻と観測地点が追加で必要になる理由を説明せよ。

ここまでに、データの読み込み、整形、計算、可視化、物理量の表現を一通り扱った。同じ解析を別のデータへ安全に適用するには、処理を関数とモジュールへ分け、テスト可能な公開インタフェースを定める必要がある。次章では、そのためのパッケージ構成と依存関係の管理を扱う。

第 12 章

自作モジュールを作って再利用する

解析コードは最初、1 本のスクリプトとして書き始めることが多い。しかし、同じ処理を別の日のデータにも使いたくなり、さらに別のプロジェクトからも呼びたくなると、単発スクリプトのままでは管理しにくい。こういうときは、処理を自作モジュールとして切り出し、必要ならパッケージとしてインストールできる形へ整理した方がよい。

ここでいう「モジュール」は、`plotting.py` や `models.py` のような一つの Python ファイルを指す。一方、パッケージは、複数のモジュールをディレクトリへまとめ、`import` できる形にした単位である。本章では、`uv` を使ってパッケージを作り、`pip install` と `pip install -e` で利用できる状態にするまでを説明する。

12.1 なぜ自作モジュールにするのか

処理をモジュールへ分ける利点は、単にファイル数が増えることではない。役割ごとに分離でき、テストしやすくなり、エディタから補完やドキュメントも見やすくなる。特に解析では、次のような部品が再利用されやすい。

- データ読み込み関数
- フィルタ条件やカットの判定関数
- 図を作成して保存する関数
- フィット関数やモデル関数

こうした処理を `main.py` の中へすべて書いてしまうと、別スクリプトから流用するときにはコピーペーストが増える。コピー元とコピー先がずれると、片方だけバグ修正される、といった問題も起こりやすい。自作モジュールへ切り出して `from histfit_tools import save_histogram_with_fit` のように呼べる形にしておけば、修正箇所は 1 か所で済む。

12.2 uv でパッケージ開発を始める

12.2.1 最初の初期化

本書では、単発スクリプトではなく「後で import される部品」として育てたいので、uv init --package から始めるのが自然である。たとえばヒストグラム作図用のパッケージを作るなら、次のように実行する。

```
$ mkdir histfit_tools
$ cd histfit_tools
$ uv init --package
$ uv add numpy matplotlib scipy
$ uv add --dev pytest ruff
$ uv sync
```

uv add で追加した numpy、matplotlib、scipy は、このパッケージを使うときにも必要な実行時依存関係である。一方、pytest や ruff は開発者だけが使う道具なので、--dev 付きで追加する。たとえば pytest は、本章で後から示す tests/test_models.py のように、linear が期待通りの値を返すか、未知のモデル名で ValueError が正しく送出されるかなどのテストを行う際に使用する。ruff は、uv run ruff check . として実行し、未使用 import や単純な書式崩れを commit 前に見つけるための道具である。pytest や ruff によるテストは必須ではないが、将来的にパッケージを公開する予定があるなら早い段階で実装しておくことが好ましい。

最終的なディレクトリ構成は、少なくとも次のように整理したい。

histfit_tools/ の例

```
.gitignore
.python-version
README.md
pyproject.toml
src/
`-- histfit_tools/
    |-- __init__.py
    |-- models.py
    `-- plotting.py
tests/
`-- test_models.py
```

ここで src/ の下にパッケージを置く構成を src レイアウトという。これは、作業ディレクトリ直下にあるファイルをたまたま import してしまう事故を防ぎやすい。パッケージとして配布する前提なら、この構成にしておく方が後で困りにくい。

12.2.2 pyproject.toml の役割

パッケージ開発では、`pyproject.toml` が中心になる。ここにはパッケージ名、Python バージョン条件、依存関係、ビルド方法などを書く。上の例なら、少なくとも次のような内容になる。

pyproject.toml

```
[project]
name = "histfit_tools"
version = "0.1.0"
description = "Histogram plotting and fitting utilities."
readme = "README.md"
requires-python = ">=3.12"
dependencies = [
    "matplotlib>=3.10",
    "numpy>=2.0",
    "scipy>=1.15",
]

[build-system]
requires = ["hatchling"]
build-backend = "hatchling.build"

[dependency-groups]
dev = [
    "pytest>=8.3",
    "ruff>=0.11",
]
```

`dependencies` に書いた内容は、このパッケージを `pip install` したときに一緒に解決される。つまり、「自分の環境でたまたま入っているから動く」のではなく、「このパッケージを使うにはこれが必要だ」と宣言していることになる。

一方、`uv.lock` は開発側で環境を厳密にそろえるためのファイルである。自分のリポジトリへ戻ってきたときや、共同研究者が同じ環境を再現したいときには `uv lock` や `uv sync` が役に立つ。しかし、利用者が `pip install` するときに参照するのは基本的に `pyproject.toml` であり、`uv.lock` ではない。この違いは重要である。

12.3 具体例: ヒストグラムとフィットをまとめたパッケージ

ここでは、1 次元データを与えると、`numpy.histogram` でヒストグラムを作り、`matplotlib` の `errorbar` で描き、必要なら `scipy.optimize.curve_fit` でフィットした曲線を重ねて保存する関数を作る。要求仕様は次の通りとする。

- データ列と保存先を受け取る
- bins か bin_width のどちらかでビンを指定できる
- fit=False にするとフィットせずにヒストグラムだけ保存する
- linear、quadratic、power_law、gaussian を名前で選べる
- 自分で定義した関数もそのまま渡せる
- フィット初期値 p0 も引数から与えられる

12.3.1 モデル関数を分ける

まず、フィット関数群を models.py へ切り出す。

src/histfit_tools/models.py

```

1 from collections.abc import Callable
2
3 import numpy as np
4
5 ModelFunc = Callable[..., np.ndarray]
6
7
8 def linear(x: np.ndarray, a: float, b: float) -> np.ndarray:
9     """一次関数  $y = a x + b$  を返す。"""
10    return a * x + b
11
12
13 def quadratic(x: np.ndarray, a: float, b: float, c: float) -> np.ndarray:
14     """二次関数  $y = a x^2 + b x + c$  を返す。"""
15    return a * x**2 + b * x + c
16
17
18 def power_law(x: np.ndarray, amplitude: float, alpha: float) -> np.ndarray:
19     """べき関数  $\text{amplitude} * x^\alpha$  を返す。 $x > 0$  を仮定する。"""
20    return amplitude * np.power(x, alpha)
21
22
23 def gaussian(
24     x: np.ndarray,
25     amplitude: float,
26     mean: float,
27     sigma: float,
28     offset: float,
29 ) -> np.ndarray:
30     """ガウス関数と定数オフセットを返す。"""
31     z = (x - mean) / sigma
32     return amplitude * np.exp(-0.5 * z**2) + offset
33
34
35 NAMED_MODELS: dict[str, ModelFunc] = {

```

```

36     "linear": linear,
37     "quadratic": quadratic,
38     "power_law": power_law,
39     "gaussian": gaussian,
40 }
41
42
43 def resolve_model(model: str | ModelFunc) -> ModelFunc:
44     """名前または callable からフィット関数を返す。"""
45     if callable(model):
46         return model
47
48     try:
49         return NAMED_MODELS[model]
50     except KeyError as exc:
51         names = ", ".join(sorted(NAMED_MODELS))
52         raise ValueError(f"Unknown model: {model}. Choose from {names}.") from exc

```

モデル関数を別ファイルへ置いておくと、作図ロジックと数学的なモデルが分離される。その結果、後で指数減衰やローレンツ関数を追加したくなったときも、`plotting.py` を大きく書き換えずに済む。

12.3.2 pytest で最小のテストを書く

`tests/test_models.py` は、`models.py` の公開関数が最低限期待通りに振る舞うかを確認するためのファイルである。ここでは、式の値そのもの、ガウス関数の基本的な形、未知のモデル名に対する例外の 3 点だけを確認しておく。

tests/test_models.py

```

1 import numpy as np
2 import pytest
3
4 from histfit_tools.models import gaussian, linear, resolve_model
5
6
7 def test_linear_returns_expected_values() -> None:
8     x = np.array([0.0, 1.0, 2.0])
9     y = linear(x, 2.0, 1.0)
10    assert np.allclose(y, [1.0, 3.0, 5.0])
11
12
13 def test_gaussian_has_peak_at_mean() -> None:
14     x = np.array([4.0, 5.0, 6.0])
15     y = gaussian(x, amplitude=10.0, mean=5.0, sigma=1.0, offset=2.0)
16     assert y[1] == pytest.approx(12.0)
17     assert y[1] > y[0]
18     assert y[0] == pytest.approx(y[2])
19

```

```

20
21 def test_resolve_model_rejects_unknown_name() -> None:
22     with pytest.raises(ValueError):
23         resolve_model("lorentzian")

```

最初のテストは、一次関数の実装が式通りかを直接確かめている。2 つ目は、ガウス関数が平均値の位置で最大になり、左右対称になることを見ている。3 つ目は、誤ったモデル名を黙って通さず、利用者へ早い段階でエラーを返すことを確かめるテストである。

実行は次のように行う。

```

$ uv run pytest
$ uv run ruff check .

```

ここで `pytest` は関数の振る舞いを確かめ、`ruff` は未使用 `import` や単純な書式の乱れを確認する。今回のような小さなパッケージでも、この 2 つがあるだけで「動いているつもり」の状態を減らしやすい。

12.3.3 ヒストグラムを作って保存する関数

次に、実際の主役である `save_histogram_with_fit` を `plotting.py` に書く。

```

src/histfit_tools/plotting.py
1 from collections.abc import Sequence
2 from pathlib import Path
3
4 import matplotlib.pyplot as plt
5 import numpy as np
6 from numpy.typing import ArrayLike
7 from scipy.optimize import curve_fit
8
9 from .models import ModelFunc, resolve_model
10
11
12 def _make_bin_edges(
13     values: np.ndarray,
14     bins: int | Sequence[float] | None,
15     bin_width: float | None,
16     data_range: tuple[float, float] | None,
17 ) -> np.ndarray:
18     if bins is not None and bin_width is not None:
19         raise ValueError("Specify either bins or bin_width, not both.")
20
21     if data_range is None:
22         low = float(np.min(values))
23         high = float(np.max(values))
24     else:
25         low, high = data_range
26

```

```

27     if low == high:
28         high = low + 1.0
29
30     if bin_width is not None:
31         if bin_width <= 0:
32             raise ValueError("bin_width must be positive.")
33         return np.arange(low, high + bin_width, bin_width)
34
35     if bins is None:
36         bins = 40
37
38     if isinstance(bins, int):
39         return np.linspace(low, high, bins + 1)
40
41     return np.asarray(bins, dtype=float)
42
43
44 def save_histogram_with_fit(
45     data: ArrayLike,
46     output_path: str | Path,
47     *,
48     bins: int | Sequence[float] | None = 40,
49     bin_width: float | None = None,
50     data_range: tuple[float, float] | None = None,
51     fit_model: str | ModelFunc = "gaussian",
52     fit: bool = True,
53     p0: Sequence[float] | None = None,
54     xlabel: str = "x",
55     ylabel: str = "Counts",
56     title: str | None = None,
57 ) -> Path:
58     """ヒストグラムと誤差棒を描き、必要ならフィット線も重ねて保存する。
59
60     Args:
61         data: 1 次元データ列。
62         output_path: 保存先ファイル名。
63         bins: ビン数またはビン境界列。
64         bin_width: ビン幅。bins と同時には指定しない。
65         data_range: ヒストグラム範囲。
66         fit_model: モデル名または自作 callable。
67         fit: False ならフィットを実行しない。
68         p0: curve_fit へ渡す初期値。
69         xlabel: x 軸ラベル。
70         ylabel: y 軸ラベル。
71         title: 図タイトル。
72
73     Returns:
74         保存したファイルの Path。
75     """

```

```
76 values = np.asarray(data, dtype=float)
77 if values.ndim != 1:
78     raise ValueError("data must be one-dimensional.")
79
80 bin_edges = _make_bin_edges(values, bins, bin_width, data_range)
81 counts, bin_edges = np.histogram(values, bins=bin_edges)
82 centers = 0.5 * (bin_edges[:-1] + bin_edges[1:])
83 errors = np.sqrt(counts)
84
85 fig, ax = plt.subplots()
86 ax.errorbar(centers, counts, yerr=errors, fmt="o", capsize=3, label="data")
87
88 if fit:
89     model = resolve_model(fit_model)
90     sigma = np.where(errors > 0, errors, 1.0)
91     params, _ = curve_fit(
92         model,
93         centers,
94         counts,
95         p0=p0,
96         sigma=sigma,
97         absolute_sigma=True,
98         maxfev=10000,
99     )
100     x_fit = np.linspace(bin_edges[0], bin_edges[-1], 400)
101     y_fit = model(x_fit, *params)
102     ax.plot(x_fit, y_fit, label="fit")
103
104 ax.set_xlabel(xlabel)
105 ax.set_ylabel(ylabel)
106 if title is not None:
107     ax.set_title(title)
108 ax.legend()
109
110 output_path = Path(output_path)
111 output_path.parent.mkdir(parents=True, exist_ok=True)
112 fig.savefig(output_path, dpi=150, bbox_inches="tight")
113 plt.close(fig)
114 return output_path
```

この関数では、ビン境界の決定と作図本体を分けている。こうしておくと、ビン幅指定のロジックだけを単独にテストしやすい。また、`fit_model` が文字列なら既定のモデル群から選び、callable ならそのまま使うので、利用者は既定モデルと自作モデルを同じ引数で扱える。

12.3.4 公開インタフェースを `__init__.py` で整える

パッケージの外から使ってほしい関数は、`__init__.py` でまとめて再公開しておくといよい。

```
src/histfit_tools/__init__.py
1  """Histogram plotting and fitting utilities."""
2
3  from .models import gaussian, linear, power_law, quadratic
4  from .plotting import save_histogram_with_fit
5
6  __all__ = [
7      "gaussian",
8      "linear",
9      "power_law",
10     "quadratic",
11     "save_histogram_with_fit",
12 ]
```

こうしておけば、利用側は `from histfit_tools import save_histogram_with_fit` と短く書ける。内部ファイル構成を後で整理し直しても、公開 API を保っている限り、利用者側のコードを壊しにくい。

12.3.5 使い方の例

実際にこのパッケージを使うスクリプトは、たとえば次のようになる。

```
1  from pathlib import Path
2
3  import numpy as np
4
5  from histfit_tools import save_histogram_with_fit
6
7  rng = np.random.default_rng(42)
8  data = rng.normal(loc=5.0, scale=1.2, size=4000)
9
10 save_histogram_with_fit(
11     data,
12     Path("figures/gaussian_fit.png"),
13     bins=35,
14     fit_model="gaussian",
15     p0=[250.0, 5.0, 1.0, 1.0],
16     xlabel="signal [mV]",
17     title="Gaussian fit example",
18 )
19
20 save_histogram_with_fit(
21     data,
22     Path("figures/hist_only.png"),
```

```

23     bin_width=0.2,
24     fit=False,
25     xlabel="signal [mV]",
26 )

```

1 つ目はガウス関数でフィットし、2 つ目は `fit=False` としてヒストグラムだけを保存している。ビン数指定とビン幅指定の両方が動くことも、この 2 例で確認できる。

1 つ目の呼出しで保存される図の例を図 12.1 に示す。ヒストグラムは入力した 4000 個の信号値を表し、曲線はビンごとの度数へ当てはめたガウス関数である。この入力は一平均 5.0 mV、標準偏差 1.2 mV の正規分布から生成しているため、凡例に示した推定値が生成条件に近いことを確認できる。図を保存できるだけでなく、既知の入力に対して妥当なパラメータが得られることまで確かめる必要がある。

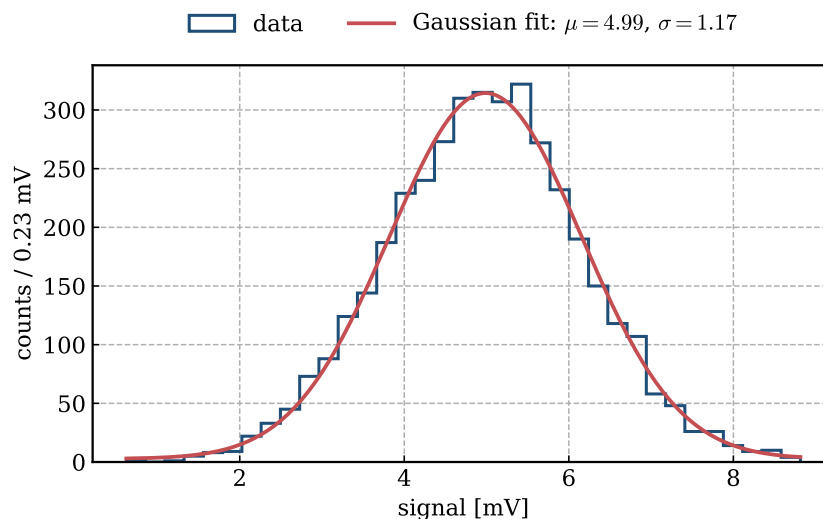


図 12.1 `save_histogram_with_fit()` の出力例。ヒストグラムは乱数シード 42 で生成した 4000 個の信号値、実線はガウス関数によるフィットを表す。

`fit_model` には、あらかじめ登録した名前だけでなく、同じ呼出し規約を持つ自作関数も渡せる。指数減衰関数を渡す例を次に示す。

```

1  import numpy as np
2
3  from histfit_tools import save_histogram_with_fit
4
5
6  def exponential_decay(
7      x: np.ndarray,
8      amplitude: float,
9      tau: float,
10     offset: float,
11 ) -> np.ndarray:
12     return amplitude * np.exp(-x / tau) + offset
13

```

```
14
15 save_histogram_with_fit(
16     data,
17     "figures/custom_fit.png",
18     bin_width=0.25,
19     fit_model=exponential_decay,
20     p0=[120.0, 3.0, 5.0],
21 )
```

このように、`fit_model` へ callable を渡せるようにしておけば、パッケージ側は柔軟性を保ちつつ、利用者側は自分の解析に必要なモデルを差し込める。

12.4 docstring、型ヒント、README

自作モジュールは、動くだけでは不十分である。後から自分で使うときにも、他人が使うときにも、「何を受け取り、何を返すか」がすぐ分かる形で書いておく方がよい。そのために有効なのが docstring、型ヒント、README である。

12.4.1 docstring と型ヒント

基礎編で見た docstring だが、パッケージではその重要性がさらに増す。特に、引数の意味や既定値、返り値、例外条件まで書いておくと、`help()` だけでなくエディタの補完でも内容が見やすくなる。

先ほどの `save_histogram_with_fit` には、`data: ArrayLike`、`output_path: str | Path`、`fit: bool = True` のような型ヒントが付いている。VS Code などのエディタでは、関数名にカーソルを合わせたときや補完候補を開いたときに、これらの型ヒントと docstring の先頭がそのまま表示されることが多い。つまり、docstring は文書であると同時に、普段のコーディング画面そのものへ表示される説明でもある。

そのため、docstring の最初の 1 行には、関数の役割を簡潔に書く方がよい。たとえば「ヒストグラムと誤差棒を描き、必要ならフィット線も重ねて保存する。」のように書けば、補完欄を見ただけでも関数の用途が分かる。

12.4.2 README に書くべきこと

README.md は、リポジトリを開いた人が最初に読む文書である。少なくとも次の 4 点は入れておきたい。

- このパッケージが何をするか
- どうインストールするか
- 最小の使用例
- 主要な依存関係

たとえば最小限の README は次のようになる。

README.md

```
# histfit_tools

Histogram plotting and fitting utilities for analysis scripts.

## Installation

```bash
pip install .
```

## Example

```python
from histfit_tools import save_histogram_with_fit
```
```

将来 GitHub へ置いたり、研究室内で URL を共有したりするときも、README があるだけで入口がかなり分かりやすくなる。docstring は関数単位の説明、README はパッケージ全体の説明、と役割を分けると整理しやすい。

12.5 pip install . と pip install -e .

12.5.1 通常インストール

作ったパッケージを別の環境から使いたいときは、パッケージのルートディレクトリで `pip install .` を実行する。

```
$ source .venv/bin/activate
$ pip install .
```

これは「いまのソースコードのスナップショットを環境へインストールする」操作である。したがって、後から `src/histfit_tools/plotting.py` を編集しても、その変更は自動では反映されない。もう一度 `pip install .` し直す必要がある。

12.5.2 editable install

開発中は、`pip install -e .` の方が便利である。

```
$ source .venv/bin/activate
$ pip install -e .
```

`-e` は `editable` の略であり、環境にコピーを置く代わりに、手元のソースディレクトリを参

照する形でインストールする。つまり、Python ファイルを編集した結果がすぐ import 側へ反映される。関数定義や docstring を調整しながら試す段階では、こちらを使う方が自然である。

ただし、editable install は「何もかも自動で再ビルドされる」という意味ではない。pyproject.toml の依存関係を書き換えたときや、後述する pybind11 のような C++ 拡張を編集したときは、再インストールや再ビルドが必要になることがある。この点は覚えておいた方がよい。

12.5.3 インストール確認

インストール後は、短い import テストを必ずしておく方がよい。

```
$ python -c "from histfit_tools import save_histogram_with_fit; print(save_histogram_with_fit)"
```

ImportError が出なければ、少なくともパッケージとして見つかっている。さらに依存関係も含めて確認したいなら、短い実データや乱数データで関数を 1 回実行して、図ファイルが保存されることまで確認すると確実である。

12.6 Git リポジトリへ置いて共有する

研究室内や共同研究で共有するなら、Git リポジトリとして管理しておく方がよい。uv init --package は既定では .git/ も作るので、通常は改めて git init し直す必要はない。

GitHub で共有する場合は、まず GitHub の Web 画面で空のリポジトリを 1 つ作る。ここではリポジトリ名と公開範囲だけを決めればよい。README や .gitignore は手元ですでに用意しているので、最初は GitHub 側で追加しない方が流れが単純である。

その準備ができれば、手元では次のように進めればよい。

```
$ git add pyproject.toml README.md src tests .python-version
$ git commit -m "Create histfit_tools package"
$ git remote add origin git@github.com:username/histfit_tools.git
$ git push -u origin main
```

uv init --vcs none のように Git 初期化を明示的に無効化して始めた場合には、最初に git init が必要になる。

.venv/ や __pycache__/ はコミットしないので、.gitignore に書いておくのがよい。Git の詳細については付録 B を参照されたい。

GitHub などへ置いたりリポジトリは、別環境からそのまま pip install できる。たとえば次のように書く。

```
$ pip install "git+ssh://git@github.com:username/histfit_tools.git"
```

タグやコミットを固定したいなら、URL の末尾に @v0.1.0 や @9f3c1a2 を付ける。

```
$ pip install "git+ssh://git@github.com/username/histfit_tools.git@v0.1.0"
```

これは再現性のために重要である。研究ノートや論文用メモには、「どのタグを入れたか」まで残しておく方が後で確実である。

12.7 依存関係をどう書くか

最後に、依存関係の考え方を整理しておく。パッケージでは「いま自分の環境で動く」ことより、「利用者の環境でも必要なものが入る」ことの方が重要である。そのため、次の3種類を区別するとよい。

実行時依存関係 パッケージを import して使うだけで必要なもの。今回なら `numpy`、`matplotlib`、`scipy` がこれに当たる。

開発用依存関係 テスト、整形、静的解析にだけ必要なもの。今回なら `pytest` や `ruff` がこれに当たる。

ローカル環境の固定 開発者どうしで完全に同じ環境を再現したいときの情報。これは `uv.lock` が担う。

ライブラリとして配布するなら、`dependencies` にはあまり厳しすぎる固定を入れない方がよい。たとえば `numpy==2.0.0` のように完全固定すると、利用者側の既存環境と衝突しやすい。むしろ `numpy>=2.0` のように「必要な下限」を書き、自分の開発環境の厳密な再現は `uv.lock` に任せる方が扱いやすい。

また、パッケージを書いていると、Jupyter、Sphinx、mkdocs など「利用者全員には不要だが、自分は使いたい」ものが増えやすい。そのとき全部を実行時依存関係へ入れてしまうと、インストールが重くなる。依存関係は増やすほど便利になる一方で、衝突や管理コストも増える。だからこそ、まずは公開 API を小さく保ち、本当に必要な依存関係だけを残す方がよい。

12.8 章末問題

1. **最小のパッケージを作る。** 空の作業ディレクトリで `uv init --package signal_tools` を実行し、生成された `pyproject.toml` と `src/signal_tools/__init__.py` を確認せよ。`--package` を付けた理由、`src` レイアウトで `import` 事故を防ぎやすい理由、`pyproject.toml` の `name` と `requires-python` の意味を説明すること。
2. **公開関数とテストを追加する。** `src/signal_tools/statistics.py` に、数値列を受け取って算術平均を返す `mean_signal(values)` を作れ。空列では `ValueError` を送出すること。`__init__.py` から再公開し、`from signal_tools import mean_signal` で呼び出せるようにする。`tests/test_statistics.py` に通常入力と空入力のテストを書き、`uv run pytest` の

末尾で 2 passed と表示されることを確認せよ。

3. **引数、返り値、例外を文書化する。** `mean_signal()` に型ヒントと複数行 docstring を追加し、`uv run python -c "from signal_tools import mean_signal; help(mean_signal)"` で表示せよ。docstring の要約、引数、返り値、例外条件がそれぞれの行に表示されたかを確認すること。
4. **通常インストールと editable install を比較する。** 別の仮想環境を 2 つ作り、一方では `pip install .`、もう一方では `pip install -e .` を実行する。インストール後に関数の戻り値を変更し、どちらの環境で変更が直ちに反映されるかを確認せよ。`-e` が何を省略する指定なのではなく、ソースディレクトリを参照可能な形で登録する指定であることを説明せよ。
5. **依存関係と生成物を分類する。** NumPy を実行時依存関係、pytest と ruff を開発用依存関係として追加し、`pyproject.toml` の該当箇所を示せ。さらに `.gitignore` に `.venv/`、`__pycache__/`、`.pytest_cache/`、`dist/` を書き、`git status --short` にこれらが現れないことを確認せよ。`uv.lock` をコミットするかどうかはプロジェクトの運用方針によるため、本課題では自分の判断と理由を記述すること。

自作モジュールによって処理を独立した関数へ分けると、各関数の正しさと処理時間を個別に調べられる。次章では、この構造を利用してボトルネックを測定し、計算量、配列演算、メモリ使用量、逐次処理の観点から実装を改善する。

第 13 章

Python 解析の高速化

13.1 まず測る

高速化を始める前に、どこが遅いのかを測る。読み込みが遅いのか、探索が遅いのか、図の作成が遅いのかを区別しないまま最適化しても、効果は見えにくい。

ここで役立つのが `time.perf_counter` である。処理の前後で時刻を取り、差を記録するだけでも、どの段階が重いかの見当が付く。大切なのは、全体時間だけでなく、読み込み、前処理、探索、書き出し、作図のように段階を分けて測ることである。

```
1 import time
2
3 t0 = time.perf_counter()
4 run_analysis()
5 t1 = time.perf_counter()
6 print(f"elapsed = {t1 - t0:.3f} s")
```

測定するときは、一度だけの結果を絶対視しないことも重要である。キャッシュや OS の状態で多少は揺れるため、条件をそろえ、複数回の傾向を見る方がよい。

13.1.1 Jupyter では `%timeit` も使える

ノートブック環境で短い式や関数の速度を見たいときは、Jupyter の `%timeit` や `%timeit` も便利である。複数回の実行を自動で繰り返し、平均的な実行時間を表示してくれるため、小さな差を見たいときに役立つ。

```
1 %timeit
2 result = array * 2
```

ただし、`%timeit` はノートブックや IPython での簡便な測定であり、スクリプト全体の読み込みや入出力を含めた時間を見るには向かない。小さな部品には `%timeit`、解析全体には

`perf_counter` というように使い分けると整理しやすい。

13.2 計算量の見積もり

高速化では、実測だけでなく理屈の見積もりも役に立つ。入力サイズを N としたとき、処理回数が N に比例するのか、 $N \log N$ 程度なのか、 N^2 なのかで、大きなデータに対する伸び方がまるで違う。

厳密な漸近解析までできなくても、「データを 2 倍にしたら計算量は何倍くらいになるか」を考えるだけで十分に有益である。高速化はテクニック集ではなく、問題の構造を考える作業である。

この点は初学者ほど見落としやすい。変数名を短くしたり、関数を減らしたりすることは、ほとんど速度に効かない。一方で、比較回数を減らす、探索範囲を絞る、配列演算へ置き換えるといった変更は、本質的に計算量や定数倍を変える。

13.2.1 オーダーとは何か

ここでいう「オーダー」とは、入力サイズ N を大きくしたときに処理回数がどのように増えるかを表す粗い記法である。通常は Big-O 記法で書き、定数倍や低次の項を落として、最終的に支配的になる項だけを見る。

$$3N + 20 \rightarrow O(N), \quad 5N \log N + 100N \rightarrow O(N \log N), \quad N^2 + N \rightarrow O(N^2)$$

よく出るオーダーには、たとえば次のようなものがある。

- $O(\log N)$: 範囲を半分ずつ絞る処理。二分探索など
- $O(N)$: 1 回走査する処理。総和、平均、閾値判定、隣接差分など
- $O(N \log N)$: ソートを含む処理。高速なソート、ソート後の探索など
- $O(N^2)$: 全組合せ比較。全ペアの力づく比較など

たとえば $N = 10^6$ なら、 $\log_2 N$ はおよそ 20、 $N \log_2 N$ はおよそ 2×10^7 、 N^2 は 10^{12} である。どちらも「増える」ことに変わりはないが、その増え方の差は圧倒的である。小さなテストデータでは目立たなくても、大きな入力では急に効いてくる。

ただし、オーダーは定数倍を完全に無視してよい、という意味ではない。同じ $O(N)$ でも 10 倍、100 倍の速度差が付くことは普通にある。ここでいう「ファクター」とは、この定数倍の差のことである。したがって、まずオーダーを疑い、その後で定数倍を詰める、という順序で考えると整理しやすい。

この中で、 $O(\log N)$ や $O(N \log N)$ は初学者には直感を持ちにくい。そこで、まず二分探索で $O(\log N)$ を見てから、ソートで $O(N \log N)$ を見る。

13.2.1.1 二分探索ではなぜ $O(\log N)$ になるか

$\log N$ は、「問題の大きさを毎回半分にできる」場面で現れる。たとえば昇順に並んだ列から目的の値を探す二分探索では、中央を見て、左半分か右半分のどちらかを捨てられる。

```
1 def binary_search(values, target):
2     left = 0
3     right = len(values) - 1
4
5     while left <= right:
6         mid = (left + right) // 2
7         if values[mid] == target:
8             return mid
9         if values[mid] < target:
10            left = mid + 1
11        else:
12            right = mid - 1
13    return -1
```

要素数が N 個あっても、1 回ごとに候補数が半分になるなら、必要な反復回数は $N \rightarrow N/2 \rightarrow N/4 \rightarrow \dots \rightarrow 1$ という形になる。これが $\log_2 N$ 回程度である。したがって、二分探索は $O(\log N)$ で動く。重要なのは、「1 個ずつ減らす」のではなく「毎回半分に減らす」ことが $\log N$ を生む、という点である。

13.2.1.2 ソートではなぜ $O(N \log N)$ になるか

ソートは、計算量の違いを理解するための基本例である。たとえば、挿入ソートやバブルソートのような初等的な方法は、一般に $O(N^2)$ で動く。一方、マージソートやヒープソートは $O(N \log N)$ であり、Python の `sorted()` も実用的な高速ソートを使っている。

ここで $N \log N$ と書くなら、その意味も説明する必要がある。典型例であるマージソートでは、まず列を半分に分け、さらにその半分へ分ける、という分割を繰り返す。この分割の深さは、およそ $\log_2 N$ 段である。一方、各段では、分かれた小列どうしを併合する過程で、全体として各要素を 1 回ずつ見るので $O(N)$ かかる。

$$\text{各段の仕事量 } O(N) \times \text{段数 } O(\log N) = O(N \log N)$$

ここで重要なのは、「ソートは前処理にすぎない」と軽視しないことである。データを一度並べ替えるだけで、その後の探索や近傍判定がずっと安くなることがある。つまり、 $O(N \log N)$ の前処理へ投資することで、後続の $O(N^2)$ を避けられるなら、全体として大幅に得をする。

データが 10 倍に増えたとき、 $O(N^2)$ のソートはおおよそ 100 倍重くなるが、 $O(N \log N)$ のソートは 10 倍強で済む。この差を理解しておく、単にソートの名前を知るだけでなく、「なぜその方法を選ぶのか」を説明できるようになる。

13.2.2 オーダーが改善する例: 重複判定

問題をはっきりさせよう。長さ N の整数列に、同じ値が 2 回以上現れるかを判定したいとする。

最も素朴な方法は、すべての組 (i, j) を比べることである。

```
1 def has_duplicate_slow(values):
2     for i in range(len(values)):
3         for j in range(i + 1, len(values)):
4             if values[i] == values[j]:
5                 return True
6     return False
```

この方法では、最悪の場合ほぼすべてのペアを比べるので $O(N^2)$ になる。データ量が 10 倍になると、比較回数はおおよそ 100 倍に増える。

しかし、いったんソートしてしまえば、同じ値どうしは必ず隣り合う。したがって、隣接要素だけを見れば重複を判定できる。

```
1 def has_duplicate_fast(values):
2     values_sorted = sorted(values)
3     for i in range(len(values_sorted) - 1):
4         if values_sorted[i] == values_sorted[i + 1]:
5             return True
6     return False
```

`sorted()` の部分は $O(N \log N)$ 、その後の 1 回走査は $O(N)$ なので、全体は $O(N \log N)$ である。元の $O(N^2)$ より明確に良い。ここで本質なのは、Python の書き方が少し変わったことではなく、「全比較」という問題の解き方をやめたことである。

Python では `set(values)` を使えば平均的には $O(N)$ で重複判定できることも多いが、ここでは「ソートすると後続の走査が安くなる」という典型的な発想を示すために、あえてソートを使った例を出している。

同じ考え方は探索でも現れる。未整列の列から目的の値を探す線形探索は $O(N)$ だが、整列済みの列なら二分探索で $O(\log N)$ にできる。したがって、「一度だけ探す」のか「何度も探す」のかによって、先にソートする価値があるかどうかとも変わる。前処理に時間を使っても、その後の探索を何度も繰り返すなら、全体としては得になることが多い。

13.2.3 オーダーは同じで定数倍だけ改善する例

今度は、 $N \times N$ 行列 A が対称行列かどうか、すなわち $A_{ij} = A_{ji}$ が成り立つかを調べる問題を考える。

最も素朴な方法は、すべての (i, j) について比較することである。

```
1 def is_symmetric_slow(A):
2     n = len(A)
```

```
3     for i in range(n):
4         for j in range(n):
5             if A[i][j] != A[j][i]:
6                 return False
7     return True
```

これは n^2 回程度の比較を行うので $O(N^2)$ である。

しかし、対称性の判定では、上三角と下三角は同じ情報を二重に見ている。したがって、片側だけ見れば十分である。

```
1 def is_symmetric_fast(A):
2     n = len(A)
3     for i in range(n):
4         for j in range(i + 1, n):
5             if A[i][j] != A[j][i]:
6                 return False
7     return True
```

こちらもオーダーは同じく $O(N^2)$ だが、比較回数はおよそ $n(n-1)/2$ 回になり、素朴な実装のほぼ半分で済む。つまり、問題の大きさに対する増え方は変わっていないが、無駄な比較を消したことで定数倍が改善している。

この例は、「同じオーダーでも速くなる」ことを示している。図 13.1 の隣接差分フィルタの比較も、基本的にはこの種類の改善である。比較回数そのものを 2 次から 1 次へ落としたのではなく、同じ種類の処理をより効率よく実装しているのである。

13.3 実装を軽くする

13.3.1 NumPy と pandas による高速化

アルゴリズムを見直した後は、NumPy や pandas へ処理を移すことで、Python レベルのループを減らせる。特に差分計算、ソート、列抽出、フィルタは、これらのライブラリが得意とする。図 13.1 からも、ベクトル化が速度に与える効果を視覚的に確認できる。

ここで注意したいのは、NumPy や pandas を使えば自動的に何でも速くなるわけではないことである。根本的に無駄な比較をしているなら、配列演算へ移しても限界がある。まず計算量を減らし、そのうえで実装をベクトル化するという順序が自然である。

13.3.2 メモリとコピーの管理

速度だけを見ていると、メモリ使用量が大きくなりすぎて逆に遅くなることがある。巨大な中間配列を何度も作る実装では、CPU よりもメモリ転送が支配的になる場合がある。

そのため、高速化では「何回ループしたか」だけでなく、「何回コピーしたか」も見る必要

がある。不要なコピーを減らし、必要な列や範囲だけを扱う設計は、速度と安定性の両方に効く。

NumPy では、スライスが元配列とメモリを共有するビューになる一方、条件抽出は新しい配列を作る。次のコードで両者を確認できる。

```
1 import numpy as np
2
3 values = np.arange(1_000_000, dtype=np.float64)
4 view = values[100:200]
5 copied = values[values % 2 == 0]
6
7 print(values.nbytes)
8 print(view.nbytes, np.shares_memory(values, view))
9 print(copied.nbytes, np.shares_memory(values, copied))
```

実行結果は次の通りである。

```
8000000
800 True
4000000 False
```

nbytes は配列要素が占めるバイト数を表す。np.shares_memory(a, b) は 2 配列が同じメモリ領域を共有するかを調べる。この例では、view 自体が参照する要素は 800 バイト分だが、元配列のメモリを共有するため、view の要素を書き換えると対応する values の要素も変わる。copied は 50 万要素の独立した配列であり、新たに 4 MB を使う。なお、nbytes は Python オブジェクト自体の管理領域を含まないため、プロセス全体のメモリ使用量と完全には一致しない。

13.3.3 分割読み込み、並列化、ストリーミング

大規模解析では、全部を一度にメモリへ載せない設計も重要である。ファイル単位で処理し、必要な情報だけを持ち回る。さらに、独立な単位に分割できるなら並列化も検討できる。

ストリーミング処理では、「今必要な範囲だけを覚える」という考え方が中心になる。全履歴が不要なら保持しない、集計済みの量は逐次更新する、といった設計が有効である。メモリ節約と速度向上が同時に得られることも多い。

例えば、1 行に 1 個の測定値が書かれた values.txt の平均は、全行をリストへ保存せずに求められる。

values.txt

```
1.5
2.0
4.0
2.5
```

```
1 def streaming_mean(path):
2     count = 0
```

```

3     total = 0.0
4
5     with open(path, "r", encoding="utf-8") as f:
6         for line_number, line in enumerate(f, start=1):
7             text = line.strip()
8             if text == "":
9                 continue
10            try:
11                value = float(text)
12            except ValueError as exc:
13                raise ValueError(f"line {line_number}: {text!r}") from exc
14            count += 1
15            total += value
16
17    if count == 0:
18        raise ValueError("input contains no numeric values")
19    return total / count
20
21
22 print(streaming_mean("values.txt"))

```

出力は 2.5 となる。引数 `path` は入力ファイルの場所であり、`enumerate(..., start=1)` はエラー箇所をファイルと同じ 1 始まりの行番号で報告するために使う。この関数が保持するのは現在行、件数、合計だけなので、入力行数が増えても集計用メモリはほぼ一定である。ただし、中央値や全体のソートのように全データの順序が必要な統計量は、この単純な方法だけでは正確に求められない。

13.4 プロファイリングと低レベル実装

13.4.1 プロファイラを使う

本格的に遅い理由を調べるなら、プロファイラも役立つ。例えば `cProfile` を使うと、どの関数に時間が使われているかを確認できる。

```

1 import cProfile
2
3 cProfile.run("run_analysis()", sort="cumtime")

```

ただし、プロファイラの数字も文脈なしには読めない。回数が多いから遅いのか、1 回が重いのか、入出力を待っているのかを区別する必要がある。測定値とコード構造を一緒に見ることが重要である。

| ncalls | tottime | cumtime | function |
|--------|---------|---------|----------------|
| 100000 | 1.25 | 1.40 | check_pair |
| 10 | 0.02 | 2.10 | read_all_files |

この結果例では、`check_pair` は 1 回は軽い回数が多く、`read_all_files` は内部の処理込みで累積時間が大きいと読める。どの列が何を意味するかを理解すると、改善方針が立てやすくなる。

13.4.2 C や C++ へ書き換えるという選択肢

Python は開発速度が高いが、計算コアのすべてを Python で書く必要はない。アルゴリズムを十分に整理したうえで、どうしても重い部分だけを C や C++ へ移す、というのは自然な選択肢である。

ただし、言語を変える前に Python 側で何が遅いかを把握しておく必要がある。ボトルネックが入出力や高水準の設計にあるなら、C++ 化しても大きな改善にならない。低レベル言語への移行は、最後の切り札として考える方がよい。

```
1 for (std::size_t i = 0; i < n; ++i) {
2     for (std::size_t j = i + 1; j < n; ++j) {
3         if (std::abs(times[j] - times[i]) <= window) {
4             ++count;
5         }
6     }
7 }
```

この書き換えは、定数倍の改善をもたらすことがある。しかし、比較回数そのものが多いすぎるなら本質的な改善ではない。アルゴリズムと実装言語を分けて考える姿勢が必要である。

13.4.3 Python から C/C++ を呼び出す

方法はいくつかある。たとえば `ctypes` で共有ライブラリを呼ぶ方法、Python C 拡張を書く方法、`pybind11` を使って C++ を Python から自然に使える形へ包む方法などがある。さらに、Python と C の中間に位置する `Cython` を使い、Python に近い構文のまま重いループだけをコンパイルする方法も、この節に含まれる重要な選択肢である。

```
1 cdef int i
2 cdef double total = 0.0
3
4 for i in range(n):
5     total += values[i]
```

ここで `cdef` は `Cython` 特有の宣言であり、変数を Python オブジェクトではなく C の整数や浮動小数点数として扱わせるために使う。これにより、ループ内部の型判定やボックス化のコストを減らせる。

どの方法を選ぶかは、必要な速度、保守性、ビルドの難しさで決まる。目安としては次のように考えるとよい。

- `ctypes`: 既にある C の共有ライブラリを手早く呼びたいときに向く。設定は比較的軽い
が、引数や戻り値の型を手で合わせる必要がある
- Python C 拡張: Python 本体の C API を直接使う最も低レベルな方法で、自由度は高い
が記述量も多い
- `pybind11`: C++ の関数やクラスを Python から自然に使える形で公開しやすい。C++ 側
の実装をすでに持っている場合に相性がよい
- Cython: Python に近い構文を保ちながら、重いループや数値計算だけへ型を付けてコン
パイルしたいときに向く

短い試作なら `ctypes`、Python らしい使い勝手まで整えたいなら `pybind11`、既存の Python 風コードを保ちながら内側だけ速くしたいなら Cython、といった選択が考えられる。重要なのは、インタフェースを狭く保ち、重い部分だけを外へ出すことである。`pybind11` の最小構成は付録 C を参照してほしい。

```
1 result = fast_module.count_pairs(times, window)
```

Python 側の呼び出しが単純であれば、内部実装を後から差し替えやすい。まず Python で正しい API を決め、その内部だけを高速化するという順が自然である。なお、Cython も強力だが、アルゴリズムそのものが無駄な場合には決定打にならない。この点は C++ 化と同じであり、まず Python 側で何が遅いのか、どのループが本当に支配的なのかを見極めた後に使う方が効果的である。

13.4.4 外部実装を参考にする時の見方

高速化を学ぶときは、外部の C や C++ の実装例を読むことが参考になる。ただし、そこで大切なのは固有名詞を覚えることではない。まず見るべきなのは、どの問題を解いているのか、どのアルゴリズムを選んでいるのか、どの部分だけを低レベル言語へ移しているのか、という点である。

高速化の章では、このような例を参照しながら、次の二つを常に区別して考える。

- アルゴリズムを変えたから速くなったのか
- 実装言語を変えたから速くなったのか

多くの場合、最初に効くのは前者である。

したがって、高速化のレポートを書くときは、「NumPy にした」「C++ にした」で終わらせない方がよい。何が変わった結果として速くなったのかを、計算量、比較回数、メモリ配置、ループの位置といった観点で説明するべきである。

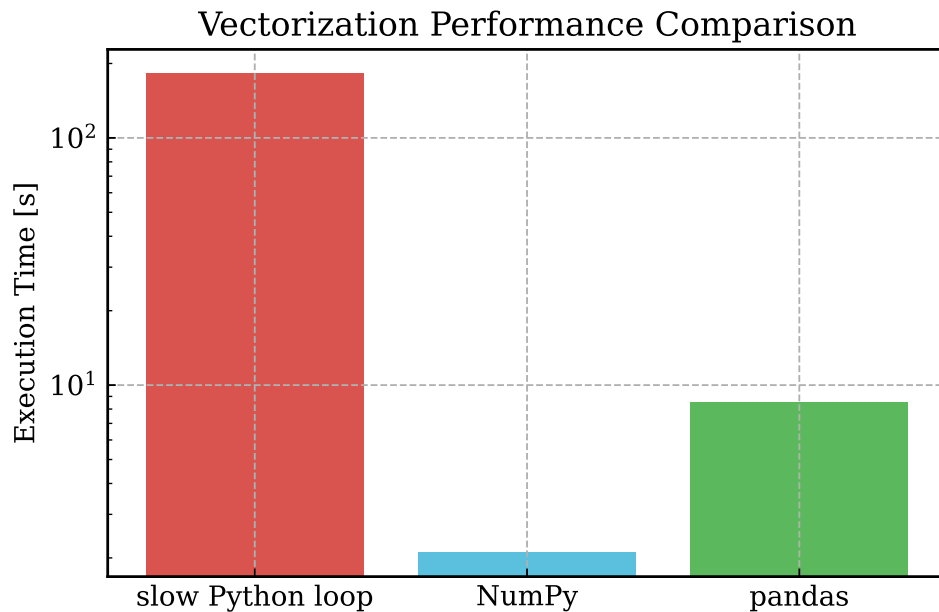


図 13.1 同じ隣接差分フィルタを、純粋な Python ループ、NumPy、pandas で実行した例。縦軸は実行時間で、対数目盛を使っている。

13.5 結果例と報告の仕方

高速化の結果は、本文中で比較できる形に整理すると読みやすい。例えば、次のような簡潔な結果でも十分に意味がある。

```
slow version : 182.4 s
vectorized   :  12.7 s
streaming    :   4.8 s
```

この表現だけでは理由が分からないので、続けて「比較回数を減らした」「Python ループを配列演算へ移した」「全件保持をやめた」といった説明を添える。数値と理由の両方がそろって初めて報告になる。

特に、高速化の報告では「 $O(N^2)$ から $O(N \log N)$ や $O(N)$ へ落とした改善」と、「同じ $O(N)$ や $O(N^2)$ のまま NumPy, Cython, C++ などで定数倍を詰めた改善」を分けて書くといい。前者は問題の解き方を変えた改善であり、後者は実装の効率を上げた改善である。この二つを混同しない方が、読者にも自分にも何が効いたのかが伝わりやすい。

```
1 import numpy as np
2
3 values = np.array(data, dtype=np.float64)
4 diffs = np.diff(values)
5 count = np.count_nonzero(diffs > 0.25)
```

図 13.1 は実行時間の大小を比較するための図であり、結果の正しさまでは保証しない。高速化の前後で同じ count が得られることをテストし、その上で入力サイズ、反復回数、実行

環境、代表値をそろえて時間を比較する必要がある。

13.6 章末問題

1. **結果を変えずにベクトル化する。** `values = np.arange(1_000_000, dtype=np.float64)` について、各要素を二乗してから総和を求める処理を Python の `for` 文と NumPy の列演算で実装せよ。`np.isclose()` で結果が一致することを先に確認し、その後で `time.perf_counter()` を使って各方法を 5 回ずつ測定せよ。最小値または中央値を報告し、1 回だけの値を比較しない理由を説明すること。
2. **入力サイズと計算量の関係を測る。** 本文の `has_duplicate_slow()` と `has_duplicate_fast()` を使い、重複を含まない入力について $N = 1000, 2000, 4000$ の実行時間を測定せよ。 N を 2 倍にしたときの時間比を求め、 $O(N^2)$ と $O(N \log N)$ の予想と比較する。計算機や実行時の状態によって時間は変わるため、絶対時間ではなく比と傾向を説明せよ。
3. **ビューとコピーの影響を確認する。** 本文の `values`、`view`、`copied` を作り、`view[0] = -1` と `copied[0] = -2` を実行した後で `values[100]` と `values[0]` を表示せよ。前者だけが変化する理由を、`np.shares_memory()` の結果と対応させて説明する。さらに `float32` と `float64` で同じ要素数の `nbytes` を比較せよ。
4. **逐次集計を検証する。** 1 から 100000 までを 1 行ずつ書いたファイルを作り、本文の `streaming_mean()` で平均を求めよ。期待値は 50000.5 である。同じファイルを全件リストへ読み込む実装も作り、結果の一致を確認せよ。両者が保持するデータ量を、入力行数 N に対するオーダーで説明すること。
5. **プロファイル結果から改善対象を選ぶ。** 自分の解析スクリプト、または 1 問目の 2 実装を関数へ分けたスクリプトを `python -m cProfile -s cumtime script.py` で測定せよ。出力の `ncalls`、`tottime`、`cumtime` の意味を説明し、累積時間の上位 3 関数を記録する。そのうち改善対象に選ぶ関数と、選ばない関数の理由をそれぞれ述べよ。

性能改善は、速いコードを書くことだけを目的としない。高速化の前後で同じ結果が得られることを検証し、入力条件と測定方法を記録して初めて、改善を再現できる。次章の総合課題では、同時観測イベントの探索を題材として、読み込み、検証、可視化、フィット、性能測定を一つの解析へ統合する。

第 14 章

課題：同時観測イベントの探索

14.1 課題の準備

この課題では、次のファイルを配布する。

- events_data.zip
- slow_event_analysis.py
- 本テキスト

events_data.zip を展開すると、events_data/ というディレクトリが得られる。日ごとに gzip 圧縮されたイベントファイルがあり、年・月ごとのディレクトリに分かれている。

課題のスクリプトやデータは、GitHub リポジトリ (https://github.com/i-komae/python_textbook) から入手できる。手元に配布物がそろっていない場合は、まずリポジトリを取得し、リポジトリ直下にある slow_event_analysis.py と events_data.zip の配置を確認する。events_data.zip を展開すると events_data/ が得られ、これを入力ディレクトリとして使う。

配布物は意図的に最小限にしてある。したがって、何を入力とし、どのファイルが正解判定の対象で、どこからどこまでを自分で実装するのかを最初に整理してから着手した方がよい。

14.1.1 まず配布コードを動かす

まずは、配布された遅いコードをそのまま動かす。

```
$ python3 slow_event_analysis.py events_data --outdir slow_output
```

全期間の処理時間は、データ量、ストレージ、計算機によって変わる。初回は数日分だけを切り出し、入力形式と出力を確認してから対象期間を広げる。

重要なのは、最初から完成版を書こうとしないことである。まず配布コードが本当に遅い

ことを自分の環境で確認し、どの出力が得られるかを見てから変更を始める方が、修正の意味が分かりやすい。

14.1.2 gzip ファイルの中身を見る

データの中身を見ずに解析を始めてはいけない。まずは 1 ファイルを開き、列の意味を確認する。

```
$ gzip -cd events_data/2024/01/events_2024_01_01.txt.gz | head
```

各行の列は次のようになっている。

```
event_id year month day hhmmss ns detector_id signal
```

列名を見たら、それぞれの型と役割を自分で言葉にしてみるとよい。整数として扱うべき列、時刻を構成する列、識別子としてだけ使う列、物理量として解釈する列が混ざっている。ここを曖昧にしたまま進むと、後でバグの原因になる。

この課題では、2 事象の時刻差を ns 単位まで含めて評価し、 $|\Delta t| \leq 250 \text{ ns}$ を満たすものを「同時観測」と定義する。したがって、hhmmss 列だけを見ればよいわけではなく、ns 列も含めて時刻を一つの量として組み立てる必要がある。秒境界や日付境界をまたぐ場合にどう振る舞うかも、この定義を満たす形で確認しなければならない。

14.1.3 小さいデータで確認する

いきなり 500 日分全体に対して試すのではなく、まずは数日分を別ディレクトリにコピーして試すとよい。

```
$ mkdir -p events_data_small/2024/01
$ cp events_data/2024/01/events_2024_01_01.txt.gz events_data_small/2024/01/
$ cp events_data/2024/01/events_2024_01_02.txt.gz events_data_small/2024/01/
$ cp events_data/2024/01/events_2024_01_03.txt.gz events_data_small/2024/01/
```

小さい入力での確認は、速度のためではなく、正しさを素早く確かめるためである。数十秒で終わる条件を作っておけば、修正してすぐ再実行できる。大きなデータを回すのは、その後で十分である。

```
n_pairs_total = 241
analysis_sec = 18.4
```

この段階では、数値そのものよりも、修正のたびに一貫した結果が出るかが大切である。件数が毎回変わる、出力ファイルの形式が揺れる、といった状態では、その先の高速化に進むべきではない。

14.2 課題で求めること

最低限、次の内容に取り組むこと。

1. 配布コードの処理時間を確認する
2. 同時観測イベント数が正しいかを検証する
3. バグを修正する
4. 高速化する
5. 同時観測イベントの時刻差分布を描き、フィットする

この並びにも意味がある。いきなり高速化から始めるのではなく、まず元のコードの振る舞いを把握し、次に正しさを確認し、その後で改善する。間違ったコードを高速化しても、間違いが速くなるだけである。

14.2.1 出力としてほしいもの

最低限ほしい出力は次の三つである。

- 同時観測イベントの一覧
- 同時観測イベントを構成する 2 事象の時刻差の分布
- 同時観測イベントどうしの発生間隔、またはそれに相当する量の分布

ここでいう「2 事象の時刻差」は、同時観測と判定した 2 つのイベントの時刻差である。たとえば $\Delta t = t_2 - t_1$ と定義してよい。ただし、符号付きで使うのか絶対値を使うのか、検出器番号で向きをそろえるのかなどは、自分で決めて明記すること。

分布を描くときは、ビン幅、横軸の範囲、単位も明記すること。図は出しただけでは不十分で、どの量をどう定義して数えたのかが分からなければ、他人は解釈できない。フィットを行うなら、どの範囲を使ったかも書くべきである。

```
coincidence_pairs.csv
pair_dt_hist.pdf
pair_mean_gap_hist.pdf
summary.txt
```

ファイル名の流儀を最初に決めておくと、後で解析を繰り返したときにも混乱しにくい。どのファイルが一覧で、どれが図で、どれが数値要約なのかが名前から分かるようにしておくとよい。

14.2.2 正しさ確認の目安

今回の配布データは 500 日分であり、正しく同時観測イベントを拾えていれば、全期間で得られる同時観測イベント数は

```
n_pairs_total = 12033
```

になる。この値は最終確認の目安として使ってよい。

ただし、この値に合ったからといって、それだけで実装が正しいとは限らない。偶然件数だけ一致している可能性もある。可能なら、個々のイベント対の内容や、ファイル境界付近の振る舞いも確認した方がよい。

```
n_pairs_total = 12033
analysis_sec = 248.6
plot_sec = 1.8
```

このような要約ファイルを出しておく、コード変更のたびに結果を比較しやすい。件数だけでなく処理時間も同じ場所へ残すと、正しさと速度を一緒に追える。

14.3 提出物と発展課題

提出物は次の二つである。

- 解析に使った Python コード
- LaTeX で書いたレポートと、その PDF

レポートには、少なくとも次の内容を書くこと。

- 元のコードの問題点
- 正しさをどう検証したか
- バグをどう修正したか
- どう高速化したか
- 改善前後の処理時間
- 作成した図
- フィット結果とその解釈

レポートを書くときは、変更点の羅列よりも、問題の発見、原因の切り分け、修正方針、検証方法、結果、考察の順で並べると読みやすい。図や表を入れるときは、本文の中で参照しながら説明すること。

1. 元のコードの観察
2. バグの特定
3. 修正内容
4. 高速化方針

5. 結果の比較

6. 図とフィットの考察

この順序で書けば、読者は「何が問題で、どう直して、何が改善したのか」を追やすい。レポートは成果物であると同時に、思考過程の記録でもある。

14.3.1 発展課題

余裕があれば、図の作成やフィットまで含めた全体処理をさらに高速化してよい。さらに、NumPy や pandas を使うだけでなく、自分でアルゴリズムを設計し直したり、C や C++ で計算コアを実装したりしてもよい。

この発展課題では、特定の解法を要求しない。pandas を深く使う方向もあれば、ストリーミング処理を突き詰める方向もあり得るし、重い部分だけを別言語へ切り出す方向もあり得る。重要なのは、自分の方針を説明できることである。

付録 A

SSH, 鍵認証, ファイル転送

A.1 SSH の基本

SSH は、離れたマシンへ安全に接続してコマンドを実行するための仕組みである。研究室や計算サーバーへ入り、大きなデータ処理や長時間ジョブを回すときによく使う。GitHub へ SSH で接続するときにも同じ仕組みを使うが、GitHub はシェルへログインする相手ではなく、Git 操作の認証先である点は区別しておいた方がよい。

A.1.1 サーバーへ接続する

最も基本的な形は次である。

```
$ ssh user@server.example.jp
```

user はサーバー上の自分のユーザー名、server.example.jp はホスト名である。ポート番号が既定の 22 でないなら -p を付ける。

```
$ ssh -p 2222 user@server.example.jp
```

接続後は、ローカルのターミナルと似た形でコマンドを打てる。ただし、そこで見えているファイルは自分の手元ではなく、サーバー側のファイルである。この区別が曖昧になると事故の原因になるので、hostname、whoami、pwd を見て今どこにいるかを確かめる習慣を持った方がよい。

A.1.2 SSH 鍵を作る

毎回パスワードを打つ代わりに、SSH 鍵を使うことが多い。鍵は公開鍵と秘密鍵の 2 つ 1 組であり、公開鍵をサーバーへ登録し、秘密鍵は自分だけが持つ。

```
$ ssh-keygen -t ed25519 -C "your_email@example.com"
```

既定値のまま進めると、多くの場合は `~/.ssh/id_ed25519` が秘密鍵、`~/.ssh/id_ed25519.pub` が公開鍵になる。`.pub` が付いている方だけを相手へ渡す。`.pub` が付いていない方は秘密鍵なので、決して共有してはいけない。

```
$ ls -l ~/.ssh
```

秘密鍵の権限が緩いと SSH が拒否することがある。その場合は次のように直す。

```
$ chmod 700 ~/.ssh
$ chmod 600 ~/.ssh/id_ed25519
$ chmod 644 ~/.ssh/id_ed25519.pub
```

A.1.3 公開鍵をサーバーへ登録する

公開鍵をサーバーへ登録する最も簡単な方法は `ssh-copy-id` である。

```
$ ssh-copy-id user@server.example.jp
```

特定の公開鍵を使いたいなら `-i` で指定できる。

```
$ ssh-copy-id -i ~/.ssh/id_ed25519.pub user@server.example.jp
```

`ssh-copy-id` は、指定した公開鍵をサーバー側の `~/.ssh/authorized_keys` へ追加する。つまり、「手元の公開鍵を安全に送り、向こうで追記する」という面倒な作業を代わりにやってくれる。

A.1.3.1 `ssh-copy-id` が使えない場合

`ssh-copy-id` が使えない環境では、公開鍵の中身を手で登録する。

```
$ cat ~/.ssh/id_ed25519.pub
```

表示された 1 行全体を、サーバー側の `~/.ssh/authorized_keys` へ追記する。サーバーへ通常のパスワードログインができるなら、自分でログインして追記してもよい。

A.1.4 `ssh-agent` を使う

パスフレーズ付きの鍵を使う場合は、`ssh-agent` に読み込ませておくことで毎回の入力を減らせる。

```
$ eval "$(ssh-agent -s)"
$ ssh-add ~/.ssh/id_ed25519
```

A.2 接続設定と GitHub 連携

A.2.1 ~/.ssh/config を書く

接続先が増えると、毎回長いホスト名やポート番号や鍵ファイル名を書くのは面倒になる。そこで ~/.ssh/config を使う。設定ファイルの中身は次のようになる。

~/.ssh/config

```
Host labserver
  HostName server.example.jp
  User ikomae
  Port 2222
  IdentityFile ~/.ssh/id_ed25519
```

これを書いておけば、以後は次のように短く書ける。

```
$ ssh labserver
```

config ファイル自体も秘密情報に近いので、`chmod 600 ~/.ssh/config` としておく方が安全である。

A.2.2 GitHub に SSH 鍵を登録する

通常のサーバーとは違って、GitHub には `ssh-copy-id` を使わない。GitHub はシェル接続先ではなく、アカウント設定に公開鍵を登録する相手だからである。

最も確実なのは、GitHub の設定画面の SSH and GPG keys へ公開鍵を登録する方法である。もし GitHub CLI `gh` を使うなら、認証時に SSH 鍵の作成や登録を案内させることもできる。

```
$ gh auth login --git-protocol ssh --web
```

GitHub CLI を使う場合は、既存の鍵を検出してアップロードするか、新しい鍵を作るかを対話的に選べる。GitHub CLI を使わない場合は、`cat ~/.ssh/id_ed25519.pub` で表示した公開鍵を Web 画面へ貼り付ければよい。

A.3 ファイル転送

小さなファイルを 1 回だけ送るなら `scp` でもよいが、解析ではディレクトリ全体を何度も同期することが多い。その場合は `rsync` を使う方が自然である。

A.3.1 scp の最小例

scp では、ローカルとリモートのどちらを送信元を書くかによって転送方向が決まる。両方向の最小例を次に示す。

```
$ scp result.csv user@server.example.jp:~/work/  
$ scp user@server.example.jp:~/work/result.csv .
```

1 行目は手元からサーバーへ送り、2 行目はサーバーから手元へ持ってくる。ディレクトリごと送りたい場合は `-r` を付ける。

A.3.2 rsync を推奨する理由

rsync は、差分だけを転送し、途中で止まっても再開しやすく、除外パターンも柔軟に書ける。そのため、解析用ディレクトリやコード一式をサーバーへ送る用途では scp より向いている。

```
$ rsync -aP ./analysis/ user@server.example.jp:~/work/analysis/
```

よく使うオプションの意味は、次のように 1 つずつ読んだ方が理解しやすい。

- a archive モードである。再帰コピーを行い、時刻や権限などもまとめて保つ。
- P `--partial --progress` の省略形である。途中ファイルを残しつつ進捗を表示する。
- v 何を転送しているか詳しく表示する。
- z 転送時に圧縮する。低速回線では有利だが、既に圧縮済みの大きなデータでは効果が薄いこともある。
- exclude** 特定のファイルやディレクトリを転送対象から除外する。

次の例では、アーカイブモードと進捗表示を有効にし、転送先で再作成できる仮想環境、キャッシュ、ビルド成果物を除外する。

```
$ rsync -aPv \  
  --exclude='.venv/' \  
  --exclude='__pycache__/' \  
  --exclude='build/' \  
  --exclude='.mypy_cache/' \  
  ./analysis/ user@server.example.jp:~/work/analysis/
```

仮想環境、キャッシュ、ビルド成果物はサーバー側で作り直せることが多いので、転送から外した方が軽くなる。

A.3.2.1 末尾のスラッシュに注意する

rsync では、送信元パスの末尾に `/` を付けるかどうかで意味が変わる。

```
$ rsync -aP analysis user@server:~/work/
```

```
$ rsync -aP analysis/ user@server:~/work/analysis/
```

1 行目は `analysis` というディレクトリごと送る。2 行目は `analysis` の中身を送る。末尾のスラッシュを取り違えると転送先の階層が 1 段ずれるため、初回は `--dry-run` を加えて転送対象を確認する。

付録 B

Git と GitHub の基本

B.1 Git の基本概念

Git は、手元のファイルの変更履歴を記録するためのソフトウェアである。これに対して GitHub は、Git リポジトリを置いて共有するためのサービスである。

- Git: ローカルマシンでも単独で使えるバージョン管理システム
- GitHub: Git リポジトリをホストし、共有、Issue、Pull Request などを提供するサービス

したがって、`git init`、`git add`、`git commit` は Git の話であり、`git push` で GitHub へ送る段階になって初めて GitHub が関係してくる。

B.1.1 リポジトリと公開範囲

リポジトリは、コード、データ、履歴をまとめて管理する単位である。GitHub へ置く場合は、公開範囲も意識する必要がある。

- `public`: 誰でも見られる
- `private`: 明示的に許可した人だけが見られる
- `internal`: 組織内だけで見られる。主に GitHub Enterprise の文脈で使う

授業用の配布物、研究中のコード、機密データを含むリポジトリでは、公開範囲の設定を誤ると問題になる。GitHub で新しいリポジトリを作るときは、まず `visibility` を確認した方がよい。

B.1.2 Git で何をしているのか

Git は、ファイルを 1 回で全部自動保存してくれるわけではない。ふつうは次の 3 段階で進む。

1. 作業中のファイルを編集する
2. 記録したい変更を `git add` で選ぶ
3. `git commit` で履歴として確定する

この 2 段階目の「いったん選ぶ場所」をステージングエリアという。`git add` と `git commit` の違いを理解するには、ここが重要である。

最初はここが見えにくいので、次の 3 層に分けて考えるとよい。

- working tree: いま自分がエディタで直接書き換えている実ファイル
- staging area (index): 次のコミットへ入れる予定の変更を一時的に集めた場所
- commit history: すでに確定して履歴へ残った状態

たとえば、ファイルを編集した直後は変更は working tree にしかない。`git add` を実行すると、その時点の内容が staging area へコピーされる。`git commit` は、その staging area の内容を履歴として確定する操作である。したがって、`git add` の後にさらにファイルを編集すると、「ステージ済みの内容」と「作業中の内容」がずれることがある。

B.2 ローカルで履歴を作る

B.2.1 最初の設定

最初に、自分の名前とメールアドレスを設定しておく。

```
$ git config --global user.name "Your Name"
$ git config --global user.email "your_email@example.com"
```

これをしておかないと、コミット時に誰の変更なのかが分からなくなる。

B.2.2 新しいリポジトリを作る

まだ Git 管理されていないディレクトリなら、`git init` で始める。

```
$ mkdir analysis_work
$ cd analysis_work
$ git init
$ git status
```

`git status` は最重要コマンドである。今どのファイルが未追跡か、どれが変更済みか、何がコミット待ちかを確認できる。迷ったらまず `git status` を見る、という習慣を付けるとよい。

B.2.3 編集、確認、記録の流れ

日常的な流れは、編集して、差分を確認して、必要なものだけをコミットする、である。

```
$ git status
$ git diff
$ git add analyze.py
$ git diff --cached
$ git commit -m "Add first analysis script"
$ git log --oneline
```

`git diff` は、まだ `add` していない変更を見る。`git diff --cached` は、`add` 済みで次のコミットへ入る変更を見る。`git log --oneline` は、これまでのコミット履歴を短く確認する。

B.2.3.1 `git status` の読み方

`git status` は、少なくとも次の 3 種類を見分けられるようになるとよい。

- untracked: まだ Git が追跡していない新規ファイル
- modified: 追跡中だが、まだ `add` していない変更
- changes to be committed: `add` 済みで、次のコミットへ入る変更

つまり、`git status` は「いま何がどの段階にあるか」を示す画面である。

B.2.3.2 `git add` は何をしているのか

`git add` は「変更を次のコミット候補として選ぶ」コマンドである。保存ではない。したがって、`git add` した後でも、コミット前にさらに編集すれば、その追加分はまだコミット対象へ入っていない。

```
$ git diff
$ git add analyze.py
$ git diff --cached
```

この 2 回の `diff` を見分けられるようになると、Git の挙動がかなり分かりやすくなる。

B.2.4 `.gitignore` を使って除外する

生成物まで毎回コミットすると、履歴が見にくくなる。Python 解析なら、仮想環境、キャッシュ、ビルド成果物などは除外することが多い。

B.2.4.1 .gitignore を書く

.gitignore は、Git に「このパターンのファイルは追跡しなくてよい」と教えるためのファイルである。リポジトリのルートに置くことが多い。Python 解析用の最小例は次のようになる。

```
.gitignore
.venv/
__pycache__/
*.pyc
build/
dist/
.pytest_cache/
```

このファイルを作ると、まだ追跡されていない .venv/ や build/ は git status に出にくくなり、誤ってコミットしにくくなる。

B.2.4.2 すでに追跡済みなら git rm --cached が必要

.gitignore は「これから追跡しない」ための設定である。すでに一度コミットしたファイルには、そのままでは効かない。

```
$ git rm --cached -r .venv __pycache__ build
$ git commit -m "Stop tracking generated files"
```

git rm --cached は、作業ディレクトリの実ファイルを消さずに、インデックスからだけ外す。そのため、「手元には残したまま Git の追跡対象から外す」という目的に向いている。

B.2.4.3 uv プロジェクトで何をコミットするか

uv を使うプロジェクトでは、通常 .venv/ はコミットしない。一方で、pyproject.toml、uv.lock、.gitignore、そしてプロジェクト単位でバージョンを固定したいなら .python-version はコミット候補になる。

B.3 GitHub と接続する

B.3.1 既存のリポジトリを持ってくる

他人のプロジェクトや GitHub 上の教材は、git clone で手元へコピーする。

```
$ git clone git@github.com:example/project.git
$ cd project
$ git remote -v
```

`git clone` は、履歴も含めて手元へ持ってくる。`git remote -v` は、どの URL が接続先として設定されているかを確認する。本書では、GitHub から取得するときも `push` するときも SSH URL を使う方針を勧める。公開鍵の準備は付録 A を参照してほしい。

B.3.2 手元のリポジトリを GitHub へ初めて載せる

ここまでは「GitHub 上にあるものを取得する」話だったが、逆に、手元で `git init` して育てたプロジェクトを自分の GitHub へ載せたい場面も多い。その場合は、まず GitHub 側で空のリポジトリを作り、次に手元のリポジトリから接続先を登録して `push` する。

B.3.2.1 GitHub 側で空のリポジトリを作る

GitHub の Web 画面で `New repository` を開き、リポジトリ名と公開範囲を決める。このとき、手元にすでに履歴があるなら、最初は空のリポジトリとして作る方が簡単である。つまり、`Add a README file`、`Add .gitignore`、`Choose a license` には最初は触れない方がよい。

最初から README や `.gitignore` を GitHub 側で作ると、リモート側に 1 個目のコミットができる。その状態で手元の履歴をそのまま `push` すると、履歴の始点が一致しないため、初心者には扱いにくい状況になりやすい。したがって、既存のローカルリポジトリを載せるときは「空の GitHub リポジトリ」を作る、という方針で覚えた方が安全である。

B.3.2.2 README や LICENSE を先に作っていた場合

すでに GitHub 側で README や LICENSE を作ってしまったなら、リモート側には先にコミットが 1 つ以上存在する。その場合は、いきなり `git push` するのではなく、まずリモート側の履歴を取り込んでから `push` しなければならない。

```
$ git branch -M main
$ git remote add origin git@github.com:owner/project.git
$ git pull origin main --allow-unrelated-histories
$ git push -u origin main
```

`--allow-unrelated-histories` が必要なのは、手元のリポジトリと GitHub 側のリポジトリが別々に作られており、履歴の出発点が一致していないからである。この `pull` によって、GitHub 側の README や LICENSE のコミットを自分の履歴へ取り込んでから、その続きとして `push` できるようになる。

もし手元にも README や LICENSE があり、同じ名前のファイルを両側で作っていたなら、`git pull` の途中で競合が起こることがある。その場合は、どちらの内容を残すかを自分で決めて修正し、`git add` と `git commit` で競合解消を確定してから、あらためて `git push -u origin main` を実行する。

B.3.2.3 接続先を登録して最初の push を行う

GitHub が表示する URL を確認し、手元のリポジトリで origin として登録する。多くの場合、次の流れで足りる。

```
$ git status
$ git branch --show-current
$ git branch -M main
$ git remote add origin git@github.com:owner/project.git
$ git remote -v
$ git push -u origin main
```

`git branch --show-current` は、いま自分がどのブランチにいるかを確認する。`git branch -M main` は、既定ブランチ名を `main` へそろえるための書き方である。`main` ではなく既存のブランチ名をそのまま使いたいなら、その名前で `push` してよい。

`git remote add origin ...` は、GitHub の URL を `origin` という別名で登録する。`git remote -v` を見れば、正しい URL が設定されたかを確認できる。`git push -u origin main` の `-u` は、以後このローカルブランチが `origin/main` を追跡する、という設定である。初回にこれを付けておけば、次回以降は `git push` や `git pull` を短く書きやすい。

B.3.2.4 origin がすでにある場合

すでに `origin` が登録されている状態で `git remote add origin ...` を実行すると、エラーになる。その場合は、いま何が設定されているかを確認してから、必要なら URL を差し替える。

```
$ git remote -v
$ git remote set-url origin git@github.com:owner/project.git
$ git remote -v
```

教材を別のリポジトリから複製して流用した場合や、HTTPS から SSH へ切り替えたい場合に、この操作が必要になることがある。

B.3.3 GitHub と同期する

ローカルでコミットしただけでは、まだ GitHub には反映されない。GitHub 側から最新を取り込むのが `pull`、手元のコミットを送るのが `push` である。

```
$ git pull --ff-only
$ git push origin main
```

`git pull --ff-only` は、自分がまだ追加のコミットを持っていないときだけ安全に進める書き方である。履歴が分岐している場合は止まるので、意図しないマージを避けやすい。

B.3.4 GitHub で使う URL には HTTPS と SSH がある

GitHub のリポジトリ URL には、少なくとも HTTPS と SSH の 2 通りがある。

```
https://github.com/owner/repo.git  
git@github.com:owner/repo.git
```

HTTPS は見慣れた URL で扱いやすい。一方 SSH は、公開鍵認証を済ませておけばパスワードやトークン入力を減らしやすい。どちらを使ってもよいが、混在させると混乱しやすいので、最初に方針を決めた方がよい。git clone、pull、push を通して運用をそろえるなら、本書では SSH を標準にする方が自然だと考える。特に、公開リポジトリを読む段階では HTTPS でも動くが、後で push や private repository の取得が必要になると認証方式が増えて煩雑になりやすい。そのため、最初から SSH へそろえておく方が混乱が少ない。

B.4 認証と最小ワークフロー

B.4.1 GitHub の認証: SSH 鍵を基本にし、必要なら PAT

GitHub では、HTTPS 経由の Git 操作で通常のアカунトパスワードは使えない。現在は、SSH 鍵を登録して使うか、個人用アクセストークン (PAT) を使うのが基本である。本書では SSH 鍵を使う方法を標準とし、HTTPS + PAT は事情がある場合の代替手段として扱う。

B.4.1.1 SSH 鍵を使う

SSH を使う場合は、付録 A を参照して公開鍵を用意し、それを GitHub の SSH and GPG keys へ登録する。GitHub CLI `gh auth login --git-protocol ssh --web` を使うと、その作業を対話的に進めやすい。

B.4.1.2 PAT を使う

SSH 鍵を使えない事情があり、HTTPS で GitHub を使う場合は、プライベートリポジトリへの push や pull で PAT を使う。

1. GitHub の Settings を開く
2. Developer settings から Personal access tokens へ進む
3. fine-grained token か classic token を作る
4. 対象リポジトリと必要最小限の権限を選ぶ
5. 発行したトークンを安全に保管する

認証時には、ユーザー名は GitHub のアカウント名、パスワード欄には通常のパスワードではなく PAT を入れる。組織アカウントでは、SAML SSO の承認が別に必要なこともある。その場合は、発行したトークンをその組織用に追加承認しないと使えない。

fine-grained token は、どのリポジトリへ、どの操作権限だけを与えるかを細かく絞れる新しい方式である。個人の通常利用では、まずこちらを選ぶ方が安全である。これに対して classic token は古い方式で、広い権限をまとめて与える形になりやすい。古いツールや特殊な用途ではまだ必要になることがあるが、理由がなければ fine-grained の方を優先した方がよい。

SAML SSO は、組織アカウントで「そのトークンが本当に組織のリソースへアクセスしてよいか」を追加で承認する仕組みである。つまり、トークンを作っただけでは終わらず、対象組織に対して明示的に利用許可を与えないと push や pull に失敗することがある。個人リポジトリしか使わないなら通常は意識しなくてよいが、大学や研究所の GitHub 組織では重要になる。

B.4.2 初心者が押さえるべき最小ワークフロー

最初のうちは、次の順序だけ守れば十分である。

1. `git status` で状況を見る
2. `git diff` で変更内容を読む
3. `git add` で記録したいファイルを選ぶ
4. `git diff --cached` でコミット内容を再確認する
5. `git commit -m "..."` で記録する
6. GitHub とやり取りするなら `git pull --ff-only` と `git push` を使う

この流れを身に付ければ、教材の取得、uv プロジェクトの再現、共同研究でのコード共有まで一通り対応できる。最初からブランチ運用や複雑な履歴操作まで覚える必要はないが、`status` と `diff` を読まずに `add` や `commit` をする癖だけは避けた方がよい。

付録 C

pybind11 による C++ 拡張 モジュール

高速化を進めると、最終的に「この部分だけは C++ で書きたい」という場面が出てくる。そのとき、Python から自然に呼べる形で C++ 関数を公開する道具として便利なのが pybind11 である。本付録では、最小限の構成で C++ 製モジュールを作り、`pip install -e .` で使えるようにする流れを示す。

C.1 どのようなときに使うか

pybind11 が向いているのは、重いループや数値計算の核は C++ で書きたいが、入出力や可視化、制御全体は Python に残したい場合である。つまり、「公開する関数の数は少なく、内側だけが重い」という構造に合っている。

具体例として、時刻列 `times` に対して、差が `window` 以下のペア数を数える関数を C++ で実装する。これは高速化章で出てきた「重い内側のループ」を外へ出す典型例である。

C.2 最小構成

ディレクトリ構成は次のようになる。

fastcount_cpp/ の例

```
pyproject.toml
CMakeLists.txt
cpp/
`-- bindings.cpp
src/
`-- fastcount_cpp/
    `-- __init__.py
```

`pyproject.toml` はビルド方法と Python パッケージの情報、`CMakeLists.txt` は C++ 拡張

のコンパイル方法を記述する。cpp/bindings.cpp に C++ の実装と Python への公開方法を書き、src/fastcount_cpp/__init__.py から利用者向けの関数を再公開する。以下では、この構成を上から順に作成する。

C.2.1 pyproject.toml

純粋な Python パッケージと違って、C++ 拡張ではビルドバックエンドも必要になる。ここでは scikit-build-core と pybind11 を使う。pybind11 は setup.py ベースでも扱えるが、この付録では C++ 側の構成を見通しよく保ちやすい CMake ベースの流儀でそろえる。

pyproject.toml

```
[build-system]
requires = [
    "pybind11>=2.13",
    "scikit-build-core>=0.10",
]
build-backend = "scikit_build_core.build"

[project]
name = "fastcount_cpp"
version = "0.1.0"
requires-python = ">=3.12"
dependencies = ["numpy>=2.0"]

[tool.scikit-build]
wheel.packages = ["src/fastcount_cpp"]
```

build-system に書いたパッケージは、拡張モジュールをコンパイルするために必要である。これは通常の実行時依存関係とは別物であり、dependencies へ混ぜない方が整理しやすい。

C.2.2 CMakeLists.txt

CMakeLists.txt には、使用する C++ 規格、コンパイル対象、インストール先を記述する。最小構成は次の通りである。

CMakeLists.txt

```
cmake_minimum_required(VERSION 3.15)
project(fastcount_cpp LANGUAGES CXX)

find_package(pybind11 CONFIG REQUIRED)

pybind11_add_module(_core cpp/bindings.cpp)
target_compile_features(_core PRIVATE cxx_std_17)
install(TARGETS _core DESTINATION fastcount_cpp)
```

pybind11_add_module が、Python から import できる共有ライブラリを作る中心になる。こ

ここで作るモジュール名を `_core` にしておくと、Python 側では `fastcount_cpp._core` として参照できる。

C.2.3 C++ の実装

`cpp/bindings.cpp` では、時刻差を数える C++ 関数を定義し、同じファイル内で `pybind11` による公開名と引数名を指定する。

```
cpp/bindings.cpp
1 #include <cmath>
2 #include <cstdint>
3 #include <vector>
4
5 #include <pybind11/pybind11.h>
6 #include <pybind11/stl.h>
7
8 std::size_t count_pairs_within_window(
9     const std::vector<double>& times,
10     double window
11 ) {
12     std::size_t count = 0;
13     const std::size_t n = times.size();
14
15     for (std::size_t i = 0; i < n; ++i) {
16         for (std::size_t j = i + 1; j < n; ++j) {
17             if (std::abs(times[j] - times[i]) <= window) {
18                 ++count;
19             }
20         }
21     }
22     return count;
23 }
24
25 namespace py = pybind11;
26
27 PYBIND11_MODULE(_core, m) {
28     m.doc() = "Count close pairs in C++.";
29     m.def(
30         "count_pairs_within_window",
31         &count_pairs_within_window,
32         py::arg("times"),
33         py::arg("window"),
34         "Return the number of pairs whose time difference is within window."
35     );
36 }
```

`pybind11/stl.h` を include しているので、Python の `list` は自動で `std::vector<double>` に変換される。つまり、Python 側からは特別な変換コードを書かずに呼べる。

C.2.4 Python 側の入口

C++ 拡張を内部モジュール `_core` として直接公開すると、利用者が実装上の名前を知る必要が生じる。そこで、パッケージの `__init__.py` から公開関数だけを再公開する。

```
src/fastcount_cpp/__init__.py
1 """Python interface for the C++ pair counter."""
2
3 from ._core import count_pairs_within_window
4
5 __all__ = ["count_pairs_within_window"]
```

この `__init__.py` を用意しておく と、利用側は `from fastcount_cpp import count_pairs_within_window` と書ける。C++ 側の内部モジュール名 `_core` を意識せず済む。

C.3 ビルドとインストール

開発中は、プロジェクトの仮想環境を作ったうえで `editable install` するのが分かりやすい。

```
$ uv sync
$ source .venv/bin/activate
$ pip install -e .
$ python -c "from fastcount_cpp import count_pairs_within_window; print(
    count_pairs_within_window([0.0, 0.1, 0.4], 0.25))"
```

最後の出力は次のようになる。

```
1
```

これは、`(0.0, 0.1)` だけが窓 `0.25` の中に入っていることを表している。

ここで重要なのは、純粋な Python の `editable install` と違い、C++ ソースを書き換えたら再ビルドが必要になることが多い点である。つまり、`bindings.cpp` を編集した後は、`pip install -e .` をもう一度実行するか、対応するビルドコマンドを走らせて更新する必要がある。

C.4 Python 側からの使い方

Python から見れば、これは普通のモジュールと同じである。

```
1 from fastcount_cpp import count_pairs_within_window
2
3 times = [0.0, 0.1, 0.4, 0.55]
4 window = 0.2
```

```
5  
6 count = count_pairs_within_window(times, window)  
7 print(count)
```

したがって、設計上は「どの関数を Python へ公開するか」を小さく保つ方がよい。まず Python 側で使いやすい関数シグネチャを決め、その中身だけを C++ へ差し替える、という順序で進めると壊れにくい。

C.5 どこまで C++ へ出すべきか

pybind11 を使えるようになると、つい多くの処理を C++ へ移したくなる。しかし、入出力や可視化、例外処理、引数整理まで全部 C++ へ押し出すと、かえって保守が難しくなる。通常は、次のように役割を分けるとよい。

- Python: ファイル読み込み、引数処理、図の保存、ワークフロー全体
- C++: 重いループ、組合せ探索、繰り返し数値計算

つまり、Python 側には分かりやすい API を残し、C++ 側には計算コアだけを押し込む。これが、pybind11 を長く使いやすく保つための基本方針である。

付録 D

正規表現の基本

正規表現は、文字列の集合を短く記述するための書き方である。grep、awk、Python の re モジュールなどで共通して使われ、ログの抽出、ファイル名の検査、検出器 ID の選別、時刻文字列の確認といった場面で役に立つ。ここでは、本文で触れた grep や awk の延長として、解析で本当に必要になる範囲をまとめる。

D.1 正規表現とは何か

D.1.1 文字どおりの一致とメタ文字

最初の一步は、普通の文字列検索と正規表現の違いを区別することである。det_A という文字列だけを探したいなら、文字をそのまま並べればよい。

```
$ grep "det_A" events.txt
```

これに対して、. や * のような特別な意味を持つ記号を使うと、「どの文字でもよい」「0 回以上くり返す」といった条件を書けるようになる。こうした特別な意味を持つ文字をメタ文字という。

```
.  
*  
+  
?  
^  
$  
[ ]  
( )  
|
```

たとえば det_. は det_A にも det_B にも一致する。. は「任意の 1 文字」を表すからである。逆に、ピリオドそのものを探したいなら、前にバックスラッシュを付けて \. のように

エスケープしなければならない。これは + や ? でも同じである。

D.1.2 ワイルドカードとの違い

シェルのワイルドカードと正規表現は似て見えるが、同じものではない。ls *.txt の * は「任意の文字列」を表すが、正規表現の * は「直前の要素の 0 回以上のくり返し」を表す。したがって、正規表現で「任意の文字列」を書きたいときは .* と書く。

表 D.1 ワイルドカードと正規表現の違い

| 場面 | 記法 | 意味 |
|------|-----------|-------------------------|
| シェル | *.txt | 任意の名前で .txt で終わるファイル |
| 正規表現 | .*\.txt\$ | 任意の文字列で始まり .txt で終わる文字列 |

表 D.1 は、見た目が似ていて混同しやすい 2 つの記法を並べたものである。この違いを混同すると、grep "*.txt" のように書いて期待通りに動かない。シェルが展開する記法なのか、正規表現エンジンが読む記法なのかを意識することが重要である。

D.2 よく使う部品

D.2.1 アンカー（行頭と行末）

^ は行頭、\$ は行末を表す。したがって、文字列の一部ではなく「行全体がその形である」ことを確かめたいときに使う。

```
$ grep '^ERROR' run.log
$ grep 'txt$' filelist.txt
$ grep '^det_A$' detectors.txt
```

1 行目は ERROR で始まる行、2 行目は txt で終わる行、3 行目は行全体がちょうど det_A である行だけを取る。特に 3 行目のように両端をアンカーで挟むと、det_AB や xdet_A が紛れ込むのを防げる。

D.2.2 文字クラス [...] と代表的な省略記法

[abc] は a か b か c の 1 文字、[A-Z] は英大文字 1 文字を表す。検出器名やセンサー番号のように、限られた候補から 1 文字だけ変わる文字列を扱うときに便利である。

```
^det_[AB]$
^run_[0-9][0-9][0-9]$
^[A-Za-z_][A-Za-z0-9_]*$
```

1 行目は `det_A` または `det_B`、2 行目は `run_001` のような 3 桁番号付きの名前、3 行目は Python 風の識別子に近い形を表す。範囲指定の `[0-9]` は非常によく使う。

Python の `re` では、数字 1 文字を `\d`、空白 1 文字を `\s` と書ける。

```
1 import re
2
3 re.fullmatch(r"det_[AB]", "det_A")
4 re.fullmatch(r"\d\d:\d\d:\d\d", "12:34:56")
5 re.search(r"\s+", "det_A 15.2")
```

ただし、`grep` や `awk` では `\d` や `\s` がそのまま使えないことがある。コマンドラインでは `[0-9]` や `[:space:]` を使う方が移植性が高い。

D.2.3 量指定子 `*`, `+`, `?`

量指定子は、直前の要素が何回くり返されるかを表す。`*` は 0 回以上、`+` は 1 回以上、`?` は 0 回または 1 回である。

```
ab*c
ab+c
colou?r
```

`ab*c` は `ac`、`abc`、`abbbc` に一致する。`ab+c` は少なくとも 1 個の `b` が必要なので `ac` には一致しない。`colou?r` は `color` と `colour` の両方に一致する。

解析では、「空白が 1 個以上続く」「符号が付くかもしれない」「小数点以下があるかもしれない」といった曖昧さを扱うときに量指定子が役立つ。

```
1 pattern = re.compile(r"^[+-]?[0-9]+(\.[0-9]+)?$")
2
3 for text in ["12", "-3.5", "+0.25", "1.2.3"]:
4     print(text, bool(pattern.fullmatch(text)))
```

このパターンは、符号付き整数や小数を表し、`1.2.3` のような壊れた文字列は弾く。

D.3 物理・データ解析での具体例

D.3.1 ログから必要な行だけを探す

実験ログやジョブログでは、まず異常行や特定の `run` だけを抜き出したい。アンカーと文字クラスを組み合わせると、かなり具体的な条件を書ける。

```
$ grep '^ERROR' run.log
$ grep '^2026-04-04 .* det_[AB] ' run.log
$ grep -E '^(WARN|ERROR)' run.log
```

3 行目では `grep -E` を使って `|` による選択を書いている。拡張正規表現を使うと括弧や `+` を素直に書けるので、条件が少し複雑になったらこちらの方が読みやすい。

D.3.2 検出器 ID やセンサー名を選別する

検出器名が `det_A`, `det_B`, `det_C` のように並んでいるなら、文字クラスや選択を使って対象を絞れる。

```
$ grep '^det_[AB]$_' detector_ids.txt
$ awk '$1 ~ /^det_[AB]$_/ {print $1, $3}' summary.txt
```

`awk` では `$1 ~ /.../` の形で、1 列目がその正規表現に一致するかどうかを判定できる。列を見ながら条件を書くときは `grep` より `awk` の方が自然なことが多い。

D.3.3 Python で抽出と変換を同時に行う

正規表現は「一致したかどうか」だけでなく、「どの部分が一致したか」を取り出すのにも向いている。Python では `re.search` や `re.fullmatch` の戻り値から、必要部分を取り出せる。

```
1 import re
2
3 line = "2026-04-04 12:34:56 det_A signal=15.2"
4 match = re.search(r"signal=(\d+\.\d+)", line)
5
6 if match:
7     signal = float(match.group(1))
8     print(signal)
```

括弧 (...) で囲んだ部分はキャプチャされ、`group(1)`、`group(2)` のように取り出せる。ログ 1 行から `run` 番号、検出器名、信号値を抜き出して数値へ変換するような処理では、この形が便利である。

D.4 Python の re と grep/awk の使い分け

D.4.1 Python では raw string を使う

Python の文字列中で `\` を多用すると読みにくくなる。そのため、正規表現を書くときは `r"..."` のような raw string を使うのが基本である。

```
1 import re
2
3 pattern = re.compile(r"^det_[A-Z]\d+$")
4 print(bool(pattern.fullmatch("det_A12")))
```

raw string を使わないと、"`\\d+`" のようにバックスラッシュを二重に書かなければならない。正規表現の意図が見えにくくなるので、特別な事情がなければ raw string を選ぶ方がよい。

D.4.2 search, match, fullmatch を使い分ける

`re.search` は文字列のどこかに一致があればよいとき、`re.match` は先頭から一致していればよいとき、`re.fullmatch` は文字列全体がその形かを確認したいときに使う。ファイル名や識別子を検証するときは、意図せず余分な文字が混ざらないよう `fullmatch` を使う方が安全である。

```
1 import re
2
3 print(re.search(r"\d+", "run_42").group())
4 print(bool(re.match(r"run", "run_42")))
5 print(bool(re.fullmatch(r"run_\d+", "run_42")))
```

D.4.3 コマンドラインで十分な場面と、Python に移すべき場面

単に条件に合う行を見つけない、件数を数えたい、特定列だけを抜きたい、といった用途なら `grep` や `awk` が速い。逆に、一致した部分を数値へ変換して次の計算へ渡したい、複数の条件分岐を組み合わせたい、抽出結果を `DataFrame` にしたい、といった場合は Python へ移した方が自然である。

- `grep`: 行全体の抽出や件数確認をすばやく行いたいとき
- `awk`: 列条件と正規表現を組み合わせたいとき
- Python `re`: 抽出した結果をそのまま解析処理へつなげたいとき

正規表現は強力だが、複雑な 1 行を無理に作ると自分でも読めなくなる。まずはアンカー、文字クラス、量指定子の 3 つで書けないかを考え、それでも足りないときに括弧や選択を足す方が安全である。解析コードと同じく、「まず読めること」を優先した方が長く使える。

付録 E

シェルスクリプトによる自動化と一括処理

第 2 章では、シェルを対話的に操作する方法を説明した。同じコマンド列を複数のファイルや条件に対して実行する場合は、操作をシェルスクリプトへ記述すると、処理順序と引数を記録したまま再実行できる。

本付録では、引数の受取り、変数の計算、ファイルの存在確認、条件分岐、配列、for ループを扱う。最後に、設定ファイルのテンプレートを sed で置換して複数条件を実行する方法を説明する。

E.1 シェルスクリプトの基礎構造

E.1.1 シバンと実行権限

シェルスクリプトの先頭には、どのシェルで解釈するかを指定するシバン (shebang) を書く。本付録の例は Bash の配列と算術式を使うため、env で bash の所在を検索する。

```
1 #!/usr/bin/env bash
2
3 echo "Hello, Shell Scripting!"
```

run.sh という名前で保存した場合は、実行権限を付けてから直接実行する。

```
$ chmod +x run.sh
$ ./run.sh
Hello, Shell Scripting!
```

chmod +x は実行権限を追加し、./run.sh は現在のディレクトリにあるファイルを実行する。最後の行がスクリプトの標準出力である。

E.1.2 コマンドライン引数

スクリプトに外から値（ファイル名や設定値）を渡すには、実行時の引数を利用する。スクリプト内からは \$1, \$2 といった特殊な変数で受け取れる。

- \$0: 実行されたスクリプトのファイル名（例: ./run.sh）
- \$1, \$2, ...: 1 番目、2 番目以降の引数
- \$#: 渡された引数の総数
- \$@: 渡された全ての引数

次のスクリプトは、二つの位置引数を検査してから入力名と出力名を表示する。

```
1 #!/bin/bash
2
3 # 引数が2つ渡されたかどうかを確認する簡易チェック
4 if [ "$#" -ne 2 ]; then
5     echo "Usage: $0 <input_file> <output_file>"
6     exit 1
7 fi
8
9 INPUT=$1
10 OUTPUT=$2
11 echo "Reading from $INPUT, writing to $OUTPUT"
```

./run.sh input.csv result.csv と実行すると、Reading from input.csv, writing to result.csv と表示される。引数が二つでなければ使用法を表示し、終了状態 1 で処理を打ち切る。

E.2 変数、演算、条件式

E.2.1 変数の定義と配列

シェルで変数を定義する際は、イコールの前後に空白を入れてはならない。また、変数の値を参照するときは先頭に \$ を付ける。

```
1 COUNT=100
2 NAME="data_analysis"
3
4 echo "The count is $COUNT."      # 展開される
5 echo 'The count is $COUNT.'      # シングルクォートでは展開されず "$COUNT" と表示される
```

Bash では、要素を空白で区切って括弧 () で囲むと配列を定義できる。各要素を引用符で囲めば、要素内の空白やワイルドカードが意図せず展開されることを防げる。

```
1 # 配列の定義
2 targets=("background" "signal" "noise")
3
4 # 配列サイズの取得
5 echo "Number of targets: ${#targets[@]}"
6
7 # 配列要素へのアクセス
8 echo "First element: ${targets[0]}"
9
10 # 配列全体を展開する ( for ループでよく使う )
11 echo "All elements: ${targets[@]}"
```

E.2.2 数値計算（算術式展開）

Bash で整数の加減乗除を行う場合は、算術式展開 `$((...))` または算術評価 `((...))` を用いる。この中では変数名の `$` を省略できる。Bash の算術評価は浮動小数点計算に対応しないため、小数を扱う場合は Python、bc、awk などを用いる。

```
1 A=10
2 B=5
3
4 # 計算式は (( )) を用いる
5 SUM=$(( A + B ))
6 MUL=$(( A * B ))
7
8 # 変数を直接加算・インクリメントする
9 (( A++ ))
10 (( B += 20 ))
```

E.2.3 条件式（test コマンド）

数値の大小比較や、ファイルが存在するかどうかの確認には、`[ ]` を用いる。大括弧の内側には必ずスペースが必要である。

数値の比較:

- `-eq`: 等しい (equal)
- `-ne`: 等しくない (not equal)
- `-gt`: より大きい (greater than)
- `-ge`: 以上 (greater than or equal)
- `-lt`: 未満 (less than)
- `-le`: 以下 (less than or equal)

文字列の比較:

- `=`, `==`: 一致する

- !=: 一致しない
- -z "\$STR": 文字列が空である

ファイルやディレクトリの確認:

- -e "\$FILE": ファイル（またはディレクトリ）が存在する
- -f "\$FILE": 通常のファイルとして存在する
- -d "\$DIR": ディレクトリとして存在する

これらの条件式は、後述する if 文の [] の中身として頻繁に登場する。

E.3 条件分岐 (if, case)

E.3.1 if 文を用いた分岐

前節の条件式を組み合わせて処理を分岐する。then と fi でブロックを囲むのがシェルの特徴である。

```
1 #!/bin/bash
2 FILE=$1
3
4 if [ -z "$FILE" ]; then
5     echo "エラー: ファイル名が指定されていません"
6     exit 1
7 fi
8
9 if [ -f "$FILE" ]; then
10    echo "ファイル $FILE を発見しました。処理を開始します。"
11 elif [ -d "$FILE" ]; then
12    echo "エラー: $FILE はディレクトリです。ファイルを指定してください。"
13    exit 1
14 else
15    echo "エラー: $FILE が存在しません。"
16    exit 1
17 fi
```

E.3.2 case 文による分岐

条件が特定の文字列パターンとの一致で表せる場合は、case 文を用いると elif を重ねずに分岐を列挙できる。拡張子の判定や、-h、--version などのオプション処理に利用できる。

```
1 #!/bin/bash
2 MODE=$1
3
4 case "$MODE" in
```

```
5     "start")
6         echo "Starting process..."
7         ;;
8     "stop" | "halt")
9         echo "Stopping process..."
10        ;;
11    *)
12        echo "Unknown mode: $MODE"
13        echo "Available modes: start, stop, halt"
14        exit 1
15        ;;
16 esac
```

各区分の終わりは `;;` で締めくくる。アスタリスク `*` は「上記以外のすべて」にマッチするワイルドカードとして機能する。

E.4 反復処理と一括自動化 (for)

E.4.1 配列やリストの要素に対する for ループ

指定した値のリストや、定義した配列の全要素に対して処理を繰り返す場合は以下のように書く。

```
1  #!/bin/bash
2
3  # 配列の全要素に対して
4  targets=("run_01" "run_02" "run_03")
5
6  for t in "${targets[@]}; do
7      echo "Processing target: $t"
8  done
9
10 # 単純な数値の手書きリストやコマンド展開に対して
11 for num in 10 20 30; do
12     echo "Number is $num"
13 done
```

E.4.2 実践: 複数ファイルのパス置換とバッチ処理

実務で最もよく使うループは「特定のディレクトリにあるファイルを順次処理する」ための構文である。ここで、入力ファイル名（例: `data/input/run01.csv`）から、出力ファイル名（例: `data/output/run01_result.png`）を自動的に作り出す技術が必要となる。

`basename` コマンドや、変数の後方一致削除（`%...`）を用いることでこれを簡潔に実装できる。

```
1 #!/bin/bash
2
3 INPUT_DIR="data/input"
4 OUTPUT_DIR="data/output"
5
6 # 出力先が存在しなければ作成する
7 if [ ! -d "$OUTPUT_DIR" ]; then
8     mkdir -p "$OUTPUT_DIR"
9 fi
10
11 # INPUT_DIR 中にあるすべての .csv ファイルに対して反復
12 for filepath in "$INPUT_DIR"/*.csv; do
13     # 該当するファイルが存在しない場合はそのままループが回ってしまうのでチェックする
14     if [ ! -f "$filepath" ]; then
15         echo "No CSV files found in $INPUT_DIR"
16         break
17     fi
18
19     echo "Found raw file: $filepath"
20
21     # basename コマンドでディレクトリパスを取り除き、ファイル名だけにする
22     # 例: "data/input/run01.csv" -> "run01.csv"
23     filename=$(basename "$filepath")
24
25     # 文字列置換: 後方の ".csv" を取り除く
26     # 例: "run01.csv" -> "run01"
27     name_no_ext="${filename%.csv}"
28
29     # 新しい出力パスを組み立て直す
30     output_path="$OUTPUT_DIR/${name_no_ext}_result.png"
31
32     echo "Running analysis Python code for: $name_no_ext"
33     # 実際の処理コマンド (例: python analyze.py <in> <out>)
34     # python analyze.py "$filepath" "$output_path"
35 done
```

この例では、入力ファイルごとに拡張子を除いた名前を作り、対応する出力パスを組み立てている。実際に解析コマンドを有効にする前に、echo で入力と出力の対応を表示し、同名ファイルを上書きしないことを確認する。

E.5 sed を用いたテンプレート書き換えと動的実行

外部の実行プログラムの中には、引数 (--input xxx) の形式ではなく、設定ファイル (例: config.ini) を読み込ませることで動作するものがある。バッチ処理で繰り返し実行したい場合、ループを回すたびに設定ファイルの特定のパラメータ (例えばシード値や閾値など)

を書き換える必要が生じる。

sed を使うと、テンプレートとして用意した設定ファイルの一部を置換し、条件ごとの実行用設定ファイルを生成できる。

E.5.1 テンプレートファイルの準備

パラメータ部分をユニークなプレースホルダー文字（例: __THRESHOLD__ や __OUTFILENAME__ のように、ファイル内の他の場所には現れない文字列）にした上で保存しておく。

config_template.ini

```
[Simulation]
Events = 10000
Threshold = __THRESHOLD__
OutputFile = __OUTFILENAME__
```

E.5.2 sed による置換の実行

シェルスクリプト内で条件値を順に取り出し、sed で置換した設定ファイルを生成して実行プログラムへ渡す。

sed 's/置換前文字列/置換後文字列/g' 入力ファイル > 出力ファイル の形式を取る。この際、sed で変数を展開したい場合はシングルクォートではなく、**ダブルクォート** ("...") で囲む必要がある点に注意する。

```
1 #!/bin/bash
2
3 # 閾値を 3 つ変えてシミュレーション用の設定ファイルを動的生成し、実行する例
4 thresholds=(0.5 1.0 2.5)
5
6 for thresh in "${thresholds[@]}; do
7
8     out_name="result_th_${thresh}.dat"
9     run_config="config_run.ini"
10
11     echo "Generating config for Threshold = $thresh"
12
13     # sedコマンドを用いて __THRESHOLD__ と __OUTFILENAME__ の 2 箇所を同時に置換する。
14     # 変数 ($thresh など) を展開させるためダブルクォートで 's/.../.../g' 全体を囲む
15     sed -e "s/__THRESHOLD__/$thresh/g" \
16         -e "s/__OUTFILENAME__/$out_name/g" \
17         config_template.ini > "$run_config"
18
19     # 置換された設定の内容を出力して確認（または直接プログラムを走らせる）
20     cat "$run_config"
21
22     # 以下で実際のプログラムを呼び出す
23     # ./my_simulation_app --config "$run_config"
```

24

25 `done`

この方法は、設定ファイルを入力とする Python プログラムだけでなく、コンパイル済みのシミュレーションプログラムにも適用できる。各反復で生成した設定ファイルを別名で保存すれば、どの条件で各結果を得たかを後から確認できる。ただし、置換値に / や & が含まれる場合は、sed の区切り文字やエスケープ方法を調整する必要がある。

参考文献

- [1] Python Software Foundation, *The Python Tutorial*, <https://docs.python.org/3/tutorial/>, 閱覽日: 2026-04-12.
- [2] Python Software Foundation, *The Python Standard Library*, <https://docs.python.org/3/library/>, 閱覽日: 2026-04-12.
- [3] Project Jupyter, *JupyterLab Documentation*, <https://jupyterlab.readthedocs.io/>, 閱覽日: 2026-04-12.
- [4] Astral, *uv Documentation*, <https://docs.astral.sh/uv/>, 閱覽日: 2026-04-12.
- [5] pyenv contributors, *pyenv README / Documentation*, <https://github.com/pyenv/pyenv>, 閱覽日: 2026-04-12.
- [6] NumPy Developers, *NumPy Documentation*, <https://numpy.org/doc/stable/>, 閱覽日: 2026-04-12.
- [7] NumPy Developers, *NumPy Quickstart*, <https://numpy.org/devdocs/user/quickstart.html>, 閱覽日: 2026-04-12.
- [8] pandas development team, *pandas Documentation*, <https://pandas.pydata.org/docs/>, 閱覽日: 2026-04-12.
- [9] pandas development team, *Getting started tutorials*, https://pandas.pydata.org/docs/getting_started/intro_tutorials/, 閱覽日: 2026-04-12.
- [10] pandas development team, *10 minutes to pandas*, https://pandas.pydata.org/pandas-docs/stable/user_guide/10min.html, 閱覽日: 2026-04-12.
- [11] pandas development team, *Indexing and selecting data*, https://pandas.pydata.org/docs/user_guide/indexing.html, 閱覽日: 2026-04-12.
- [12] pandas development team, *Working with missing data*, https://pandas.pydata.org/docs/user_guide/missing_data.html, 閱覽日: 2026-04-12.
- [13] pandas development team, *Merge, join, concatenate and compare*, https://pandas.pydata.org/docs/user_guide/merging.html, 閱覽日: 2026-04-12.
- [14] pandas development team, *Time series / date functionality*, https://pandas.pydata.org/docs/user_guide/timeseries.html, 閱覽日: 2026-04-12.
- [15] Polars project, *Polars Documentation*, <https://docs.pola.rs/>, 閱覽日: 2026-04-12.
- [16] Polars project, *Concepts*, <https://docs.pola.rs/user-guide/concepts/>, 閱覽日: 2026-04-12.
- [17] Polars project, *Expressions and contexts*, <https://docs.pola.rs/user-guide/concepts/expressions-and-contexts/>, 閱覽日: 2026-04-12.
- [18] Polars project, *Lazy API*, <https://docs.pola.rs/user-guide/concepts/lazy-api/>, 閱覽日: 2026-04-12.

- [19] Polars project, *IO*, <https://docs.pola.rs/user-guide/io/>, 閱覽日: 2026-04-12.
- [20] Polars project, *Coming from Pandas*, <https://docs.pola.rs/user-guide/migration/pandas/>, 閱覽日: 2026-04-12.
- [21] Matplotlib Development Team, *Matplotlib Documentation*, <https://matplotlib.org/stable/>, 閱覽日: 2026-04-12.
- [22] Matplotlib Development Team, *Quick start*, https://matplotlib.org/stable/users/explain/quick_start.html, 閱覽日: 2026-04-12.
- [23] Matplotlib Development Team, *Plot types*, https://matplotlib.org/stable/plot_types/index.html, 閱覽日: 2026-04-12.
- [24] Matplotlib Development Team, *Gallery*, <https://matplotlib.org/stable/gallery/index.html>, 閱覽日: 2026-04-12.
- [25] Matplotlib Development Team, *Cheatsheets*, <https://matplotlib.org/cheatsheets/>, 閱覽日: 2026-04-12.
- [26] SciPy community, *SciPy Documentation*, <https://docs.scipy.org/doc/scipy/>, 閱覽日: 2026-04-12.
- [27] SciPy community, *SciPy User Guide*, <https://docs.scipy.org/doc/scipy/tutorial/>, 閱覽日: 2026-04-12.
- [28] SciPy community, *Statistics*, <https://docs.scipy.org/doc/scipy/tutorial/stats.html>, 閱覽日: 2026-04-12.
- [29] SciPy community, *Optimization*, <https://docs.scipy.org/doc/scipy/tutorial/optimize.html>, 閱覽日: 2026-04-12.
- [30] SciPy community, *Interpolation*, <https://docs.scipy.org/doc/scipy/tutorial/interpolate.html>, 閱覽日: 2026-04-12.
- [31] SciPy community, *Signal processing*, <https://docs.scipy.org/doc/scipy/tutorial/signal.html>, 閱覽日: 2026-04-12.
- [32] SciPy community, *Integration*, <https://docs.scipy.org/doc/scipy/tutorial/integrate.html>, 閱覽日: 2026-04-12.
- [33] Astropy Project, *Astropy Documentation*, <https://docs.astropy.org/>, 閱覽日: 2026-04-12.
- [34] Astropy Project, *Getting Started*, https://docs.astropy.org/en/stable/index_getting_started.html, 閱覽日: 2026-04-12.
- [35] Astropy Project, *User Guide*, https://docs.astropy.org/en/stable/index_user_docs.html, 閱覽日: 2026-04-12.
- [36] Astropy Project, *Units and Quantities*, <https://docs.astropy.org/en/stable/units/index.html>, 閱覽日: 2026-04-12.
- [37] Astropy Project, *Time and Dates*, <https://docs.astropy.org/en/stable/time/index.html>, 閱覽日: 2026-04-12.
- [38] Astropy Project, *Coordinates*, <https://docs.astropy.org/en/stable/coordinates/index.h>

- tml, 閱覽日: 2026-04-12.
- [39] Astropy Project, *Tables*, <https://docs.astropy.org/en/stable/table/index.html>, 閱覽日: 2026-04-12.
- [40] Astropy Project, *FITS File Handling*, <https://docs.astropy.org/en/stable/io/fits/index.html>, 閱覽日: 2026-04-12.
- [41] Git project, *Git Documentation*, <https://git-scm.com/doc>, 閱覽日: 2026-04-12.
- [42] GitHub, *GitHub Docs*, <https://docs.github.com/>, 閱覽日: 2026-04-12.

索引

A

| | |
|----------|-----|
| append() | 53 |
| Astropy | 161 |
| awk | 21 |
| axis | 84 |

B

| | |
|-----------|------------|
| Bash | |
| case 文 | 232 |
| for ループ | 233 |
| if 文 | 232 |
| 配列 | 230 |
| Big-O 記法 | 186 |
| bounds | 153 |
| broadcast | → ブロードキャスト |

C

| | |
|----------------|-----|
| CamelCase | 63 |
| cat | 19 |
| cd | 12 |
| chmod | 30 |
| CMake | 217 |
| CMakeLists.txt | 218 |
| cp | 15 |
| C++ 拡張モジュール | 217 |
| cProfile | 191 |
| CSV | 70 |
| curl | 23 |
| Cython | 192 |

D

| | |
|-----------|----|
| DataFrame | 95 |
| datetime | 74 |

| | |
|------------|---------|
| dict | → 辞書 |
| dictionary | → 辞書 |
| DictReader | 70 |
| docstring | 63, 179 |
| dtype | 80 |

E

| | |
|------------------|----------|
| editable install | 180, 220 |
| elif | 55 |
| else | 55 |
| except | 65 |
| expression | 112 |

F

| | |
|-------------------|-------------|
| FileNotFoundError | 62 |
| filter | 113 |
| FITS | 166 |
| fitting | → curve_fit |
| f 文字列 | 52 |

G

| | |
|-----------------|-----|
| Git | 209 |
| git add | 211 |
| git clone | 212 |
| git commit | 211 |
| git config | 210 |
| git diff | 211 |
| git init | 210 |
| git pull | 214 |
| git push | 214 |
| git remote | 213 |
| git rm --cached | 212 |
| git status | 211 |

| | |
|------------------|----------|
| GitHub | 181, 209 |
| HTTPS | 215 |
| SSH | 215 |
| 認証 | 215 |
| .gitignore | 211 |
| glob | 69 |
| grep | 21 |
| GridSpec | 130 |
| group_by | 114 |
| groupby | 99 |
| gzip | 24 |

H

| | |
|-------------------------|-----|
| Hammer projection | 125 |
| head | 20 |
| help() | 63 |

I

| | |
|--------------|----|
| if | 55 |
| iloc | 97 |
| import | 60 |

J

| | |
|------------------|-----|
| join | 115 |
| JSON | 72 |
| JupyterLab | 41 |

L

| | |
|-----------------|-------|
| LazyFrame | 111 |
| less | 19 |
| linestyle | 139 |
| list | → リスト |
| loc | 97 |
| ls | 10 |

M

| | |
|----------------------------|-----|
| marker | 139 |
| MathText | 143 |
| Matplotlib | 121 |
| Matplotlib | |
| .mplstyle | 142 |
| Axes | 129 |
| axhline() | 123 |
| axvline() | 123 |
| errorbar() | 123 |
| Figure | 129 |
| hist2d() | 124 |
| imshow() | 124 |
| LogNorm | 128 |
| plot() | 123 |
| savefig() | 133 |
| scatter() | 123 |
| mkdir | 14 |
| Mollweide projection | 125 |
| mv | 16 |

N

| | |
|--------------------|-----|
| NaN | 116 |
| ndarray | 79 |
| None | 60 |
| NumPy | 79 |
| NumPy | |
| np.arange | 82 |
| np.array | 82 |
| np.diff | 83 |
| np.histogram | 83 |
| np.linspace | 82 |
| np.mean | 88 |
| np.ones | 82 |
| np.sort | 83 |
| np.std | 88 |
| np.zeros | 82 |

O

| | |
|----------------|-----|
| object 型 | 105 |
| open | 67 |

P

| | |
|-------------------------|----------|
| p0 | 153 |
| pandas | 95 |
| category | 101 |
| chunksize | 103 |
| concat() | 100 |
| datetime | 101 |
| describe() | 96 |
| dropna() | 99 |
| fillna() | 99 |
| index | 97 |
| isna() | 99 |
| iterrows() | 106 |
| itertuples() | 106 |
| merge() | 100 |
| Parquet | 102 |
| pivot_table() | 100 |
| read_csv() | 102 |
| to_latex() | 104 |
| Parquet | 102, 117 |
| 個人用アクセストークン (PAT) | 215 |
| PATH | 26 |
| pathlib | 68 |
| PEP 8 | 63 |
| time.perf_counter | 185 |
| pickle | 73 |
| pip | 40 |
| install | 180 |
| install -e | 180 |
| Polars | 111 |
| collect() | 117 |
| eager API | 112 |
| lazy API | 112 |

| | |
|-----------------------|----------|
| map_elements() | 118 |
| null | 116 |
| Parquet | 117 |
| pivot() | 115 |
| scan_parquet() | 117 |
| pop() | 53 |
| pwd | 10 |
| pybind11 | 192, 217 |
| PYBIND11_MODULE | 219 |
| pyenv | 36 |
| pyproject.toml | 42, 171 |
| pytest | 170 |
| Python | 35 |

Q

| | |
|----------------|-----|
| QTable | 165 |
| Quantity | 161 |

R

| | |
|------------------|-----|
| raw string | 226 |
| rcParams | 141 |
| re モジュール | 226 |
| fullmatch | 227 |
| match | 227 |
| search | 227 |
| rm | 17 |
| rsync | 206 |
| 末尾のスラッシュ | 206 |
| ruff | 170 |

S

| | |
|-------------------------|-----|
| scikit-build-core | 217 |
| SciPy | 149 |
| SciPy | |
| scipy.constants | 149 |
| curve_fit | 149 |

| | |
|-------------------------|--------|
| scipy.integrate | 149 |
| scipy.interpolate | 149 |
| scipy.signal | 149 |
| scipy.stats | 149 |
| scp | 206 |
| sed | 234 |
| select | 113 |
| Series | 95 |
| SettingWithCopy | 105 |
| shape | 80 |
| sigma | 154 |
| SkyCoord | 164 |
| slice | → スライス |
| snake_case | 63 |
| sort() | 53 |
| sorted() | 53 |
| src レイアウト | 170 |
| SSH | 203 |
| ssh | 203 |
| ~/.ssh/config | 205 |
| ssh-agent | 204 |
| ssh-copy-id | 204 |
| ssh-keygen | 203 |
| subplot_mosaic | 130 |

T

| | |
|-----------------|-------|
| Table | 165 |
| tail | 20 |
| tar | 25 |
| tee | 30 |
| test | 231 |
| Time | 163 |
| TimeDelta | 164 |
| %timeit | 185 |
| try | 65 |
| tuple | → タプル |
| type() | 49 |

| | |
|-----------------|----|
| TypeError | 62 |
|-----------------|----|

U

| | |
|------------------|---------|
| UPPER_CASE | 63 |
| uv | 42, 169 |
| uv.lock | 43 |

V

| | |
|------------------|----|
| ValueError | 62 |
| venv | 39 |

W

| | |
|--------------------|-----|
| wc | 20 |
| wget | 23 |
| with_columns | 113 |
| WSL | 7 |

Z

| | |
|-----------|----|
| zip | 25 |
|-----------|----|

ア

| | |
|------------------|--------------|
| アクセス権 | 30 |
| アルゴリズム | 186 |
| アンカー（正規表現） | 224 |
| 依存関係 | 42, 171 |
| インデックス | 54 |
| エラーバー | → errorbar() |
| 演算子 | 51 |
| オーダー | → Big-O 記法 |

カ

| | |
|------------|-----|
| 鍵認証 | 203 |
| 仮想環境 | 39 |

| | |
|-----------|---------------------------|
| 型 | 49 |
| 型ヒント | 179 |
| カラーバー | → <code>colorbar()</code> |
| カラーマップ | 125 |
| 環境変数 | 26 |
| 関数 | 59 |
| 返り値 | 59 |
| 引数 | 59, 59 |
| キーワード引数 | 59 |
| 計算量 | 186 |
| 欠損値 | 99 |
| 公開 API | 177 |
| 公開鍵 | 203 |
| 公開範囲 | 209 |
| 高速化 | 185 |
| コピー | 189 |
| コマンドライン引数 | 230 |

サ

| | |
|-----------|--------------------------|
| 算術式展開 | 231 |
| 参照線 | → <code>axvline()</code> |
| 散布図 | → <code>scatter()</code> |
| シェル | 7 |
| シェルスクリプト | 229 |
| 時間窓 | 198 |
| 時刻差 | 198 |
| 辞書 | 52 |
| 値 | 52 |
| キー | 52 |
| 四則演算 | 51 |
| シバン | 229 |
| 条件分岐 | 55 |
| 剰余 | 51 |
| 初期値 | → <code>p0</code> |
| 真偽値 | 49 |
| ステージングエリア | 210 |
| ストーリーミング | 190 |

| | |
|------|-----------------------|
| スライス | 54, 85 |
| 正規表現 | 223 |
| 整数 | 49 |
| 整数除算 | 51 |
| 線図 | → <code>plot()</code> |
| 添字 | → インデックス |

タ

| | |
|---------|-----|
| ターミナル | 7 |
| 対数軸 | 132 |
| 正しさの検証 | 200 |
| タプル | 52 |
| デフォルト引数 | 59 |
| 投影法 | 125 |
| 同時観測 | 197 |
| トレースバック | 62 |

ナ

| | |
|-----|----|
| 入出力 | 67 |
|-----|----|

ハ

| | |
|----------|--------------------------|
| パイプ | 22 |
| 配列 | → <code>ndarray</code> |
| パス | 13 |
| パッケージ | 169 |
| バッチ処理 | 233 |
| 凡例 | → <code>legend()</code> |
| ヒートマップ | → <code>imshow()</code> |
| 比較演算子 | 55 |
| ヒストグラム | 82, 121 |
| 秘密鍵 | 203 |
| ビュー | 189 |
| フィッティング | → <code>curve_fit</code> |
| 浮動小数点数 | 49 |
| 負の添字 | 54 |
| ブロードキャスト | 87 |

| | |
|--------------|----------|
| プロファイラ | 191 |
| 分割読込み | 103, 190 |
| 並列化 | 190 |
| べき乗 | 51 |
| ベクトル化 | 189 |
| 変数 | 49 |
| ベンチマーク | 201 |
| ボトルネック | 191 |

マ

| | |
|-----------------|---------|
| マジックナンバー | 46, 64 |
| メソッド | 53 |
| メタ文字 | 223 |
| メモリ管理 | 189 |
| 文字クラス | 224 |
| モジュール | 60, 169 |
| 文字列 | 49, 52 |
| f 文字列 | 52 |
| replace() | 52 |
| split() | 52 |
| strip() | 52 |

ヤ

| | |
|------------------------|----|
| ユニバーサル関数 (ufunc) | 83 |
|------------------------|----|

ラ

| | |
|-----------------|-----|
| リスト | 52 |
| リスト内包表記 | 58 |
| リポジトリ | 209 |
| リモートリポジトリ | 213 |
| 量指定子 | 225 |
| 例外 | 62 |
| 例外処理 | 65 |
| ロックファイル | 43 |
| 論理演算子 | 55 |

ワ

| | |
|----------------|---------|
| ワーキングツリー | 210 |
| ワイルドカード | 18, 224 |